7. Library

7.1 Introduction

7.1.1 Definitions of terms

1 A *string* is a contiguous sequence of characters terminated by and including the first null character. The term *multibyte string* is sometimes used instead to emphasize special processing given to multibyte characters contained in the string or to avoid confusion with a wide string. A *pointer to a string* is a pointer to its initial (lowest addressed) character. The *length of a string* is the number of bytes preceding the null character and the *value of a string* is the sequence of the values of the contained characters, in order.

2 The *decimal-point character* is the character used by functions that convert floating-point numbers to or from character sequences to denote the beginning of the fractional part of such character sequences.[212)] It is represented in the text and examples by a period, but may be changed by the **setlocale** function.

3 A *null wide character* is a wide character with code value zero.

4 A *wide string* is a contiguous sequence of wide characters terminated by and including the first null wide character. A *pointer to a wide string* is a pointer to its initial (lowest addressed) wide character. The *length of a wide string* is the number of wide characters preceding the null wide character and the *value of a wide string* is the sequence of code values of the contained wide characters, in order.

5 A *shift sequence* is a contiguous sequence of bytes within a multibyte string that (potentially) causes a change in shift state (see 5.2.2). A shift sequence shall not have a corresponding wide character; it is instead taken to be an adjunct to an adjacent multibyte character.[213)] In this clause, "*white-space character*" refers to (execution) white-space character as defined by **isspace**. "*White-space wide character*" refers to (execution) white-space wide character as defined by **iswspace**.

Forward references: character handling (7.4), the **setlocale** function (7.11.1.1).

7.1.2 Standard headers

1 Each library function is declared in a *header*,[214)] whose contents are made available by the **#include** preprocessing directive. The header declares a set of related functions, plus any types and additional macros needed to facilitate the use of such related functions. In addition to the provisions given in this clause, an implementation that defines **__STDC_IEC_60559_BFP__**, **__STDC_IEC_559__**, or **__STDC_IEC_60559_DFP__** shall conform to the specifications in Annex F, one that defines **__STDC_IEC_60559_COMPLEX__** or **__STDC_IEC_559_COMPLEX__** shall conform to the specifications in Annex G, one that defines **__STDC_IEC_60559_TYPES__** shall conform to the specifications in Annex H and one that defines **__STDC_LIB_EXT1__** shall conform to the specifications in Annex K, and those Annexes should be read as if they were merged into the parallel structure of named subclauses of this clause. Declarations of types described here, in Annex H, or in Annex K, shall not include type qualifiers, unless explicitly stated otherwise.

2 An implementation that does not support decimal floating types (6.10.10.4) may not support interfaces or aspects of interfaces that are specific to these types.

3 The standard headers are[215)]

[212)] The functions that make use of the decimal-point character are the numeric conversion functions (7.24.1, 7.31.4.1) and the formatted input/output functions (7.23.6, 7.31.2).

[213)] For state-dependent encodings, the values for **MB_CUR_MAX** and **MB_LEN_MAX** are thus required to be large enough to count all the bytes in any complete multibyte character plus at least one adjacent shift sequence of maximum length. Whether these counts provide for more than one shift sequence is the implementation's choice.

[214)] A header is not necessarily a source file, nor are the < and > delimited sequences in header names necessarily valid source file names.

[215)] The headers <complex.h>, <stdatomic.h>, and <threads.h> are conditional features that implementations need not support; see 6.10.10.4.

 © ISO 2024 — All rights reserved

`<assert.h>`	`<setjmp.h>`	`<stdlib.h>`
`<complex.h>`	`<signal.h>`	`<stdnoreturn.h>`
`<ctype.h>`	`<stdalign.h>`	`<string.h>`
`<errno.h>`	`<stdarg.h>`	`<tgmath.h>`
`<fenv.h>`	`<stdatomic.h>`	`<threads.h>`
`<float.h>`	`<stdbit.h>`	`<time.h>`
`<inttypes.h>`	`<stdbool.h>`	`<uchar.h>`
`<iso646.h>`	`<stdckdint.h>`	`<wchar.h>`
`<limits.h>`	`<stddef.h>`	`<wctype.h>`
`<locale.h>`	`<stdint.h>`	
`<math.h>`	`<stdio.h>`	

4 If a file with the same name as one of the preceding entries in < and > delimited sequences, not provided as part of the implementation, is placed in any of the standard places that are searched for included source files, the behavior is undefined.

5 Standard headers may be included in any order; each may be included more than once in a given scope, with no effect different from being included only once, except that the effect of including `<assert.h>` depends on the definition of **`NDEBUG`** (see 7.2). If used, a header shall be included outside of any external declaration or definition, and it shall first be included before the first reference to any of the functions or objects it declares, or to any of the types or macros it defines. However, if an identifier is declared or defined in more than one header, the second and subsequent associated headers may be included after the initial reference to the identifier. The program shall not have any macros with names lexically identical to keywords currently defined prior to the inclusion of the header or when any macro defined in the header is expanded.

6 Some standard headers define or declare identifiers that had not been present in previous versions of this document. To allow implementations and users to adapt to that situation, they also define a version macro for feature test of the form **`__STDC_VERSION_`***XXXX*`_H__` which expands to `202311L`, where `XXXX` is the all-caps spelling of the corresponding header `<xxxx.h>`.

7 Any definition of an object-like macro described in this clause or Annex F, Annex G, Annex H, or Annex K shall expand to code that is fully protected by parentheses where necessary, so that it groups in an arbitrary expression as if it were a single identifier.

8 Any declaration of a library function shall have external linkage.

9 A summary of the contents of the standard headers is given in Annex B.

Forward references: diagnostics (7.2).

7.1.3 Reserved identifiers

1 Each header declares or defines all identifiers listed in its associated subclause, and optionally declares or defines identifiers listed in its associated future library directions subclause and identifiers which are always reserved either for any use or for use as file scope identifiers.

— All potentially reserved identifiers (including ones listed in the future library directions) that are provided by an implementation with an external definition are reserved for any use. An implementation shall not provide an external definition of a potentially reserved identifier unless that identifier is reserved for a use where it would have external linkage.[216)] All other potentially reserved identifiers that are provided by an implementation (including in the form of a macro) are reserved for any use when the associated header is included. No other potentially reserved identifiers are reserved.[217)]

— Each macro name in any of the following subclauses (including the future library directions) is reserved for use as specified if any of its associated headers is included; unless explicitly stated otherwise (see 7.1.4).

[216)] All library functions have external linkage.

[217)] A potentially reserved identifier becomes a reserved identifier when an implementation begins using it or a future standard reserves it, but is otherwise available for use by the programmer.

© ISO 2024 — All rights reserved

— All identifiers with external linkage in any of the following subclauses (including the future library directions) and **errno** are always reserved for use as identifiers with external linkage.[218)]

— Each identifier with file scope listed in any of the following subclauses (including the future library directions) is reserved for use as a macro name and as an identifier with file scope in the same name space if any of its associated headers is included.

7.1.4 Use of library functions

1 Each of the following statements applies unless explicitly stated otherwise in the detailed descriptions that follow:

— If an argument to a function has an invalid value (such as a value outside the domain of the function, or a pointer outside the address space of the program, or a null pointer, or a pointer to non-modifiable storage when the corresponding parameter is not const-qualified) or a type (after default argument promotion) not expected by a function with a variable number of arguments, the behavior is undefined.

— If a function argument is described as being an array, the pointer passed to the function shall have a value such that all address computations and accesses to objects (that would be valid if the pointer did point to the first element of such an array) are valid.[219)]

— Any function declared in a header may be additionally implemented as a function-like macro defined in the header, so if a library function is declared explicitly when its header is included, one of the techniques shown later in the next subclause can be used to ensure the declaration is not affected by such a macro. Any macro definition of a function can be suppressed locally by enclosing the name of the function in parentheses, because the name is then not followed by the left parenthesis that indicates expansion of a macro function name. For the same syntactic reason, it is permitted to take the address of a library function even if it is also defined as a macro.[220)] The use of **#undef** to remove any macro definition will also ensure that an actual function is referred to.

— Any invocation of a library function that is implemented as a macro shall expand to code that evaluates each of its arguments exactly once, fully protected by parentheses where necessary, so it is generally safe to use arbitrary expressions as arguments.[221)]

— Likewise, those function-like macros described in the following subclauses may be invoked in an expression anywhere a function with a compatible return type could be called.[222)]

— All object-like macros listed as expanding to integer constant expressions shall additionally be suitable for use in conditional expression inclusion preprocessing directives.

[218)] The list of reserved identifiers with external linkage includes **math_errhandling**, **setjmp**, **va_copy**, and **va_end**.

[219)] This includes, for example, passing a valid pointer that points one-past-the-end of an array along with a size of 0, or using any valid pointer with a size of 0.

[220)] This means that an implementation is required to provide an actual function for each library function, even if it also provides a macro for that function.

[221)] However, such macros may not contain the sequence points that the corresponding function calls do.

[222)] Because external identifiers and some macro names beginning with an underscore are reserved, implementations can provide special semantics for such names. For example, the identifier _BUILTIN_abs could be used to indicate generation of in-line code for the **abs** function. Thus, the appropriate header could specify

```
#define abs(x) _BUILTIN_abs(x)
```

for a compiler whose code generator will accept it.

In this manner, a user desiring to guarantee that a given library function such as **abs** will be a genuine function can write

```
#undef abs
```

whether the implementation's header provides a macro implementation of **abs** or a built-in implementation. The prototype for the function, which precedes and is hidden by any macro definition, is thereby revealed also.

 © ISO 2024 — All rights reserved

2 Provided that a library function can be declared without reference to any type defined in a header, it is also permissible to declare the function and use it without including its associated header.

3 There is a sequence point immediately before a library function returns.

4 The functions in the standard library are not guaranteed to be reentrant and may modify objects with static or thread storage duration.[223)]

5 Unless explicitly stated otherwise in the detailed descriptions that follow, library functions shall prevent data races as follows: A library function shall not directly or indirectly access objects accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function's arguments. A library function shall not directly or indirectly modify objects accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function's non-const arguments.[224)] Implementations may share their own internal objects between threads if the objects are not visible to users and are protected against data races.

6 Unless otherwise specified, library functions shall perform all operations solely within the current thread if those operations have effects that are visible to users.[225)]

7 EXAMPLE The function **atoi** can be used in any of several ways:

— by use of its associated header (possibly generating a macro expansion)

```
#include <stdlib.h>
const char *str;
/* ... */
i = atoi(str);
```

— by use of its associated header (assuredly generating a true function reference)

```
#include <stdlib.h>
#undef atoi
const char *str;
/* ... */
i = atoi(str);
```

or

```
#include <stdlib.h>
const char *str;
/* ... */
i = (atoi)(str);
```

— by explicit declaration

```
extern int atoi(const char *);
const char *str;
/* ... */
i = atoi(str);
```

[223)] Thus, a signal handler cannot, in general, call standard library functions.

[224)] This means, for example, that an implementation is not permitted to use a **static** object for internal purposes without synchronization because it could cause a data race even in programs that do not explicitly share objects between threads. Similarly, an implementation of **memcpy** is not permitted to copy bytes beyond the specified length of the destination object and then restore the original values because it could cause a data race if the program shared those bytes between threads.

[225)] This allows implementations to parallelize operations if there are no visible side effects.

© ISO 2024 — All rights reserved

7.2 Diagnostics <assert.h>

7.2.1 General

1 The header <assert.h> defines the **assert** and **__STDC_VERSION_ASSERT_H__** macros and refers to another macro,

```
NDEBUG
```

which is *not* defined by <assert.h>. If **NDEBUG** is defined as a macro name at the point in the source file where <assert.h> is included, the **assert** macro is defined simply as

```
#define assert(...) ((void)0)
```

The **assert** macro is redefined according to the current state of **NDEBUG** each time that <assert.h> is included.

2 The **assert** macro shall be implemented as a macro with an ellipsis parameter, not as an actual function. If the macro definition is suppressed to access an actual function, the behavior is undefined.

3 NOTE Nevertheless, when **NDEBUG** is not defined, the macro acts as a function taking one parameter as indicated by the prototype as given later in this subclause (7.2.2.1). For both **assert**() and **assert**(1, 1), the number of arguments does not agree with the number of parameters. A diagnostic is required by 6.5.3.3.

4 The macro

```
__STDC_VERSION_ASSERT_H__
```

is an integer constant expression with a value equivalent to 202311L.

7.2.2 Program diagnostics

7.2.2.1 The assert macro

Synopsis

1
```
#include <assert.h>
void assert(scalar expression);
```

Description

2 The **assert** macro puts diagnostic tests into programs; it expands to a void expression. When it is executed, if expression (which shall have a scalar type) is false (that is, compares equal to 0), the **assert** macro writes information about the particular invocation that failed (including the text of the argument, the name of the source file, the source line number, and the name of the enclosing function — the latter are respectively the values of the preprocessing macros **__FILE__** and **__LINE__** and of the identifier **__func__**) on the standard error stream in an implementation-defined format.[226)]

It then calls the **abort** function.

Returns

3 The **assert** macro returns no value.

Forward references: the **abort** function (7.24.4.1).

[226)] The message written can be of the form:

```
Assertion failed: expression, function abc, file xyz, line nnn.
```

 © ISO 2024 — All rights reserved

7.3 Complex arithmetic <complex.h>

7.3.1 Introduction

1 The header <complex.h> defines macros and declares functions that support complex arithmetic.[227)]

2 Implementations that define the macro **__STDC_NO_COMPLEX__** may not provide this header nor support any of its facilities.

3 The macro

```
__STDC_VERSION_COMPLEX_H__
```

is an integer constant expression with a value equivalent to `202311L`.

4 Each synopsis, other than for the **CMPLX** macros, specifies a family of functions consisting of a principal function with one or more **double complex** parameters and a **double complex** or **double** return value; and other functions with the same name but with **f** and **l** suffixes which are corresponding functions with **float** and **long double** parameters and return values.

5 The macro

```
complex
```

expands to **_Complex**; the macro

```
_Complex_I
```

expands to an arithmetic constant expression of type **float _Complex**, with the value of the imaginary unit.[228)]

6 The macros

```
imaginary
```

and

```
_Imaginary_I
```

are defined if and only if the implementation supports imaginary types;[229)] and, if defined, they expand to **_Imaginary** and an arithmetic constant expression of type **float _Imaginary** with the value of the imaginary unit.

7 The macro

```
I
```

expands to either **_Imaginary_I** or **_Complex_I**. If **_Imaginary_I** is not defined, **I** shall expand to **_Complex_I**.

8 Notwithstanding the provisions of 7.1.3, a program may undefine and perhaps then redefine the macros **complex**, **imaginary**, and **I**.

Forward references: the **CMPLX** macros (7.3.9.3), ISO/IEC 60559-compatible complex arithmetic (Annex G).

7.3.2 Conventions

1 Values are interpreted as radians, not degrees. An implementation may set **errno** but is not required to do so.

[227)] See "future library directions" (7.33.1).
[228)] The imaginary unit is a number i such that $i^2 = -1$.
[229)] A specification for imaginary types is in Annex G.

© ISO 2024 — All rights reserved

7.3.3 Branch cuts

1 Some of the following functions have branch cuts, across which the function is discontinuous. For implementations with a signed zero (including all ISO/IEC 60559 implementations) that follow the specifications of Annex G, the sign of zero distinguishes one side of a cut from another so that the function is continuous (except for format limitations) as the cut is approached from either side. For example, for the square root function, which has a branch cut along the negative real axis, the top of the cut, with imaginary part `+0`, maps to the positive imaginary axis, and the bottom of the cut, with imaginary part `-0`, maps to the negative imaginary axis.

2 Implementations that do not support a signed zero (see Annex F) cannot distinguish the sides of branch cuts. These implementations shall map a cut so that the function is continuous as the cut is approached coming around the finite endpoint of the cut in a counter clockwise direction. (Branch cuts for the functions specified here have just one finite endpoint.) For example, in the square root function, coming counter clockwise around the finite endpoint of the cut along the negative real axis approaches the cut from above, so that the cut maps to the positive imaginary axis.

7.3.4 The `CX_LIMITED_RANGE` pragma

Synopsis

1
```
#include <complex.h>
#pragma STDC CX_LIMITED_RANGE on-off-switch
```

Description

2 The usual mathematical formulas for complex multiply, divide, and absolute value are problematic because of their treatment of infinities and because of undue overflow and underflow. The `CX_LIMITED_RANGE` pragma can be used to inform the implementation that (where the state is "on") the usual mathematical formulas are acceptable.[230)] The pragma can occur either outside external declarations or preceding all explicit declarations and statements inside a compound statement. When outside external declarations, the pragma takes effect from its occurrence until another `CX_LIMITED_RANGE` pragma is encountered, or until the end of the translation unit. When inside a compound statement, the pragma takes effect from its occurrence until another `CX_LIMITED_RANGE` pragma is encountered (including within a nested compound statement), or until the end of the compound statement; at the end of a compound statement the state for the pragma is restored to its condition just before the compound statement. If this pragma is used in any other context, the behavior is undefined. The default state for the pragma is "off".

7.3.5 Trigonometric functions

7.3.5.1 The `cacos` functions

Synopsis

1
```
#include <complex.h>
double complex cacos(double complex z);
float complex cacosf(float complex z);
long double complex cacosl(long double complex z);
```

Description

2 The `cacos` functions compute the complex arc cosine of `z`, with branch cuts outside the interval $[-1, +1]$ along the real axis.

[230)] The purpose of the pragma is to allow the implementation to use the formulas:

$$\begin{aligned} (x+iy)\times(u+iv) &= (xu-yv)+i(yu+xv) \\ (x+iy)\,/\,(u+iv) &= [(xu+yv)+i(yu-xv)]/(u^2+v^2) \\ |x+iy| &= \sqrt{x^2+y^2} \end{aligned}$$

where the programmer can determine they are safe.

 © ISO 2024 — All rights reserved

Returns

3 The **cacos** functions return the complex arc cosine value, in the range of a strip mathematically unbounded along the imaginary axis and in the interval $[0, \pi]$ along the real axis.

7.3.5.2 The casin functions

Synopsis

1
```
#include <complex.h>
double complex casin(double complex z);
float complex casinf(float complex z);
long double complex casinl(long double complex z);
```

Description

2 The **casin** functions compute the complex arc sine of z, with branch cuts outside the interval $[-1, +1]$ along the real axis.

Returns

3 The **casin** functions return the complex arc sine value, in the range of a strip mathematically unbounded along the imaginary axis and in the interval $[-\frac{\pi}{2}, +\frac{\pi}{2}]$ along the real axis.

7.3.5.3 The catan functions

Synopsis

1
```
#include <complex.h>
double complex catan(double complex z);
float complex catanf(float complex z);
long double complex catanl(long double complex z);
```

Description

2 The **catan** functions compute the complex arc tangent of z, with branch cuts outside the interval $[-i, +i]$ along the imaginary axis.

Returns

3 The **catan** functions return the complex arc tangent value, in the range of a strip mathematically unbounded along the imaginary axis and in the interval $[-\frac{\pi}{2}, +\frac{\pi}{2}]$ along the real axis.

7.3.5.4 The ccos functions

Synopsis

1
```
#include <complex.h>
double complex ccos(double complex z);
float complex ccosf(float complex z);
long double complex ccosl(long double complex z);
```

Description

2 The **ccos** functions compute the complex cosine of z.

Returns

3 The **ccos** functions return the complex cosine value.

7.3.5.5 The csin functions

Synopsis

1
```
#include <complex.h>
double complex csin(double complex z);
float complex csinf(float complex z);
long double complex csinl(long double complex z);
```

© ISO 2024 — All rights reserved

Description

2 The **csin** functions compute the complex sine of z.

Returns

3 The **csin** functions return the complex sine value.

7.3.5.6 The ctan functions

Synopsis

1
```
#include <complex.h>
double complex ctan(double complex z);
float complex ctanf(float complex z);
long double complex ctanl(long double complex z);
```

Description

2 The **ctan** functions compute the complex tangent of z.

Returns

3 The **ctan** functions return the complex tangent value.

7.3.6 Hyperbolic functions

7.3.6.1 The cacosh functions

Synopsis

1
```
#include <complex.h>
double complex cacosh(double complex z);
float complex cacoshf(float complex z);
long double complex cacoshl(long double complex z);
```

Description

2 The **cacosh** functions compute the complex arc hyperbolic cosine of z, with a branch cut at values less than 1 along the real axis.

Returns

3 The **cacosh** functions return the complex arc hyperbolic cosine value, in the range of a half-strip of nonnegative values along the real axis and in the interval $[-i\pi, +i\pi]$ along the imaginary axis.

7.3.6.2 The casinh functions

Synopsis

1
```
#include <complex.h>
double complex casinh(double complex z);
float complex casinhf(float complex z);
long double complex casinhl(long double complex z);
```

Description

2 The **casinh** functions compute the complex arc hyperbolic sine of z, with branch cuts outside the interval $[-i, +i]$ along the imaginary axis.

Returns

3 The **casinh** functions return the complex arc hyperbolic sine value, in the range of a strip mathematically unbounded along the real axis and in the interval $[-\frac{i\pi}{2}, +\frac{i\pi}{2}]$ along the imaginary axis.

7.3.6.3 The catanh functions

Synopsis

1
```
#include <complex.h>
```

 © ISO 2024 — All rights reserved

```
double complex catanh(double complex z);
float complex catanhf(float complex z);
long double complex catanhl(long double complex z);
```

Description

2 The **catanh** functions compute the complex arc hyperbolic tangent of z, with branch cuts outside the interval $[-1, +1]$ along the real axis.

Returns

3 The **catanh** functions return the complex arc hyperbolic tangent value, in the range of a strip mathematically unbounded along the real axis and in the interval $[-\frac{i\pi}{2}, +\frac{i\pi}{2}]$ along the imaginary axis.

7.3.6.4 The ccosh functions

Synopsis

1
```
#include <complex.h>
double complex ccosh(double complex z);
float complex ccoshf(float complex z);
long double complex ccoshl(long double complex z);
```

Description

2 The **ccosh** functions compute the complex hyperbolic cosine of z.

Returns

3 The **ccosh** functions return the complex hyperbolic cosine value.

7.3.6.5 The csinh functions

Synopsis

1
```
#include <complex.h>
double complex csinh(double complex z);
float complex csinhf(float complex z);
long double complex csinhl(long double complex z);
```

Description

2 The **csinh** functions compute the complex hyperbolic sine of z.

Returns

3 The **csinh** functions return the complex hyperbolic sine value.

7.3.6.6 The ctanh functions

Synopsis

1
```
#include <complex.h>
double complex ctanh(double complex z);
float complex ctanhf(float complex z);
long double complex ctanhl(long double complex z);
```

Description

2 The **ctanh** functions compute the complex hyperbolic tangent of z.

Returns

3 The **ctanh** functions return the complex hyperbolic tangent value.

7.3.7 Exponential and logarithmic functions

7.3.7.1 The cexp functions

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <complex.h>
double complex cexp(double complex z);
float complex cexpf(float complex z);
long double complex cexpl(long double complex z);
```

Description

2 The **cexp** functions compute the complex base-e exponential of z.

Returns

3 The **cexp** functions return the complex base-e exponential value.

7.3.7.2 The clog functions

Synopsis

1
```
#include <complex.h>
double complex clog(double complex z);
float complex clogf(float complex z);
long double complex clogl(long double complex z);
```

Description

2 The **clog** functions compute the complex natural (base-e) logarithm of z, with a branch cut along the negative real axis.

Returns

3 The **clog** functions return the complex natural logarithm value, in the range of a strip mathematically unbounded along the real axis and in the interval $[-i\pi, +i\pi]$ along the imaginary axis.

7.3.8 Power and absolute-value functions

7.3.8.1 The cabs functions

Synopsis

1
```
#include <complex.h>
double cabs(double complex z);
float cabsf(float complex z);
long double cabsl(long double complex z);
```

Description

2 The **cabs** functions compute the complex absolute value (also called norm, modulus, or magnitude) of z.

Returns

3 The **cabs** functions return the complex absolute value.

7.3.8.2 The cpow functions

Synopsis

1
```
#include <complex.h>
double complex cpow(double complex x, double complex y);
float complex cpowf(float complex x, float complex y);
long double complex cpowl(long double complex x, long double complex y);
```

Description

2 The **cpow** functions compute the complex power function x^y, with a branch cut for the first parameter along the negative real axis.

© ISO 2024 — All rights reserved

Returns

3 The **cpow** functions return the complex power function value.

7.3.8.3 The csqrt functions

Synopsis

1
```
#include <complex.h>
double complex csqrt(double complex z);
float complex csqrtf(float complex z);
long double complex csqrtl(long double complex z);
```

Description

2 The **csqrt** functions compute the complex square root of z, with a branch cut along the negative real axis.

Returns

3 The **csqrt** functions return the complex square root value, in the range of the right half-plane (including the imaginary axis).

7.3.9 Manipulation functions

7.3.9.1 The carg functions

Synopsis

1
```
#include <complex.h>
double carg(double complex z);
float cargf(float complex z);
long double cargl(long double complex z);
```

Description

2 The **carg** functions compute the argument (also called phase (which is an angle)) of z, with a branch cut along the negative real axis.

Returns

3 The **carg** functions return the value of the argument in the interval $[-\pi, +\pi]$.

7.3.9.2 The cimag functions

Synopsis

1
```
#include <complex.h>
double cimag(double complex z);
float cimagf(float complex z);
long double cimagl(long double complex z);
```

Description

2 The **cimag** functions compute the imaginary part of z.[231)]

Returns

3 The **cimag** functions return the imaginary part value (as a real).

7.3.9.3 The CMPLX macros

Synopsis

1
```
#include <complex.h>
double complex CMPLX(double x, double y);
float complex CMPLXF(float x, float y);
```

[231)] For a complex variable z, z and **CMPLX**(**creal**(z), **cimag**(z)) are equivalent expressions. If imaginary types are supported, z and **creal**(z)+**cimag**(z)***I** are equivalent expressions.

© ISO 2024 — All rights reserved

```
      long double complex CMPLXL(long double x, long double y);
```

Description

2 The **CMPLX** macros expand to an expression of the specified complex type, with the real part having the (converted) value of x and the imaginary part having the (converted) value of y. The resulting expression shall be suitable for use as an initializer for an object with static or thread storage duration, provided both arguments are likewise suitable. The resulting expression shall be an arithmetic constant expression, provided both arguments are arithmetic constant expressions.

Returns

3 The **CMPLX** macros return the complex value $x + iy$.

4 NOTE These macros act as if the implementation supported imaginary types and the definitions were:

```
#define CMPLX(x, y)  ((double complex)((double)(x) + \
                              _Imaginary_I * (double)(y)))
#define CMPLXF(x, y) ((float complex)((float)(x) + \
                              _Imaginary_I * (float)(y)))
#define CMPLXL(x, y) ((long double complex)((long double)(x) + \
                              _Imaginary_I * (long double)(y)))
```

7.3.9.4 The conj functions

Synopsis

1
```
      #include <complex.h>
      double complex conj(double complex z);
      float complex conjf(float complex z);
      long double complex conjl(long double complex z);
```

Description

2 The **conj** functions compute the complex conjugate of z, by negating the sign of its imaginary part.

Returns

3 The **conj** functions return the complex conjugate value.

7.3.9.5 The cproj functions

Synopsis

1
```
      #include <complex.h>
      double complex cproj(double complex z);
      float complex cprojf(float complex z);
      long double complex cprojl(long double complex z);
```

Description

2 The **cproj** functions compute a projection of z onto the Riemann sphere where z projects to z except that all complex infinities (even those with one infinite part and one NaN part) project to positive infinity on the real axis. If z has an infinite part, then **cproj**(z) is equivalent to

```
      INFINITY + I * copysign(0.0, cimag(z))
```

Returns

3 The **cproj** functions return the value of the projection onto the Riemann sphere.

7.3.9.6 The creal functions

Synopsis

1
```
      #include <complex.h>
```

 © ISO 2024 — All rights reserved

```
double creal(double complex z);
float crealf(float complex z);
long double creall(long double complex z);
```

Description

2 The **creal** functions compute the real part of z.[232)]

Returns

3 The **creal** functions return the real part value.

[232)] For a complex variable z, z and **CMPLX**(**creal**(z), **cimag**(z)) are equivalent expressions. If imaginary types are supported, z and **creal**(z)+**cimag**(z)∗**I** are equivalent expressions.

© ISO 2024 — All rights reserved

7.4 Character handling <ctype.h>

7.4.1 General

1 The header <ctype.h> declares several functions useful for classifying and mapping characters.[233] In all cases the argument is an **int**, the value of which shall be representable as an **unsigned char** or shall equal the value of the macro **EOF**. If the argument has any other value, the behavior is undefined.

2 The behavior of these functions is affected by the current locale. Those functions that have locale-specific aspects only when not in the "C" locale are noted subsequently in this subclause.

3 The term *printing character* refers to a member of a locale-specific set of characters, each of which occupies one printing position on a display device; the term *control character* refers to a member of a locale-specific set of characters that are not printing characters.[234] All letters and digits are printing characters.

Forward references:

EOF (7.23.1), localization (7.11).

7.4.2 Character classification functions

1 The functions in this subclause return nonzero (true) if and only if the value of the argument c conforms to that in the description of the function.

7.4.2.1 The isalnum function

Synopsis

1
```
#include <ctype.h>
int isalnum(int c);
```

Description

2 The **isalnum** function tests for any character for which **isalpha** or **isdigit** is true.

7.4.2.2 The isalpha function

Synopsis

1
```
#include <ctype.h>
int isalpha(int c);
```

Description

2 The **isalpha** function tests for any character for which **isupper** or **islower** is true, or any character that is one of a locale-specific set of alphabetic characters for which none of **iscntrl**, **isdigit**, **ispunct**, or **isspace** is true.[235] In the "C" locale, **isalpha** returns true only for the characters for which **isupper** or **islower** is true.

7.4.2.3 The isblank function

Synopsis

1
```
#include <ctype.h>
int isblank(int c);
```

Description

2 The **isblank** function tests for any character that is a standard blank character or is one of a locale-specific set of characters for which **isspace** is true and that is used to separate words within a line

[233] See "future library directions" (7.33.2).

[234] In an implementation that uses the seven-bit US ASCII character set, the printing characters are those whose values lie from 0x20 (space) through 0x7E (tilde); the control characters are those whose values lie from 0 (NUL) through 0x1F (US), and the character 0x7F (DEL).

[235] The functions **islower** and **isupper** test true or false separately for each of these additional characters; all four combinations are possible.

© ISO 2024 — All rights reserved

of text. The standard blank characters are the following: space (`' '`), and horizontal tab (`'\t'`). In the `"C"` locale, **`isblank`** returns true only for the standard blank characters.

7.4.2.4 The `iscntrl` function

Synopsis

1
```
#include <ctype.h>
int iscntrl(int c);
```

Description

2 The **`iscntrl`** function tests for any control character.

7.4.2.5 The `isdigit` function

Synopsis

1
```
#include <ctype.h>
int isdigit(int c);
```

Description

2 The **`isdigit`** function tests for any decimal-digit character (as defined in 5.2.1).

7.4.2.6 The `isgraph` function

Synopsis

1
```
#include <ctype.h>
int isgraph(int c);
```

Description

2 The **`isgraph`** function tests for any printing character except space (`' '`).

7.4.2.7 The `islower` function

Synopsis

1
```
#include <ctype.h>
int islower(int c);
```

Description

2 The **`islower`** function tests for any character that is a lowercase letter or is one of a locale-specific set of characters for which none of **`iscntrl`**, **`isdigit`**, **`ispunct`**, or **`isspace`** is true. In the `"C"` locale, **`islower`** returns true only for the lowercase letters (as defined in 5.2.1).

7.4.2.8 The `isprint` function

Synopsis

1
```
#include <ctype.h>
int isprint(int c);
```

Description

2 The **`isprint`** function tests for any printing character including space (`' '`).

7.4.2.9 The `ispunct` function

Synopsis

1
```
#include <ctype.h>
int ispunct(int c);
```

© ISO 2024 — All rights reserved

Description

2 The **ispunct** function tests for any printing character that is one of a locale-specific set of punctuation characters for which neither **isspace** nor **isalnum** is true. In the "C" locale, **ispunct** returns true for every printing character for which neither **isspace** nor **isalnum** is true.

7.4.2.10 The isspace function

Synopsis

1
```
#include <ctype.h>
int isspace(int c);
```

Description

2 The **isspace** function tests for any character that is a standard white-space character or is one of a locale-specific set of characters for which **isalnum** is false. The standard white-space characters are the following: space (' '), form feed ('\f'), new-line ('\n'), carriage return ('\r'), horizontal tab ('\t'), and vertical tab ('\v'). In the "C" locale, **isspace** returns true only for the standard white-space characters.

7.4.2.11 The isupper function

Synopsis

1
```
#include <ctype.h>
int isupper(int c);
```

Description

2 The **isupper** function tests for any character that is an uppercase letter or is one of a locale-specific set of characters for which none of **iscntrl**, **isdigit**, **ispunct**, or **isspace** is true. In the "C" locale, **isupper** returns true only for the uppercase letters (as defined in 5.2.1).

7.4.2.12 The isxdigit function

Synopsis

1
```
#include <ctype.h>
int isxdigit(int c);
```

Description

2 The **isxdigit** function tests for any hexadecimal-digit character (as defined in 6.4.4.2).

7.4.3 Character case mapping functions

7.4.3.1 The tolower function

Synopsis

1
```
#include <ctype.h>
int tolower(int c);
```

Description

2 The **tolower** function converts an uppercase letter to a corresponding lowercase letter.

Returns

3 If the argument is a character for which **isupper** is true and there are one or more corresponding characters, as specified by the current locale, for which **islower** is true, the **tolower** function returns one of the corresponding characters (always the same one for any given locale); otherwise, the argument is returned unchanged.

7.4.3.2 The toupper function

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <ctype.h>
int toupper(int c);
```

Description

2 The **toupper** function converts a lowercase letter to a corresponding uppercase letter.

Returns

3 If the argument is a character for which **islower** is true and there are one or more corresponding characters, as specified by the current locale, for which **isupper** is true, the **toupper** function returns one of the corresponding characters (always the same one for any given locale); otherwise, the argument is returned unchanged.

© ISO 2024 — All rights reserved

7.5 Errors <errno.h>

1 The header <errno.h> defines several macros, all relating to the reporting of error conditions.

2 The macros are

```
EDOM
EILSEQ
ERANGE
```

which expand to integer constant expressions with type **int**, distinct positive values, and which are suitable for use in conditional expression inclusion preprocessing directives; and

```
errno
```

which expands to a modifiable lvalue[236] that has type **int** and thread storage duration, the value of which is set to a positive error number by several library functions. If a macro definition is suppressed to access an actual object, or a program defines an identifier with the name **errno**, the behavior is undefined.

3 The value of **errno** in the initial thread is zero at program startup (the initial representation of the object designated by **errno** in other threads is indeterminate), but is never set to zero by any library function.[237] The value of **errno** may be set to nonzero by a library function call whether or not there is an error, provided the use of **errno** is not documented in the description of the function in this document.

4 Additional macro definitions, beginning with **E** and a digit or **E** and an uppercase letter,[238] may also be specified by the implementation.

[236] The macro **errno** need not be the identifier of an object. Expansion to a modifiable lvalue resulting from a function call (for example, `(*errno())`) is a viable implementation strategy.

[237] Thus, a program that uses **errno** for error checking would set it to zero before a library function call, then inspect it before a subsequent library function call. Of course, a library function can save the value of **errno** on entry and then set it to zero, as long as the original value is restored if **errno**'s value is still zero just before the return.

[238] See "future library directions" (7.33.3).

© ISO 2024 — All rights reserved

7.6 Floating-point environment <fenv.h>

1 The header <fenv.h> defines several macros, and declares types and functions that provide access to the floating-point environment. The *floating-point environment* refers collectively to any floating-point status flags and control modes supported by the implementation.[239)]

A *floating-point status flag* is a system variable whose value is set (but never cleared) when a *floating-point exception* is raised, which occurs as a side effect of exceptional floating-point arithmetic to provide auxiliary information.[240)] A *floating-point control mode* is a system variable whose value may be set by the user to affect the subsequent behavior of floating-point arithmetic.

2 A floating-point control mode may be *constant* (7.6.2) or *dynamic*. The *dynamic floating-point environment* includes the dynamic floating-point control modes and the floating-point status flags.

3 The dynamic floating-point environment has thread storage duration. The initial state for a thread's dynamic floating-point environment is the current state of the dynamic floating-point environment of the thread that creates it. It is initialized at the time of the thread's creation.

4 Certain programming conventions support the intended model of use for the dynamic floating-point environment:[241)]

— a function call does not alter its caller's floating-point control modes, clear its caller's floating-point status flags, nor depend on the state of its caller's floating-point status flags unless the function is so documented;

— a function call is assumed to require default floating-point control modes, unless its documentation promises otherwise;

— a function call is assumed to have the potential for raising floating-point exceptions, unless its documentation promises otherwise.

5 The feature test macro **__STDC_VERSION_FENV_H__** expands to the token 202311L.

6 The type

```
fenv_t
```

represents the entire dynamic floating-point environment.

7 The type

```
femode_t
```

represents the collection of dynamic floating-point control modes supported by the implementation, including the dynamic rounding direction mode.

8 The type

```
fexcept_t
```

represents the floating-point status flags collectively, including any status the implementation associates with the flags.

9 Each of the macros

[239)] This header is designed to support the floating-point exception status flags and rounding-direction control modes required by ISO/IEC 60559, and other similar floating-point state information. It is also designed to facilitate code portability among all systems.

[240)] A floating-point status flag is not an object and can be set more than once within an expression.

[241)] With these conventions, a programmer can safely assume default floating-point control modes (or be unaware of them). The responsibilities associated with accessing the floating-point environment fall on the programmer or program that does so explicitly.

© ISO 2024 — All rights reserved

```
FE_DIVBYZERO
FE_INEXACT
FE_INVALID
FE_OVERFLOW
FE_UNDERFLOW
```

is defined if and only if the implementation supports the floating-point exception by means of the functions in 7.6.4.[242] Additional implementation-defined floating-point exceptions, with macro definitions beginning with **FE_** and an uppercase letter,[243] may also be specified by the implementation. The defined macros expand to integer constant expressions with values such that bitwise ORs of all combinations of the macros result in distinct values, and furthermore, bitwise ANDs of all combinations of the macros result in zero.[244]

10 Decimal floating-point operations and ISO/IEC 60559 binary floating-point operations (Annex F) access the same floating-point exception status flags.

11 The macro

```
FE_DFL_MODE
```

represents the default state for the collection of dynamic floating-point control modes supported by the implementation – and has type "pointer to const-qualified **femode_t**". Additional implementation-defined states for the dynamic mode collection, with macro definitions beginning with **FE_** and an uppercase letter, and having type "pointer to const-qualified **femode_t**", may also be specified by the implementation.

12 The macro

```
FE_ALL_EXCEPT
```

is the bitwise OR of all floating-point exception macros defined by the implementation. If no such macros are defined, **FE_ALL_EXCEPT** shall be defined as `0`.

13 Each of the macros

```
FE_DOWNWARD
FE_TONEAREST
FE_TONEARESTFROMZERO
FE_TOWARDZERO
FE_UPWARD
```

is defined if and only if the implementation supports getting and setting the represented rounding direction by means of the **fegetround** and **fesetround** functions. The defined macros expand to integer constant expressions whose values are distinct nonnegative values. Additional implementation-defined rounding directions, with macro definitions beginning with **FE_** and an uppercase letter,[245] may also be specified by the implementation.[246]

14 If the implementation supports decimal floating types, each of the macros

```
FE_DEC_DOWNWARD
FE_DEC_TONEAREST
FE_DEC_TONEARESTFROMZERO
FE_DEC_TOWARDZERO
```

[242] The implementation supports a floating-point exception if there are circumstances where a call to at least one of the functions in 7.6.4, using the macro as the appropriate argument, will succeed. It is not necessary for all the functions to succeed all the time.

[243] See "future library directions" (7.33.4).

[244] The macros are typically distinct powers of two.

[245] See "future library directions" (7.33.4).

[246] Even though the rounding direction macros can expand to constants corresponding to the values of **FLT_ROUNDS**, they are not required to do so.

© ISO 2024 — All rights reserved

```
FE_DEC_UPWARD
```

is defined for use with the **fe_dec_getround** and **fe_dec_setround** functions for getting and setting the dynamic rounding direction mode for decimal floating-point operations. The decimal rounding direction affects all (inexact) operations that produce a result of decimal floating type and all operations that produce an integer or character sequence result and have an operand of decimal floating type, unless stated otherwise. The macros expand to integer constant expressions whose values are distinct nonnegative values.

15 During translation, constant rounding direction modes for decimal floating-point arithmetic are in effect where specified. Elsewhere, during translation the decimal rounding direction mode is **FE_DEC_TONEAREST**.

16 At program startup the dynamic rounding direction mode for decimal floating-point arithmetic is initialized to **FE_DEC_TONEAREST**.

17 The macro

```
FE_DFL_ENV
```

represents the default dynamic floating-point environment — the one installed at program startup — and has type "pointer to const-qualified **fenv_t**". It can be used as an argument to `<fenv.h>` functions that manage the dynamic floating-point environment.

18 Additional implementation-defined environments, with macro definitions beginning with **FE_** and an uppercase letter,[247] and having type "pointer to const-qualified **fenv_t**", may also be specified by the implementation.

7.6.1 The FENV_ACCESS pragma

Synopsis

1
```
#include <fenv.h>
#pragma STDC FENV_ACCESS on-off-switch
```

Description

2 The **FENV_ACCESS** pragma provides a means to inform the implementation when a program can access the floating-point environment to test floating-point status flags or run under non-default floating-point control modes.[248] The pragma shall occur either outside external declarations or preceding all explicit declarations and statements inside a compound statement. When outside external declarations, the pragma takes effect from its occurrence until another **FENV_ACCESS** pragma is encountered, or until the end of the translation unit. When inside a compound statement, the pragma takes effect from its occurrence until another **FENV_ACCESS** pragma is encountered (including within a nested compound statement), or until the end of the compound statement. At the end of a compound statement, the state for the pragma is restored to its condition just before the compound statement. If this pragma is used in any other context, the behavior is undefined. If part of a program tests floating-point status flags or establishes or is executed with non-default floating-point mode settings using any means other than the **FENV_ROUND** pragmas, but was translated with the state for the **FENV_ACCESS** pragma "off", the behavior is undefined. The default state ("on" or "off") for the pragma is implementation-defined. (When execution passes from a part of the program translated with **FENV_ACCESS** "off" to a part translated with **FENV_ACCESS** "on", the state of the floating-point status flags is unspecified and the floating-point control modes have their default settings.)

3 EXAMPLE

```
#include <fenv.h>
```

[247] See "future library directions" (7.33.4).

[248] The purpose of the **FENV_ACCESS** pragma is to allow certain optimizations that could subvert flag tests and mode changes (e.g. global common subexpression elimination, code motion, and constant folding). In general, if the state of **FENV_ACCESS** is "off", the translator can assume that the flags are not tested, and that default modes are in effect, except where specified otherwise by an **FENV_ROUND** pragma.

© ISO 2024 — All rights reserved

```
void f(double x)
{
      #pragma STDC FENV_ACCESS ON
      void g(double);
      void h(double);
      /* ... */
      g(x + 1);
      h(x + 1);
      /* ... */
}
```

4 If the function `g` can depend on status flags set as a side effect of the first `x + 1`, or if the second `x + 1` can depend on control modes set as a side effect of the call to function `g`, then the program has to contain an appropriately placed invocation of **`#pragma STDC FENV_ACCESS ON`** as shown.[249)]

7.6.2 The FENV_ROUND pragma

Synopsis

1

```
#include <fenv.h>
#pragma STDC FENV_ROUND direction
#pragma STDC FENV_ROUND FE_DYNAMIC
```

Description

2 The **FENV_ROUND** pragma provides a means to specify a constant rounding direction for floating-point operations for standard floating types within a translation unit or compound statement. The pragma shall occur either outside external declarations or before all explicit declarations and statements inside a compound statement. When outside external declarations, the pragma takes effect from its occurrence until another **FENV_ROUND** pragma is encountered, or until the end of the translation unit. When inside a compound statement, the pragma takes effect from its occurrence until another **FENV_ROUND** pragma is encountered (including within a nested compound statement), or until the end of the compound statement; at the end of a compound statement the static rounding mode is restored to its condition just before the compound statement. If this pragma is used in any other context, its behavior is undefined.

3 *direction* shall be: one of the names of the supported rounding direction macros for operations for standard floating types (7.6), to specify a constant rounding mode; or, **FE_DYNAMIC**, to specify dynamic rounding. If any other value is specified, the behavior is undefined. If no **FENV_ROUND** pragma is in effect, or the specified *direction* is **FE_DYNAMIC**, rounding is according to the mode specified by the dynamic floating-point environment, which is the dynamic rounding mode that was established either at thread creation or by a call to **fesetround**, **fesetmode**, **fesetenv**, or **feupdateenv**. If the *direction* **FE_DYNAMIC** is specified and **FENV_ACCESS** is "off", the translator may assume that the default rounding mode is in effect.

4 The **FENV_ROUND** pragma affects operations for standard floating types. Within the scope of an **FENV_ROUND** pragma establishing a constant rounding mode, floating-point operators, implicit conversions (including the conversion of a value represented in a format wider than its semantic types to its semantic type, as done by classification macros), and invocations of functions indicated in Table 7.1, for which macro replacement has not been suppressed (7.1.4), shall be evaluated according to the specified constant rounding mode (as though no constant mode was specified and the corresponding dynamic rounding mode had been established by a call to **fesetround**). Invocations of functions for which macro replacement has been suppressed and invocations of functions other than those indicated in Table 7.1 shall not be affected by constant rounding modes – they are affected by (and affect) only the dynamic mode. Floating constants (6.4.4.3) of a standard floating type that occur in the scope of a constant rounding mode shall be interpreted according to that mode.

[249)] The side effects impose a temporal ordering that requires two evaluations of `x + 1`. On the other hand, without the **`#pragma STDC FENV_ACCESS ON`** pragma, and assuming the default state is "off", just one evaluation of `x + 1` would suffice.

 © ISO 2024 — All rights reserved

Table 7.1: Functions affected by constant rounding modes – for standard floating types

Header	Function families
<math.h>	**acos**, **acospi**, **asin**, **asinpi**, **atan**, **atan2**, **atan2pi**, **atanpi**
<math.h>	**cos**, **cospi**, **sin**, **sinpi**, **tan**, **tanpi**
<math.h>	**acosh**, **asinh**, **atanh**
<math.h>	**cosh**, **sinh**, **tanh**
<math.h>	**exp**, **exp10**, **exp10m1**, **exp2**, **exp2m1**, **expm1**
<math.h>	**log**, **log10**, **log10p1**, **log1p**, **log2**, **log2p1**, **logp1**
<math.h>	**scalbn**, **scalbln**, **ldexp**
<math.h>	**cbrt**, **compoundn**, **hypot**, **pow**, **pown**, **powr**, **rootn**, **rsqrt**, **sqrt**
<math.h>	**erf**, **erfc**
<math.h>	**lgamma**, **tgamma**
<math.h>	**rint**, **nearbyint**, **lrint**, **llrint**
<math.h>	**fdim**
<math.h>	**fma**
<math.h>	**fadd**, **dadd**, **fsub**, **dsub**, **fmul**, **dmul**, **fdiv**, **ddiv**, **ffma**, **dfma**, **fsqrt**, **dsqrt**
<stdlib.h>	**atof**, **strfrom**, **strto**
<wchar.h>	**wcsto**
<stdio.h>	**printf** and **scanf** families
<wchar.h>	**wprintf** and **wscanf** families

A function family listed in Table 7.1 indicates the functions for all standard floating types, where the function family is represented by the name of the functions without a suffix. For example, **acos** indicates the functions **acos**, **acosf**, and **acosl**.

NOTE Constant rounding modes could be implemented using dynamic rounding modes as illustrated in the following example, except that this method does not interpret inexact floating constants according to the constant rounding mode as required.

```
{
      #pragma STDC FENV_ROUND direction
      // compiler inserts:
      // #pragma STDC FENV_ACCESS ON
      // int __savedrnd;
      // __savedrnd = __swapround(direction);
      ...  operations affected by constant rounding mode ...
      // compiler inserts:
      // __savedrnd = __swapround(__savedrnd);
      ...  operations not affected by constant rounding mode ...
      // compiler inserts:
      // __savedrnd = __swapround(__savedrnd);
      ...  operations affected by constant rounding mode ...
      // compiler inserts:
      // __swapround(__savedrnd);
}
```

where `__swapround` is defined by:

```
static inline int __swapround(const int new) {
      const int old = fegetround();
      fesetround(new);
      return old;
}
```

7.6.3 The FENV_DEC_ROUND pragma

Synopsis

1

```
#include <fenv.h>
#ifdef __STDC_IEC_60559_DFP__
#pragma STDC FENV_DEC_ROUND dec-direction
#endif
```

Description

2 The **FENV_DEC_ROUND** pragma is a decimal floating-point analog of the **FENV_ROUND** pragma. If **FLT_RADIX** is not 10, the **FENV_DEC_ROUND** pragma affects operators, functions, and floating constants only for decimal floating types. The affected functions are listed in Table 7.2. If **FLT_RADIX** is 10, whether the **FENV_ROUND** and **FENV_DEC_ROUND** pragmas alter the rounding direction of both standard and decimal floating-point operations is implementation-defined. *dec-direction* shall be one of the decimal rounding direction macro names (**FE_DEC_DOWNWARD**, **FE_DEC_TONEAREST**, **FE_DEC_TONEARESTFROMZERO**, **FE_DEC_TOWARDZERO**, and **FE_DEC_UPWARD**) defined in 7.6, to specify a constant rounding mode, or **FE_DEC_DYNAMIC**, to specify dynamic rounding. The corresponding dynamic rounding mode can be established by a call to **fe_dec_setround**.

Table 7.2: Functions affected by constant rounding modes – for decimal floating types

Header	Function families
<math.h>	**acos**, **acospi**, **asin**, **asinpi**, **atan**, **atan2**, **atan2pi**, **atanpi**
<math.h>	**cos**, **cospi**, **sin**, **sinpi**, **tan**, **tanpi**
<math.h>	**acosh**, **asinh**, **atanh**
<math.h>	**cosh**, **sinh**, **tanh**
<math.h>	**exp**, **exp10**, **exp10m1**, **exp2**, **exp2m1**, **expm1**
<math.h>	**log**, **log10**, **log10p1**, **log1p**, **log2**, **log2p1**, **logp1**
<math.h>	**scalbn**, **scalbln**, **ldexp**
<math.h>	**cbrt**, **compoundn**, **hypot**, **pow**, **pown**, **powr**, **rootn**, **rsqrt**, **sqrt**
<math.h>	**erf**, **erfc**
<math.h>	**lgamma**, **tgamma**
<math.h>	**rint**, **nearbyint**, **lrint**, **llrint**
<math.h>	**quantize**
<math.h>	**fdim**
<math.h>	**fma**
<math.h>	**d32add**, **d64add**, **d32sub**, **d64sub**, **d32mul**, **d64mul**, **d32div**, **d64div**, **d32fma**, **d64fma**, **d32sqrt**, **d64sqrt**
<stdlib.h>	**strfrom**, **strto**
<wchar.h>	**wcsto**
<stdio.h>	**printf** and **scanf** families
<wchar.h>	**wprintf** and **wscanf** families

A function family listed in Table 7.2 indicates the functions for all decimal floating types, where the function family is represented by the name of the functions without a suffix. For example, **acos** indicates the functions **acosd32**, **acosd64**, and **acosd128**.

7.6.4 Floating-point exceptions

1 The following functions provide access to the floating-point status flags.[250)] The **int** input argument for the functions represents a subset of floating-point exceptions, and can be zero or the bitwise OR of one or more floating-point exception macros, for example **FE_OVERFLOW** | **FE_INEXACT**. For other argument values, the behavior of these functions is undefined.

7.6.4.1 The feclearexcept function

[250)] The functions **fetestexcept**, **feraiseexcept**, and **feclearexcept** support the basic abstraction of flags that are either set or clear. An implementation can endow floating-point status flags with more information — for example, the address of the code which first raised the floating-point exception; the functions **fegetexceptflag** and **fesetexceptflag** deal with the full content of flags.

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <fenv.h>
int feclearexcept(int excepts);
```

Description

2 The **feclearexcept** function attempts to clear the supported floating-point exceptions represented by its argument.

Returns

3 The **feclearexcept** function returns zero if the `excepts` argument is zero or if all the specified exceptions were successfully cleared. Otherwise, it returns a nonzero value.

7.6.4.2 The fegetexceptflag function

Synopsis

1
```
#include <fenv.h>
int fegetexceptflag(fexcept_t *flagp, int excepts);
```

Description

2 The **fegetexceptflag** function attempts to store an implementation-defined representation of the states of the floating-point status flags indicated by the argument `excepts` in the object pointed to by the argument `flagp`.

Returns

3 The **fegetexceptflag** function returns zero if the representation was successfully stored. Otherwise, it returns a nonzero value.

7.6.4.3 The feraiseexcept function

Synopsis

1
```
#include <fenv.h>
int feraiseexcept(int excepts);
```

Description

2 The **feraiseexcept** function attempts to raise the supported floating-point exceptions represented by its argument.[251)] The order in which these floating-point exceptions are raised is unspecified, except as stated in F.8.7. Whether the **feraiseexcept** function additionally raises the "inexact" floating-point exception whenever it raises the "overflow" or "underflow" floating-point exception is implementation-defined.

Returns

3 The **feraiseexcept** function returns zero if the `excepts` argument is zero or if all the specified exceptions were successfully raised. Otherwise, it returns a nonzero value.

Recommended practice

Implementation extensions associated with raising a floating-point exception (for example, enabled traps or ISO/IEC 60559 alternate exception handling) should be honored by this function.

7.6.4.4 The fesetexcept function

Synopsis

1
```
#include <fenv.h>
int fesetexcept(int excepts);
```

[251)] The effect is intended to be similar to that of floating-point exceptions raised by arithmetic operations. Hence, implementation extensions associated with raising a floating-point exception (for example, enabled traps or ISO/IEC 60559 alternate exception handling) should be honored. The specification in F.8.7 is in the same spirit.

© ISO 2024 — All rights reserved

Description

2 The **fesetexcept** function attempts to set the supported floating-point exception flags represented by its argument. This function does not clear any floating-point exception flags. This function changes the state of the floating-point exception flags, but does not cause any other side effects that can be associated with raising floating-point exceptions.[252)]

Returns

3 The **fesetexcept** function returns zero if all the specified exceptions were successfully set or if the excepts argument is zero. Otherwise, it returns a nonzero value.

7.6.4.5 The fesetexceptflag function

Synopsis

1
```
#include <fenv.h>
int fesetexceptflag(const fexcept_t *flagp, int excepts);
```

Description

2 The **fesetexceptflag** function attempts to set the floating-point status flags indicated by the argument `excepts` to the states stored in the object pointed to by `flagp`. The value of `*flagp` shall have been set by a previous call to **fegetexceptflag** whose second argument represented at least those floating-point exceptions represented by the argument `excepts`. Like **fesetexcept**, this function does not raise floating-point exceptions, but only sets the state of the flags.

Returns

3 The **fesetexceptflag** function returns zero if the `excepts` argument is zero or if all the specified flags were successfully set to the appropriate state. Otherwise, it returns a nonzero value.

7.6.4.6 The fetestexceptflag function

Synopsis

1
```
#include <fenv.h>
int fetestexceptflag(const fexcept_t *flagp, int excepts);
```

Description

2 The **fetestexceptflag** function determines which of a specified subset of the floating-point exception flags are set in the object pointed to by `flagp`. The value of `*flagp` shall have been set by a previous call to **fegetexceptflag** whose second argument represented at least those floating-point exceptions represented by the argument `excepts`. The `excepts` argument specifies the floating-point status flags to be queried.

Returns

3 The **fetestexceptflag** function returns the value of the bitwise OR of the floating-point exception macros included in `excepts` corresponding to the floating-point exceptions set in `*flagp`.

7.6.4.7 The fetestexcept function

Synopsis

1
```
#include <fenv.h>
int fetestexcept(int excepts);
```

Description

2 The **fetestexcept** function determines which of a specified subset of the floating-point exception flags are currently set. The `excepts` argument specifies the floating-point status flags to be queried.[253)]

[252)] Implementation extensions like traps for floating-point exceptions and ISO/IEC 60559 exception handling do not occur.
[253)] This mechanism allows testing several floating-point exceptions with just one function call.

 © ISO 2024 — All rights reserved

Returns

3 The **fetestexcept** function returns the value of the bitwise OR of the floating-point exception macros corresponding to the currently set floating-point exceptions included in excepts.

4 EXAMPLE Call f if "invalid" is set, then g if "overflow" is set:

```
#include <fenv.h>
/* ... */
{
      #pragma STDC FENV_ACCESS ON
      int set_excepts;
      feclearexcept(FE_INVALID | FE_OVERFLOW);
      // maybe raise exceptions
      set_excepts = fetestexcept(FE_INVALID | FE_OVERFLOW);
      if (set_excepts & FE_INVALID) f();
      if (set_excepts & FE_OVERFLOW) g();
      /* ... */
}
```

7.6.5 Rounding and other control modes

1 The **fegetround** and **fesetround** functions provide control of rounding direction modes. The **fegetmode** and **fesetmode** functions manage all the implementation's dynamic floating-point control modes collectively.

7.6.5.1 The fegetmode function

Synopsis

1
```
#include <fenv.h>
int fegetmode(femode_t *modep);
```

Description

2 The **fegetmode** function attempts to store all the dynamic floating-point control modes in the object pointed to by modep.

Returns

3 The **fegetmode** function returns zero if the modes were successfully stored. Otherwise, it returns a nonzero value.

7.6.5.2 The fegetround function

Synopsis

1
```
#include <fenv.h>
int fegetround(void);
```

Description

2 The **fegetround** function gets the current value of the dynamic rounding direction mode.

Returns

3 The **fegetround** function returns the value of the rounding direction macro representing the current dynamic rounding direction or a negative value if there is no such rounding direction macro or the current dynamic rounding direction is not determinable.

7.6.5.3 The fe_dec_getround function

Synopsis

1
```
#include <fenv.h>
#ifdef __STDC_IEC_60559_DFP__
int fe_dec_getround(void);
#endif
```

© ISO 2024 — All rights reserved

Description

2 The **fe_dec_getround** function gets the current value of the dynamic rounding direction mode for decimal floating-point operations.

Returns

3 The **fe_dec_getround** function returns the value of the rounding direction macro representing the current dynamic rounding direction for decimal floating-point operations, or a negative value if there is no such rounding macro or the current rounding direction is not determinable.

7.6.5.4 The fesetmode function

Synopsis

1
```
#include <fenv.h>
int fesetmode(const femode_t *modep);
```

Description

2 The **fesetmode** function attempts to establish the dynamic floating-point modes represented by the object pointed to by modep. The argument modep shall point to an object set by a call to **fegetmode**, or equal **FE_DFL_MODE** or a dynamic floating-point mode state macro defined by the implementation.

Returns

The fesetmode **fesetmode** function returns zero if the modes were successfully established. Otherwise, it returns a nonzero value.

7.6.5.5 The fesetround function

Synopsis

1
```
#include <fenv.h>
int fesetround(int rnd);
```

Description

2 The **fesetround** function establishes the rounding direction represented by its argument rnd. If the argument is not equal to the value of a rounding direction macro, the rounding direction is not changed.

Returns

3 The **fesetround** function returns zero if and only if the dynamic rounding direction mode was set to the requested rounding direction.

4 EXAMPLE Save, set, and restore the rounding direction. Report an error and abort if setting the rounding direction fails.

```
#include <fenv.h>
#include <assert.h>

void f(int rnd_dir)
{
      #pragma STDC FENV_ACCESS ON
      int save_round;
      int setround_ok;
      save_round = fegetround();
      setround_ok = fesetround(rnd_dir);
      assert(setround_ok == 0);
      /* ... */
      fesetround(save_round);
      /* ... */
}
```

 © ISO 2024 — All rights reserved

7.6.5.6 The `fe_dec_setround` function

Synopsis

1
```
#include <fenv.h>
#ifdef __STDC_IEC_60559_DFP__
int fe_dec_setround(int rnd);
#endif
```

Description

2 The **`fe_dec_setround`** function sets the dynamic rounding direction mode for decimal floating-point operations to be the rounding direction represented by its argument `rnd`. If the argument is not equal to the value of a decimal rounding direction macro, the rounding direction is not changed.

3 If **`FLT_RADIX`** is not 10, the rounding direction altered by the **`fesetround`** function is independent of the rounding direction altered by the **`fe_dec_setround`** function; otherwise if **`FLT_RADIX`** is 10, whether the **`fesetround`** and **`fe_dec_setround`** functions alter the rounding direction of both standard and decimal floating-point operations is implementation-defined.

Returns

4 The **`fe_dec_setround`** function returns a zero value if and only if the argument is equal to a decimal rounding direction macro (that is, if and only if the dynamic rounding direction mode for decimal floating-point operations was set to the requested rounding direction).

7.6.6 Environment

1 The functions in this section manage the floating-point environment — status flags and control modes — as one entity.

7.6.6.1 The `fegetenv` function

Synopsis

1
```
#include <fenv.h>
int fegetenv(fenv_t *envp);
```

Description

2 The **`fegetenv`** function attempts to store the current dynamic floating-point environment in the object pointed to by `envp`.

Returns

3 The **`fegetenv`** function returns zero if the environment was successfully stored. Otherwise, it returns a nonzero value.

7.6.6.2 The `feholdexcept` function

Synopsis

1
```
#include <fenv.h>
int feholdexcept(fenv_t *envp);
```

Description

2 The **`feholdexcept`** function saves the current dynamic floating-point environment in the object pointed to by `envp`, clears the floating-point status flags, and then installs a *non-stop* (continue on floating-point exceptions) mode, if available, for all floating-point exceptions.[254)]

[254)] ISO/IEC 60559 systems have a default non-stop mode, and typically at least one other mode for trap handling or aborting; if the system provides only the non-stop mode then installing it is trivial. For such systems, the **`feholdexcept`** function can be used in conjunction with the **`feupdateenv`** function to write routines that hide spurious floating-point exceptions from their callers.

© ISO 2024 — All rights reserved

Returns

3 The **feholdexcept** function returns zero if and only if non-stop floating-point exception handling was successfully installed.

7.6.6.3 The fesetenv function

Synopsis

1
```
#include <fenv.h>
int fesetenv(const fenv_t *envp);
```

Description

2 The **fesetenv** function attempts to establish the dynamic floating-point environment represented by the object pointed to by envp. The argument envp shall point to an object set by a call to **fegetenv** or **feholdexcept**, or equal a dynamic floating-point environment macro. Note that **fesetenv** merely installs the state of the floating-point status flags represented through its argument, and does not raise these floating-point exceptions.

Returns

3 The **fesetenv** function returns zero if the environment was successfully established. Otherwise, it returns a nonzero value.

7.6.6.4 The feupdateenv function

Synopsis

1
```
#include <fenv.h>
int feupdateenv(const fenv_t *envp);
```

Description

2 The **feupdateenv** function attempts to save the currently raised floating-point exceptions in its automatic storage, install the dynamic floating-point environment represented by the object pointed to by envp, and then raise the saved floating-point exceptions. The argument envp shall point to an object set by a call to **feholdexcept** or **fegetenv**, or equal a dynamic floating-point environment macro.

Returns

3 The **feupdateenv** function returns zero if all the actions were successfully carried out. Otherwise, it returns a nonzero value.

4 EXAMPLE Hide spurious underflow floating-point exceptions:

```
#include <fenv.h>
double f(double x)
{
      #pragma STDC FENV_ACCESS ON
      double result;
      fenv_t save_env;
      if (feholdexcept(&save_env))
            return /* indication of an environmental problem */;
      // compute result
      if (/* test spurious underflow */)
            if (feclearexcept(FE_UNDERFLOW))
                  return /* indication of an environmental problem */;
      if (feupdateenv(&save_env))
            return /* indication of an environmental problem */;
      return result;
}
```

 © ISO 2024 — All rights reserved

7.7 Characteristics of floating types `<float.h>`

1 The header `<float.h>` defines several macros that expand to various limits and parameters of the real floating types.

2 The macro

```
__STDC_VERSION_FLOAT_H__
```

is an integer constant expression with a value equivalent to `202311L`.

3 The rest of the macros, their meanings, and the constraints (or restrictions) on their values are listed in 5.2.5.3.3 and 5.2.5.3.4. A summary is given in Annex E.

© ISO 2024 — All rights reserved

7.8 Format conversion of integer types <inttypes.h>

1 The header <inttypes.h> includes the header <stdint.h> and extends it with additional facilities provided by hosted implementations.

2 It defines the macro

```
__STDC_VERSION_INTTYPES_H__
```

which is an integer constant expression with a value equivalent to 202311L.

3 It declares functions for manipulating greatest-width integers and converting numeric character strings to greatest-width integers, and it declares the type

```
imaxdiv_t
```

which is a structure type that is the type of the value returned by the **imaxdiv** function. For each type declared in <stdint.h>, it defines corresponding macros for conversion specifiers for use with the formatted input/output functions.[255)]

Forward references: integer types <stdint.h> (7.22), formatted input/output functions (7.23.6), formatted wide character input/output functions (7.31.2).

7.8.1 Macros for format specifiers

1 Each of the following object-like macros expands to a character string literal containing a conversion specifier, possibly modified by a length modifier, suitable for use within the format argument of a formatted input/output function when converting the corresponding integer type. These macro names have the general form of **PRI** (character string literals for the **fprintf** and **fwprintf** family) or **SCN** (character string literals for the **fscanf** and **fwscanf** family),[256)] followed by the conversion specifier, followed by a name corresponding to a similar type name in 7.22.1. In these names, *N* represents the width of the type as described in 7.22.1. For example, **PRIdFAST32** can be used in a format string to print the value of an integer of type **int_fast32_t**. The functions in the **fprintf** and **fwprintf** families shall behave as if they use **va_arg** with a type argument naming the type resulting from applying the default argument promotions to the type corresponding to the macro and then convert the result of the **va_arg** expansion to the type corresponding to the macro.

2 The **fprintf** macros for signed integers are:

PRId*N* **PRIdLEAST***N* **PRIdFAST***N* **PRIdMAX** **PRIdPTR**
PRIi*N* **PRIiLEAST***N* **PRIiFAST***N* **PRIiMAX** **PRIiPTR**

3 The **fprintf** macros for unsigned integers are:

PRIb*N* **PRIbLEAST***N* **PRIbFAST***N* **PRIbMAX** **PRIbPTR**
PRIo*N* **PRIoLEAST***N* **PRIoFAST***N* **PRIoMAX** **PRIoPTR**
PRIu*N* **PRIuLEAST***N* **PRIuFAST***N* **PRIuMAX** **PRIuPTR**
PRIx*N* **PRIxLEAST***N* **PRIxFAST***N* **PRIxMAX** **PRIxPTR**
PRIX*N* **PRIXLEAST***N* **PRIXFAST***N* **PRIXMAX** **PRIXPTR**

4 The following **fprintf** macros for unsigned integer types are optional:

PRIB*N* **PRIBLEAST***N* **PRIBFAST***N* **PRIBMAX** **PRIBPTR**

They shall be defined if the implementation supports the B specifier as indicated in 7.23.6.1 and 7.31.2.1; otherwise they shall not be defined.

5 The **fscanf** macros for signed integers are:

SCNd*N* **SCNdLEAST***N* **SCNdFAST***N* **SCNdMAX** **SCNdPTR**
SCNi*N* **SCNiLEAST***N* **SCNiFAST***N* **SCNiMAX** **SCNiPTR**

6 The **fscanf** macros for unsigned integers are:

[255)] See "future library directions" (7.33.6).
[256)] For any given type, the corresponding macros for **fprintf** and **fscanf** functions may be distinct.

 © ISO 2024 — All rights reserved

SCNb*N*	**SCNbLEAST***N*	**SCNbFAST***N*	**SCNbMAX**	**SCNbPTR**
SCNo*N*	**SCNoLEAST***N*	**SCNoFAST***N*	**SCNoMAX**	**SCNoPTR**
SCNu*N*	**SCNuLEAST***N*	**SCNuFAST***N*	**SCNuMAX**	**SCNuPTR**
SCNx*N*	**SCNxLEAST***N*	**SCNxFAST***N*	**SCNxMAX**	**SCNxPTR**

7 For each type that the implementation provides in `<stdint.h>`, the corresponding **fprintf** macros shall be defined and the corresponding **fscanf** macros shall be defined unless the implementation does not have a suitable **fscanf** length modifier for the type.

8 EXAMPLE

```
#include <inttypes.h>
#include <wchar.h>
int main(void)
{
      uintmax_t i = UINTMAX_MAX;    // this type always exists
      wprintf(L"The largest integer value is %020"
            PRIxMAX "\n", i);
      return 0;
}
```

7.8.2 Functions for greatest-width integer types

7.8.2.1 The imaxabs function

Synopsis

1
```
#include <inttypes.h>
intmax_t imaxabs(intmax_t j);
```

Description

2 The **imaxabs** function computes the absolute value of an integer `j`. If the result cannot be represented, the behavior is undefined.[257)]

Returns

3 The **imaxabs** function returns the absolute value.

7.8.2.2 The imaxdiv function

Synopsis

1
```
#include <inttypes.h>
imaxdiv_t imaxdiv(intmax_t numer, intmax_t denom);
```

Description

2 The **imaxdiv** function computes `numer / denom` and `numer % denom` in a single operation.

Returns

3 The **imaxdiv** function returns a structure of type **imaxdiv_t** comprising both the quotient and the remainder. The structure shall contain (in either order) the members `quot` (the quotient) and `rem` (the remainder), each of which has type **intmax_t**. If either part of the result cannot be represented, the behavior is undefined.

7.8.2.3 The strtoimax and strtoumax functions

Synopsis

1
```
#include <inttypes.h>
intmax_t strtoimax(const char * restrict nptr, char ** restrict endptr, int base);
uintmax_t strtoumax(const char * restrict nptr, char ** restrict endptr, int base);
```

[257)] The absolute value of the most negative number is not representable.

© ISO 2024 — All rights reserved

Description

2 The **strtoimax** and **strtoumax** functions are equivalent to the **strtol**, **strtoll**, **strtoul**, and **strtoull** functions, except that the initial portion of the string is converted to **intmax_t** and **uintmax_t** representation, respectively.

Returns

3 The **strtoimax** and **strtoumax** functions return the converted value, if any. If no conversion could be performed, zero is returned. If the correct value is outside the range of representable values, **INTMAX_MAX**, **INTMAX_MIN**, or **UINTMAX_MAX** is returned (according to the return type and sign of the value, if any), and the value of the macro **ERANGE** is stored in **errno**.

Forward references: the **strtol**, **strtoll**, **strtoul**, and **strtoull** functions (7.24.1.7).

7.8.2.4 The wcstoimax and wcstoumax functions

Synopsis

1
```
#include <stddef.h>            // for wchar_t
#include <inttypes.h>
intmax_t wcstoimax(const wchar_t *restrict nptr, wchar_t **restrict endptr, int base);
uintmax_t wcstoumax(const wchar_t *restrict nptr, wchar_t **restrict endptr, int base);
```

Description

2 The **wcstoimax** and **wcstoumax** functions are equivalent to the **wcstol**, **wcstoll**, **wcstoul**, and **wcstoull** functions except that the initial portion of the wide string is converted to **intmax_t** and **uintmax_t** representation, respectively.

Returns

3 The **wcstoimax** function returns the converted value, if any. If no conversion could be performed, zero is returned. If the correct value is outside the range of representable values, **INTMAX_MAX**, **INTMAX_MIN**, or **UINTMAX_MAX** is returned (according to the return type and sign of the value, if any), and the value of the macro **ERANGE** is stored in **errno**.

Forward references: the **wcstol**, **wcstoll**, **wcstoul**, and **wcstoull** functions (7.31.4.1.4).

© ISO 2024 — All rights reserved

7.9 Alternative spellings `<iso646.h>`

1 The header `<iso646.h>` defines the following eleven macros (on the left) that expand to the corresponding tokens (on the right):

```
and      &&
and_eq   &=
bitand   &
bitor    |
compl    ~
not      !
not_eq   !=
or       ||
or_eq    |=
xor      ^
xor_eq   ^=
```

© ISO 2024 — All rights reserved

7.10 Characteristics of integer types `<limits.h>`

1 The header `<limits.h>` defines several macros that expand to various limits and parameters of the standard integer types.

2 The macro

```
__STDC_VERSION_LIMITS_H__
```

is an integer constant expression with a value equivalent to `202311L`.

3 The rest of the macros, their meanings, and the constraints (or restrictions) on their values are listed in 5.2.5.3.2. A summary is given in Annex E.

© ISO 2024 — All rights reserved

7.11 Localization <locale.h>

1 The header <locale.h> declares two functions, one type, and defines several macros.

2 The type is

```
struct lconv
```

which contains members related to the formatting of numeric values. The structure shall contain at least the following members, in any order. The semantics of the members and their normal ranges are explained in 7.11.2.1. In the "C" locale, the members shall have the values specified in the comments.

```
char *decimal_point;       // "."
char *thousands_sep;       // ""
char *grouping;            // ""
char *mon_decimal_point;   // ""
char *mon_thousands_sep;   // ""
char *mon_grouping;        // ""
char *positive_sign;       // ""
char *negative_sign;       // ""
char *currency_symbol;     // ""
char frac_digits;          // CHAR_MAX
char p_cs_precedes;        // CHAR_MAX
char n_cs_precedes;        // CHAR_MAX
char p_sep_by_space;       // CHAR_MAX
char n_sep_by_space;       // CHAR_MAX
char p_sign_posn;          // CHAR_MAX
char n_sign_posn;          // CHAR_MAX
char *int_curr_symbol;     // ""
char int_frac_digits;      // CHAR_MAX
char int_p_cs_precedes;    // CHAR_MAX
char int_n_cs_precedes;    // CHAR_MAX
char int_p_sep_by_space;   // CHAR_MAX
char int_n_sep_by_space;   // CHAR_MAX
char int_p_sign_posn;      // CHAR_MAX
char int_n_sign_posn;      // CHAR_MAX
```

3 The macros defined are **NULL** (described in 7.21); and

```
LC_ALL
LC_COLLATE
LC_CTYPE
LC_MONETARY
LC_NUMERIC
LC_TIME
```

which expand to integer constant expressions with distinct values, suitable for use as the first argument to the **setlocale** function.[258] Additional macro definitions, beginning with the characters **LC_** and an uppercase letter,[259] may also be specified by the implementation.

7.11.1 Locale control

7.11.1.1 The setlocale function

Synopsis

1

```
#include <locale.h>
char *setlocale(int category, const char *locale);
```

[258] ISO/IEC 9945 specifies locale and charmap formats that can be used to specify locales for C.
[259] See "future library directions" (7.33.7).

© ISO 2024 — All rights reserved

Description

2 The **setlocale** function selects the appropriate portion of the program's locale as specified by the category and locale arguments. The **setlocale** function may be used to change or query the program's entire current locale or portions thereof. The value **LC_ALL** for category names the program's entire locale; the other values for category name only a portion of the program's locale. **LC_COLLATE** affects the behavior of the **strcoll** and **strxfrm** functions. **LC_CTYPE** affects the behavior of the character handling functions[260)] and the multibyte and wide character functions. **LC_MONETARY** affects the monetary formatting information returned by the **localeconv** function. **LC_NUMERIC** affects the decimal-point character for the formatted input/output functions and the string conversion functions, as well as the nonmonetary formatting information returned by the **localeconv** function. **LC_TIME** affects the behavior of the **strftime** and **wcsftime** functions.

3 A value of "C" for locale specifies the minimal environment for C translation; a value of "" for locale specifies the locale-specific native environment. Other implementation-defined strings may be passed as the second argument to **setlocale**.

4 At program startup, the equivalent of

```
setlocale(LC_ALL, "C");
```

is executed.

5 A call to the **setlocale** function may introduce a data race with other calls to the **setlocale** function or with calls to functions that are affected by the current locale. The implementation shall behave as if no library function calls the **setlocale** function.

Returns

6 If a pointer to a string is given for locale and the selection can be honored, the **setlocale** function returns a pointer to the string associated with the specified category for the new locale. If the selection cannot be honored, the **setlocale** function returns a null pointer and the program's locale is not changed.

7 A null pointer for locale causes the **setlocale** function to return a pointer to the string associated with the category for the program's current locale; the program's locale is not changed.[261)]

8 The pointer to string returned by the **setlocale** function is such that a subsequent call with that string value and its associated category will restore that part of the program's locale. The string pointed to shall not be modified by the program. The behavior is undefined if the returned value is used after a subsequent call to the **setlocale** function, or after the thread which called the **setlocale** function to obtain the returned value has exited.

Forward references: formatted input/output functions (7.23.6), multibyte/wide character conversion functions (7.24.7), multibyte/wide string conversion functions (7.24.8), numeric conversion functions (7.24.1), the **strcoll** function (7.26.4.3), the **strftime** function (7.29.3.5), the **strxfrm** function (7.26.4.5).

7.11.2 Numeric formatting convention inquiry

7.11.2.1 The localeconv function

Synopsis

1
```
#include <locale.h>
struct lconv *localeconv(void);
```

Description

2 The **localeconv** function sets the components of an object with type **struct lconv** with values appropriate for the formatting of numeric quantities (monetary and otherwise) according to the rules of the current locale.

[260)] The only functions in 7.4 whose behavior is not affected by the current locale are **isdigit** and **isxdigit**.

[261)] The implementation is thus required to arrange to encode in a string the various categories due to a heterogeneous locale when category has the value **LC_ALL**.

 © ISO 2024 — All rights reserved

3 The members of the structure with type **char** ∗ are pointers to strings, any of which (except **decimal_point**) can point to "", to indicate that the value is not available in the current locale or is of zero length. Apart from **grouping** and **mon_grouping**, the strings shall start and end in the initial shift state. The members with type **char** are nonnegative numbers, any of which can be **CHAR_MAX** to indicate that the value is not available in the current locale. The members include the following:

char ∗**decimal_point**
: The decimal-point character used to format nonmonetary quantities.

char ∗**thousands_sep**
: The character used to separate groups of digits before the decimal-point character in formatted nonmonetary quantities.

char ∗**grouping**
: A string whose elements indicate the size of each group of digits in formatted nonmonetary quantities.

char ∗**mon_decimal_point**
: The decimal-point used to format monetary quantities.

char ∗**mon_thousands_sep**
: The separator for groups of digits before the decimal-point in formatted monetary quantities.

char ∗**mon_grouping**
: A string whose elements indicate the size of each group of digits in formatted monetary quantities.

char ∗**positive_sign**
: The string used to indicate a nonnegative-valued formatted monetary quantity.

char ∗**negative_sign**
: The string used to indicate a negative-valued formatted monetary quantity.

char ∗**currency_symbol**
: The local currency symbol applicable to the current locale.

char **frac_digits**
: The number of fractional digits (those after the decimal-point) to be displayed in a locally formatted monetary quantity.

char **p_cs_precedes**
: Set to 1 or 0 if the **currency_symbol** respectively precedes or succeeds the value for a nonnegative locally formatted monetary quantity.

char **n_cs_precedes**
: Set to 1 or 0 if the **currency_symbol** respectively precedes or succeeds the value for a negative locally formatted monetary quantity.

char **p_sep_by_space**
: Set to a value indicating the separation of the **currency_symbol**, the sign string, and the value for a nonnegative locally formatted monetary quantity.

char **n_sep_by_space**
: Set to a value indicating the separation of the **currency_symbol**, the sign string, and the value for a negative locally formatted monetary quantity.

char **p_sign_posn**
: Set to a value indicating the positioning of the **positive_sign** for a nonnegative locally formatted monetary quantity.

© ISO 2024 — All rights reserved

char **n_sign_posn**
: Set to a value indicating the positioning of the **negative_sign** for a negative locally formatted monetary quantity.

char *__int_curr_symbol__
: The international currency symbol applicable to the current locale. The first three characters contain the alphabetic international currency symbol in accordance with those specified in ISO 4217. The fourth character (immediately preceding the null character) is the character used to separate the international currency symbol from the monetary quantity.

char **int_frac_digits**
: The number of fractional digits (those after the decimal-point) to be displayed in an internationally formatted monetary quantity.

char **int_p_cs_precedes**
: Set to 1 or 0 if the **int_curr_symbol** respectively precedes or succeeds the value for a nonnegative internationally formatted monetary quantity.

char **int_n_cs_precedes**
: Set to 1 or 0 if the **int_curr_symbol** respectively precedes or succeeds the value for a negative internationally formatted monetary quantity.

char **int_p_sep_by_space**
: Set to a value indicating the separation of the **int_curr_symbol**, the sign string, and the value for a nonnegative internationally formatted monetary quantity.

char **int_n_sep_by_space**
: Set to a value indicating the separation of the **int_curr_symbol**, the sign string, and the value for a negative internationally formatted monetary quantity.

char **int_p_sign_posn**
: Set to a value indicating the positioning of the **positive_sign** for a nonnegative internationally formatted monetary quantity.

char **int_n_sign_posn**
: Set to a value indicating the positioning of the **negative_sign** for a negative internationally formatted monetary quantity.

4 The elements of **grouping** and **mon_grouping** are interpreted according to the following:

CHAR_MAX No further grouping is to be performed.

`0` The previous element is to be repeatedly used for the remainder of the digits.

other The integer value is the number of digits that compose the current group. The next element is examined to determine the size of the next group of digits before the current group.

5 The values of **p_sep_by_space**, **n_sep_by_space**, **int_p_sep_by_space**, and **int_n_sep_by_space** are interpreted according to the following:

`0` No space separates the currency symbol and value.

`1` If the currency symbol and sign string are adjacent, a space separates them from the value; otherwise, a space separates the currency symbol from the value.

`2` If the currency symbol and sign string are adjacent, a space separates them; otherwise, a space separates the sign string from the value.

© ISO 2024 — All rights reserved

For **int_p_sep_by_space** and **int_n_sep_by_space**, the fourth character of **int_curr_symbol** is used instead of a space.

6 The values of **p_sign_posn**, **n_sign_posn**, **int_p_sign_posn**, and **int_n_sign_posn** are interpreted according to the following:

`0` Parentheses surround the quantity and currency symbol.

`1` The sign string precedes the quantity and currency symbol.

`2` The sign string succeeds the quantity and currency symbol.

`3` The sign string immediately precedes the currency symbol.

`4` The sign string immediately succeeds the currency symbol.

7 The implementation shall behave as if no library function calls the **localeconv** function.

Returns

8 The **localeconv** function returns a pointer to the filled-in object. The structure pointed to by the return value shall not be modified by the program, but may be overwritten by a subsequent call to the **localeconv** function. In addition, calls to the **setlocale** function with categories **LC_ALL**, **LC_MONETARY**, or **LC_NUMERIC** may overwrite the contents of the structure.

9 EXAMPLE 1 The following table illustrates rules which could have been used by four countries to format monetary quantities.

	Local format		International format	
Country	Positive	Negative	Positive	Negative
Country1	`1.234,56 mk`	`-1.234,56 mk`	`FIM 1.234,56`	`FIM -1.234,56`
Country2	`L.1.234`	`-L.1.234`	`ITL 1.234`	`-ITL 1.234`
Country3	`f 1.234,56`	`f -1.234,56`	`NLG 1.234,56`	`NLG -1.234,56`
Country4	`SFrs.1,234.56`	`SFrs.1,234.56C`	`CHF 1,234.56`	`CHF 1,234.56C`

10 For these four countries, the respective values for the monetary members of the structure returned by **localeconv** could be:

	Country1	Country2	Country3	Country4
mon_decimal_point	`","`	`""`	`","`	`"."`
mon_thousands_sep	`"."`	`"."`	`"."`	`","`
mon_grouping	`"\3"`	`"\3"`	`"\3"`	`"\3"`
positive_sign	`""`	`""`	`""`	`""`
negative_sign	`"-"`	`"-"`	`"-"`	`"C"`
currency_symbol	`"mk"`	`"L."`	`"\u0192"`	`"SFrs."`
frac_digits	2	0	2	2
p_cs_precedes	0	1	1	1
n_cs_precedes	0	1	1	1
p_sep_by_space	1	0	1	0
n_sep_by_space	1	0	2	0
p_sign_posn	1	1	1	1
n_sign_posn	1	1	4	2
int_curr_symbol	`"FIM "`	`"ITL "`	`"NLG "`	`"CHF "`
int_frac_digits	2	0	2	2
int_p_cs_precedes	1	1	1	1
int_n_cs_precedes	1	1	1	1
int_p_sep_by_space	1	1	1	1
int_n_sep_by_space	2	1	2	1
int_p_sign_posn	1	1	1	1
int_n_sign_posn	4	1	4	2

© ISO 2024 — All rights reserved

11 EXAMPLE 2 The following table illustrates how the `cs_precedes`, `sep_by_space`, and `sign_posn` members affect the formatted value.

p_cs_precedes	p_sign_posn	p_sep_by_space		
		0	1	2
0	0	(1.25$)	(1.25 $)	(1.25$)
	1	+1.25$	+1.25 $	+ 1.25$
	2	1.25$+	1.25 $+	1.25$ +
	3	1.25+$	1.25 +$	1.25+ $
	4	1.25$+	1.25 $+	1.25$ +
1	0	($1.25)	($ 1.25)	($1.25)
	1	+$1.25	+$ 1.25	+ $1.25
	2	$1.25+	$ 1.25+	$1.25 +
	3	+$1.25	+$ 1.25	+ $1.25
	4	$+1.25	$+ 1.25	$ +1.25

© ISO 2024 — All rights reserved

7.12 Mathematics <math.h>

1 The header <math.h> declares two types and many mathematical functions and defines several macros. Most synopses specify a family of functions consisting of a principal function with one or more **double** parameters, a **double** return value, or both; and other functions with the same name but with **f** and **l** suffixes, which are corresponding functions with **float** and **long double** parameters, return values, or both.[262] Integer arithmetic functions and conversion functions are discussed later.

2 The feature test macro **__STDC_VERSION_MATH_H__** expands to the token 202311L.

3 The types

```
float_t
double_t
```

are floating types such that the values of **float** and **double** are subsets of the values of **float_t** and **double_t**, respectively, and such that the values of **float_t** are a subset of the values of **double_t**. If **FLT_EVAL_METHOD** equals 0, **float_t** and **double_t** are **float** and **double**, respectively; if **FLT_EVAL_METHOD** equals 1, they are both **double**; if **FLT_EVAL_METHOD** equals 2, they are both **long double**; and for other values of **FLT_EVAL_METHOD**, they are otherwise implementation-defined.[263] If they are not real floating types, the behavior is implementation-defined.

4 The types

```
_Decimal32_t
_Decimal64_t
```

are decimal floating types at least as wide as **_Decimal32** and **_Decimal64**, respectively, and such that **_Decimal64_t** is at least as wide as **_Decimal32_t**. They are present only if the implementation defines **__STDC_IEC_60559_DFP__** and additionally the user code defines **__STDC_WANT_IEC_60559_EXT__** before any inclusion of <math.h>. If **DEC_EVAL_METHOD** equals 0, **_Decimal32_t** and **_Decimal64_t** are **_Decimal32** and **_Decimal64**, respectively; if **DEC_EVAL_METHOD** equals 1, they are both **_Decimal64**; if **DEC_EVAL_METHOD** equals 2, they are both **_Decimal128**; and for other values of **DEC_EVAL_METHOD**, they are otherwise implementation-defined.

5 The macro

```
HUGE_VAL
```

expands to a **double** arithmetic constant expression, not necessarily representable as a **float**, whose value is the maximum value returned by library functions when a floating result of type **double** overflows under the default rounding mode, either maximum finite number in the type or positive or unsigned infinity. The macros

```
HUGE_VALF
HUGE_VALL
```

are respectively **float** and **long double** analogs of **HUGE_VAL**.[264]

6 The macros in this paragraph are only present if the implementation defines **__STDC_IEC_60559_DFP__** and additionally the user code defines **__STDC_WANT_IEC_60559_EXT__** before any inclusion of <math.h>. The macro

[262] Particularly on systems with wide expression evaluation, a <math.h> function can pass arguments and return values in wider format than the synopsis prototype indicates.

[263] The types **float_t** and **double_t** are intended to be the implementation's most efficient types at least as wide as **float** and **double**, respectively. For **FLT_EVAL_METHOD** equal 0, 1, or 2, the type **float_t** is the narrowest type used by the implementation to evaluate floating expressions.

[264] **HUGE_VAL**, **HUGE_VALF**, and **HUGE_VALL** can be positive infinities in an implementation that supports infinities.

© ISO 2024 — All rights reserved

```
HUGE_VAL_D32
```

expands to an arithmetic constant expression of type **_Decimal32** representing positive infinity. The macros

```
HUGE_VAL_D64
HUGE_VAL_D128
```

are respectively **_Decimal64** and **_Decimal128** analogs of **HUGE_VAL_D32**.

7 The macro

```
INFINITY
```

is defined if and only if the implementation supports an infinity for the type **float**. It expands to an arithmetic constant expression of type **float** representing positive or unsigned infinity.

8 The macro

```
DEC_INFINITY
```

expands to an arithmetic constant expression of type **_Decimal32** representing positive infinity.

9 The macro

```
NAN
```

is defined if and only if the implementation supports quiet NaNs for the **float** type. It expands to an arithmetic constant expression of type **float** representing a quiet NaN.

10 The macro

```
DEC_NAN
```

expands to an arithmetic constant expression of type **_Decimal32** representing a quiet NaN.

11 Use of the macros **INFINITY**, **DEC_INFINITY**, **NAN**, and **DEC_NAN** in `<math.h>` is an obsolescent feature. Instead, use the same macros in `<float.h>`.

12 The *number classification macros*

```
FP_INFINITE
FP_NAN
FP_NORMAL
FP_SUBNORMAL
FP_ZERO
```

represent mutually exclusive kinds of floating-point values. They expand to integer constant expressions with distinct values. Additional implementation-defined floating-point classifications, with macro definitions beginning with **FP_** and an uppercase letter, may also be specified by the implementation.

13 The *math rounding direction macros*

```
FP_INT_UPWARD
FP_INT_DOWNWARD
FP_INT_TOWARDZERO
FP_INT_TONEARESTFROMZERO
FP_INT_TONEAREST
```

represent the rounding directions of the functions **ceil**, **floor**, **trunc**, **round**, and **roundeven**, respectively, that convert to integral values in floating-point formats. They expand to integer

 © ISO 2024 — All rights reserved

constant expressions with distinct values suitable for use as the second argument to the **fromfp**, **ufromfp**, **fromfpx**, and **ufromfpx** functions.

14 The macro

```
FP_FAST_FMA
```

is optionally defined. If defined, it indicates that the **fma** function generally executes about as fast as, or faster than, a multiply and an add of **double** operands.[265] The macros

```
FP_FAST_FMAF
FP_FAST_FMAL
```

are, respectively, **float** and **long double** analogs of **FP_FAST_FMA**. If defined, these macros expand to the integer constant 1.

15 The macros

```
FP_FAST_FMAD32
FP_FAST_FMAD64
FP_FAST_FMAD128
```

are, respectively, **_Decimal32**, **_Decimal64**, and **_Decimal128** analogs of **FP_FAST_FMA**.

16 Each of the macros

FP_FAST_FADD	**FP_FAST_DSUBL**	**FP_FAST_FDIVL**	**FP_FAST_FFMA**
FP_FAST_FADDL	**FP_FAST_FMUL**	**FP_FAST_DDIVL**	**FP_FAST_FFMAL**
FP_FAST_DADDL	**FP_FAST_FMULL**	**FP_FAST_FSQRT**	**FP_FAST_DFMAL**
FP_FAST_FSUB	**FP_FAST_DMULL**	**FP_FAST_FSQRTL**	
FP_FAST_FSUBL	**FP_FAST_FDIV**	**FP_FAST_DSQRTL**	

is optionally defined. If defined, it indicates that the corresponding function generally executes about as fast, or faster, than the corresponding operation or function of the argument type with result type the same as the argument type followed by conversion to the narrower type. For **FP_FAST_FFMA**, **FP_FAST_FFMAL**, and **FP_FAST_DFMAL**, the comparison is to a call to **fma** or **fmal** followed by a conversion, not to separate multiply, add, and conversion. If defined, these macros expand to the integer constant 1.

17 The macros

FP_FAST_D32ADDD64	**FP_FAST_D32MULD64**	**FP_FAST_D32FMAD64**
FP_FAST_D32ADDD128	**FP_FAST_D32MULD128**	**FP_FAST_D32FMAD128**
FP_FAST_D64ADDD128	**FP_FAST_D64MULD128**	**FP_FAST_D64FMAD128**
FP_FAST_D32SUBD64	**FP_FAST_D32DIVD64**	**FP_FAST_D32SQRTD64**
FP_FAST_D32SUBD128	**FP_FAST_D32DIVD128**	**FP_FAST_D32SQRTD128**
FP_FAST_D64SUBD128	**FP_FAST_D64DIVD128**	**FP_FAST_D64SQRTD128**

are analogs of **FP_FAST_FADD**, **FP_FAST_FADDL**, **FP_FAST_DADDL**, etc., for decimal floating types.

18 The macros

```
FP_ILOGB0
FP_ILOGBNAN
```

expand to integer constant expressions whose values are returned by **ilogb**(x) if x is zero or NaN, respectively. The value of **FP_ILOGB0** shall be either **INT_MIN** or **-INT_MAX**. The value of **FP_ILOGBNAN** shall be either **INT_MAX** or **INT_MIN**.

[265] Typically, the **FP_FAST_FMA** macro is defined if and only if the **fma** function is implemented directly with a hardware multiply-add instruction. Software implementations are expected to be substantially slower.

© ISO 2024 — All rights reserved

19 The macros

```
FP_LLOGB0
FP_LLOGBNAN
```

expand to integer constant expressions whose values are returned by **llogb**(x) if x is zero or NaN, respectively. The value of **FP_LLOGB0** shall be **LONG_MIN** if the value of **FP_ILOGB0** is **INT_MIN**, and shall be -**LONG_MAX** if the value of **FP_ILOGB0** is -**INT_MAX**. The value of **FP_LLOGBNAN** shall be **LONG_MAX** if the value of **FP_ILOGBNAN** is **INT_MAX**, and shall be **LONG_MIN** if the value of **FP_ILOGBNAN** is **INT_MIN**.

20 The macros

```
MATH_ERRNO
MATH_ERREXCEPT
```

expand to the integer constants 1 and 2, respectively; the macro

```
math_errhandling
```

expands to an expression that has type **int** and the value **MATH_ERRNO**, **MATH_ERREXCEPT**, the bitwise OR of both, or 0; the value shall not be 0 in a hosted implementation. The value of **math_errhandling** is constant for the duration of the program. It is unspecified whether **math_errhandling** is a macro or an identifier with external linkage. If a macro definition is suppressed or a program defines an identifier with the name **math_errhandling**, the behavior is undefined. If the expression **math_errhandling** & **MATH_ERREXCEPT** can be nonzero, the implementation shall define the macros **FE_DIVBYZERO**, **FE_INVALID**, and **FE_OVERFLOW** in <fenv.h>.

7.12.1 Treatment of error conditions

1 The behavior of each of the functions in <math.h> is specified for all representable values of its input arguments, except where explicitly stated otherwise. Each function shall execute as if it were a single operation without raising **SIGFPE** and without generating any of the floating-point exceptions "invalid", "divide-by-zero", or "overflow" except to reflect the result of the function.

2 For all functions, a *domain error* occurs if and only if an input argument is outside the domain over which the mathematical function is defined. The description of each function lists any required domain errors; an implementation may define additional domain errors, provided that such errors are consistent with the mathematical definition of the function.[266)] Whether a signaling NaN input causes a domain error is implementation-defined. On a domain error, the function returns an implementation-defined value; if the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero, the integer expression **errno** acquires the value **EDOM**; if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero, the "invalid" floating-point exception is raised.

3 Similarly, a *pole error* (also known as a singularity or infinitary) occurs if and only if the mathematical function has an exact infinite result as the finite input argument(s) are approached in the limit (for example, **log**(0.0)). The description of each function lists any required pole errors; an implementation may define additional pole errors, provided that such errors are consistent with the mathematical definition of the function. On a pole error, the function returns an implementation-defined value; if the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero, the integer expression **errno** acquires the value **ERANGE**; if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero, the "divide-by-zero" floating-point exception is raised.

4 Likewise, a *range error* occurs if and only if the result overflows or underflows, as defined below. The description of each function lists any required range errors; an implementation may define additional range errors, provided that such errors are consistent with the mathematical definition of

[266)] In an implementation that supports infinities, this allows an infinity as an argument to be a domain error if the mathematical domain of the function does not include the infinity.

© ISO 2024 — All rights reserved

the function and are the result of either overflow or underflow.[267)]

5 A floating result overflows if a finite result value with ordinary accuracy[268)] would have magnitude (absolute value) too large for the representation with full precision in the specified type. A result that is exactly an infinity does not overflow. If a floating result overflows and default rounding is in effect, then the function returns the value of the macro **HUGE_VAL**, **HUGE_VALF**, or **HUGE_VALL** according to the return type, with the same sign as the correct value of the function; however, for the types with reduced-precision representations of numbers beyond the overflow threshold, the function may return a representation of the result with less than full precision for the type. If a floating result overflows and default rounding is in effect and the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero, then the integer expression **errno** acquires the value **ERANGE**. If a floating result overflows, and the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero, the "overflow" floating-point exception is raised (regardless of whether default rounding is in effect).

6 The result underflows if a nonzero result value with ordinary accuracy would have magnitude (absolute value) less than the minimum normalized number in the type; however a zero result that is specified to be an exact zero does not underflow. Also, a result with ordinary accuracy and the magnitude of the minimum normalized number may underflow.[269)] If the result underflows, the function returns an implementation-defined value whose magnitude is no greater than the smallest normalized positive number in the specified type; if the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero, whether **errno** acquires the value **ERANGE** is implementation-defined; if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero, whether the "underflow" floating-point exception is raised is implementation-defined.

7 If a domain, pole, or range error occurs and the integer expression **math_errhandling** & **MATH_ERRNO** is zero,[270)] then **errno** shall either be set to the value corresponding to the error or left unmodified. If no such error occurs, **errno** shall be left unmodified regardless of the setting of **math_errhandling**.

7.12.2 The FP_CONTRACT pragma

Synopsis

1
```
#include <math.h>
#pragma STDC FP_CONTRACT on-off-switch
```

Description

2 The **FP_CONTRACT** pragma can be used to allow (if the state is "on") or disallow (if the state is "off") the implementation to contract expressions (6.5.1). Each pragma can occur either outside external declarations or preceding all explicit declarations and statements inside a compound statement. When outside external declarations, the pragma takes effect from its occurrence until another **FP_CONTRACT** pragma is encountered, or until the end of the translation unit. When inside a compound statement, the pragma takes effect from its occurrence until another **FP_CONTRACT** pragma is encountered (including within a nested compound statement), or until the end of the compound statement; at the end of a compound statement the state for the pragma is restored to its condition just before the compound statement. If this pragma is used in any other context, the behavior is undefined. The default state ("on" or "off") for the pragma is implementation-defined.

7.12.3 Classification macros

1 Floating-point values can be classified as NaN, infinite, normal, subnormal, or zero, or into other implementation-defined categories. Numbers whose magnitude is at least $b^{e_{\min}-1}$ (the minimum magnitude of normalized floating-point numbers in the type) and at most $(1 - b^{-p})b^{e_{\max}}$ (the maximum magnitude of normalized floating-point numbers in the type), where b, p, $e_{\min}$, and $e_{\max}$

[267)] Range errors that are required or implementation-defined shall or may be reported, as specified in this subclause.
[268)] Ordinary accuracy is determined by the implementation. It refers to the accuracy of the function where results are not compromised by extreme magnitude.
[269)] The term underflow here is intended to encompass both "gradual underflow" as in ISO/IEC 60559 and also "flush-to-zero" underflow. ISO/IEC 60559 underflow can occur in cases where the magnitude of the rounded result (accurate to the full precision of the type) equals the minimum normalized number in the format.
[270)] Math errors are being indicated by the floating-point exception flags rather than by **errno**.

© ISO 2024 — All rights reserved

are as in 5.2.5.3.3, are classified as normal. Larger magnitude finite numbers represented with full precision in the type may also be classified as normal. Nonzero numbers whose magnitude is less than $b^{e_{\min}-1}$ are classified as subnormal.

2 In the synopses in this subclause, *real-floating* indicates that the argument shall be an expression of real floating type.

7.12.3.1 The `fpclassify` macro

Synopsis

1
```
#include <math.h>
int fpclassify(real-floating x);
```

Description

2 The **`fpclassify`** macro classifies its argument value as NaN, infinite, normal, subnormal, zero, or into another implementation-defined category. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then classification is based on the type of the argument.[271)]

Returns

3 The **`fpclassify`** macro returns the value of the number classification macro appropriate to the value of its argument.

7.12.3.2 The `iscanonical` macro

Synopsis

1
```
#include <math.h>
int iscanonical(real-floating x);
```

Description

2 The **`iscanonical`** macro determines whether its argument value is canonical (5.2.5.3.3). First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then, determination is based on the type of the argument.

Returns

3 The **`iscanonical`** macro returns a nonzero value if and only if its argument is canonical.

7.12.3.3 The `isfinite` macro

Synopsis

1
```
#include <math.h>
int isfinite(real-floating x);
```

Description

2 The **`isfinite`** macro determines whether its argument has a finite value (zero, subnormal, or normal, and not infinite or NaN). First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.

Returns

3 The **`isfinite`** macro returns a nonzero value if and only if its argument has a finite value.

7.12.3.4 The `isinf` macro

Synopsis

1
```
#include <math.h>
```

[271)] Since an expression can be evaluated with more range and precision than its type has, it is important to know the type that classification is based on. For example, a normal **`long double`** value can become subnormal when converted to **`double`**, and zero when converted to **`float`**.

 © ISO 2024 — All rights reserved

```
int isinf(real-floating x);
```

Description

2 The **isinf** macro determines whether its argument value is (positive or negative) infinity. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.

Returns

3 The **isinf** macro returns a nonzero value if and only if its argument has an infinite value.

7.12.3.5 The isnan macro

Synopsis

1
```
#include <math.h>
int isnan(real-floating x);
```

Description

2 The **isnan** macro determines whether its argument value is a NaN. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.[272)]

Returns

3 The **isnan** macro returns a nonzero value if and only if its argument has a NaN value.

7.12.3.6 The isnormal macro

Synopsis

1
```
#include <math.h>
int isnormal(real-floating x);
```

Description

2 The **isnormal** macro determines whether its argument value is normal (neither zero, subnormal, infinite, nor NaN). First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.

Returns

3 The **isnormal** macro returns a nonzero value if and only if its argument has a normal value.

7.12.3.7 The signbit macro

Synopsis

1
```
#include <math.h>
int signbit(real-floating x);
```

Description

2 The **signbit** macro determines whether the sign of its argument value is negative.[273)] If the argument value is an unsigned zero, its sign is regarded as positive. Otherwise, if the argument value is unsigned, the result value (zero or nonzero) is implementation-defined.

Returns

3 The **signbit** macro returns a nonzero value if and only if the sign of its argument value is determined to be negative.

7.12.3.8 The issignaling macro

[272)] For the **isnan** macro, the type for determination does not matter unless the implementation supports NaNs in the evaluation type but not in the semantic type.

[273)] The **signbit** macro determines the sign of all values, including infinities, zeros, and NaNs.

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <math.h>
int issignaling(real-floating x);
```

Description

2 The **issignaling** macro determines whether its argument value is a signaling NaN.

Returns

3 The **issignaling** macro returns a nonzero value if and only if its argument is a signaling NaN.[274)]

7.12.3.9 The issubnormal macro

Synopsis

1
```
#include <math.h>
int issubnormal(real-floating x);
```

Description

2 The **issubnormal** macro determines whether its argument value is subnormal. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then determination is based on the type of the argument.

Returns

3 The **issubnormal** macro returns a nonzero value if and only if its argument is subnormal.

7.12.3.10 The iszero macro

Synopsis

1
```
#include <math.h>
int iszero(real-floating x);
```

Description

2 The **iszero** macro determines whether its argument value is (positive, negative, or unsigned) zero. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then, determination is based on the type of the argument.

Returns

3 The **iszero** macro returns a nonzero value if and only if its argument is zero.

7.12.4 Trigonometric functions

7.12.4.1 The acos functions

Synopsis

1
```
#include <math.h>
double acos(double x);
float acosf(float x);
long double acosl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 acosd32(_Decimal32 x);
_Decimal64 acosd64(_Decimal64 x);
_Decimal128 acosd128(_Decimal128 x);
#endif
```

[274)] F.3 specifies that **issignaling** (and all the other classification macros), raise no floating-point exception if the argument is a variable, or any other expression whose value is represented in the format of its semantic type, even if the value is a signaling NaN.

 © ISO 2024 — All rights reserved

Description

2 The **acos** functions compute the principal value of the arc cosine of x. A domain error occurs for arguments not in the interval $[-1, +1]$.

Returns

3 The **acos** functions return $\arccos$ x in the interval $[0, \pi]$ radians.

7.12.4.2 The asin functions

Synopsis

1
```
#include <math.h>
double asin(double x);
float asinf(float x);
long double asinl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 asind32(_Decimal32 x);
_Decimal64 asind64(_Decimal64 x);
_Decimal128 asind128(_Decimal128 x);
#endif
```

Description

2 The **asin** functions compute the principal value of the arc sine of x. A domain error occurs for arguments not in the interval $[-1, +1]$. A range error occurs if nonzero x is too close to zero.

Returns

3 The **asin** functions return $\arcsin$ x in the interval $[-\frac{\pi}{2}, +\frac{\pi}{2}]$ radians.

7.12.4.3 The atan functions

Synopsis

1
```
#include <math.h>
double atan(double x);
float atanf(float x);
long double atanl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 atand32(_Decimal32 x);
_Decimal64 atand64(_Decimal64 x);
_Decimal128 atand128(_Decimal128 x);
#endif
```

Description

2 The **atan** functions compute the principal value of the arc tangent of x. A range error occurs if nonzero x is too close to zero.

Returns

3 The **atan** functions return $\arctan$ x in the interval $[-\frac{\pi}{2}, +\frac{\pi}{2}]$ radians.

7.12.4.4 The atan2 functions

Synopsis

1
```
#include <math.h>
double atan2(double y, double x);
float atan2f(float y, float x);
long double atan2l(long double y, long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 atan2d32(_Decimal32 y, _Decimal32 x);
_Decimal64 atan2d64(_Decimal64 y, _Decimal64 x);
_Decimal128 atan2d128(_Decimal128 y, _Decimal128 x);
#endif
```

© ISO 2024 — All rights reserved

Description

2 The **atan2** functions compute the value of the arc tangent of y/x, using the signs of both arguments to determine the quadrant of the return value. A domain error may occur if both arguments are zero. A range error occurs if x is positive and nonzero $\frac{y}{x}$ is too close to zero.

Returns

3 The **atan2** functions return $\arctan(y/x)$ in the interval $[-\pi, +\pi]$ radians.

7.12.4.5 The cos functions

Synopsis

1
```
#include <math.h>
double cos(double x);
float cosf(float x);
long double cosl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 cosd32(_Decimal32 x);
_Decimal64 cosd64(_Decimal64 x);
_Decimal128 cosd128(_Decimal128 x);
#endif
```

Description

2 The **cos** functions compute the cosine of x (measured in radians).

Returns

3 The **cos** functions return $\cos x$.

7.12.4.6 The sin functions

Synopsis

1
```
#include <math.h>
double sin(double x);
float sinf(float x);
long double sinl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 sind32(_Decimal32 x);
_Decimal64 sind64(_Decimal64 x);
_Decimal128 sind128(_Decimal128 x);
#endif
```

Description

2 The **sin** functions compute the sine of x (measured in radians). A range error occurs if nonzero x is too close to zero.

Returns

3 The **sin** functions return $\sin x$.

7.12.4.7 The tan functions

Synopsis

1
```
#include <math.h>
double tan(double x);
float tanf(float x);
long double tanl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 tand32(_Decimal32 x);
_Decimal64 tand64(_Decimal64 x);
_Decimal128 tand128(_Decimal128 x);
#endif
```


© ISO 2024 — All rights reserved

Description

2 The **tan** functions return the tangent of x (measured in radians). A range error occurs if nonzero x is too close to zero.

Returns

3 The **tan** functions return $\tan$ x.

7.12.4.8 The acospi functions

Synopsis

1
```
#include <math.h>
double acospi(double x);
float acospif(float x);
long double acospil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 acospid32(_Decimal32 x);
_Decimal64 acospid64(_Decimal64 x);
_Decimal128 acospid128(_Decimal128 x);
#endif
```

Description

2 The **acospi** functions compute the principal value of the arc cosine of x, divided by π, thus measuring the angle in half-revolutions. A domain error occurs for arguments not in the interval $[-1, +1]$.

Returns

3 The **acospi** functions return $\arccos(\mathrm{x})/\pi$ in the interval $[0, 1]$.

7.12.4.9 The asinpi functions

Synopsis

1
```
#include <math.h>
double asinpi(double x);
float asinpif(float x);
long double asinpil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 asinpid32(_Decimal32 x);
_Decimal64 asinpid64(_Decimal64 x);
_Decimal128 asinpid128(_Decimal128 x);
#endif
```

Description

2 The **asinpi** functions compute the principal value of the arc sine of x, divided by π, thus measuring the angle in half-revolutions. A domain error occurs for arguments not in the interval $[-1, +1]$. A range error occurs if nonzero x is too close to zero.

Returns

3 The **asinpi** functions return $\arcsin(\mathrm{x})/\pi$ in the interval $[-\frac{1}{2}, +\frac{1}{2}]$.

7.12.4.10 The atanpi functions

Synopsis

1
```
#include <math.h>
double atanpi(double x);
float atanpif(float x);
long double atanpil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 atanpid32(_Decimal32 x);
_Decimal64 atanpid64(_Decimal64 x);
```

© ISO 2024 — All rights reserved

```
_Decimal128 atanpid128(_Decimal128 x);
#endif
```

Description

2 The **atanpi** functions compute the principal value of the arc tangent of x, divided by π, thus measuring the angle in half-revolutions. A range error occurs if nonzero x is too close to zero.

Returns

3 The **atanpi** functions return $\arctan(x)/\pi$. in the interval $[-\frac{1}{2}, +\frac{1}{2}]$.

7.12.4.11 The atan2pi functions

Synopsis

1
```
#include <math.h>
double atan2pi(double y, double x);
float atan2pif(float y, float x);
long double atan2pil(long double y, long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 atan2pid32(_Decimal32 y, _Decimal32 x);
_Decimal64 atan2pid64(_Decimal64 y, _Decimal64 x);
_Decimal128 atan2pid128(_Decimal128 y, _Decimal128 x);
#endif
```

Description

2 The **atan2pi** functions compute the angle, measured in half-revolutions, subtended at the origin by the point (x, y) and the positive x-axis. Thus, the **atan2pi** functions compute $\arctan(\frac{y}{x})/\pi$, in the range $[-1, +1]$. A domain error may occur if both arguments are zero. A range error occurs if x is positive and nonzero $\frac{y}{x}$ is too close to zero.

Returns

3 The **atan2pi** functions return the computed angle, in the interval $[-1, +1]$.

7.12.4.12 The cospi functions

Synopsis

1
```
#include <math.h>
double cospi(double x);
float cospif(float x);
long double cospil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 cospid32(_Decimal32 x);
_Decimal64 cospid64(_Decimal64 x);
_Decimal128 cospid128(_Decimal128 x);
#endif
```

Description

2 The **cospi** functions compute the cosine of $\pi \times x$, thus regarding x as a measurement in half-revolutions.

Returns

3 The **cospi** functions return $\cos(\pi \times x)$.

7.12.4.13 The sinpi functions

Synopsis

1
```
#include <math.h>
double sinpi(double x);
float sinpif(float x);
```


© ISO 2024 — All rights reserved

```
long double sinpil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 sinpid32(_Decimal32 x);
_Decimal64 sinpid64(_Decimal64 x);
_Decimal128 sinpid128(_Decimal128 x);
#endif
```

Description

2 The **sinpi** functions compute the sine of $\pi \times$ x, thus regarding x as a measurement in half-revolutions. A range error occurs if nonzero x is too close to zero.

Returns

3 The **sinpi** functions return $\sin(\pi \times \mathrm{x})$.

7.12.4.14 The tanpi functions

Synopsis

1
```
#include <math.h>
double tanpi(double x);
float tanpif(float x);
long double tanpil(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 tanpid32(_Decimal32 x);
_Decimal64 tanpid64(_Decimal64 x);
_Decimal128 tanpid128(_Decimal128 x);
#endif
```

Description

2 The **tanpi** functions compute the tangent of $\pi \times$ x, thus regarding x as a measurement in half-revolutions. A range error occurs if nonzero x is too close to zero. A pole error may occur if $|x|$ is $(n + 0.5)$ for integer n.

Returns

3 The **tanpi** functions return $\tan(\pi \times \mathrm{x})$.

7.12.5 Hyperbolic functions

7.12.5.1 The acosh functions

Synopsis

1
```
#include <math.h>
double acosh(double x);
float acoshf(float x);
long double acoshl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 acoshd32(_Decimal32 x);
_Decimal64 acoshd64(_Decimal64 x);
_Decimal128 acoshd128(_Decimal128 x);
#endif
```

Description

2 The **acosh** functions compute the (nonnegative) arc hyperbolic cosine of x. A domain error occurs for arguments less than 1.

Returns

3 The **acosh** functions return $\operatorname{arcosh} \mathrm{x}$ in the interval $[0, +\infty]$.

7.12.5.2 The asinh functions

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <math.h>
double asinh(double x);
float asinhf(float x);
long double asinhl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 asinhd32(_Decimal32 x);
_Decimal64 asinhd64(_Decimal64 x);
_Decimal128 asinhd128(_Decimal128 x);
#endif
```

Description

2 The **asinh** functions compute the arc hyperbolic sine of x. A range error occurs if nonzero x is too close to zero.

Returns

3 The **asinh** functions return $\operatorname{arsinh} x$.

7.12.5.3 The atanh functions

Synopsis

1
```
#include <math.h>
double atanh(double x);
float atanhf(float x);
long double atanhl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 atanhd32(_Decimal32 x);
_Decimal64 atanhd64(_Decimal64 x);
_Decimal128 atanhd128(_Decimal128 x);
#endif
```

Description

2 The **atanh** functions compute the arc hyperbolic tangent of x. A domain error occurs for arguments not in the interval $[-1, +1]$. A pole error may occur if the argument equals -1 or +1. A range error occurs if nonzero x is too close to zero.

Returns

3 The **atanh** functions return $\operatorname{artanh} x$.

7.12.5.4 The cosh functions

Synopsis

1
```
#include <math.h>
double cosh(double x);
float coshf(float x);
long double coshl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 coshd32(_Decimal32 x);
_Decimal64 coshd64(_Decimal64 x);
_Decimal128 coshd128(_Decimal128 x);
#endif
```

Description

2 The **cosh** functions compute the hyperbolic cosine of x. A range error occurs if the magnitude of finite x is too large.

Returns

3 The **cosh** functions return $\cosh x$.

© ISO 2024 — All rights reserved

7.12.5.5 The `sinh` functions

Synopsis

1
```
#include <math.h>
double sinh(double x);
float sinhf(float x);
long double sinhl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 sinhd32(_Decimal32 x);
_Decimal64 sinhd64(_Decimal64 x);
_Decimal128 sinhd128(_Decimal128 x);
#endif
```

Description

2 The **`sinh`** functions compute the hyperbolic sine of `x`. A range error occurs if the magnitude of finite `x` is too large or if nonzero `x` is too close to zero.

Returns

3 The **`sinh`** functions return $\sinh$ `x`.

7.12.5.6 The `tanh` functions

Synopsis

1
```
#include <math.h>
double tanh(double x);
float tanhf(float x);
long double tanhl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 tanhd32(_Decimal32 x);
_Decimal64 tanhd64(_Decimal64 x);
_Decimal128 tanhd128(_Decimal128 x);
#endif
```

Description

2 The **`tanh`** functions compute the hyperbolic tangent of `x`. A range error occurs if nonzero `x` is too close to zero.

Returns

3 The **`tanh`** functions return $\tanh$ `x`.

7.12.6 Exponential and logarithmic functions

7.12.6.1 The `exp` functions

Synopsis

1
```
#include <math.h>
double exp(double x);
float expf(float x);
long double expl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 expd32(_Decimal32 x);
_Decimal64 expd64(_Decimal64 x);
_Decimal128 expd128(_Decimal128 x);
#endif
```

Description

2 The **`exp`** functions compute the base-e exponential of `x`. A range error occurs if the magnitude of finite `x` is too large.

© ISO 2024 — All rights reserved

Returns

3 The **exp** functions return e^x.

7.12.6.2 The exp10 functions

Synopsis

1
```
#include <math.h>
double exp10(double x);
float exp10f(float x);
long double exp10l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 exp10d32(_Decimal32 x);
_Decimal64 exp10d64(_Decimal64 x);
_Decimal128 exp10d128(_Decimal128 x);
#endif
```

Description

2 The **exp10** functions compute the base-10 exponential of x. A range error occurs if the magnitude of finite x is too large.

Returns

3 The **exp10** functions return 10^x.

7.12.6.3 The exp10m1 functions

Synopsis

1
```
#include <math.h>
double exp10m1(double x);
float exp10m1f(float x);
long double exp10m1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 exp10m1d32(_Decimal32 x);
_Decimal64 exp10m1d64(_Decimal64 x);
_Decimal128 exp10m1d128(_Decimal128 x);
#endif
```

Description

2 The **exp10m1** functions compute the base-10 exponential of the argument, minus 1. A range error occurs if positive finite x is too large or if nonzero x is too close to zero.

Returns

3 The **exp10m1** functions return $10^x - 1$.

7.12.6.4 The exp2 functions

Synopsis

1
```
#include <math.h>
double exp2(double x);
float exp2f(float x);
long double exp2l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 exp2d32(_Decimal32 x);
_Decimal64 exp2d64(_Decimal64 x);
_Decimal128 exp2d128(_Decimal128 x);
#endif
```

Description

2 The **exp2** functions compute the base-2 exponential of x. A range error occurs if the magnitude of finite x is too large.

 © ISO 2024 — All rights reserved

Returns

3 The **exp2** functions return 2^x.

7.12.6.5 The exp2m1 functions

Synopsis

1
```
#include <math.h>
double exp2m1(double x);
float exp2m1f(float x);
long double exp2m1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 exp2m1d32(_Decimal32 x);
_Decimal64 exp2m1d64(_Decimal64 x);
_Decimal128 exp2m1d128(_Decimal128 x);
#endif
```

Description

2 The **exp2m1** functions compute the base-2 exponential of the argument, minus 1. A range error occurs if positive finite x is too large or if nonzero x is too close to zero.

Returns

3 The **exp2m1** functions return $2^x - 1$.

7.12.6.6 The expm1 functions

Synopsis

1
```
#include <math.h>
double expm1(double x);
float expm1f(float x);
long double expm1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 expm1d32(_Decimal32 x);
_Decimal64 expm1d64(_Decimal64 x);
_Decimal128 expm1d128(_Decimal128 x);
#endif
```

Description

2 The **expm1** functions compute the base-e exponential of the argument, minus 1. A range error occurs if positive finite x is too large or if nonzero x is too close to zero.[275)]

Returns

3 The **expm1** functions return $e^x - 1$.

7.12.6.7 The frexp functions

Synopsis

1
```
#include <math.h>
double frexp(double value, int *p);
float frexpf(float value, int *p);
long double frexpl(long double value, int *p);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 frexpd32(_Decimal32 value, int *p);
_Decimal64 frexpd64(_Decimal64 value, int *p);
_Decimal128 frexpd128(_Decimal128 value, int *p);
#endif
```

[275)] For small magnitude x, **expm1**(x) is expected to be more accurate than **exp**(x) - 1.

© ISO 2024 — All rights reserved

Description

2 The **frexp** functions break a floating-point number into a normalized fraction and an integer exponent. They store the integer in the **int** object pointed to by p. If the return type of the function is a standard floating type, the exponent is an integral power of 2. If the return type of the function is a decimal floating type, the exponent is an integral power of 10.

Returns

3 If value is not a floating-point number or if the integral power is outside the range of **int**, the results are unspecified. Otherwise, the **frexp** functions return the value x, such that x has a magnitude in the interval $[\frac{1}{2}, 1)$ or zero, and value equals x $\times$ 2^{*p}, when the return type of the function is a standard floating type; or x has a magnitude in the interval $[1/10, 1)$ or zero, and value equals x $\times$ 10^{*p}, when the return type of the function is a decimal floating type. If value is zero, both parts of the result are zero.

7.12.6.8 The ilogb functions

Synopsis

1
```
#include <math.h>
int ilogb(double x);
int ilogbf(float x);
int ilogbl(long double x);
#ifdef __STDC_IEC_60559_DFP__
int ilogbd32(_Decimal32 x);
int ilogbd64(_Decimal64 x);
int ilogbd128(_Decimal128 x);
#endif
```

Description

2 The **ilogb** functions extract the exponent of x as a signed **int** value. If x is zero they compute the value **FP_ILOGB0**; if x is infinite they compute the value **INT_MAX**; if x is a NaN they compute the value **FP_ILOGBNAN**; otherwise, they are equivalent to calling the corresponding **logb** function and converting the returned value to type **int**. A domain error or range error may occur if x is zero, infinite, or NaN. If the correct value is outside the range of the return type, the numeric result is unspecified and a domain error or range error may occur.

Returns

3 The **ilogb** functions return the exponent of x as a signed **int** value.

Forward references: the **logb** functions (7.12.6.17).

7.12.6.9 The ldexp functions

Synopsis

1
```
#include <math.h>
double ldexp(double x, int p);
float ldexpf(float x, int p);
long double ldexpl(long double x, int p);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 ldexpd32(_Decimal32 x, int p);
_Decimal64 ldexpd64(_Decimal64 x, int p);
_Decimal128 ldexpd128(_Decimal128 x, int p);
#endif
```

Description

2 The **ldexp** functions multiply a floating-point number by an integral power of 2 when the return type of the function is a standard floating type, or by an integral power of 10 when the return type of the function is a decimal floating type. A range error occurs for some finite x, depending on p.

 © ISO 2024 — All rights reserved

Returns

3 The **ldexp** functions return $x \times 2^p$ when the return type of the function is a standard floating type, or return $x \times 10^p$ when the return type of the function is a decimal floating type.

7.12.6.10 The llogb functions

Synopsis

1
```
#include <math.h>
long int llogb(double x);
long int llogbf(float x);
long int llogbl(long double x);
#ifdef __STDC_IEC_60559_DFP__
long int llogbd32(_Decimal32 x);
long int llogbd64(_Decimal64 x);
long int llogbd128(_Decimal128 x);
#endif
```

Description

2 The **llogb** functions extract the exponent of x as a signed **long int** value. If x is zero they compute the value **FP_LLOGB0**; if x is infinite they compute the value **LONG_MAX**; if x is a NaN they compute the value **FP_LLOGBNAN**; otherwise, they are equivalent to calling the corresponding **logb** function and converting the returned value to type **long int**. A domain error or range error may occur if x is zero, infinite, or NaN. If the correct value is outside the range of the return type, the numeric result is unspecified.

Returns

3 The **llogb** functions return the exponent of x as a signed **long int** value.

Forward references: the **logb** functions (7.12.6.17).

7.12.6.11 The log functions

Synopsis

1
```
#include <math.h>
double log(double x);
float logf(float x);
long double logl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 logd32(_Decimal32 x);
_Decimal64 logd64(_Decimal64 x);
_Decimal128 logd128(_Decimal128 x);
#endif
```

Description

2 The **log** functions compute the base-e (natural) logarithm of x. A domain error occurs if the argument is less than zero. A pole error may occur if the argument is zero.

Returns

3 The **log** functions return $\log_e x$.

7.12.6.12 The log10 functions

Synopsis

1
```
#include <math.h>
double log10(double x);
float log10f(float x);
long double log10l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log10d32(_Decimal32 x);
_Decimal64 log10d64(_Decimal64 x);
```

© ISO 2024 — All rights reserved

```
_Decimal128 log10d128(_Decimal128 x);
#endif
```

Description

2 The **log10** functions compute the base-10 (common) logarithm of x. A domain error occurs if the argument is less than zero. A pole error may occur if the argument is zero.

Returns

3 The **log10** functions return $\log_{10} x$.

7.12.6.13 The log10p1 functions

Synopsis

1
```
#include <math.h>
double log10p1(double x);
float log10p1f(float x);
long double log10p1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log10p1d32(_Decimal32 x);
_Decimal64 log10p1d64(_Decimal64 x);
_Decimal128 log10p1d128(_Decimal128 x);
#endif
```

Description

2 The **log10p1** functions compute the base-10 logarithm of 1 plus the argument. A domain error occurs if the argument is less than -1. A pole error may occur if the argument equals -1. A range error occurs if nonzero x is too close to zero.

Returns

3 The **log10p1** functions return $\log_{10}(1 + x)$.

7.12.6.14 The log1p and logp1 functions

Synopsis

1
```
#include <math.h>
double log1p(double x);
float log1pf(float x);
long double log1pl(long double x);
double logp1(double x);
float logp1f(float x);
long double logp1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log1pd32(_Decimal32 x);
_Decimal64 log1pd64(_Decimal64 x);
_Decimal128 log1pd128(_Decimal128 x);
_Decimal32 logp1d32(_Decimal32 x);
_Decimal64 logp1d64(_Decimal64 x);
_Decimal128 logp1d128(_Decimal128 x);
#endif
```

Description

2 The **log1p** functions are equivalent to the **logp1** functions.[276] These functions compute the base-e (natural) logarithm of 1 plus the argument.[277] A domain error occurs if the argument is less than -1. A pole error may occur if the argument equals -1. A range error occurs if nonzero x is too close to zero.

[276] The **logp1** functions are preferred for name consistency with the **log10p1** and **log2p1** functions.
[277] For small magnitude x, **logp1**(x) is expected to be more accurate than **log**(1 + x).

© ISO 2024 — All rights reserved

Returns

3 The **log1p** and **logp1** functions return $\log_e(1 + \texttt{x})$.

7.12.6.15 The log2 functions

Synopsis

1
```
#include <math.h>
double log2(double x);
float log2f(float x);
long double log2l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log2d32(_Decimal32 x);
_Decimal64 log2d64(_Decimal64 x);
_Decimal128 log2d128(_Decimal128 x);
#endif
```

Description

2 The **log2** functions compute the base-2 logarithm of x. A domain error occurs if the argument is less than zero. A pole error may occur if the argument is zero.

Returns

3 The **log2** functions return $\log_2 \texttt{x}$.

7.12.6.16 The log2p1 functions

Synopsis

1
```
#include <math.h>
double log2p1(double x);
float log2p1f(float x);
long double log2p1l(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 log2p1d32(_Decimal32 x);
_Decimal64 log2p1d64(_Decimal64 x);
_Decimal128 log2p1d128(_Decimal128 x);
#endif
```

Description

2 The **log2p1** functions compute the base-2 logarithm of 1 plus the argument. A domain error occurs if the argument is less than -1. A pole error may occur if the argument equals -1. A range error occurs if nonzero x is too close to zero.

Returns

3 The **log2p1** functions return $\log_2(1+\texttt{x})$.

7.12.6.17 The logb functions

Synopsis

1
```
#include <math.h>
double logb(double x);
float logbf(float x);
long double logbl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 logbd32(_Decimal32 x);
_Decimal64 logbd64(_Decimal64 x);
_Decimal128 logbd128(_Decimal128 x);
#endif
```

© ISO 2024 — All rights reserved

Description

2 The **logb** functions extract the exponent of x, as a signed integer value in floating-point format. If x is subnormal it is treated as though it were normalized; thus, for positive finite x,

$$1 \leq \mathtt{x} \times b^{-\mathtt{logb(x)}} < b$$

where $b =$ **FLT_RADIX** if the return type of the function is a standard floating type, or $b = 10$ if the return type of the function is a decimal floating type. A domain error or pole error may occur if the argument is zero.

Returns

3 The **logb** functions return the signed exponent of x.

7.12.6.18 The modf functions

Synopsis

1
```
#include <math.h>
double modf(double value, double *iptr);
float modff(float value, float *iptr);
long double modfl(long double value, long double *iptr);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 modfd32(_Decimal32 x, _Decimal32 *iptr);
_Decimal64 modfd64(_Decimal64 x, _Decimal64 *iptr);
_Decimal128 modfd128(_Decimal128 x, _Decimal128 *iptr);
#endif
```

Description

2 The **modf** functions break the argument value into integral and fractional parts, each of which has the same type and sign as the argument. They store the integral part (in floating-point format) in the object pointed to by iptr.

Returns

3 The **modf** functions return the signed fractional part of value.

7.12.6.19 The scalbn and scalbln functions

Synopsis

1
```
#include <math.h>
double scalbn(double x, int n);
float scalbnf(float x, int n);
long double scalbnl(long double x, int n);
double scalbln(double x, long int n);
float scalblnf(float x, long int n);
long double scalblnl(long double x, long int n);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 scalbnd32(_Decimal32 x, int n);
_Decimal64 scalbnd64(_Decimal64 x, int n);
_Decimal128 scalbnd128(_Decimal128 x, int n);
_Decimal32 scalblnd32(_Decimal32 x, long int n);
_Decimal64 scalblnd64(_Decimal64 x, long int n);
_Decimal128 scalblnd128(_Decimal128 x, long int n);
#endif
```

Description

2 The **scalbn** and **scalbln** functions compute $\mathtt{x} \times b^{\mathtt{n}}$, where $b =$ **FLT_RADIX** if the return type of the function is a standard floating type, or $b = 10$ if the return type of the function is a decimal floating type. A range error occurs for some finite x, depending on n.

Returns

3 The **scalbn** and **scalbln** functions return $\mathtt{x} \times b^{\mathtt{n}}$.

© ISO 2024 — All rights reserved

7.12.7 Power and absolute-value functions

7.12.7.1 The `cbrt` functions

Synopsis

1
```
#include <math.h>
double cbrt(double x);
float cbrtf(float x);
long double cbrtl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 cbrtd32(_Decimal32 x);
_Decimal64 cbrtd64(_Decimal64 x);
_Decimal128 cbrtd128(_Decimal128 x);
#endif
```

Description

2 The **`cbrt`** functions compute the real cube root of `x`.

Returns

3 The **`cbrt`** functions return $x^{\frac{1}{3}}$.

7.12.7.2 The `compoundn` functions

Synopsis

1
```
#include <math.h>
double compoundn(double x, long long int n);
float compoundnf(float x, long long int n);
long double compoundnl(long double x, long long int n);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 compoundnd32(_Decimal32 x, long long int n);
_Decimal64 compoundnd64(_Decimal64 x, long long int n);
_Decimal128 compoundnd128(_Decimal128 x, long long int n);
#endif
```

Description

2 The **`compoundn`** functions compute 1 plus `x`, raised to the power `n`. A domain error occurs if $x < -1$. Depending on `n`, a range error occurs if either positive finite `x` is too large or if `x` is too near but not equal to `-1`. A pole error may occur if `x` equals -1 and $n < 0$.

Returns

3 The **`compoundn`** functions return $(1 + x)^n$.

7.12.7.3 The `fabs` functions

Synopsis

1
```
#include <math.h>
double fabs(double x);
float fabsf(float x);
long double fabsl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fabsd32(_Decimal32 x);
_Decimal64 fabsd64(_Decimal64 x);
_Decimal128 fabsd128(_Decimal128 x);
#endif
```

Description

2 The **`fabs`** functions compute the absolute value of `x`.

© ISO 2024 — All rights reserved

Returns

3 The **fabs** functions return |x|.

7.12.7.4 The hypot functions

Synopsis

1
```
#include <math.h>
double hypot(double x, double y);
float hypotf(float x, float y);
long double hypotl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 hypotd32(_Decimal32 x, _Decimal32 y);
_Decimal64 hypotd64(_Decimal64 x, _Decimal64 y);
_Decimal128 hypotd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **hypot** functions compute the square root of the sum of the squares of x and y, without undue overflow or underflow. A range error occurs for some finite arguments.

Returns

3 The **hypot** functions return $\sqrt{x^2 + y^2}$.

7.12.7.5 The pow functions

Synopsis

1
```
#include <math.h>
double pow(double x, double y);
float powf(float x, float y);
long double powl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 powd32(_Decimal32 x, _Decimal32 y);
_Decimal64 powd64(_Decimal64 x, _Decimal64 y);
_Decimal128 powd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **pow** functions compute x raised to the power y. A domain error occurs if x is finite and less than zero and y is finite and not an integer value. A domain error may occur if x is zero and y is zero. Depending on y, a range error occurs if either the magnitude of nonzero finite x is too large or too near zero. A domain error or pole error may occur if x is zero and y is less than zero.

Returns

3 The **pow** functions return x^y.

7.12.7.6 The pown functions

Synopsis

1
```
#include <math.h>
double pown(double x, long long int n);
float pownf(float x, long long int n);
long double pownl(long double x, long long int n);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 pownd32(_Decimal32 x, long long int n);
_Decimal64 pownd64(_Decimal64 x, long long int n);
_Decimal128 pownd128(_Decimal128 x, long long int n);
#endif
```


© ISO 2024 — All rights reserved

Description

2 The **pown** functions compute x raised to the nth power. A pole error may occur if x equals 0 and n < 0. Depending on n, a range error occurs if either the magnitude of nonzero finite x is too large or too near zero.

Returns

3 The **pown** functions return x^n.

7.12.7.7 The powr functions

Synopsis

1
```
#include <math.h>
double powr(double y, double x);
float powrf(float y, float x);
long double powrl(long double y, long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 powrd32(_Decimal32 y, _Decimal32 x);
_Decimal64 powrd64(_Decimal64 y, _Decimal64 x);
_Decimal128 powrd128(_Decimal128 y, _Decimal128 x);
#endif
```

Description

2 The **powr** functions compute x raised to the power y as $e^{y \log_e x}$.[278)] A domain error occurs if x < 0 or if x and y are both zero. Depending on y, a range error occurs if either positive nonzero finite x is too large or too near zero. A pole error may occur if x equals zero and finite y < 0.

Returns

3 The **powr** functions return $e^{y \log_e x}$.

7.12.7.8 The rootn functions

Synopsis

1
```
#include <math.h>
double rootn(double x, long long int n);
float rootnf(float x, long long int n);
long double rootnl(long double x, long long int n);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 rootnd32(_Decimal32 x, long long int n);
_Decimal64 rootnd64(_Decimal64 x, long long int n);
_Decimal128 rootnd128(_Decimal128 x, long long int n);
#endif
```

Description

2 The **rootn** functions compute the principal n^{th} root of x. A domain error occurs if n is 0 or if x < 0 and n is even. If n is -1, a range error occurs if either the magnitude of nonzero finite x is too large or too near zero. A pole error may occur if x equals zero and n < 0.

Returns

3 The **rootn** functions return $x^{\frac{1}{n}}$.

7.12.7.9 The rsqrt functions

Synopsis

1
```
#include <math.h>
double rsqrt(double x);
float rsqrtf(float x);
```

[278)] Restricting the domain to that of the formula $e^{y \log_e x}$ is intended to better meet expectations for a continuous power function and to allow implementations with fewer tests for special cases.

© ISO 2024 — All rights reserved

```
long double rsqrtl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 rsqrtd32(_Decimal32 x);
_Decimal64 rsqrtd64(_Decimal64 x);
_Decimal128 rsqrtd128(_Decimal128 x);
#endif
```

Description

2 The **rsqrt** functions compute the reciprocal of the nonnegative square root of the argument. A domain error occurs if the argument is less than zero. A pole error may occur if the argument equals zero.

Returns

3 The **rsqrt** functions return $\frac{1}{\sqrt{x}}$.

7.12.7.10 The sqrt functions

Synopsis

1
```
#include <math.h>
double sqrt(double x);
float sqrtf(float x);
long double sqrtl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 sqrtd32(_Decimal32 x);
_Decimal64 sqrtd64(_Decimal64 x);
_Decimal128 sqrtd128(_Decimal128 x);
#endif
```

Description

2 The **sqrt** functions compute the nonnegative square root of x. A domain error occurs if the argument is less than zero.

Returns

3 The **sqrt** functions return $\sqrt{x}$.

7.12.8 Error and gamma functions

7.12.8.1 The erf functions

Synopsis

1
```
#include <math.h>
double erf(double x);
float erff(float x);
long double erfl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 erfd32(_Decimal32 x);
_Decimal64 erfd64(_Decimal64 x);
_Decimal128 erfd128(_Decimal128 x);
#endif
```

Description

2 The **erf** functions compute the error function of x. A range error occurs if nonzero x is too close to zero.

Returns

3 The **erf** functions return $\operatorname{erf} x = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\,dt$.

 © ISO 2024 — All rights reserved

7.12.8.2 The `erfc` functions

Synopsis

1

```
#include <math.h>
double erfc(double x);
float erfcf(float x);
long double erfcl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 erfcd32(_Decimal32 x);
_Decimal64 erfcd64(_Decimal64 x);
_Decimal128 erfcd128(_Decimal128 x);
#endif
```

Description

2 The **`erfc`** functions compute the complementary error function of `x`. A range error occurs if positive finite `x` is too large.

Returns

3 The **`erfc`** functions return $\operatorname{erfc} x = 1 - \operatorname{erf} x = \frac{2}{\sqrt{\pi}} \int_{x}^{\infty} e^{-t^2}\,dt$.

7.12.8.3 The `lgamma` functions

Synopsis

1

```
#include <math.h>
double lgamma(double x);
float lgammaf(float x);
long double lgammal(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 lgammad32(_Decimal32 x);
_Decimal64 lgammad64(_Decimal64 x);
_Decimal128 lgammad128(_Decimal128 x);
#endif
```

Description

2 The **`lgamma`** functions compute the natural logarithm of the absolute value of gamma of `x`. A range error occurs if positive finite `x` is too large. A pole error may occur if `x` is a negative integer or zero.

Returns

3 The **`lgamma`** functions return $\log_e |\Gamma(x)|$.

7.12.8.4 The `tgamma` functions

Synopsis

1

```
#include <math.h>
double tgamma(double x);
float tgammaf(float x);
long double tgammal(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 tgammad32(_Decimal32 x);
_Decimal64 tgammad64(_Decimal64 x);
_Decimal128 tgammad128(_Decimal128 x);
#endif
```

Description

2 The **`tgamma`** functions compute the gamma function of `x`. A domain error or pole error may occur if `x` is a negative integer or zero. A range error occurs for some finite `x` less than zero, if positive finite `x` is too large, or nonzero `x` is too close to zero.

© ISO 2024 — All rights reserved

Returns

3 The **tgamma** functions return $\Gamma(x)$.

7.12.9 Nearest integer functions

7.12.9.1 The ceil functions

Synopsis

1
```
#include <math.h>
double ceil(double x);
float ceilf(float x);
long double ceill(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 ceild32(_Decimal32 x);
_Decimal64 ceild64(_Decimal64 x);
_Decimal128 ceild128(_Decimal128 x);
#endif
```

Description

2 The **ceil** functions compute the smallest integer value not less than x.

Returns

3 The **ceil** functions return $\lceil x \rceil$, expressed as a floating-point number.

7.12.9.2 The floor functions

Synopsis

1
```
#include <math.h>
double floor(double x);
float floorf(float x);
long double floorl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 floord32(_Decimal32 x);
_Decimal64 floord64(_Decimal64 x);
_Decimal128 floord128(_Decimal128 x);
#endif
```

Description

2 The **floor** functions compute the largest integer value not greater than x.

Returns

3 The **floor** functions return $\lfloor x \rfloor$, expressed as a floating-point number.

7.12.9.3 The nearbyint functions

Synopsis

1
```
#include <math.h>
double nearbyint(double x);
float nearbyintf(float x);
long double nearbyintl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nearbyintd32(_Decimal32 x);
_Decimal64 nearbyintd64(_Decimal64 x);
_Decimal128 nearbyintd128(_Decimal128 x);
#endif
```

Description

2 The **nearbyint** functions round their argument to an integer value in floating-point format, using the current rounding direction and without raising the "inexact" floating-point exception.

© ISO 2024 — All rights reserved

Returns

3 The **nearbyint** functions return the rounded integer value.

7.12.9.4 The rint functions

Synopsis

1
```
#include <math.h>
double rint(double x);
float rintf(float x);
long double rintl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 rintd32(_Decimal32 x);
_Decimal64 rintd64(_Decimal64 x);
_Decimal128 rintd128(_Decimal128 x);
#endif
```

Description

2 The **rint** functions differ from the **nearbyint** functions (7.12.9.3) only in that the **rint** functions may raise the "inexact" floating-point exception if the result differs in value from the argument.

Returns

3 The **rint** functions return the rounded integer value.

7.12.9.5 The lrint and llrint functions

Synopsis

1
```
#include <math.h>
long int lrint(double x);
long int lrintf(float x);
long int lrintl(long double x);
long long int llrint(double x);
long long int llrintf(float x);
long long int llrintl(long double x);
#ifdef __STDC_IEC_60559_DFP__
long int lrintd32(_Decimal32 x);
long int lrintd64(_Decimal64 x);
long int lrintd128(_Decimal128 x);
long long int llrintd32(_Decimal32 x);
long long int llrintd64(_Decimal64 x);
long long int llrintd128(_Decimal128 x);
#endif
```

Description

2 The **lrint** and **llrint** functions round their argument to the nearest integer value, rounding according to the current rounding direction. If the rounded value is outside the range of the return type, the numeric result is unspecified and a domain error or range error may occur.

Returns

3 The **lrint** and **llrint** functions return the rounded integer value.

7.12.9.6 The round functions

Synopsis

1
```
#include <math.h>
double round(double x);
float roundf(float x);
long double roundl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 roundd32(_Decimal32 x);
```

© ISO 2024 — All rights reserved

```
_Decimal64 roundd64(_Decimal64 x);
_Decimal128 roundd128(_Decimal128 x);
#endif
```

Description

2 The **round** functions round their argument to the nearest integer value in floating-point format, rounding halfway cases away from zero, regardless of the current rounding direction.

Returns

3 The **round** functions return the rounded integer value.

7.12.9.7 The lround and llround functions

Synopsis

1
```
#include <math.h>
long int lround(double x);
long int lroundf(float x);
long int lroundl(long double x);
long long int llround(double x);
long long int llroundf(float x);
long long int llroundl(long double x);
#ifdef __STDC_IEC_60559_DFP__
long int lroundd32(_Decimal32 x);
long int lroundd64(_Decimal64 x);
long int lroundd128(_Decimal128 x);
long long int llroundd32(_Decimal32 x);
long long int llroundd64(_Decimal64 x);
long long int llroundd128(_Decimal128 x);
#endif
```

Description

2 The **lround** and **llround** functions round their argument to the nearest integer value, rounding halfway cases away from zero, regardless of the current rounding direction. If the rounded value is outside the range of the return type, the numeric result is unspecified and a domain error or range error may occur.

Returns

3 The **lround** and **llround** functions return the rounded integer value.

7.12.9.8 The roundeven functions

Synopsis

1
```
#include <math.h>
double roundeven(double x);
float roundevenf(float x);
long double roundevenl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 roundevend32(_Decimal32 x);
_Decimal64 roundevend64(_Decimal64 x);
_Decimal128 roundevend128(_Decimal128 x);
#endif
```

Description

2 The **roundeven** functions round their argument to the nearest integer value in floating-point format, rounding halfway cases to even (that is, to the nearest value that is an even integer), regardless of the current rounding direction.

 © ISO 2024 — All rights reserved

Returns

3 The **roundeven** functions return the rounded integer value.

7.12.9.9 The trunc functions

Synopsis

1
```
#include <math.h>
double trunc(double x);
float truncf(float x);
long double truncl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 truncd32(_Decimal32 x);
_Decimal64 truncd64(_Decimal64 x);
_Decimal128 truncd128(_Decimal128 x);
#endif
```

Description

2 The **trunc** functions round their argument to the integer value, in floating format, nearest to but no larger in magnitude than the argument.

Returns

3 The **trunc** functions return the truncated integer value.

7.12.9.10 The fromfp and ufromfp functions

Synopsis

1
```
#include <math.h>
double fromfp(double x, int rnd, unsigned int width);
float fromfpf(float x, int rnd, unsigned int width);
long double fromfpl(long double x, int rnd, unsigned int width);
double ufromfp(double x, int rnd, unsigned int width);
float ufromfpf(float x, int rnd, unsigned int width);
long double ufromfpl(long double x, int rnd, unsigned int width);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fromfpd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 fromfpd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 fromfpd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 ufromfpd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 ufromfpd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 ufromfpd128(_Decimal128 x, int rnd, unsigned int width);
#endif
```

Description

2 The **fromfp** and **ufromfp** functions round `x`, using the math rounding direction indicated by `rnd`, to a signed or unsigned integer, respectively. If `width` is nonzero and the resulting integer is within the range

— $[-2^{(\texttt{width}-1)}, 2^{(\texttt{width}-1)} - 1]$, for signed

— $[0, 2^{\texttt{width}} - 1]$, for unsigned

then the functions return the integer value (represented in floating type). Otherwise, if `width` is zero or `x` does not round to an integer within the range, the functions return a NaN (of the type of the `x` argument, if available), else the value of `x`, and a domain error occurs. If the value of the `rnd` argument is not equal to the value of a math rounding direction macro (7.12), the direction of rounding is unspecified. The **fromfp** and **ufromfp** functions do not raise the "inexact" floating-point exception.

© ISO 2024 — All rights reserved

Returns

3 The **fromfp** and **ufromfp** functions return the rounded integer value.

4 EXAMPLE 1 Upward rounding of **double** x to type **int**, without raising the "inexact" floating-point exception, is achieved by

```
(int)fromfp(x, FP_INT_UPWARD, INT_WIDTH)
```

5 EXAMPLE 2 Unsigned integer wrapping is not performed in

```
ufromfp(-3.0, FP_INT_UPWARD, UINT_WIDTH) /* domain error */
```

7.12.9.11 The fromfpx and ufromfpx functions

Synopsis

1

```
#include <math.h>
double fromfpx(double x, int rnd, unsigned int width);
float fromfpxf(float x, int rnd, unsigned int width);
long double fromfpxl(long double x, int rnd, unsigned int width);
double ufromfpx(double x, int rnd, unsigned int width);
float ufromfpxf(float x, int rnd, unsigned int width);
long double ufromfpxl(long double x, int rnd, unsigned int width);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fromfpxd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 fromfpxd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 fromfpxd128(_Decimal128 x, int rnd, unsigned int width);
_Decimal32 ufromfpxd32(_Decimal32 x, int rnd, unsigned int width);
_Decimal64 ufromfpxd64(_Decimal64 x, int rnd, unsigned int width);
_Decimal128 ufromfpxd128(_Decimal128 x, int rnd, unsigned int width);
#endif
```

Description

2 The **fromfpx** and **ufromfpx** functions differ from the **fromfp** and **ufromfp** functions, respectively, only in that the **fromfpx** and **ufromfpx** functions raise the "inexact" floating-point exception if a rounded result not exceeding the specified width differs in value from the argument x.

Returns

3 The **fromfpx** and **ufromfpx** functions return the rounded integer value.

4 NOTE Conversions to integer types that are not required to raise the inexact exception can be done simply by rounding to integral value in floating type and then converting to the target integer type. For example, the conversion of **long double** x to **uint64_t**, using upward rounding, is done by

```
(uint64_t)ceill(x)
```

7.12.10 Remainder functions

7.12.10.1 The fmod functions

Synopsis

1

```
#include <math.h>
double fmod(double x, double y);
float fmodf(float x, float y);
long double fmodl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmodd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmodd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmodd128(_Decimal128 x, _Decimal128 y);
#endif
```


© ISO 2024 — All rights reserved

Description

2 The **fmod** functions compute the floating-point remainder of x/y.

Returns

3 The **fmod** functions return the value $x - ny$, for some integer n such that, if y is nonzero, the result has the same sign as x and magnitude less than the magnitude of y. If y is zero, whether a domain error occurs or the **fmod** functions return zero is implementation-defined.

7.12.10.2 The remainder functions

Synopsis

1
```
#include <math.h>
double remainder(double x, double y);
float remainderf(float x, float y);
long double remainderl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 remainderd32(_Decimal32 x, _Decimal32 y);
_Decimal64 remainderd64(_Decimal64 x, _Decimal64 y);
_Decimal128 remainderd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **remainder** functions compute the remainder x REM y required by ISO/IEC 60559.[279)]

Returns

3 The **remainder** functions return x REM y. If y is zero, whether a domain error occurs or the functions return zero is implementation-defined.

7.12.10.3 The remquo functions

Synopsis

1
```
#include <math.h>
double remquo(double x, double y, int *quo);
float remquof(float x, float y, int *quo);
long double remquol(long double x, long double y, int *quo);
```

Description

2 The **remquo** functions compute the same remainder as the **remainder** functions. In the object pointed to by quo they store a value whose magnitude is congruent modulo 2^n to the magnitude of the integral quotient of x/y, where n is an implementation-defined integer greater than or equal to 3. If the value stored is not zero, its sign is the sign of x/y.

Returns

3 The **remquo** functions return x REM y. If y is zero, the value stored in the object pointed to by quo is unspecified and whether a domain error occurs or the functions return zero is implementation-defined.

4 NOTE There are no decimal floating-point versions of the **remquo** functions.

7.12.11 Manipulation functions

7.12.11.1 The copysign functions

Synopsis

1
```
#include <math.h>
double copysign(double x, double y);
```

[279)] "When $y \neq 0$, the remainder $r = x$ REM y is defined regardless of the rounding mode by the mathematical relation $r = x - ny$, where n is the integer nearest the exact value of $\frac{x}{y}$; whenever $|n - \frac{x}{y}| = \frac{1}{2}$, then n is even. If $r = 0$, its sign shall be that of x." This definition is applicable for all implementations.

© ISO 2024 — All rights reserved

```
float copysignf(float x, float y);
long double copysignl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 copysignd32(_Decimal32 x, _Decimal32 y);
_Decimal64 copysignd64(_Decimal64 x, _Decimal64 y);
_Decimal128 copysignd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **copysign** functions produce a value with the magnitude of x and the sign of y. If x or y is an unsigned value, the sign (if any) of the result is implementation-defined. On implementations that represent a signed zero but do not treat negative zero consistently in arithmetic operations, the **copysign** functions should regard the sign of zero as positive.

Returns

3 The **copysign** functions return a value with the magnitude of x and the sign of y.

7.12.11.2 The nan functions

Synopsis

1

```
#include <math.h>
double nan(const char *tagp);
float nanf(const char *tagp);
long double nanl(const char *tagp);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nand32(const char *tagp);
_Decimal64 nand64(const char *tagp);
_Decimal128 nand128(const char *tagp);
#endif
```

Description

2 The **nan**, **nanf**, and **nanl** functions convert the string pointed to by tagp according to the following rules. The call **nan**("*n-char-sequence*") is equivalent to **strtod**("NAN(*n-char-sequence*)", **nullptr**); the call **nan**("") is equivalent to **strtod**("NAN()", **nullptr**). If tagp does not point to an empty string or an n-char sequence, the call is equivalent to **strtod**("NAN", **nullptr**). Calls to **nanf** and **nanl** are equivalent to the corresponding calls to **strtof** and **strtold**.

Returns

3 The **nan** functions return a quiet NaN, if available, with content indicated through tagp. If the implementation does not support quiet NaNs, the functions return zero.

Forward references: the **strtod**, **strtof**, and **strtold** functions (7.24.1.5).

7.12.11.3 The nextafter functions

Synopsis

1

```
#include <math.h>
double nextafter(double x, double y);
float nextafterf(float x, float y);
long double nextafterl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nextafterd32(_Decimal32 x, _Decimal32 y);
_Decimal64 nextafterd64(_Decimal64 x, _Decimal64 y);
_Decimal128 nextafterd128(_Decimal128 x, _Decimal128 y);
#endif
```


© ISO 2024 — All rights reserved

Description

2 The **nextafter** functions determine the next representable value, in the return type of the function, after x in the direction of y, where x and y are first converted to the return type of the function.[280] The **nextafter** functions return y if x equals y.

A range error occurs if the magnitude of x is the largest finite value representable in the type and the result is infinite or not representable in the type. If x != y, a range error occurs for either subnormal or zero results.

Returns

3 The **nextafter** functions return the next representable value in the specified format after x in the direction of y.

7.12.11.4 The nexttoward functions

Synopsis

1
```
#include <math.h>
double nexttoward(double x, long double y);
float nexttowardf(float x, long double y);
long double nexttowardl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nexttowardd32(_Decimal32 x, _Decimal128 y);
_Decimal64 nexttowardd64(_Decimal64 x, _Decimal128 y);
_Decimal128 nexttowardd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **nexttoward** functions are equivalent to the **nextafter** functions except that the second parameter has type **long double** or **_Decimal128** and the functions return y converted to the return type of the function if x equals y.[281]

Returns

3 The **nexttoward** functions return the next representable value in the specified format after x in the direction of y.

7.12.11.5 The nextup functions

Synopsis

1
```
#include <math.h>
double nextup(double x);
float nextupf(float x);
long double nextupl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nextupd32(_Decimal32 x);
_Decimal64 nextupd64(_Decimal64 x);
_Decimal128 nextupd128(_Decimal128 x);
#endif
```

Description

2 The **nextup** functions determine the next representable value, in the return type of the function, greater than x. If x is the negative number of least magnitude in the type of x, **nextup**(x) is -0 if the type has signed zeros and is 0 otherwise. If x is zero, **nextup**(x) is the positive number of least magnitude in the type of x. If x is the positive number (finite or infinite) of maximum magnitude in the type, **nextup**(x) is x.

[280] The argument values are converted to the return type of the function, even by a macro implementation of the function.
[281] The result of the **nexttoward** functions is determined in the return type of the function, without loss of range or precision in a floating second argument.

© ISO 2024 — All rights reserved

Returns

3 The nextup functions return the next representable value in the specified type greater than x.

7.12.11.6 The nextdown functions

Synopsis

1
```
#include <math.h>
double nextdown(double x);
float nextdownf(float x);
long double nextdownl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 nextdownd32(_Decimal32 x);
_Decimal64 nextdownd64(_Decimal64 x);
_Decimal128 nextdownd128(_Decimal128 x);
#endif
```

Description

2 The nextdown functions determine the next representable value, in the return type of the function, less than x. If x is the positive number of least magnitude in the type of x, nextdown(x) is +0 if the type has signed zeros and is 0 otherwise. If x is zero, nextdown(x) is the negative number of least magnitude in the type of x. If x is the negative number (finite or infinite) of maximum magnitude in the type, nextdown(x) is x.

Returns

3 The nextdown functions return the next representable value in the specified type less than x.

7.12.11.7 The canonicalize functions

Synopsis

1
```
#include <math.h>
int canonicalize(double *cx, const double *x);
int canonicalizef(float *cx, const float *x);
int canonicalizel(long double *cx, const long double *x);
#ifdef __STDC_IEC_60559_DFP__
int canonicalized32(_Decimal32 *cx, const _Decimal32 *x);
int canonicalized64(_Decimal64 *cx, const _Decimal64 *x);
int canonicalized128(_Decimal128 *cx, const _Decimal128 *x);
#endif
```

Description

2 The canonicalize functions attempt to produce a canonical version of the floating-point representation in the object pointed to by the argument x, as if to a temporary object of the specified type, and store the canonical result in the object pointed to by the argument cx.[282] If the input *x is a signaling NaN, the canonicalize functions are intended to store a canonical quiet NaN. If a canonical result is not produced the object pointed to by cx is unchanged.

Returns

3 The canonicalize functions return zero if a canonical result is stored in the object pointed to by cx. Otherwise they return a nonzero value.

7.12.12 Maximum, minimum, and positive difference functions

7.12.12.1 The fdim functions

Synopsis

1
```
#include <math.h>
double fdim(double x, double y);
```

[282] Arguments x and cx may point to the same object.

 © ISO 2024 — All rights reserved

```
float fdimf(float x, float y);
long double fdiml(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fdimd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fdimd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fdimd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fdim** functions determine the *positive difference* between their arguments:

$$\begin{cases} x - y & \text{if } x > y \\ +0 & \text{if } x \leq y \end{cases}$$

A range error occurs for some finite arguments.

Returns

3 The **fdim** functions return the positive difference value.

7.12.12.2 The fmax functions

Synopsis

1
```
#include <math.h>
double fmax(double x, double y);
float fmaxf(float x, float y);
long double fmaxl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaxd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaxd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaxd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fmax** functions determine the maximum numeric value of their arguments.[283)]

Returns

3 The **fmax** functions return the maximum numeric value of their arguments.

7.12.12.3 The fmin functions

Synopsis

1
```
#include <math.h>
double fmin(double x, double y);
float fminf(float x, float y);
long double fminl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmind32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmind64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmind128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fmin** functions determine the minimum numeric value of their arguments.[284)]

[283)] Quiet NaN arguments are treated as missing data: if one argument is a quiet NaN and the other numeric, then the **fmax** functions choose the numeric value. See F.10.9.2.

[284)] The **fmin** functions are analogous to the **fmax** functions in their treatment of quiet NaNs.

© ISO 2024 — All rights reserved

Returns

3 The **fmin** functions return the minimum numeric value of their arguments.

4 NOTE The **fmax** and **fmin** functions are similar to the **fmaximum_num** and **fminimum_num** functions, though may differ in which signed zero is returned when the arguments are differently signed zeros and in their treatment of signaling NaNs (see F.10.9.5).

7.12.12.4 The fmaximum functions

Synopsis

1
```
#include <math.h>
double fmaximum(double x, double y);
float fmaximumf(float x, float y);
long double fmaximuml(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaximumd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximumd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximumd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fmaximum** functions determine the maximum value of their arguments. For these functions, $+0$ is considered greater than -0. These functions differ from the **fmaximum_num** functions only in their treatment of NaN arguments (see F.10.9.4, F.10.9.5).

Returns

3 The **fmaximum** functions return the maximum value of their arguments.

7.12.12.5 The fminimum functions

Synopsis

1
```
#include <math.h>
double fminimum(double x, double y);
float fminimumf(float x, float y);
long double fminimuml(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fminimumd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimumd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimumd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fminimum** functions determine the minimum value of their arguments. For these functions, -0 is considered less than $+0$. These functions differ from the **fminimum_num** functions only in their treatment of NaN arguments (see F.10.9.4, F.10.9.5).

Returns

3 The **fminimum** functions return the minimum value of their arguments.

7.12.12.6 The fmaximum_mag functions

Synopsis

1
```
#include <math.h>
double fmaximum_mag(double x, double y);
float fmaximum_magf(float x, float y);
long double fmaximum_magl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaximum_magd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_magd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_magd128(_Decimal128 x, _Decimal128 y);
```

 © ISO 2024 — All rights reserved

```
#endif
```

Description

2 The **fmaximum_mag** functions determine the value of the argument of maximum magnitude: x if $|x| > |y|$, y if $|y| > |x|$, and **fmaximum**(x, y) otherwise. These functions differ from the **fmaximum_mag_num** functions only in their treatment of NaN arguments (see F.10.9.4, F.10.9.5).

Returns

3 The **fmaximum_mag** functions return the value of the argument of maximum magnitude.

7.12.12.7 The fminimum_mag functions

Synopsis

1
```
#include <math.h>
double fminimum_mag(double x, double y);
float fminimum_magf(float x, float y);
long double fminimum_magl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fminimum_magd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_magd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_magd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fminimum_mag** functions determine the value of the argument of minimum magnitude: x if $|x| < |y|$, y if $|y| < |x|$, and **fminimum**(x, y) otherwise. These functions differ from the **fminimum_mag_num** functions only in their treatment of NaN arguments (see F.10.9.4, F.10.9.5).

Returns

3 The **fminimum_mag** functions return the value of the argument of minimum magnitude.

7.12.12.8 The fmaximum_num functions

Synopsis

1
```
#include <math.h>
double fmaximum_num(double x, double y);
float fmaximum_numf(float x, float y);
long double fmaximum_numl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaximum_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_numd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fmaximum_num** functions determine the maximum value of their numeric arguments. They determine the number if one argument is a number and the other is a NaN. These functions differ from the **fmaximum** functions only in their treatment of NaN arguments (see F.10.9.4, F.10.9.5).

Returns

3 The **fmaximum_num** functions return the maximum value of their numeric arguments.

7.12.12.9 The fminimum_num functions

Synopsis

1
```
#include <math.h>
double fminimum_num(double x, double y);
float fminimum_numf(float x, float y);
```

© ISO 2024 — All rights reserved

```
long double fminimum_numl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fminimum_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_numd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fminimum_num** functions determine the minimum value of their numeric arguments. They determine the number if one argument is a number and the other is a NaN. These functions differ from the **fminimum** functions only in their treatment of NaN arguments (see F.10.9.4, F.10.9.5).

Returns

3 The **fminimum_num** functions return the minimum value of their numeric arguments.

7.12.12.10 The fmaximum_mag_num functions

Synopsis

1
```
#include <math.h>
double fmaximum_mag_num(double x, double y);
float fmaximum_mag_numf(float x, float y);
long double fmaximum_mag_numl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmaximum_mag_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fmaximum_mag_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fmaximum_mag_numd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fmaximum_mag_num** functions determine the value of a numeric argument of maximum magnitude. They determine the number if one argument is a number and the other is a NaN. These functions differ from the **fmaximum_mag** functions only in their treatment of NaN arguments (see F.10.9.4, F.10.9.5).

Returns

3 The **fmaximum_mag_num** functions return the value of a numeric argument of maximum magnitude.

7.12.12.11 The fminimum_mag_num functions

Synopsis

1
```
#include <math.h>
double fminimum_mag_num(double x, double y);
float fminimum_mag_numf(float x, float y);
long double fminimum_mag_numl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fminimum_mag_numd32(_Decimal32 x, _Decimal32 y);
_Decimal64 fminimum_mag_numd64(_Decimal64 x, _Decimal64 y);
_Decimal128 fminimum_mag_numd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **fminimum_mag_num** functions determine the value of a numeric argument of minimum magnitude. They determine the number if one argument is a number and the other is a NaN. These functions differ from the **fminimum_mag** functions only in their treatment of NaN arguments (see F.10.9.4, F.10.9.5).

Returns

3 The **fminimum_mag_num** functions return the value of a numeric argument of minimum magnitude.

© ISO 2024 — All rights reserved

7.12.13 Fused multiply-add

7.12.13.1 The `fma` functions

Synopsis

1
```
#include <math.h>
double fma(double x, double y, double z);
float fmaf(float x, float y, float z);
long double fmal(long double x, long double y, long double z);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 fmad32(_Decimal32 x, _Decimal32 y, _Decimal32 z);
_Decimal64 fmad64(_Decimal64 x, _Decimal64 y, _Decimal64 z);
_Decimal128 fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
#endif
```

Description

2 The **fma** functions compute $(x \times y) + z$, rounded as one ternary operation: they compute the value (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error occurs for some infinite arguments.

Returns

3 The **fma** functions return $(x \times y) + z$, rounded as one ternary operation.

7.12.14 Functions that round result to narrower type

1 The functions in this subclause round their results to the return type, which is typically narrower[285)] than the parameter types.

7.12.14.1 Add and round to narrower type

Synopsis

1
```
#include <math.h>
float fadd(double x, double y);
float faddl(long double x, long double y);
double daddl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32addd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32addd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64addd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 These functions compute the sum of $x + y$, rounded to the return type of the function. They compute the sum (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error may occur for infinite arguments.

Returns

3 These functions return the sum of $x + y$, rounded to the return type of the function.

7.12.14.2 Subtract and round to narrower type

Synopsis

1
```
#include <math.h>
float fsub(double x, double y);
float fsubl(long double x, long double y);
double dsubl(long double x, long double y);
```

[285)] In some cases the destination type may not be narrower than the parameter types. For example, **double** potentially is not narrower than **long double**.

© ISO 2024 — All rights reserved

```
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32subd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32subd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64subd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 These functions compute the difference of x − y, rounded to the return type of the function. They compute the difference (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error may occur for infinite arguments.

Returns

3 These functions return the difference of x − y, rounded to the return type of the function.

7.12.14.3 Multiply and round to narrower type

Synopsis

1
```
#include <math.h>
float fmul(double x, double y);
float fmull(long double x, long double y);
double dmull(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32muld64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32muld128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64muld128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 These functions compute the product x × y, rounded to the return type of the function. They compute the product (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error occurs for one infinite argument and one zero argument.

Returns

3 These functions return the product of x × y, rounded to the return type of the function.

7.12.14.4 Divide and round to narrower type

Synopsis

1
```
#include <math.h>
float fdiv(double x, double y);
float fdivl(long double x, long double y);
double ddivl(long double x, long double y);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32divd64(_Decimal64 x, _Decimal64 y);
_Decimal32 d32divd128(_Decimal128 x, _Decimal128 y);
_Decimal64 d64divd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 These functions compute the quotient x ÷ y, rounded to the return type of the function. They compute the quotient (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error occurs for either both arguments infinite or both arguments zero. A pole error occurs for a finite x and a zero y.

Returns

3 These functions return the quotient x ÷ y, rounded to the return type of the function.

 © ISO 2024 — All rights reserved

7.12.14.5 Fused multiply-add and round to narrower type

Synopsis

1
```
#include <math.h>
float ffma(double x, double y, double z);
float ffmal(long double x, long double y, long double z);
double dfmal(long double x, long double y, long double z);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32fmad64(_Decimal64 x, _Decimal64 y, _Decimal64 z);
_Decimal32 d32fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
_Decimal64 d64fmad128(_Decimal128 x, _Decimal128 y, _Decimal128 z);
#endif
```

Description

2 These functions compute $(x \times y) + z$ (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite arguments. A domain error may occur for an infinite argument.

Returns

3 These functions return $(x \times y) + z$, rounded to the return type of the function.

7.12.14.6 Square root rounded to narrower type

Synopsis

1
```
#include <math.h>
float fsqrt(double x);
float fsqrtl(long double x);
double dsqrtl(long double x);
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32sqrtd64(_Decimal64 x);
_Decimal32 d32sqrtd128(_Decimal128 x);
_Decimal64 d64sqrtd128(_Decimal128 x);
#endif
```

Description

2 These functions compute the square root of `x`, rounded to the return type of the function. They compute the square root (as if) to infinite precision and round once to the return type, according to the current rounding mode. A range error occurs for some finite positive arguments. A domain error occurs if the argument is less than zero.

Returns

3 These functions return the nonnegative square root of `x`, rounded to the return type of the function.

7.12.15 Quantum and quantum exponent functions

7.12.15.1 The `quantized`*N* functions

Synopsis

1
```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 quantized32(_Decimal32 x, _Decimal32 y);
_Decimal64 quantized64(_Decimal64 x, _Decimal64 y);
_Decimal128 quantized128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The `quantized`*N* functions compute, if possible, a value with the numerical value of `x` and the quantum exponent of `y`. If the quantum exponent is being increased, the value shall be correctly rounded; if the result does not have the same value as `x`, the "inexact" floating-point exception shall

© ISO 2024 — All rights reserved

be raised. If the quantum exponent is being decreased and the significand of the result has more digits than the type would allow, the result is NaN, the "invalid" floating-point exception is raised, and a domain error occurs. If one or both operands are NaN the result is NaN. Otherwise if only one operand is infinite, the result is NaN, the "invalid" floating-point exception is raised, and a domain error occurs. If both operands are infinite, the result is **DEC_INFINITY** with the sign of x, converted to the return type of the function. The **quantized***N* functions do not raise the "overflow" and "underflow" floating-point exceptions.

Returns

3 The **quantized***N* functions return a value with the numerical value of x (except for any rounding) and the quantum exponent of y.

7.12.15.2 The samequantumd*N* functions

Synopsis

1
```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
bool samequantumd32(_Decimal32 x, _Decimal32 y);
bool samequantumd64(_Decimal64 x, _Decimal64 y);
bool samequantumd128(_Decimal128 x, _Decimal128 y);
#endif
```

Description

2 The **samequantumd***N* functions determine if the quantum exponents of x and y are the same. If both x and y are NaN, or both infinite, they have the same quantum exponents; if exactly one operand is infinite or exactly one operand is NaN, they do not have the same quantum exponents. The **samequantumd***N* functions raise no floating-point exception.

Returns

3 The **samequantumd***N* functions return nonzero (**true**) when x and y have the same quantum exponents, zero (**false**) otherwise.

7.12.15.3 The quantumd*N* functions

Synopsis

1
```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 quantumd32(_Decimal32 x);
_Decimal64 quantumd64(_Decimal64 x);
_Decimal128 quantumd128(_Decimal128 x);
#endif
```

Description

2 The **quantumd***N* functions compute the quantum (5.2.5.3.4) of a finite argument. If x is infinite, the result is $+\infty$.

Returns

3 The **quantumd***N* functions return the quantum of x.

7.12.15.4 The llquantexpd*N* functions

Synopsis

1
```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
long long int llquantexpd32(_Decimal32 x);
long long int llquantexpd64(_Decimal64 x);
long long int llquantexpd128(_Decimal128 x);
#endif
```


© ISO 2024 — All rights reserved

Description

2 The **llquantexpd***N* functions compute the quantum exponent (5.2.5.3.4) of a finite argument. If x is infinite or NaN, they compute **LLONG_MIN**, the "invalid" floating-point exception is raised, and a domain error occurs.

Returns

3 The **llquantexpd***N* functions return the quantum exponent of x.

7.12.16 Decimal re-encoding functions

1 ISO/IEC 60559 specifies two different schemes to encode significands in the object representation of a decimal floating-point object: one based on decimal encoding (which packs three decimal digits into 10 bits), the other based on binary encoding (as a binary integer). An implementation may use either of these encoding schemes for its decimal floating types. The re-encoding functions in this subclause provide conversions between external decimal data with a given encoding scheme and the implementation's corresponding decimal floating type.

7.12.16.1 The encodedecd*N* functions

Synopsis

1
```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
void encodedecd32(unsigned char encptr[restrict static 4],
      const _Decimal32 * restrict xptr);
void encodedecd64(unsigned char encptr[restrict static 8],
      const _Decimal64 * restrict xptr);
void encodedecd128(unsigned char encptr[restrict static 16],
      const _Decimal128 * restrict xptr);
#endif
```

Description

2 The **encodedecd***N* functions convert *xptr into an ISO/IEC 60559 decimal*N* encoding in the encoding scheme based on decimal encoding of the significand and store the resulting encoding as an $N/8$ element array, with 8 bits per array element, in the object pointed to by encptr. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2). These functions preserve the value of *xptr and raise no floating-point exceptions. If *xptr is non-canonical, these functions may or may not produce a canonical encoding.

Returns

3 The **encodedecd***N* functions return no value.

7.12.16.2 The decodedecd*N* functions

Synopsis

1
```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
void decodedecd32(_Decimal32 * restrict xptr,
      const unsigned char encptr[restrict static 4]);
void decodedecd64(_Decimal64 * restrict xptr,
      const unsigned char encptr[restrict static 8]);
void decodedecd128(_Decimal128 * restrict xptr,
      const unsigned char encptr[restrict static 16]);
#endif
```

Description

2 The **decodedecd***N* functions interpret the $N/8$ element array pointed to by encptr as an ISO/IEC 60559 decimal*N* encoding, with 8 bits per array element, in the encoding scheme based on decimal encoding of the significand. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2). These functions convert the given encoding into a value

© ISO 2024 — All rights reserved

of the decimal floating type, and store the result in the object pointed to by `xptr`. These functions preserve the encoded value and raise no floating-point exceptions. If the encoding is non-canonical, these functions may or may not produce a canonical representation.

Returns

3 The **decodededecd***N* functions return no value.

7.12.16.3 The **encodebind***N* functions

Synopsis

1
```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
void encodebind32(unsigned char encptr[restrict static 4],
      const _Decimal32 * restrict xptr);
void encodebind64(unsigned char encptr[restrict static 8],
      const _Decimal64 * restrict xptr);
void encodebind128(unsigned char encptr[restrict static 16],
      const _Decimal128 * restrict xptr);
#endif
```

Description

2 The **encodebind***N* functions convert ∗`xptr` into an ISO/IEC 60559 decimal*N* encoding in the encoding scheme based on binary encoding of the significand and store the resulting encoding as an $N/8$ element array, with 8 bits per array element, in the object pointed to by `encptr`. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2). These functions preserve the value of ∗`xptr` and raise no floating-point exceptions. If ∗`xptr` is non-canonical, these functions may or may not produce a canonical encoding.

Returns

3 The **encodebind***N* functions return no value.

7.12.16.4 The **decodebind***N* functions

Synopsis

1
```
#include <math.h>
#ifdef __STDC_IEC_60559_DFP__
void decodebind32(_Decimal32 * restrict xptr,
      const unsigned char encptr[restrict static 4]);
void decodebind64(_Decimal64 * restrict xptr,
      const unsigned char encptr[restrict static 8]);
void decodebind128(_Decimal128 * restrict xptr,
      const unsigned char encptr[restrict static 16]);
#endif
```

Description

2 The **decodebind***N* functions interpret the $N/8$ element array pointed to by `encptr` as an ISO/IEC 60559 decimal*N* encoding, with 8 bits per array element, in the encoding scheme based on binary encoding of the significand. The order of bytes in the array follows the endianness specified with **__STDC_ENDIAN_NATIVE__** (7.18.2). These functions convert the given encoding into a value of decimal floating type, and store the result in the object pointed to by `xptr`. These functions preserve the encoded value and raise no floating-point exceptions. If the encoding is non-canonical, these functions may or may not produce a canonical representation.

Returns

3 The **decodebind***N* functions return no value.

 © ISO 2024 — All rights reserved

7.12.17 Comparison macros

1 The relational and equality operators support the usual mathematical relationships between numeric values. For any ordered pair of numeric values exactly one of the relationships — *less*, *greater*, and *equal* — is true. Relational operators may raise the "invalid" floating-point exception when argument values are NaNs. For a NaN and a numeric value, or for two NaNs, just the *unordered* relationship is true.[286] 7.12.17.1 through 7.12.17.6 provide macros that are quiet versions of the relational operators: the macros do not raise the "invalid" floating-point exception as an effect of quiet NaN arguments. The comparison macros facilitate writing efficient code that accounts for quiet NaNs without suffering the "invalid" floating-point exception. In the synopses in this subclause, *real-floating* indicates that the argument shall be an expression of real floating type[287] (both arguments are not required to have the same type).[288] If either argument has decimal floating type, the other argument shall have decimal floating type as well.

7.12.17.1 The `isgreater` macro

Synopsis

1
```
#include <math.h>
int isgreater(real-floating x, real-floating y);
```

Description

2 The **`isgreater`** macro determines whether its first argument is greater than its second argument. The value of **`isgreater`**`(x,y)` is always equal to `(x) > (y)`.

However, unlike `(x) > (y)`, **`isgreater`**`(x,y)` does not raise the "invalid" floating-point exception when `x` and `y` are unordered and neither is a signaling NaN.

Returns

3 The **`isgreater`** macro returns the value of `(x) > (y)`.

7.12.17.2 The `isgreaterequal` macro

Synopsis

1
```
#include <math.h>
int isgreaterequal(real-floating x, real-floating y);
```

Description

2 The **`isgreaterequal`** macro determines whether its first argument is greater than or equal to its second argument. The value of **`isgreaterequal`**`(x,y)` is always equal to `(x) >= (y)`.

However, unlike `(x) >= (y)`, **`isgreaterequal`**`(x,y)` does not raise the "invalid" floating-point exception when `x` and `y` are unordered and neither is a signaling NaN.

Returns

3 The **`isgreaterequal`** macro returns the value of `(x) >= (y)`.

7.12.17.3 The `isless` macro

Synopsis

1
```
#include <math.h>
int isless(real-floating x, real-floating y);
```

[286] ISO/IEC 60559 requires that the built-in relational operators raise the "invalid" floating-point exception if the operands compare unordered, as an error indicator for programs written without consideration of NaNs; the result in these cases is false.
[287] If any argument is of integer type, or any other type that is not a real floating type, the behavior is undefined.
[288] Whether an argument represented in a format wider than its semantic type is converted to the semantic type is unspecified.

© ISO 2024 — All rights reserved

Description

2 The **isless** macro determines whether its first argument is less than its second argument. The value of **isless**(x,y) is always equal to (x) < (y).

However, unlike (x) < (y), **isless**(x,y) does not raise the "invalid" floating-point exception when x and y are unordered and neither is a signaling NaN.

Returns

3 The **isless** macro returns the value of (x) < (y).

7.12.17.4 The **islessequal** macro

Synopsis

1
```
#include <math.h>
int islessequal(real-floating x, real-floating y);
```

Description

2 The **islessequal** macro determines whether its first argument is less than or equal to its second argument. The value of **islessequal**(x,y) is always equal to (x) <= (y).

However, unlike (x) <= (y), **islessequal**(x,y) does not raise the "invalid" floating-point exception when x and y are unordered and neither is a signaling NaN.

Returns

3 The **islessequal** macro returns the value of (x) <= (y).

7.12.17.5 The **islessgreater** macro

Synopsis

1
```
#include <math.h>
int islessgreater(real-floating x, real-floating y);
```

Description

2 The **islessgreater** macro determines whether its first argument is less than or greater than its second argument. The **islessgreater**(x,y) macro is similar to (x) < (y) || (x) > (y).

However, **islessgreater**(x,y) does not raise the "invalid" floating-point exception when x and y are unordered and neither is a signaling NaN (nor does it evaluate x and y twice).

Returns

3 The **islessgreater** macro returns the value of (x) < (y) || (x) > (y).

7.12.17.6 The **isunordered** macro

Synopsis

1
```
#include <math.h>
int isunordered(real-floating x, real-floating y);
```

Description

2 The **isunordered** macro determines whether its arguments are unordered. It does not raise the "invalid" floating-point exception when x and y are unordered and neither is a signaling NaN.

Returns

3 The **isunordered** macro returns 1 if its arguments are unordered and 0 otherwise.

7.12.17.7 The **iseqsig** macro

Synopsis

1
```
#include <math.h>
```

 © ISO 2024 — All rights reserved

```
int iseqsig(real-floating x, real-floating y);
```

Description

2 The **iseqsig** macro determines whether its arguments are equal. If an argument is a NaN, a domain error occurs for the macro, as if a domain error occurred for a function (7.12.1).

Returns

3 The **iseqsig** macro returns 1 if its arguments are equal and 0 otherwise.

© ISO 2024 — All rights reserved

7.13 Non-local jumps <setjmp.h>

1 The header <setjmp.h> defines the macros **setjmp** and **__STDC_VERSION_SETJMP_H__**, and declares one function and one type, for bypassing the normal function call and return discipline.[289)]

2 The macro

```
__STDC_VERSION_SETJMP_H__
```

is an integer constant expression with a value equivalent to 202311L.

3 The type declared is

```
jmp_buf
```

which is an array type suitable for holding the information needed to restore a calling environment. The environment of an invocation of the **setjmp** macro consists of information sufficient for a call to the **longjmp** function to return execution to the correct block and invocation of that block, were it called recursively. It does not include the state of the floating-point environment, of open files, or of any other component of the abstract machine.

4 It is unspecified whether **setjmp** is a macro or an identifier declared with external linkage. If a macro definition is suppressed to access an actual function, or a program defines an external identifier with the name **setjmp**, the behavior is undefined.

7.13.1 Save calling environment

7.13.1.1 The setjmp macro

Synopsis

1
```
#include <setjmp.h>
int setjmp(jmp_buf env);
```

Description

2 The **setjmp** macro saves its calling environment in its **jmp_buf** argument for later use by the **longjmp** function.

Returns

3 If the return is from a direct invocation, the **setjmp** macro returns the value zero. If the return is from a call to the **longjmp** function, the **setjmp** macro returns a nonzero value.

Environmental limits

4 An invocation of the **setjmp** macro shall appear only in one of the following contexts:

— the entire controlling expression of a selection or iteration statement;

— one operand of a relational or equality operator with the other operand an integer constant expression, with the resulting expression being the entire controlling expression of a selection or iteration statement;

— the operand of a unary ! operator with the resulting expression being the entire controlling expression of a selection or iteration statement; or

— the entire expression of an expression statement (possibly cast to **void**).

5 If the invocation appears in any other context, the behavior is undefined.

7.13.2 Restore calling environment

7.13.2.1 The longjmp function

[289)] These functions are useful for dealing with unusual conditions encountered in a low-level function of a program.

 © ISO 2024 — All rights reserved

Synopsis

1
```
#include <setjmp.h>
[[noreturn]] void longjmp(jmp_buf env, int val);
```

Description

2 The **longjmp** function restores the environment saved by the most recent invocation of the **setjmp** macro in the same invocation of the program with the corresponding **jmp_buf** argument. If there has been no such invocation, or if the invocation was from another thread of execution, or if the function containing the invocation of the **setjmp** macro has terminated execution[290] in the interim, or if the invocation of the **setjmp** macro was within the scope of an identifier with variably modified type and execution has left that scope in the interim, the behavior is undefined.

3 All accessible objects have values, and all other components of the abstract machine[291] have state, as of the time the **longjmp** function was called, except that the representation of objects of automatic storage duration that are local to the function containing the invocation of the corresponding **setjmp** macro that do not have volatile-qualified type and have been changed between the **setjmp** invocation and **longjmp** call is indeterminate.

Returns

4 After **longjmp** is completed, thread execution continues as if the corresponding invocation of the **setjmp** macro had just returned the value specified by `val`. The **longjmp** function cannot cause the **setjmp** macro to return the value 0; if `val` is 0, the **setjmp** macro returns the value 1.

5 EXAMPLE The **longjmp** function that returns control back to the point of the **setjmp** invocation can cause memory associated with a variable length array object to be squandered.

```
#include <setjmp.h>
jmp_buf buf;
void g(int n);
void h(int n);
int n = 6;

void f(void)
{
      int x[n];          // valid:  f is not terminated
      setjmp(buf);
      g(n);
}

void g(int n)
{
      int a[n];          // a may remain allocated
      h(n);
}

void h(int n)
{
      int b[n];          // b may remain allocated
      longjmp(buf, 2);   // can cause memory loss
}
```

[290] For example, by executing a **return** statement or because another **longjmp** call has caused a transfer to a **setjmp** invocation in a function earlier in the set of nested calls.
[291] This includes, but is not limited to, the floating-point environment and the state of open files.

© ISO 2024 — All rights reserved

7.14 Signal handling <signal.h>

1 The header <signal.h> declares a type and two functions and defines several macros, for handling various *signals* (conditions that may be reported during program execution).

2 The type defined is

```
sig_atomic_t
```

which is the (possibly volatile-qualified) integer type of an object that can be accessed as an atomic entity, even in the presence of asynchronous interrupts.

3 The macros defined are

```
SIG_DFL
SIG_ERR
SIG_IGN
```

which expand to constant expressions with distinct values that have type compatible with the second argument to, and the return value of, the **signal** function, and whose values compare unequal to the address of any declarable function; and the following, which expand to positive integer constant expressions with type **int** and distinct values that are the signal numbers, each corresponding to the specified condition:

SIGABRT abnormal termination, such as is initiated by the **abort** function

SIGFPE an erroneous arithmetic operation, such as zero divide or an operation resulting in overflow

SIGILL detection of an invalid function image, such as an invalid instruction

SIGINT receipt of an interactive attention signal

SIGSEGV an invalid access to storage

SIGTERM a termination request sent to the program

4 An implementation is not required to generate any of these signals, except as a result of explicit calls to the **raise** function. Additional signals and pointers to undeclarable functions, with macro definitions beginning, respectively, with the letters **SIG** and an uppercase letter or with **SIG_** and an uppercase letter,[292] may also be specified by the implementation. The complete set of signals, their semantics, and their default handling is implementation-defined; all signal numbers shall be positive.

7.14.1 Specify signal handling

7.14.1.1 The signal function

Synopsis

1
```
#include <signal.h>
void (*signal(int sig, void (*func)(int)))(int);
```

Description

2 The **signal** function chooses one of three ways in which receipt of the signal number sig is to be subsequently handled. If the value of func is **SIG_DFL**, default handling for that signal will occur. If the value of func is **SIG_IGN**, the signal will be ignored. Otherwise, func shall point to a function to be called when that signal occurs. An invocation of such a function because of a signal, or (recursively) of any further functions called by that invocation (other than functions in the standard library),[293] is called a *signal handler*.

[292] See "future library directions" (7.33.9). The names of the signal numbers reflect the following terms (respectively): abort, floating-point exception, illegal instruction, interrupt, segmentation violation, and termination.

[293] This includes functions called indirectly via standard library functions (e.g. a **SIGABRT** handler called via the **abort** function).

 © ISO 2024 — All rights reserved

3 When a signal occurs and `func` points to a function, it is implementation-defined whether the equivalent of `signal(sig, SIG_DFL);` is executed or the implementation prevents some implementation-defined set of signals (at least including `sig`) from occurring until the current signal handling has completed; in the case of **SIGILL**, the implementation may alternatively define that no action is taken. Then the equivalent of `(*func)(sig);` is executed. If and when the function returns, if the value of `sig` is **SIGFPE**, **SIGILL**, **SIGSEGV**, or any other implementation-defined value corresponding to a computational exception, the behavior is undefined; otherwise the program will resume execution at the point it was interrupted.

4 If the signal occurs as the result of calling the **abort** or **raise** function, the signal handler shall not call the **raise** function.

5 If the signal occurs other than as the result of calling the **abort** or **raise** function, the behavior is undefined if the signal handler refers to any object with static or thread storage duration that is not a lock-free atomic object and that is not declared with the **constexpr** storage-class specifier other than by assigning a value to an object declared as **volatile** **sig_atomic_t**, or the signal handler calls any function in the standard library other than

— the **abort** function,

— the **_Exit** function,

— the **quick_exit** function,

— the functions in <stdatomic.h> (except where explicitly stated otherwise) when the atomic arguments are lock-free,

— the **atomic_is_lock_free** function with any atomic argument, or

— the **signal** function with the first argument equal to the signal number corresponding to the signal that caused the invocation of the handler. Furthermore, if such a call to the **signal** function results in a **SIG_ERR** return, the object designated by **errno** has an indeterminate representation.[294)]

6 At program startup, the equivalent of

```
signal(sig, SIG_IGN);
```

may be executed for some signals selected in an implementation-defined manner; the equivalent of

```
signal(sig, SIG_DFL);
```

is executed for all other signals defined by the implementation.

7 Use of this function in a multi-threaded program results in undefined behavior. The implementation shall behave as if no library function calls the **signal** function.

Returns

8 If the request can be honored, the **signal** function returns the value of `func` for the most recent successful call to **signal** for the specified signal `sig`. Otherwise, a value of **SIG_ERR** is returned and a positive value is stored in **errno**.

Forward references: the **abort** function (7.24.4.1), the **exit** function (7.24.4.4), the **_Exit** function (7.24.4.5), the **quick_exit** function (7.24.4.7).

7.14.2 Send signal

7.14.2.1 The raise function

[294)] If any signal is generated by an asynchronous signal handler, the behavior is undefined.

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <signal.h>
int raise(int sig);
```

Description

2 The **raise** function carries out the actions described in 7.14.1.1 for the signal `sig`. If a signal handler is called, the **raise** function shall not return until after the signal handler does.

Returns

3 The **raise** function returns zero if successful, nonzero if unsuccessful.

© ISO 2024 — All rights reserved

7.15 Alignment <stdalign.h>

1 The header <stdalign.h> provides no content.

© ISO 2024 — All rights reserved

7.16 Variable arguments <stdarg.h>

1 The header <stdarg.h> declares a type and defines five macros, for advancing through a list of arguments whose number and types are not known to the called function when it is translated.

2 The macro

```
__STDC_VERSION_STDARG_H__
```

is an integer constant expression with a value equivalent to 202311L.

3 A function may be called with a variable number of arguments of varying types if its parameter type list ends with an ellipsis.

4 The type declared is

```
va_list
```

which is a complete object type suitable for holding information needed by the macros **va_start**, **va_arg**, **va_end**, and **va_copy**. If access to the varying arguments is desired, the called function shall declare an object (generally referred to as ap in this subclause) having type **va_list**. The object ap may be passed as an argument to another function; if that function invokes the **va_arg** macro with parameter ap, the representation of ap in the calling function is indeterminate and shall be passed to the **va_end** macro prior to any further reference to ap.[295)]

7.16.1 Variable argument list access macros

1 The **va_start** and **va_arg** macros described in this subclause shall be implemented as macros, not functions. It is unspecified whether **va_copy** and **va_end** are macros or identifiers declared with external linkage. If a macro definition is suppressed to access an actual function, or a program defines an external identifier with the same name, the behavior is undefined. Each invocation of the **va_start** and **va_copy** macros shall be matched by a corresponding invocation of the **va_end** macro in the same function.

7.16.1.1 The va_arg macro

Synopsis

1
```
#include <stdarg.h>
type va_arg(va_list ap, type);
```

Description

2 The **va_arg** macro expands to an expression that has the specified type and the value of the next argument in the call. The parameter ap shall have been initialized by the **va_start** or **va_copy** macro (without an intervening invocation of the **va_end** macro for the same ap). Each invocation of the **va_arg** macro modifies ap so that the values of successive arguments are returned in turn. The behavior is undefined if there is no actual next argument. The parameter *type* shall be an object type name. If *type* is not compatible with the type of the actual next argument (as promoted according to the default argument promotions), the behavior is undefined, except for the following cases:

— both types are pointers to qualified or unqualified versions of compatible types;

— one type is compatible with a signed integer type, the other type is compatible with the corresponding unsigned integer type, and the value is representable in both types;

— one type is pointer to qualified or unqualified **void** and the other is a pointer to a qualified or unqualified character type;

— or, the type of the next argument is **nullptr_t** and *type* is a pointer type that has the same representation and alignment requirements as a pointer to a character type.[296)]

[295)] A pointer to a **va_list** can be created and passed to another function, in which case the original function can make further use of the original list after the other function returns.

[296)] Such types are in particular pointers to qualified or unqualified versions of **void**.

 © ISO 2024 — All rights reserved

Returns

3 The first invocation of the **va_arg** macro after that of the **va_start** macro returns the value of the first argument without an explicit parameter, which matches the position of the `...` in the parameter list. Successive invocations return the values of the remaining arguments in succession.

7.16.1.2 The va_copy macro

Synopsis

1
```
#include <stdarg.h>
void va_copy(va_list dest, va_list src);
```

Description

2 The **va_copy** macro initializes `dest` as a copy of `src`, as if the **va_start** macro had been applied to `dest` followed by the same sequence of uses of the **va_arg** macro as had previously been used to reach the present state of `src`. Neither the **va_copy** nor **va_start** macro shall be invoked to reinitialize `dest` without an intervening invocation of the **va_end** macro for the same `dest`.

Returns

3 The **va_copy** macro returns no value.

7.16.1.3 The va_end macro

Synopsis

1
```
#include <stdarg.h>
void va_end(va_list ap);
```

Description

2 The **va_end** macro facilitates a normal return from the function whose variable argument list was referred to by the expansion of the **va_start** macro, or the function containing the expansion of the **va_copy** macro, that initialized the **va_list** `ap`. The **va_end** macro may modify `ap` so that it is no longer usable (without being reinitialized by the **va_start** or **va_copy** macro). If there is no corresponding invocation of the **va_start** or **va_copy** macro, or if the **va_end** macro is not invoked before the return, the behavior is undefined.

Returns

3 The **va_end** macro returns no value.

7.16.1.4 The va_start macro

Synopsis

1
```
#include <stdarg.h>
void va_start(va_list ap, ...);
```

Description

2 The **va_start** macro shall be invoked before any access to the unnamed arguments.

3 The **va_start** macro initializes `ap` for subsequent use by the **va_arg** and **va_end** macros. Neither the **va_start** nor **va_copy** macro shall be invoked to reinitialize `ap` without an intervening invocation of the **va_end** macro for the same `ap`.

4 Only the first argument passed to **va_start** is evaluated. If any additional arguments expand to include unbalanced parentheses, or a preprocessing token that does not convert to a token, the behavior is undefined.

5 NOTE The macro allows additional arguments to be passed for **va_start** for compatibility with older versions of the library only.

Returns

6 The **va_start** macro returns no value.

© ISO 2024 — All rights reserved

Recommended practice

7 Additional arguments beyond the first given to the **va_start** macro may be expanded and used in unspecified contexts where they are unevaluated. For example, an implementation diagnoses potentially erroneous input for an invocation of **va_start** such as:

```
#include <stdarg.h>

void miaou (...) {
      va_list vl;
      va_start(vl, 1, 3.0, "12", xd); // diagnostic encouraged
      /* ... */
      va_end(vl);
}
```

Simultaneously, **va_start** usage consistent with older revisions of this document should not produce a diagnostic:

```
#include <stdarg.h>

void neigh (int last_arg, ...) {
      va_list vl;
      va_start(vl, last_arg); // no diagnostic
      /* ... */
      va_end(vl);
}
```

8 EXAMPLE 1 The function `f1` gathers into an array a list of arguments that are pointers to strings (but not more than `MAXARGS` arguments), then passes the array as a single argument to function `f2`. The number of pointers is specified by the first argument to `f1`.

```
#include <stdarg.h>
#define MAXARGS   31

void f1(int n_ptrs, ...)
{
      va_list ap;
      char *array[MAXARGS];
      int ptr_no = 0;

      if (n_ptrs > MAXARGS)
            n_ptrs = MAXARGS;
      va_start(ap);
      while (ptr_no  <  n_ptrs)
            array[ptr_no++] = va_arg(ap, char *);
      va_end(ap);
      f2(n_ptrs, array);
}
```

Each call to `f1` is required to have visible the definition of the function or a declaration such as

```
void f1(int, ...);
```

9 EXAMPLE 2 The function `f3` is similar, but saves the status of the variable argument list after the indicated number of arguments; after `f2` has been called once with the whole list, the trailing part of the list is gathered again and passed to function `f4`.

```
#include <stdarg.h>
#define MAXARGS 31

void f3(int n_ptrs, int f4_after, ...)
```


© ISO 2024 — All rights reserved

```
{
      va_list ap, ap_save;
      char *array[MAXARGS];
      int ptr_no = 0;
      if (n_ptrs > MAXARGS)
            n_ptrs = MAXARGS;
      va_start(ap);
      while (ptr_no < n_ptrs) {
            array[ptr_no++] = va_arg(ap, char *);
            if (ptr_no == f4_after)
                  va_copy(ap_save, ap);
      }
      va_end(ap);
      f2(n_ptrs, array);

      // Now process the saved copy.

      n_ptrs -= f4_after;
      ptr_no = 0;
      while (ptr_no < n_ptrs)
            array[ptr_no++] = va_arg(ap_save, char *);
      va_end(ap_save);
      f4(n_ptrs, array);
}
```

10 EXAMPLE 3 The function `f5` is similar to `f1`, but instead of passing an explicit number of strings as the first argument, the argument list is terminated with a null pointer.

```
#include <stdarg.h>

#define MAXARGS 31

void f5(...)
{
      va_list ap;
      char *array[MAXARGS];
      int ptr_no = 0;
      va_start(ap);
      while (ptr_no < MAXARGS)
      {
            char *ptr = va_arg(ap, char *);
            if (!ptr)
                  break;
            array[ptr_no++] = ptr;
      }
      va_end(ap);
      f6(ptr_no, array);
}
```

Each call to `f5` is required to have visible the definition of the function or a declaration such as

```
void f5(...);
```

and implicitly requires the last argument to be a null pointer.

© ISO 2024 — All rights reserved

7.17 Atomics <stdatomic.h>

7.17.1 Introduction

1 The header <stdatomic.h> defines several macros and declares several types and functions for performing atomic operations on data shared between threads.[297)]

2 Implementations that define the macro __STDC_NO_ATOMICS__ may not provide this header nor support any of its facilities.

3 The macro

```
__STDC_VERSION_STDATOMIC_H__
```

is an integer constant expression with a value equivalent to 202311L.

4 The macros defined are the *atomic lock-free macros*

```
ATOMIC_BOOL_LOCK_FREE
ATOMIC_CHAR_LOCK_FREE
ATOMIC_CHAR8_T_LOCK_FREE
ATOMIC_CHAR16_T_LOCK_FREE
ATOMIC_CHAR32_T_LOCK_FREE
ATOMIC_WCHAR_T_LOCK_FREE
ATOMIC_SHORT_LOCK_FREE
ATOMIC_INT_LOCK_FREE
ATOMIC_LONG_LOCK_FREE
ATOMIC_LLONG_LOCK_FREE
ATOMIC_POINTER_LOCK_FREE
```

which expand to constant expressions suitable for use in conditional expression inclusion preprocessing directives and which indicate the lock-free property of the corresponding atomic types (both signed and unsigned); and

```
ATOMIC_FLAG_INIT
```

which expands to an initializer for an object of type **atomic_flag**.

5 The types include

```
memory_order
```

which is an enumerated type whose enumerators identify memory ordering constraints;

```
atomic_flag
```

which is a structure type representing a lock-free, primitive atomic flag; and several atomic analogs of integer types.

6 In the following synopses:

— An *A* refers to an atomic type.

— A *C* refers to its corresponding non-atomic type.

— An *M* refers to the type of the other argument for arithmetic operations. For atomic integer types, *M* is *C*. For atomic pointer types, *M* is **ptrdiff_t**.

— The functions not ending in **_explicit** have the same semantics as the corresponding **_explicit** function with **memory_order_seq_cst** for the **memory_order** argument.

[297)] See "future library directions" (7.33.10).

 © ISO 2024 — All rights reserved

7 It is unspecified whether any generic function declared in `<stdatomic.h>` is a macro or an identifier declared with external linkage. If a macro definition is suppressed to access an actual function, or a program defines an external identifier with the name of a generic function, the behavior is undefined.

8 NOTE Many operations are volatile-qualified. The "volatile as device register" semantics have not changed in the standard. This qualification means that volatility is preserved when applying these operations to volatile objects.

7.17.2 Initialization

1 An atomic object with automatic storage duration that is not initialized or such an object with allocated storage duration initially has an indeterminate representation; equally, a non-atomic store to any byte of the representation (either directly or, for example, by calls to **`memcpy`** or **`memset`**) makes any atomic object have an indeterminate representation. Explicit or default initialization for atomic objects with static or thread storage duration that do not have the type **`atomic_flag`** is guaranteed to produce a valid state.[298)]

2 Concurrent access to an atomic object before it is set to a valid state, even via an atomic operation, constitutes a data race. If a signal occurs other than as the result of calling the **`abort`** or **`raise`** functions, the behavior is undefined if the signal handler reads or modifies an atomic object that has an indeterminate representation.

3 EXAMPLE The following definition ensure valid states for `guide` and `head` regardless if these are found in file scope or block scope. Thus any atomic operation that is performed on them after their initialization has been met is well defined.

```
_Atomic int guide = 42;
static _Atomic(void*) head;
```

7.17.2.1 The `atomic_init` generic function

Synopsis

1

```
#include <stdatomic.h>
void atomic_init(volatile A *obj, C value);
```

Description

2 The **`atomic_init`** generic function initializes the atomic object pointed to by `obj` to the value `value`, while also initializing any additional state that the implementation may need to carry for the atomic object. If the object has no declared type, after the call the effective type is the atomic type *`A`*.

3 Although this function initializes an atomic object, it does not avoid data races; concurrent access to the object being initialized, even via an atomic operation, constitutes a data race.

4 If a signal occurs other than as the result of calling the **`abort`** or **`raise`** functions, the behavior is undefined if the signal handler calls the **`atomic_init`** generic function.

Returns

5 The **`atomic_init`** generic function returns no value.

6 EXAMPLE

```
atomic_int guide;
atomic_init(&guide, 42);
```

7.17.3 Order and consistency

1 The enumerated type **`memory_order`** specifies the detailed regular (non-atomic) memory synchronization operations as defined in 5.1.2.5 and may provide for operation ordering. Its enumeration constants are as follows:[299)]

[298)] See "future library directions" (7.33.10).
[299)] See "future library directions" (7.33.10).

© ISO 2024 — All rights reserved

```
    memory_order_relaxed
    memory_order_consume
    memory_order_acquire
    memory_order_release
    memory_order_acq_rel
    memory_order_seq_cst
```

2 For **memory_order_relaxed**, no operation orders memory.

3 For **memory_order_release**, **memory_order_acq_rel**, and **memory_order_seq_cst**, a store operation performs a release operation on the affected memory location.

4 For **memory_order_acquire**, **memory_order_acq_rel**, and **memory_order_seq_cst**, a load operation performs an acquire operation on the affected memory location.

5 For **memory_order_consume**, a load operation performs a consume operation on the affected memory location.

6 There shall be a single total order S on all **memory_order_seq_cst** operations, consistent with the "happens before" order and modification orders for all affected locations, such that each **memory_order_seq_cst** operation B that loads a value from an atomic object M observes one of the following values:

— the result of the last modification A of M that precedes B in S, if it exists, or

— if A exists, the result of some modification of M that is not **memory_order_seq_cst** and that does not happen before A, or

— if A does not exist, the result of some modification of M that is not **memory_order_seq_cst**.

7 NOTE 1 Although it is not explicitly required that S include lock operations, it can always be extended to an order that does include lock and unlock operations, since the ordering between those is already included in the "happens before" ordering.

8 NOTE 2 Atomic operations specifying **memory_order_relaxed** are relaxed only with respect to memory ordering. Implementations still guarantee that any given atomic access to a particular atomic object is indivisible with respect to all other atomic accesses to that object.

9 For an atomic operation B that reads the value of an atomic object M, if there is a **memory_order_seq_cst** fence X sequenced before B, then B observes either the last **memory_order_seq_cst** modification of M preceding X in the total order S or a later modification of M in its modification order.

10 For atomic operations A and B on an atomic object M, where A modifies M and B takes its value, if there is a **memory_order_seq_cst** fence X such that A is sequenced before X and B follows X in S, then B observes either the effects of A or a later modification of M in its modification order.

11 For atomic modifications A and B of an atomic object M, B occurs later than A in the modification order of M if:

— there is a **memory_order_seq_cst** fence X such that A is sequenced before X, and X precedes B in S, or

— there is a **memory_order_seq_cst** fence Y such that Y is sequenced before B, and A precedes Y in S, or

— there are **memory_order_seq_cst** fences X and Y such that A is sequenced before X, Y is sequenced before B, and X precedes Y in S.

12 Atomic read-modify-write operations shall always read the last value (in the modification order) stored before the write associated with the read-modify-write operation.

13 An atomic store shall only store a value that has been computed from constants and program input values by a finite sequence of program evaluations, such that each evaluation observes the stored

 © ISO 2024 — All rights reserved

values of the objects as computed by the last prior assignment in the sequence. The ordering of evaluations in this sequence shall be such that

— If an evaluation B observes a value computed by A in a different thread, then B does not happen before A.

— If an evaluation A is included in the sequence, then all evaluations that assign to the same object and happen before A are also included.

14 NOTE 3 The second requirement disallows "out-of-thin-air", or "speculative" stores of atomics when relaxed atomics are used. Since unordered operations are involved, evaluations can appear in this sequence out of thread order. For example, with `x` and `y` initially zero,

```
// Thread 1:
r1 = atomic_load_explicit(&y, memory_order_relaxed);
atomic_store_explicit(&x, r1, memory_order_relaxed);

// Thread 2:
r2 = atomic_load_explicit(&x, memory_order_relaxed);
atomic_store_explicit(&y, 42, memory_order_relaxed);
```

is allowed to produce `r1 == 42 && r2 == 42`. The sequence of evaluations justifying this consists of:

```
atomic_store_explicit(&y, 42, memory_order_relaxed);
r1 = atomic_load_explicit(&y, memory_order_relaxed);
atomic_store_explicit(&x, r1, memory_order_relaxed);
r2 = atomic_load_explicit(&x, memory_order_relaxed);
```

On the other hand,

```
// Thread 1:
r1 = atomic_load_explicit(&y, memory_order_relaxed);
atomic_store_explicit(&x, r1, memory_order_relaxed);

// Thread 2:
r2 = atomic_load_explicit(&x, memory_order_relaxed);
atomic_store_explicit(&y, r2, memory_order_relaxed);
```

is not allowed to produce `r1 == 42 && r2 == 42`, since there is no sequence of evaluations that results in the computation of 42. In the absence of "relaxed" operations and read-modify-write operations with weaker than `memory_order_acq_rel` ordering, the second requirement has no impact.

Recommended practice

15 The requirements do not forbid `r1 == 42 && r2 == 42` in the following example, with `x` and `y` initially zero:

```
// Thread 1:
r1 = atomic_load_explicit(&x, memory_order_relaxed);
if (r1 == 42)
      atomic_store_explicit(&y, r1, memory_order_relaxed);

// Thread 2:
r2 = atomic_load_explicit(&y, memory_order_relaxed);
if (r2 == 42)
      atomic_store_explicit(&x, 42, memory_order_relaxed);
```

However, this is not useful behavior, and implementations should not allow it.

16 Implementations should make atomic stores visible to atomic loads within a reasonable amount of time.

© ISO 2024 — All rights reserved

7.17.3.1 The `kill_dependency` macro

Synopsis

1
```
#include <stdatomic.h>
type kill_dependency(type y);
```

Description

2 The `kill_dependency` macro terminates a dependency chain; the argument does not carry a dependency to the return value.

Returns

3 The `kill_dependency` macro returns the value of `y`.

7.17.4 Fences

1 This subclause introduces synchronization primitives called *fences*. Fences can have acquire semantics, release semantics, or both. A fence with acquire semantics is called an *acquire fence*; a fence with release semantics is called a *release fence*.

2 A release fence A synchronizes with an acquire fence B if there exist atomic operations X and Y, both operating on some atomic object M, such that A is sequenced before X, X modifies M, Y is sequenced before B, and Y reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation.

3 A release fence A synchronizes with an atomic operation B that performs an acquire operation on an atomic object M if there exists an atomic operation X such that A is sequenced before X, X modifies M, and B reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation.

4 An atomic operation A that is a release operation on an atomic object M synchronizes with an acquire fence B if there exists some atomic operation X on M such that X is sequenced before B and reads the value written by A or a value written by any side effect in the release sequence headed by A.

7.17.4.1 The `atomic_thread_fence` function

Synopsis

1
```
#include <stdatomic.h>
void atomic_thread_fence(memory_order order);
```

Description

2 Depending on the value of `order`, this operation:

— has no effects, if `order == memory_order_relaxed`;

— is an acquire fence, if `order == memory_order_acquire` or `order == memory_order_consume`;

— is a release fence, if `order == memory_order_release`;

— is both an acquire fence and a release fence, if `order == memory_order_acq_rel`;

— is a sequentially consistent acquire and release fence, if `order == memory_order_seq_cst`.

Returns

3 The `atomic_thread_fence` function returns no value.

7.17.4.2 The `atomic_signal_fence` function

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdatomic.h>
void atomic_signal_fence(memory_order order);
```

Description

2 Equivalent to `atomic_thread_fence(order)`, except that the resulting ordering constraints are established only between a thread and a signal handler executed in the same thread.

3 NOTE 1 The `atomic_signal_fence` function can be used to specify the order in which actions performed by the thread become visible to the signal handler.

4 NOTE 2 Compiler optimizations and reorderings of loads and stores are inhibited in the same way as with `atomic_thread_fence`, but the hardware fence instructions that `atomic_thread_fence` would have inserted are not emitted.

Returns

5 The `atomic_signal_fence` function returns no value.

7.17.5 Lock-free property

1 The atomic lock-free macros indicate the lock-free property of integer and address atomic types. A value of 0 indicates that the type is never lock-free; a value of 1 indicates that the type is sometimes lock-free; a value of 2 indicates that the type is always lock-free.

Recommended practice

2 Operations that are lock-free should also be *address-free*. That is, atomic operations on the same memory location via two different addresses will communicate atomically. The implementation should not depend on any per-process state. This restriction enables communication via memory mapped into a process more than once and memory shared between two processes.

7.17.5.1 The `atomic_is_lock_free` generic function

Synopsis

1
```
#include <stdatomic.h>
bool atomic_is_lock_free(const volatile A *obj);
```

Description

2 The `atomic_is_lock_free` generic function indicates whether atomic operations on objects of the type pointed to by `obj` are lock-free.

Returns

3 The `atomic_is_lock_free` generic function returns nonzero (true) if and only if atomic operations on objects of the type pointed to by the argument are lock-free. In any given program execution, the result of the lock-free query shall be consistent for all pointers of the same type.[300)]

7.17.6 Atomic integer types

1 For each line in the following table,[301)] the atomic type name is declared as a type that has the same representation and alignment requirements as the corresponding direct type.[302)]

Atomic type name	Direct type
`atomic_bool`	`_Atomic bool`
`atomic_char`	`_Atomic char`
`atomic_schar`	`_Atomic signed char`
`atomic_uchar`	`_Atomic unsigned char`

[300)] `obj` can be a null pointer.
[301)] See "future library directions" (7.33.10).
[302)] The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and members of unions.

© ISO 2024 — All rights reserved

Atomic type name	Direct type
atomic_short	**_Atomic short**
atomic_ushort	**_Atomic unsigned short**
atomic_int	**_Atomic int**
atomic_uint	**_Atomic unsigned int**
atomic_long	**_Atomic long**
atomic_ulong	**_Atomic unsigned long**
atomic_llong	**_Atomic long long**
atomic_ullong	**_Atomic unsigned long long**
atomic_char8_t	**_Atomic char8_t**
atomic_char16_t	**_Atomic char16_t**
atomic_char32_t	**_Atomic char32_t**
atomic_wchar_t	**_Atomic wchar_t**
atomic_int_least8_t	**_Atomic int_least8_t**
atomic_uint_least8_t	**_Atomic uint_least8_t**
atomic_int_least16_t	**_Atomic int_least16_t**
atomic_uint_least16_t	**_Atomic uint_least16_t**
atomic_int_least32_t	**_Atomic int_least32_t**
atomic_uint_least32_t	**_Atomic uint_least32_t**
atomic_int_least64_t	**_Atomic int_least64_t**
atomic_uint_least64_t	**_Atomic uint_least64_t**
atomic_int_fast8_t	**_Atomic int_fast8_t**
atomic_uint_fast8_t	**_Atomic uint_fast8_t**
atomic_int_fast16_t	**_Atomic int_fast16_t**
atomic_uint_fast16_t	**_Atomic uint_fast16_t**
atomic_int_fast32_t	**_Atomic int_fast32_t**
atomic_uint_fast32_t	**_Atomic uint_fast32_t**
atomic_int_fast64_t	**_Atomic int_fast64_t**
atomic_uint_fast64_t	**_Atomic uint_fast64_t**
atomic_intptr_t	**_Atomic intptr_t**
atomic_uintptr_t	**_Atomic uintptr_t**
atomic_size_t	**_Atomic size_t**
atomic_ptrdiff_t	**_Atomic ptrdiff_t**
atomic_intmax_t	**_Atomic intmax_t**
atomic_uintmax_t	**_Atomic uintmax_t**

2 Conversions to **atomic_bool** behave the same as conversions to **bool**.

Recommended practice

3 The representation of an atomic integer type is not required to have the same size as the corresponding regular type but it should have the same size whenever possible, as it eases effort required to port existing code.

7.17.7 Operations on atomic types

1 There are only a few kinds of operations on atomic types, though there are many instances of those kinds. This subclause specifies each general kind.

7.17.7.1 The atomic_store generic functions

Synopsis

1
```
#include <stdatomic.h>
void atomic_store(volatile A *object, C desired);
void atomic_store_explicit(volatile A *object, C desired, memory_order order);
```

 © ISO 2024 — All rights reserved

Description

2 The `order` argument shall not be **`memory_order_acquire`**, **`memory_order_consume`**, nor **`memory_order_acq_rel`**. Atomically replace the value pointed to by `object` with the value of `desired`. Memory is affected according to the value of `order`.

Returns

3 The **`atomic_store`** generic functions return no value.

7.17.7.2 The `atomic_load` generic functions

Synopsis

1
```
#include <stdatomic.h>
C atomic_load(const volatile A *object);
C atomic_load_explicit(const volatile A *object, memory_order order);
```

Description

2 The `order` argument shall not be **`memory_order_release`** nor **`memory_order_acq_rel`**. Memory is affected according to the value of `order`.

Returns

3 Atomically returns the value pointed to by `object`.

7.17.7.3 The `atomic_exchange` generic functions

Synopsis

1
```
#include <stdatomic.h>
C atomic_exchange(volatile A *object, C desired);
C atomic_exchange_explicit(volatile A *object, C desired, memory_order order);
```

Description

2 Atomically replace the value pointed to by `object` with `desired`. Memory is affected according to the value of `order`. These operations are read-modify-write operations (5.1.2.5).

Returns

3 Atomically returns the value pointed to by `object` immediately before the effects.

7.17.7.4 The `atomic_compare_exchange` generic functions

Synopsis

1
```
#include <stdatomic.h>
bool atomic_compare_exchange_strong(volatile A *object, C *expected, C desired);
bool atomic_compare_exchange_strong_explicit(volatile A *object, C *expected,
      C desired, memory_order success, memory_order failure);
bool atomic_compare_exchange_weak(volatile A *object, C *expected, C desired);
bool atomic_compare_exchange_weak_explicit(volatile A *object, C *expected,
      C desired, memory_order success, memory_order failure);
```

Description

2 The `failure` argument shall not be **`memory_order_release`** nor **`memory_order_acq_rel`**. The `failure` argument shall be no stronger than the success argument.

3 Atomically, compares the contents of the memory pointed to by `object` for equality with that pointed to by `expected`, and if true, replaces the contents of the memory pointed to by `object` with `desired`, and if false, updates the contents of the memory pointed to by `expected` with that pointed to by `object`. Further, if the comparison is true, memory is affected according to the value of `success`, and if the comparison is false, memory is affected according to the value of `failure`. These operations are atomic read-modify-write operations (5.1.2.5).

4 NOTE 1 For example, the effect of **`atomic_compare_exchange_strong`** is

© ISO 2024 — All rights reserved

```
      if (memcmp(object, expected, sizeof(*object)) == 0)
            memcpy(object, &desired, sizeof(*object));
      else
            memcpy(expected, object, sizeof(*object));
```

5 A weak compare-and-exchange operation may fail spuriously. That is, even when the contents of memory referred to by `expected` and `object` are equal, it may return zero and store back to `expected` the same memory contents that were originally there.

6 NOTE 2 This spurious failure enables implementation of compare-and-exchange on a broader class of machines; e.g. load-locked store-conditional machines.

© ISO 2024 — All rights reserved

7 EXAMPLE A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop.

```
exp = atomic_load(&cur);
do {
      des = function(exp);
} while (!atomic_compare_exchange_weak(&cur, &exp, des));
```

When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable.

Returns

8 The result of the comparison.

7.17.7.5 The `atomic_fetch` and modify generic functions

1 The following operations perform arithmetic and bitwise computations. All these operations are applicable to an object of any atomic integer type other than **_Atomic bool**, **atomic_bool**, or the atomic version of an enumeration with underlying type **bool**. The key, operator, and computation correspondence is:

key	op	computation
add	+	addition
sub	-	subtraction
or	|	bitwise inclusive or
xor	^	bitwise exclusive or
and	&	bitwise and

Synopsis

2
```
#include <stdatomic.h>
C atomic_fetch_key(volatile A *object, M operand);
C atomic_fetch_key_explicit(volatile A *object, M operand, memory_order order);
```

Description

3 Atomically replaces the value pointed to by `object` with the result of the computation applied to the value pointed to by `object` and the given operand. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (5.1.2.5). For signed integer types, arithmetic performs silent wraparound on integer overflow; there are no undefined results. For address types, the result may be an undefined address, but the operations otherwise have no undefined behavior.

Returns

4 Atomically, the value pointed to by `object` immediately before the effects.

7.17.8 Atomic flag type and operations

1 The **atomic_flag** type provides the classic test-and-set functionality. It has two states, set and clear.

2 Operations on an object of type **atomic_flag** shall be lock free.

3 NOTE Hence, as per 7.17.5, the operations should also be address-free. No other type requires lock-free operations, so the **atomic_flag** type is the minimum hardware-implemented type needed to conform to this document. The remaining types can be emulated with **atomic_flag**, though with less than ideal properties.

4 The macro **ATOMIC_FLAG_INIT** may be used to initialize an **atomic_flag** to the clear state. An **atomic_flag** that is not explicitly initialized with **ATOMIC_FLAG_INIT** has initially an indeterminate representation.

5 EXAMPLE

© ISO 2024 — All rights reserved

```
      atomic_flag guard = ATOMIC_FLAG_INIT;
```

7.17.8.1 The `atomic_flag_test_and_set` functions

Synopsis

1
```
#include <stdatomic.h>
      bool atomic_flag_test_and_set(volatile atomic_flag *object);
      bool atomic_flag_test_and_set_explicit(volatile atomic_flag *object,
            memory_order order);
```

Description

2 Atomically places the atomic flag pointed to by `object` in the set state and returns the value corresponding to the immediately preceding state. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (5.1.2.5).

Returns

3 The **`atomic_flag_test_and_set`** functions return the value that corresponds to the state of the atomic flag immediately before the effects. The return value true corresponds to the set state and the return value false corresponds to the clear state.

7.17.8.2 The `atomic_flag_clear` functions

Synopsis

1
```
      #include <stdatomic.h>
      void atomic_flag_clear(volatile atomic_flag *object);
      void atomic_flag_clear_explicit(volatile atomic_flag *object,
            memory_order order);
```

Description

2 The `order` argument shall not be **`memory_order_acquire`** nor **`memory_order_acq_rel`**. Atomically places the atomic flag pointed to by `object` into the clear state. Memory is affected according to the value of `order`.

Returns

3 The **`atomic_flag_clear`** functions return no value.

© ISO 2024 — All rights reserved

7.18 Bit and byte utilities <stdbit.h>

7.18.1 General

1 The header <stdbit.h> defines the following macros, types, and functions, to work with the byte and bit representation of many types, typically integer types. This header makes available the **size_t** type name (7.21) and any uintN_t, intN_t, uint_leastN_t, or int_leastN_t type names defined by the implementation (7.22).

2 The macro

```
__STDC_VERSION_STDBIT_H__
```

is an integer constant expression with a value equivalent to 202311L.

3 The *most significant index* is the 0-based index counting from the most significant bit, 0, to the least significant bit, $w - 1$, where w is the width of the type that is having its most significant index computed.

4 The *least significant index* is the 0-based index counting from the least significant bit, 0, to the most significant bit, $w - 1$, where w is the width of the type that is having its least significant index computed.

5 It is unspecified whether any generic function declared in <stdbit.h> is a macro or an identifier declared with external linkage. If a macro definition is suppressed to access an actual function, or a program defines an external identifier with the name of a generic function, the behavior is unspecified.

7.18.2 Endian

1 Two common methods of byte ordering in multi-byte scalar types are *little-endian* and *big-endian*. Little-endian is a format for storage or transmission of binary data in which the least significant byte is placed first, with the rest in ascending order. Or, that the least significant byte is stored at the smallest memory address. Big-endian is a format for storage or transmission of binary data in which the most significant byte is placed first, with the rest in descending order. Or, that the most significant byte is stored at the smallest memory address. Other byte orderings are also possible.

2 The macros are:

```
__STDC_ENDIAN_LITTLE__
```

which represents a method of byte order storage in which the least significant byte is placed first and the rest are in ascending order, and is an integer constant expression;

```
__STDC_ENDIAN_BIG__
```

which represents a method of byte order storage in which the most significant byte is placed first and the rest are in descending order, and is an integer constant expression;

```
__STDC_ENDIAN_NATIVE__ /* see following description */
```

which represents the method of byte order storage for the execution environment and is an integer constant expression. **__STDC_ENDIAN_NATIVE__** describes the endianness of the execution environment with respect to bit-precise integer types, standard integer types, and extended integer types which do not have padding bits.

3 **__STDC_ENDIAN_NATIVE__** shall expand to an integer constant expression whose value is equivalent to the value of **__STDC_ENDIAN_LITTLE__** if the execution environment is little-endian. Otherwise, **__STDC_ENDIAN_NATIVE__** shall expand to an integer constant expression whose value is equivalent to the value of **__STDC_ENDIAN_BIG__** if the execution environment is big-endian. If the execution environment is neither little-endian nor big-endian, it then has some other implementation-defined byte order and the macro **__STDC_ENDIAN_NATIVE__** shall expand to an integer constant expression whose value is different from the values of

© ISO 2024 — All rights reserved

__STDC_ENDIAN_LITTLE__ and **__STDC_ENDIAN_BIG__**. The values of the integer constant expressions for **__STDC_ENDIAN_LITTLE__** and **__STDC_ENDIAN_BIG__** are not equal.

7.18.3 Count Leading Zeros

Synopsis

1
```
#include <stdbit.h>
unsigned int stdc_leading_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_leading_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_leading_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_leading_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_leading_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_leading_zeros(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the number of consecutive 0 bits in `value`, starting from the most significant bit.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.4 Count Leading Ones

Synopsis

1
```
#include <stdbit.h>
unsigned int stdc_leading_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_leading_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_leading_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_leading_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_leading_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_leading_ones(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the number of consecutive 1 bits in `value`, starting from the most significant bit.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.5 Count Trailing Zeros

 © ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdbit.h>
unsigned int stdc_trailing_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_trailing_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_trailing_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_trailing_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_trailing_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_trailing_zeros(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the number of consecutive 0 bits in `value`, starting from the least significant bit.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.6 Count Trailing Ones

Synopsis

1
```
#include <stdbit.h>
unsigned int stdc_trailing_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_trailing_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_trailing_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_trailing_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_trailing_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_trailing_ones(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the number of consecutive 1 bits in `value`, starting from the least significant bit.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.7 First Leading Zero

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdbit.h>
unsigned int stdc_first_leading_zero_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_leading_zero_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_leading_zero_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_leading_zero_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_leading_zero_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_leading_zero(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the most significant index of the first 0 bit in `value`, plus 1. If it is not found, this function returns 0.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.8 First Leading One

Synopsis

1
```
#include <stdbit.h>
unsigned int stdc_first_leading_one_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_leading_one_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_leading_one_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_leading_one_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_leading_one_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_leading_one(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the most significant index of the first 1 bit in `value`, plus 1. If it is not found, this function returns 0.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.9 First Trailing Zero

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdbit.h>
unsigned int stdc_first_trailing_zero_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_trailing_zero_ui(unsigned int value) [[unsequenced]];
unsigned int
stdc_first_trailing_zero_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_trailing_zero_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_trailing_zero(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the least significant index of the first 0 bit in `value`, plus 1. If it is not found, this function returns 0.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.10 First Trailing One

Synopsis

1
```
#include <stdbit.h>
unsigned int stdc_first_trailing_one_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_first_trailing_one_us(unsigned short value) [[unsequenced]];
unsigned int stdc_first_trailing_one_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_first_trailing_one_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_first_trailing_one_ull(unsigned long long int value) [[unsequenced]];
generic_return_type
stdc_first_trailing_one(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the least significant index of the first 1 bit in `value`, plus 1. If it is not found, this function returns 0.

The type-generic function (marked by its *generic_value_type* argument) returns the appropriate value based on the type of the input value, so long as it is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

© ISO 2024 — All rights reserved

7.18.11 Count Zeros

Synopsis

1

```
#include <stdbit.h>
unsigned int stdc_count_zeros_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_count_zeros_us(unsigned short value) [[unsequenced]];
unsigned int stdc_count_zeros_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_count_zeros_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_count_zeros_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_count_zeros(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the total number of 0 bits within the given `value`.

3 The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.12 Count Ones

Synopsis

1

```
#include <stdbit.h>
unsigned int stdc_count_ones_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_count_ones_us(unsigned short value) [[unsequenced]];
unsigned int stdc_count_ones_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_count_ones_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_count_ones_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_count_ones(generic_value_type value) [[unsequenced]];
```

Returns

2 Returns the total number of 1 bits within the given `value`.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.13 Single-bit Check

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdbit.h>
bool stdc_has_single_bit_uc(unsigned char value) [[unsequenced]];
bool stdc_has_single_bit_us(unsigned short value) [[unsequenced]];
bool stdc_has_single_bit_ui(unsigned int value) [[unsequenced]];
bool stdc_has_single_bit_ul(unsigned long int value) [[unsequenced]];
bool stdc_has_single_bit_ull(unsigned long long int value) [[unsequenced]];
bool stdc_has_single_bit(generic_value_type value) [[unsequenced]];
```

Returns

2 The **stdc_has_single_bit** functions return **true** if and only if there is a single 1 bit in `value`.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

7.18.14 Bit Width

Synopsis

1
```
#include <stdbit.h>
unsigned int stdc_bit_width_uc(unsigned char value) [[unsequenced]];
unsigned int stdc_bit_width_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_width_ui(unsigned int value) [[unsequenced]];
unsigned int stdc_bit_width_ul(unsigned long int value) [[unsequenced]];
unsigned int
stdc_bit_width_ull(unsigned long long int value) [[unsequenced]];
generic_return_type stdc_bit_width(generic_value_type value) [[unsequenced]];
```

Description

2 The **stdc_bit_width** functions compute the smallest number of bits needed to store `value`.

Returns

3 The **stdc_bit_width** functions return 0 if `value` is 0. Otherwise, they return $1 + \lfloor \log_2(\texttt{value}) \rfloor$.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

The *generic_return_type* type shall be a suitable large unsigned integer type capable of representing the computed result.

7.18.15 Bit Floor

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdbit.h>
unsigned char stdc_bit_floor_uc(unsigned char value) [[unsequenced]];
unsigned short stdc_bit_floor_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_floor_ui(unsigned int value) [[unsequenced]];
unsigned long int stdc_bit_floor_ul(unsigned long int value) [[unsequenced]];
unsigned long long int
stdc_bit_floor_ull(unsigned long long int value) [[unsequenced]];
generic_value_type stdc_bit_floor(generic_value_type value) [[unsequenced]];
```

Description

2 The **stdc_bit_floor** functions compute the largest integral power of 2 that is not greater than `value`.

Returns

3 The **stdc_bit_floor** functions return 0 if `value` is 0. Otherwise, they return the largest integral power of 2 that is not greater than `value`.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

7.18.16 Bit Ceiling

Synopsis

1
```
#include <stdbit.h>
unsigned char stdc_bit_ceil_uc(unsigned char value) [[unsequenced]];
unsigned short stdc_bit_ceil_us(unsigned short value) [[unsequenced]];
unsigned int stdc_bit_ceil_ui(unsigned int value) [[unsequenced]];
unsigned long int stdc_bit_ceil_ul(unsigned long int value) [[unsequenced]];
unsigned long long int
stdc_bit_ceil_ull(unsigned long long int value) [[unsequenced]];
generic_value_type stdc_bit_ceil(generic_value_type value) [[unsequenced]];
```

Description

2 The **stdc_bit_ceil** functions compute the smallest integral power of 2 that is not less than `value`. If the computation does not fit in the given return type, they return `0`.

Returns

3 The **stdc_bit_ceil** functions return the smallest integral power of 2 that is not less than `value` or `0` if such a value is not representable in the return type.

The type-generic function (marked by its *generic_value_type* argument) returns the previously described result for a given input value so long as the *generic_value_type* is a:

— standard unsigned integer type, excluding **bool**;

— extended unsigned integer type;

— or, bit-precise unsigned integer type whose width matches a standard or extended integer type, excluding **bool**.

© ISO 2024 — All rights reserved

7.19 Boolean type and values `<stdbool.h>`

1 The header `<stdbool.h>` provides the obsolescent macro **`__bool_true_false_are_defined`** which expands to the integer constant `1`.

© ISO 2024 — All rights reserved

7.20 Checked Integer Arithmetic <stdckdint.h>

1 The header <stdckdint.h> defines several macros for performing checked integer arithmetic.

2 The macro

```
__STDC_VERSION_STDCKDINT_H__
```

is an integer constant expression with a value equivalent to 202311L.

7.20.1 Checked Integer Operation Type-generic Macros

Synopsis

1
```
#include <stdckdint.h>
bool ckd_add(type1 *result, type2 a, type3 b);
bool ckd_sub(type1 *result, type2 a, type3 b);
bool ckd_mul(type1 *result, type2 a, type3 b);
```

Description

2 These type-generic macros perform addition, subtraction, or multiplication of the mathematical values of a and b, storing the result of the operation in *result, (that is, *result is assigned the result of computing a + b, a - b, or a * b). Each operation is performed as if both operands were represented in a signed integer type with infinite range, and the result was then converted from this integer type to *type1*.

3 Both *type2* and *type3* shall be any integer type other than "plain" **char**, **bool**, a bit-precise integer type, or an enumerated type, and they may or may not be the same. *result shall be a modifiable lvalue of any integer type other than "plain" **char**, **bool**, a bit-precise integer type, or an enumerated type.

Recommended practice

4 It is recommended to produce a diagnostic message if *type2* or *type3* are not suitable integer types, or if *result is not a modifiable lvalue of a suitable integer type.

Returns

5 If these type-generic macros return **false**, the value assigned to *result correctly represents the mathematical result of the operation. Otherwise, these type-generic macros return **true**. In this case, the value assigned to *result is the mathematical result of the operation wrapped around to the width of *result.

6 EXAMPLE If a and b are values of type **signed int**, and result is a **signed long**, then

```
ckd_sub(&result, a, b);
```

indicates if a - b can be expressed as a **signed long**. If **signed long** has a greater width than **signed int**, this is the case and this macro invocation returns **false**.

© ISO 2024 — All rights reserved

7.21 Common definitions <stddef.h>

1 The header <stddef.h> defines the following macros and declares the following types. Some are also defined in other headers, as noted in their respective subclauses.

2 The macro

```
__STDC_VERSION_STDDEF_H__
```

is an integer constant expression with a value equivalent to 202311L.

3 The types are

```
ptrdiff_t
```

which is the signed integer type of the result of subtracting two pointers;

```
size_t
```

which is the unsigned integer type of the result of the **sizeof** operator;

```
max_align_t
```

which is an object type whose alignment is the greatest fundamental alignment;

```
wchar_t
```

which is an integer type whose range of values can represent distinct codes for all members of the largest extended character set specified among the supported locales; the null character shall have the code value zero. Each member of the basic character set shall have a code value equal to its value when used as the lone character in an integer character constant if an implementation does not define **__STDC_MB_MIGHT_NEQ_WC__**; and,

```
nullptr_t
```

which is the type of the **nullptr** predefined constant, see the subsequent description in the following subclauses.

4 The macros are

```
NULL
```

which expands to an implementation-defined null pointer constant;

```
unreachable()
```

which expands to a void expression that invokes undefined behavior if it is reached during execution; and

```
offsetof(type, member-designator)
```

which expands to an integer constant expression that has type **size_t**, the value of which is the offset in bytes, to the subobject (designated by *member-designator*), from the beginning of any object of type *type*. The type and member designator shall be such that given

```
static type t;
```

then the expression &(t. *member-designator*) evaluates to an address constant. If the specified type name contains a comma not between matching parentheses or if the specified member is a bit-field, the behavior is undefined.

© ISO 2024 — All rights reserved

Recommended practice

5 The types used for **size_t** and **ptrdiff_t** should not have an integer conversion rank greater than that of **signed long int** unless the implementation supports objects large enough to make this necessary.

7.21.1 The unreachable macro

Synopsis

1
```
#include <stddef.h>
void unreachable(void);
```

Description

2 An invocation of the function-like macro **unreachable** indicates that the particular flow control that leads to the invocation will never be taken; it receives no arguments and expands to a void expression. The program execution shall not reach such an invocation.

Returns

3 If a macro invocation **unreachable**() is reached during execution, the behavior is undefined.

4 EXAMPLE 1 The following program assumes that each execution is provided with at least one command line argument. The behavior of an execution with no arguments is undefined.

```
#include <stddef.h>
#include <stdio.h>

int main (int argc, char* argv[static argc + 1]) {
      if (argc <= 2)
            unreachable();
      else
            return printf("%s: we see %s", argv[0], argv[1]);

      return puts("this should never be reached");
}
```

Here, the **static** array size expression and the annotation of the control flow with **unreachable** indicates that the pointed-to parameter array argv will hold at least three elements, regardless of the circumstances. A possible optimization is that the resulting executable never performs the comparison and unconditionally executes a tail call to **printf** that never returns to the **main** function. In particular, the entire call and reference to **puts** can be omitted from the executable. No diagnostic is expected.

Note that because argv and argc's values are controlled by the implementation and cannot be deterministically accounted for by the program, this program runs a high risk of engaging in completely undefined behavior.

5 EXAMPLE 2 The following code expresses the expectation that the argument to the function will be one of the three enumerator values despite an enumeration type allowing other, non-enumerated values to be passed. Some implementations may diagnose the lack of a **return** statement after the **switch**, but use of the **unreachable** macro signals information to the implementation that this scenario should not be possible, allowing for better diagnostic and optimization properties.

```
      enum Colors { Red, Green, Blue };
      int get_channel_index(enum Colors c) {
            switch (c) {
                  case Red: return 0;
                  case Green: return 1;
                  case Blue: return 2;
            }
            unreachable();
      }
```

7.21.2 The nullptr_t type

 © ISO 2024 — All rights reserved

Synopsis

1
```
#include <stddef.h>
typedef typeof_unqual(nullptr) nullptr_t;
```

Description

2 The **nullptr_t** type is the type of the **nullptr** predefined constant. It has only a very limited use in contexts where this type is needed to distinguish **nullptr** from other expression types. It is an unqualified complete scalar type that is different from all pointer or arithmetic types and is neither an atomic or array type and has exactly one value, **nullptr**. Default or empty initialization of an object of this type is equivalent to an initialization by **nullptr**.

3 The size and alignment of **nullptr_t** is the same as for a pointer to character type. An object representation of the value **nullptr** is the same as the object representation of a null pointer value of type **void**∗. An lvalue conversion of an object of type **nullptr_t** with such an object representation has the value **nullptr**; if the object representation is different, the behavior is undefined.[303)]

4 NOTE Because it is considered to be a scalar type, **nullptr_t** may appear in many context where **(void∗)0** would be valid, for example,

- as the operand of **alignas**, **sizeof** or typeof operators,
- as the operand of an implicit or explicit conversion to a pointer type,
- as the assignment expression in an assignment or initialization of an object of type **nullptr_t**,
- as an argument to a parameter of type **nullptr_t** or in a variable argument list,
- as a void expression,
- as the operand of an implicit or explicit conversion to **bool**,
- as an operand of a **_Generic** primary expression,
- as an operand of the !, &&, || or conditional operators, or
- as the controlling expression of an **if** or iteration statement.

[303)] Thus, during the whole program execution an object of type **nullptr_t** evaluates to the assumed value **nullptr**.

© ISO 2024 — All rights reserved

7.22 Integer types <stdint.h>

1 The header <stdint.h> declares sets of integer types having specified widths, and defines corresponding sets of macros.[304] It also defines macros that specify limits of integer types corresponding to types defined in other standard headers.

2 Types are defined in the following categories:

— integer types having certain exact widths;

— integer types having at least certain specified widths;

— fastest integer types having at least certain specified widths;

— integer types wide enough to hold pointers to objects;

— integer types having greatest width.

(Some of these types may denote the same type.)

3 Corresponding macros specify limits of the declared types and construct suitable constants.

4 For each type described herein that the implementation provides,[305] <stdint.h> shall declare that typedef name and define the associated macros. Conversely, for each type described herein that the implementation does not provide, <stdint.h> shall not declare that typedef name nor shall it define the associated macros. An implementation shall provide those types described as "required", but may not provide any of the others (described as "optional"). None of the types shall be defined as a synonym for a bit-precise integer type.

5 The feature test macro **__STDC_VERSION_STDINT_H__** expands to the token 202311L.

7.22.1 Integer types

1 When typedef names differing only in the absence or presence of the initial u are defined, they shall denote corresponding signed and unsigned types as described in 6.2.5; an implementation providing one of these corresponding types shall also provide the other.

2 In the following descriptions, the symbol *N* represents an unsigned decimal integer with no leading zeros (e.g. 8 or 24, but not 04 or 048).

7.22.1.1 Exact-width integer types

1 The typedef name **int***N***_t** designates a signed integer type with width *N* and no padding bits. Thus, **int8_t** denotes such a signed integer type with a width of exactly 8 bits.

2 The typedef name **uint***N***_t** designates an unsigned integer type with width *N* and no padding bits. Thus, uint24_t denotes such an unsigned integer type with a width of exactly 24 bits.

3 If an implementation provides standard or extended integer types with a particular width and no padding bits, it shall define the corresponding typedef names.

7.22.1.2 Minimum-width integer types

1 The typedef name **int_least***N***_t** designates a signed integer type with a width of at least *N*, such that no signed integer type with lesser size has at least the specified width. Thus, **int_least32_t** denotes a signed integer type with a width of at least 32 bits.

2 The typedef name **uint_least***N***_t** designates an unsigned integer type with a width of at least *N*, such that no unsigned integer type with lesser size has at least the specified width. Thus, **uint_least16_t** denotes an unsigned integer type with a width of at least 16 bits.

3 If the typedef name **int***N***_t** is defined, **int_least***N***_t** designates the same type. If the typedef name **uint***N***_t** is defined, **uint_least***N***_t** designates the same type.

4 The following types are required:

[304] See "future library directions" (7.33.14).

[305] Some of these types can denote implementation-defined extended integer types.

 © ISO 2024 — All rights reserved

int_least8_t	**uint_least8_t**
int_least16_t	**uint_least16_t**
int_least32_t	**uint_least32_t**
int_least64_t	**uint_least64_t**

All other types of this form are optional.

7.22.1.3 Fastest minimum-width integer types

1 Each of the following types designates an integer type that is usually fastest[306] to operate with among all integer types that have at least the specified width.

2 The typedef name **int_fast**_N_**_t** designates the fastest signed integer type with a width of at least *N*. The typedef name **uint_fast**_N_**_t** designates the fastest unsigned integer type with a width of at least *N*.

3 The following types are required:

int_fast8_t	**uint_fast8_t**
int_fast16_t	**uint_fast16_t**
int_fast32_t	**uint_fast32_t**
int_fast64_t	**uint_fast64_t**

All other types of this form are optional.

7.22.1.4 Integer types capable of holding object pointers

1 The following type designates a signed integer type, other than a bit-precise integer type, with the property that any valid pointer to **void** can be converted to this type, then converted back to pointer to **void**, and the result will compare equal to the original pointer:

```
intptr_t
```

The following type designates an unsigned integer type, other than a bit-precise integer type, with the property that any valid pointer to **void** can be converted to this type, then converted back to pointer to **void**, and the result will compare equal to the original pointer:

```
uintptr_t
```

These types are optional.

7.22.1.5 Greatest-width integer types

1 The following type designates a signed integer type, other than a bit-precise integer type, capable of representing any value of any signed integer type with the possible exceptions of signed bit-precise integer types and of signed extended integer types that are wider than **long long** and that are referred by the type definition for an exact width integer type:

```
intmax_t
```

The following type designates the unsigned integer type that corresponds to **intmax_t**:[307]

```
uintmax_t
```

These types are required.

[306] The designated type is not guaranteed to be fastest for all purposes; if the implementation has no clear grounds for choosing one type over another, it will simply pick some integer type satisfying the signedness and width requirements.

[307] Thus this type is capable of representing any value of any unsigned integer type with the possible exception of bit-precise integer types and particular extended integer types that are wider than **unsigned long long**.

© ISO 2024 — All rights reserved

7.22.2 Widths of specified-width integer types

1 The following object-like macros specify the width of the types declared in `<stdint.h>`. Each macro name corresponds to a similar type name in 7.22.1.

2 Each instance of any defined macro shall be replaced by a constant expression suitable for use in **`#if`** preprocessing directives. Its implementation-defined value shall be equal to or greater than the value given in the subsequent subclauses, except where stated to be exactly the given value. An implementation shall define only the macros corresponding to those typedef names it actually provides.[308)]

7.22.2.1 Width of exact-width integer types

1

INT*N*_WIDTH	exactly *N*
UINT*N*_WIDTH	exactly *N*

7.22.2.2 Width of minimum-width integer types

1

INT_LEAST*N*_WIDTH	exactly **UINT_LEAST*N*_WIDTH**
UINT_LEAST*N*_WIDTH	*N*

7.22.2.3 Width of fastest minimum-width integer types

1

INT_FAST*N*_WIDTH	exactly **UINT_FAST*N*_WIDTH**
UINT_FAST*N*_WIDTH	*N*

7.22.2.4 Width of integer types capable of holding object pointers

1

INTPTR_WIDTH	exactly **UINTPTR_WIDTH**
UINTPTR_WIDTH	16

7.22.2.5 Width of greatest-width integer types

1

INTMAX_WIDTH	exactly **UINTMAX_WIDTH**
UINTMAX_WIDTH	64

7.22.3 Width of other integer types

1 The following object-like macros specify the width of integer types corresponding to types defined in other standard headers.

2 Each instance of these macros shall be replaced by a constant expression suitable for use in **`#if`** preprocessing directives. Its implementation-defined value shall be equal to or greater than the corresponding value given in the following subclauses. An implementation shall define only the macros corresponding to those typedef names it actually provides.[309)]

7.22.3.1 Width of `ptrdiff_t`

1

PTRDIFF_WIDTH	16

7.22.3.2 Width of `sig_atomic_t`

1

SIG_ATOMIC_WIDTH	8

[308)] The exact-width and pointer-holding integer types are optional.
[309)] A freestanding implementation need not provide all these types.

© ISO 2024 — All rights reserved

7.22.3.3 Width of `size_t`

1

```
SIZE_WIDTH                      16
```

7.22.3.4 Width of `wchar_t`

1

```
WCHAR_WIDTH                     8
```

7.22.3.5 Width of `wint_t`

1

```
WINT_WIDTH                      16
```

7.22.4 Macros for integer constants

1 The following function-like macros expand to integer constants suitable for initializing objects that have integer types corresponding to types defined in `<stdint.h>`. Each macro name corresponds to a similar type name in 7.22.1.2 or 7.22.1.5.

2 The argument in any instance of these macros shall be an unsuffixed integer constant (as defined in 6.4.4.2) with a value that does not exceed the limits for the corresponding type.

3 Each invocation of one of these macros shall expand to an integer constant expression. The type of the expression shall have the same type as would an expression of the corresponding type converted according to the integer promotions. The value of the expression shall be that of the argument. If the value is in the range of the type **`intmax_t`** (for a signed type) or the type **`uintmax_t`** (for an unsigned type), see 7.22.1.5, the expression is suitable for use in conditional expression inclusion preprocessing directives.

7.22.4.1 Macros for minimum-width integer constants

1 The macro **`INT`***N*`_C(`*value*`)` expands to an integer constant expression corresponding to the type **`int_least`***N*`_t`. The macro **`UINT`***N*`_C(`*value*`)` expands to an integer constant expression corresponding to the type **`uint_least`***N*`_t`. For example, if **`uint_least64_t`** is a name for the type **`unsigned long long int`**, then **`UINT64_C(0x123)`** can expand to the integer constant `0x123ULL`.

7.22.4.2 Macros for greatest-width integer constants

1 The following macro expands to an integer constant expression having the value specified by its argument and the type **`intmax_t`**:

```
INTMAX_C(value)
```

The following macro expands to an integer constant expression having the value specified by its argument and the type **`uintmax_t`**:

```
UINTMAX_C(value)
```

7.22.5 Maximal and minimal values of integer types

1 For all integer types for which there is a macro with suffix **`_WIDTH`** holding the width, maximum macros with suffix **`_MAX`** and, for all signed types, minimum macros with suffix **`_MIN`** are defined as by 5.2.5.3. If it is unspecified if a type is signed or unsigned and the implementation has it as an unsigned type, a minimum macro with extension **`_MIN`** and value `0` of the corresponding type, converted according to the integer promotions, is defined.

© ISO 2024 — All rights reserved

7.23 Input/output <stdio.h>

7.23.1 Introduction

1 The header <stdio.h> defines several macros, and declares three types and many functions for performing input and output.

2 The macro

```
__STDC_VERSION_STDIO_H__
```

is an integer constant expression with a value equivalent to 202311L.

3 The types declared are **size_t** (described in 7.21);

```
FILE
```

which is an object type capable of recording all the information needed to control a stream, including its file position indicator, a pointer to its associated buffer (if any), an *error indicator* that records whether a read/write error has occurred, and an *end-of-file indicator* that records whether the end of the file has been reached; and

```
fpos_t
```

which is a complete object type other than an array type capable of recording all the information needed to specify uniquely every position within a file.

4 The macros are **NULL** (described in 7.21);

```
_IOFBF
_IOLBF
_IONBF
```

which expand to integer constant expressions with distinct values, suitable for use as the third argument to the **setvbuf** function;

```
BUFSIZ
```

which expands to an integer constant expression that is the size of the buffer used by the **setbuf** function;

```
EOF
```

which expands to an integer constant expression, with type **int** and a negative value, that is returned by several functions to indicate *end-of-file*, that is, no more input from a stream;

```
FOPEN_MAX
```

which expands to an integer constant expression that is the minimum number of files that the implementation guarantees can be open simultaneously;

```
FILENAME_MAX
```

which expands to an integer constant expression that is the size needed for an array of **char** large enough to hold the longest file name string that the implementation guarantees can be opened or, if the implementation imposes no practical limit on the length of file name strings, the recommended size of an array intended to hold a file name string;[310)]

[310)] Of course, file name string contents are subject to other system-specific constraints; therefore *all* possible strings of length **FILENAME_MAX** cannot be expected to be opened successfully.

 © ISO 2024 — All rights reserved

```
_PRINTF_NAN_LEN_MAX
```

which expands to an integer constant expression (suitable for use in conditional expression inclusion preprocessing directives) that is the maximum number of characters output for any

[-]NAN(*n-char-sequence*)

sequence.[311] If an implementation has no support for NaNs, it shall be 0. **_PRINTF_NAN_LEN_MAX** shall be less than 64;

```
L_tmpnam
```

which expands to an integer constant expression that is the size needed for an array of **char** large enough to hold a temporary file name string generated by the **tmpnam** function;

```
SEEK_CUR
SEEK_END
SEEK_SET
```

which expand to integer constant expressions with distinct values, suitable for use as the third argument to the **fseek** function;

```
TMP_MAX
```

which expands to an integer constant expression that is the minimum number of unique file names that can be generated by the **tmpnam** function;

```
stderr
stdin
stdout
```

which are expressions of type "pointer to **FILE**" that point to the **FILE** objects associated, respectively, with the standard error, input, and output streams.

5 The header <wchar.h> declares functions for wide character input and output. The wide character input/output functions described in that subclause provide operations analogous to most of those described here, except that the fundamental units internal to the program are wide characters. The external representation (in the file) is a sequence of generalized multibyte characters, as described further in 7.23.3.

6 The input/output functions are given the following collective terms:

— The *wide character input functions* — those functions described in 7.31 that perform input into wide characters and wide strings: **fgetwc**, **fgetws**, **getwc**, **getwchar**, **fwscanf**, **wscanf**, **vfwscanf**, and **vwscanf**.

— The *wide character output functions* — those functions described in 7.31 that perform output from wide characters and wide strings: **fputwc**, **fputws**, **putwc**, **putwchar**, **fwprintf**, **wprintf**, **vfwprintf**, and **vwprintf**.

— The *wide character input/output functions* — the union of the **ungetwc** function, the wide character input functions, and the wide character output functions.

— The *byte input/output functions* — those functions described in this subclause that perform input/output: **fgetc**, **fgets**, **fprintf**, **fputc**, **fputs**, **fread**, **fscanf**, **fwrite**, **getc**, **getchar**, **printf**, **putc**, **putchar**, **puts**, **scanf**, **ungetc**, **vfprintf**, **vfscanf**, **vprintf**, and **vscanf**.

Forward references: files (7.23.3), the **fseek** function (7.23.9.2), streams (7.23.2), the **tmpnam** function (7.23.4.4), <wchar.h> (7.31).

[311] If the implementation only uses the [-]NAN style, then **_PRINTF_NAN_LEN_MAX** would have the value 4.

© ISO 2024 — All rights reserved

7.23.2 Streams

1 Input and output, whether to or from physical devices such as terminals and tape drives, or whether to or from files supported on structured storage devices, are mapped into logical data *streams*, whose properties are more uniform than their various inputs and outputs. Two forms of mapping are supported, for *text streams* and for *binary streams*.[312)]

2 A text stream is an ordered sequence of characters composed into *lines*, each line consisting of zero or more characters plus a terminating new-line character. Whether the last line requires a terminating new-line character is implementation-defined. Characters may have to be added, altered, or deleted on input and output to conform to differing conventions for representing text in the host environment. Thus, there is not required to be a one-to-one correspondence between the characters in a stream and those in the external representation. Data read in from a text stream will necessarily compare equal to the data that were earlier written out to that stream only if:

— the data consist only of printing characters and the control characters horizontal tab and new-line;

— no new-line character is immediately preceded by space characters;

— and, the last character is a new-line character.

Whether space characters that are written out immediately before a new-line character appear when read in is implementation-defined.

3 A binary stream is an ordered sequence of characters that can transparently record internal data. Data read in from a binary stream shall compare equal to the data that were earlier written out to that stream, under the same implementation. Such a stream may, however, have an implementation-defined number of null characters appended to the end of the stream.

4 Each stream has an *orientation*. After a stream is associated with an external file, but before any operations are performed on it, the stream is unoriented. Once a wide character input/output function has been applied to an unoriented stream, the stream becomes a *wide-oriented stream*. Similarly, once a byte input/output function has been applied to an unoriented stream, the stream becomes a *byte-oriented stream*. Only a call to the **freopen** function or the **fwide** function can otherwise alter the orientation of a stream. (A successful call to **freopen** removes any orientation.)[313)]

5 Byte input/output functions shall not be applied to a wide-oriented stream and wide character input/output functions shall not be applied to a byte-oriented stream. The remaining stream operations do not affect, and are not affected by, a stream's orientation, except for the following additional restrictions:

— Binary wide-oriented streams have the file-positioning restrictions ascribed to both text and binary streams.

— For wide-oriented streams, after a successful call to a file-positioning function that leaves the file position indicator prior to the end-of-file, a wide character output function can overwrite a partial multibyte character; any file contents beyond the byte(s) written may henceforth not consist of valid multibyte characters.

6 Each wide-oriented stream has an associated **mbstate_t** object that stores the current parse state of the stream. A successful call to **fgetpos** stores a representation of the value of this **mbstate_t** object as part of the value of the **fpos_t** object. A later successful call to **fsetpos** using the same stored **fpos_t** value restores the value of the associated **mbstate_t** object as well as the position within the controlled stream.

7 Each stream has an associated lock that is used to prevent data races when multiple threads of execution access a stream, and to restrict the interleaving of stream operations performed by multiple

[312)] An implementation need not distinguish between text streams and binary streams. In such an implementation, there need be no new-line characters in a text stream nor any limit to the length of a line.

[313)] The three predefined streams **stdin**, **stdout**, and **stderr** are unoriented at program startup.

 © ISO 2024 — All rights reserved

threads. Only one thread may hold this lock at a time. The lock is reentrant: a single thread may hold the lock multiple times at a given time.

8 All functions that read, write, position, or query the position of a stream lock the stream before accessing it. They release the lock associated with the stream when the access is complete.

Environmental limits

9 An implementation shall support text files with lines containing at least 254 characters, including the terminating new-line character. The value of the macro **`BUFSIZ`** shall be at least 256.

Forward references: the **`freopen`** function (7.23.5.4), the **`fwide`** function (7.31.3.5), **`mbstate_t`** (7.31.1), the **`fgetpos`** function (7.23.9.1), the **`fsetpos`** function (7.23.9.3).

7.23.3 Files

1 A stream is associated with an external file (which may be a physical device) by *opening* a file, which may involve *creating* a new file. Creating an existing file causes its former contents to be discarded, if necessary. If a file can support positioning requests (such as a disk file, as opposed to a terminal), then a *file position indicator* associated with the stream is positioned at the start (character number zero) of the file, unless the file is opened with append mode in which case it is implementation-defined whether the file position indicator is initially positioned at the beginning or the end of the file. The file position indicator is maintained by subsequent reads, writes, and positioning requests, to facilitate an orderly progression through the file.

2 Binary files are not truncated, except as defined in 7.23.5.3. Whether a write on a text stream causes the associated file to be truncated beyond that point is implementation-defined.

3 When a stream is *unbuffered*, characters are intended to appear from the source or at the destination as soon as possible. Otherwise characters may be accumulated and transmitted to or from the host environment as a block. When a stream is *fully buffered*, characters are intended to be transmitted to or from the host environment as a block when a buffer is filled. When a stream is *line buffered*, characters are intended to be transmitted to or from the host environment as a block when a new-line character is encountered. Furthermore, characters are intended to be transmitted as a block to the host environment when a buffer is filled, when input is requested on an unbuffered stream, or when input is requested on a line buffered stream that requires the transmission of characters from the host environment. Support for these characteristics is implementation-defined, and may be affected via the **`setbuf`** and **`setvbuf`** functions.

4 A file may be disassociated from a controlling stream by *closing* the file. Output streams are flushed (any unwritten buffer contents are transmitted to the host environment) before the stream is disassociated from the file. The lifetime of a **`FILE`** object ends when the associated file is closed (including the standard text streams). Whether a file of zero length (on which no characters have been written by an output stream) actually exists is implementation-defined.

5 The file may be subsequently reopened, by the same or another program execution, and its contents reclaimed or modified (if it can be repositioned at its start). If the **`main`** function returns to its original caller, or if the **`exit`** function is called, all open files are closed (hence all output streams are flushed) before program termination. Other paths to program termination, such as calling the **`abort`** function, are not required to close all files properly.

6 The address of the **`FILE`** object used to control a stream may be significant; a copy of a **`FILE`** object is not required to serve in place of the original.

7 At program startup, three text streams are predefined and are already opened — *standard input* (for reading conventional input), *standard output* (for writing conventional output), and *standard error* (for writing diagnostic output). As initially opened, the standard error stream is not fully buffered; the standard input and standard output streams are fully buffered if and only if the stream can be determined not to refer to an interactive device.

8 Functions that open additional (nontemporary) files require a *file name*, which is a string. The rules for composing valid file names are implementation-defined. Whether the same file can be simultaneously open multiple times is also implementation-defined.

© ISO 2024 — All rights reserved

9 Although both text and binary wide-oriented streams are conceptually sequences of wide characters, the external file associated with a wide-oriented stream is a sequence of multibyte characters, generalized as follows:

— Multibyte encodings within files may contain embedded null bytes (unlike multibyte encodings valid for use internal to the program).

— A file is not required to begin nor end in the initial shift state.[314)]

10 Moreover, the encodings used for multibyte characters may differ among files. Both the nature and choice of such encodings are implementation-defined.

11 The wide character input functions read multibyte characters from the stream and convert them to wide characters as if they were read by successive calls to the **fgetwc** function. Each conversion occurs as if by a call to the **mbrtowc** function, with the conversion state described by the stream's own **mbstate_t** object. The byte input functions read characters from the stream as if by successive calls to the **fgetc** function.

12 The wide character output functions convert wide characters to multibyte characters and write them to the stream as if they were written by successive calls to the **fputwc** function. Each conversion occurs as if by a call to the **wcrtomb** function, with the conversion state described by the stream's own **mbstate_t** object. The byte output functions write characters to the stream as if by successive calls to the **fputc** function.

13 In some cases, some of the byte input/output functions also perform conversions between multibyte characters and wide characters. These conversions also occur as if by calls to the **mbrtowc** and **wcrtomb** functions.

14 An *encoding error* occurs if the character sequence presented to the underlying **mbrtowc** function does not form a valid (generalized) multibyte character, or if the code value passed to the underlying **wcrtomb** does not correspond to a valid (generalized) multibyte character. The wide character input/output functions and the byte input/output functions store the value of the macro **EILSEQ** in **errno** if and only if an encoding error occurs.

Environmental limits

15 The value of **FOPEN_MAX** shall be at least eight, including the three standard text streams.

Forward references: the **exit** function (7.24.4.4), the **fgetc** function (7.23.7.1), the **fopen** function (7.23.5.3), the **fputc** function (7.23.7.3), the **setbuf** function (7.23.5.5), the **setvbuf** function (7.23.5.6), the **fgetwc** function (7.31.3.1), the **fputwc** function (7.31.3.3), conversion state (7.31.6), the **mbrtowc** function (7.31.6.3.2), the **wcrtomb** function (7.31.6.3.3).

7.23.4 Operations on files

7.23.4.1 The remove function

Synopsis

1
```
#include <stdio.h>
int remove(const char *filename);
```

Description

2 The **remove** function causes the file whose name is the string pointed to by `filename` to be no longer accessible by that name. A subsequent attempt to open that file using that name will fail, unless it is created anew. If the file is open, the behavior of the **remove** function is implementation-defined.

Returns

3 The **remove** function returns zero if the operation succeeds, nonzero if it fails.

[314)] Setting the file position indicator to end-of-file, as with **fseek**(`file, 0, SEEK_END`), has undefined behavior for a binary stream (because of possible trailing null characters) or for any stream with state-dependent encoding that does not assuredly end in the initial shift state.

 © ISO 2024 — All rights reserved

7.23.4.2 The `rename` function

Synopsis

1
```
#include <stdio.h>
int rename(const char *old, const char *new);
```

Description

2 The **rename** function causes the file whose name is the string pointed to by `old` to be henceforth known by the name given by the string pointed to by `new`. The file named `old` is no longer accessible by that name. If a file named by the string pointed to by `new` exists prior to the call to the **rename** function, the behavior is implementation-defined.

Returns

3 The **rename** function returns zero if the operation succeeds, nonzero if it fails,[315)] in which case if the file existed previously it is still known by its original name.

7.23.4.3 The `tmpfile` function

Synopsis

1
```
#include <stdio.h>
FILE *tmpfile(void);
```

Description

2 The **tmpfile** function creates a temporary binary file that is different from any other existing file and that will automatically be removed when it is closed or at program termination. If the program terminates abnormally, whether an open temporary file is removed is implementation-defined. The file is opened for update with `"wb+"` mode.

Recommended practice

3 It should be possible to open at least **TMP_MAX** temporary files during the lifetime of the program (this limit may be shared with **tmpnam**) and there should be no limit on the number simultaneously open other than this limit and any limit on the number of open files (**FOPEN_MAX**).

Returns

4 The **tmpfile** function returns a pointer to the stream of the file that it created. If the file cannot be created, the **tmpfile** function returns a null pointer.

Forward references: the **fopen** function (7.23.5.3).

7.23.4.4 The `tmpnam` function

Synopsis

1
```
#include <stdio.h>
char *tmpnam(char *s);
```

Description

2 The **tmpnam** function generates a string that is a valid file name and that is not the same as the name of an existing file.[316)] The function is potentially capable of generating at least **TMP_MAX** different strings, but any or all of them may already be in use by existing files and thus not be suitable return values.

3 The **tmpnam** function generates a different string each time it is called.

[315)] Among the reasons the implementation could cause the **rename** function to fail are that the file is open or that it is necessary to copy its contents to effectuate its renaming.

[316)] Files created using strings generated by the **tmpnam** function are temporary only in the sense that their names are not expected to collide with those generated by conventional naming rules for the implementation. It is still necessary to use the **remove** function to remove such files when their use is ended, and before program termination.

© ISO 2024 — All rights reserved

4 Calls to the **tmpnam** function with a null pointer argument may introduce data races with each other. The implementation shall behave as if no library function calls the **tmpnam** function.

Returns

5 If no suitable string can be generated, the **tmpnam** function returns a null pointer. Otherwise, if the argument is a null pointer, the **tmpnam** function leaves its result in an internal static object and returns a pointer to that object (subsequent calls to the **tmpnam** function may modify the same object). If the argument is not a null pointer, it is assumed to point to an array of at least **L_tmpnam** **char**s; the **tmpnam** function writes its result in that array and returns the argument as its value.

Environmental limits

6 The value of the macro **TMP_MAX** shall be at least 25.

7.23.5 File access functions

7.23.5.1 The fclose function

Synopsis

1
```
#include <stdio.h>
int fclose(FILE *stream);
```

Description

2 A successful call to the **fclose** function causes the stream pointed to by stream to be flushed and the associated file to be closed. Any unwritten buffered data for the stream are delivered to the host environment to be written to the file; any unread buffered data are discarded. Whether the call succeeds or not, the stream is disassociated from the file and any buffer set by the **setbuf** or **setvbuf** function is disassociated from the stream (and deallocated if it was automatically allocated).

Returns

3 The **fclose** function returns zero if the stream was successfully closed, or **EOF** if any errors were detected.

7.23.5.2 The fflush function

Synopsis

1
```
#include <stdio.h>
int fflush(FILE *stream);
```

Description

2 If stream points to an output stream or an update stream in which the most recent operation was not input, the **fflush** function causes any unwritten data for that stream to be delivered to the host environment to be written to the file; otherwise, the behavior is undefined.

3 If stream is a null pointer, the **fflush** function performs this flushing action on all streams for which the behavior is defined previously in this subclause.

Returns

4 The **fflush** function sets the error indicator for the stream and returns **EOF** if a write error occurs, otherwise it returns zero.

Forward references: the **fopen** function (7.23.5.3).

7.23.5.3 The fopen function

Synopsis

1
```
#include <stdio.h>
FILE *fopen(const char * restrict filename, const char * restrict mode);
```

 © ISO 2024 — All rights reserved

Description

2 The **fopen** function opens the file whose name is the string pointed to by `filename`, and associates a stream with it.

3 The argument `mode` points to a string. If the string is one of the following, the file is open in the indicated mode. Otherwise, the behavior is undefined.[317)]

`r`	open text file for reading
`w`	truncate to zero length or create text file for writing
`wx`	create text file for writing
`a`	append; open or create text file for writing at end-of-file
`rb`	open binary file for reading
`wb`	truncate to zero length or create binary file for writing
`wbx`	create binary file for writing
`ab`	append; open or create binary file for writing at end-of-file
`r+`	open text file for update (reading and writing)
`w+`	truncate to zero length or create text file for update
`w+x`	create text file for update
`a+`	append; open or create text file for update, writing at end-of-file
`r+b` *or* `rb+`	open binary file for update (reading and writing)
`w+b` *or* `wb+`	truncate to zero length or create binary file for update
`w+bx` *or* `wb+x`	create binary file for update
`a+b` *or* `ab+`	append; open or create binary file for update, writing at end-of-file

4 Opening a file with read mode (`'r'` as the first character in the `mode` argument) fails if the file does not exist or cannot be read.

5 Opening a file with exclusive mode (`'x'` as the last character in the `mode` argument) fails if the file already exists or cannot be created. The check for the existence of the file and the creation of the file if it does not exist is atomic with respect to other threads and other concurrent program executions. If the implementation is not capable of performing the check for the existence of the file and the creation of the file atomically, it shall fail instead of performing a non-atomic check and creation.

6 Opening a file with append mode (`'a'` as the first character in the `mode` argument) causes all subsequent writes to the file to be forced to the then current end-of-file at the point of buffer flush or actual write, regardless of intervening calls to the **fseek**, **fsetpos**, or **rewind** functions. Incrementing the current end-of-file by the amount of data written is atomic with respect to other threads writing to the same file provided the file was also opened in append mode. If the implementation is not capable of incrementing the current end-of-file atomically, it shall fail instead of performing non-atomic end-of-file writes. In some implementations, opening a binary file with append mode (`'b'` as the second or third character in the previously described list of `mode` argument values) may initially position the file position indicator for the stream beyond the last data written, because of null character padding.

7 When a file is opened with update mode (`'+'` as the second or third character in the previously described list of `mode` argument values), both input and output may be performed on the associated stream. However, output shall not be directly followed by input without an intervening call to the **fflush** function or to a file positioning function (**fseek**, **fsetpos**, or **rewind**), and input shall not be directly followed by output without an intervening call to a file positioning function, unless the input operation encounters end-of-file. Opening (or creating) a text file with update mode may instead open (or create) a binary stream in some implementations.

8 When opened, a stream is fully buffered if and only if it can be determined not to refer to an interactive device. The error and end-of-file indicators for the stream are cleared.

[317)] If the string begins with one of the listed mode sequences, the implementation can choose to ignore the remaining characters, or it can use them to select different kinds of a file (some of which may not conform to the properties in 7.23.2).

© ISO 2024 — All rights reserved

Returns

9 The **fopen** function returns a pointer to the object controlling the stream. If the open operation fails, **fopen** returns a null pointer.

Forward references: file positioning functions (7.23.9).

7.23.5.4 The freopen function

Synopsis

1
```
#include <stdio.h>
FILE *freopen(const char * restrict filename, const char * restrict mode,
      FILE * restrict stream);
```

Description

2 The **freopen** function opens the file whose name is the string pointed to by `filename` and associates the stream pointed to by `stream` with it. The `mode` argument is used just as in the **fopen** function.[318)]

3 If `filename` is a null pointer, the **freopen** function attempts to change the mode of the stream to that specified by `mode`, as if the name of the file currently associated with the stream had been used. It is implementation-defined which changes of mode are permitted (if any), and under what circumstances.

4 The **freopen** function first attempts to close any file that is associated with the specified stream. Failure to close the file is ignored. The error and end-of-file indicators for the stream are cleared.

Returns

5 The **freopen** function returns a null pointer if the open operation fails. Otherwise, **freopen** returns the value of `stream`.

7.23.5.5 The setbuf function

Synopsis

1
```
#include <stdio.h>
void setbuf(FILE * restrict stream, char * restrict buf);
```

Description

2 Except that it returns no value, the **setbuf** function is equivalent to the **setvbuf** function invoked with the values **_IOFBF** for `mode` and **BUFSIZ** for `size`, or (if `buf` is a null pointer), with the value **_IONBF** for `mode`.

Returns

3 The **setbuf** function returns no value.

Forward references: the **setvbuf** function (7.23.5.6).

7.23.5.6 The setvbuf function

Synopsis

1
```
#include <stdio.h>
int setvbuf(FILE * restrict stream, char * restrict buf, int mode, size_t size);
```

Description

2 The **setvbuf** function may be used only after the stream pointed to by `stream` has been associated with an open file and before any other operation (other than an unsuccessful call to **setvbuf**) is performed on the stream. The argument `mode` determines how `stream` will be buffered, as follows:

[318)] The primary use of the **freopen** function is to change the file associated with a standard text stream (**stderr**, **stdin**, or **stdout**), as those identifiers need not be modifiable lvalues to which the value returned by the **fopen** function could be assigned.

 © ISO 2024 — All rights reserved

_IOFBF causes input/output to be fully buffered;

_IOLBF causes input/output to be line buffered;

_IONBF causes input/output to be unbuffered.

If `buf` is not a null pointer, the array it points to may be used instead of a buffer allocated by the **`setvbuf`** function[319)] and the argument `size` specifies the size of the array; otherwise, `size` may determine the size of a buffer allocated by the **`setvbuf`** function. The members of the array at any time have unspecified values.

Returns

3 The **`setvbuf`** function returns zero on success, or nonzero if an invalid value is given for `mode` or if the request cannot be honored.

7.23.6 Formatted input/output functions

1 The formatted input/output functions shall behave as if there is a sequence point after the actions associated with each specifier.[320)]

7.23.6.1 The `fprintf` function

Synopsis

1
```
#include <stdio.h>
int fprintf(FILE * restrict stream, const char * restrict format, ...);
```

Description

2 The **`fprintf`** function writes output to the stream pointed to by `stream`, under control of the string pointed to by `format` that specifies how subsequent arguments are converted for output. If there are insufficient arguments for the format, the behavior is undefined. If the format is exhausted while arguments remain, the excess arguments are evaluated (as always) but are otherwise ignored. The **`fprintf`** function returns when the end of the format string is encountered.

3 The format shall be a multibyte character sequence, beginning and ending in its initial shift state. The format is composed of zero or more directives: ordinary multibyte characters (not `%`), which are copied unchanged to the output stream; and conversion specifications, each of which results in fetching zero or more subsequent arguments, converting them, if applicable, according to the corresponding conversion specifier, and then writing the result to the output stream.

4 Each conversion specification is introduced by the character `%`. After the `%`, the following appear in sequence:

— Zero or more *flags* (in any order) that modify the meaning of the conversion specification.

— An optional minimum *field width*. If the converted value has fewer characters than the field width, it is padded with spaces (by default) on the left (or right, if the left adjustment flag, described later, has been given) to the field width. The field width takes the form of an asterisk `*` (described later) or a nonnegative decimal integer.[321)]

— An optional *precision* that gives the minimum number of digits to appear for the `b`, `B`, `d`, `i`, `o`, `u`, `x`, and `X` conversions, the number of digits to appear after the decimal-point character for `a`, `A`, `e`, `E`, `f`, and `F` conversions, the maximum number of significant digits for the `g` and `G` conversions, or the maximum number of bytes to be written for `s` conversions. The precision takes the form of a period (`.`) followed either by an asterisk `*` (described later) or by an optional nonnegative decimal integer; if only the period is specified, the precision is taken as zero. If a precision appears with any other conversion specifier, the behavior is undefined.

[319)] The buffer has to have a lifetime at least as great as the open stream, so not closing the stream before a buffer that has automatic storage duration is deallocated upon block exit results in undefined behavior.
[320)] The **`fprintf`** functions perform writes to memory for the `%n` specifier.
[321)] Note that `0` is taken as a flag, not as the beginning of a field width.

© ISO 2024 — All rights reserved

— An optional *length modifier* that specifies the size of the argument.

— A *conversion specifier* character that specifies the type of conversion to be applied.

5 As noted previously, a field width, or precision, or both, may be indicated by an asterisk. In this case, an **`int`** argument supplies the field width or precision. The arguments specifying field width, or precision, or both, shall appear (in that order) before the argument (if any) to be converted. A negative field width argument is taken as a `-` flag followed by a positive field width. A negative precision argument is taken as if the precision were omitted.

6 The flag characters and their meanings are:

`-` The result of the conversion is left-justified within the field. (It is right-justified if this flag is not specified.)

`+` The result of a signed conversion always begins with a plus or minus sign. (It begins with a sign only when a value with a negative sign is converted if this flag is not specified.)[322)]

space If the first character of a signed conversion is not a sign, or if a signed conversion results in no characters, a space is prefixed to the result. If the *space* and `+` flags both appear, the *space* flag is ignored.

`#` The result is converted to an "alternative form". For `o` conversion, it increases the precision, if and only if necessary, to force the first digit of the result to be a zero (if the value and precision are both 0, a single 0 is printed). For `b` conversion, a nonzero result has `0b` prefixed to it. For the optional `B` conversion as described later in this subclause, a nonzero result has `0B` prefixed to it. For `x` (or `X`) conversion, a nonzero result has `0x` (or `0X`) prefixed to it. For `a`, `A`, `e`, `E`, `f`, `F`, `g`, and `G` conversions, the result of converting a floating-point number always contains a decimal-point character, even if no digits follow it. (Normally, a decimal-point character appears in the result of these conversions only if a digit follows it.) For `g` and `G` conversions, trailing zeros are *not* removed from the result. For other conversions, the behavior is undefined.

`0` For `b`, `B`, `d`, `i`, `o`, `u`, `x`, `X`, `a`, `A`, `e`, `E`, `f`, `F`, `g`, and `G` conversions, leading zeros (following any indication of sign or base) are used to pad to the field width rather than performing space padding, except when converting an infinity or NaN. If the `0` and `-` flags both appear, the `0` flag is ignored. For `b`, `B`, `d`, `i`, `o`, `u`, `x`, and `X` conversions, if a precision is specified, the `0` flag is ignored. For other conversions, the behavior is undefined.

7 The length modifiers and their meanings are:

`hh` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`signed char`** or **`unsigned char`** argument (the argument will have been promoted according to the integer promotions, but its value shall be converted to **`signed char`** or **`unsigned char`** before printing); or that a following `n` conversion specifier applies to a pointer to a **`signed char`** argument.

`h` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`short int`** or **`unsigned short int`** argument (the argument will have been promoted according to the integer promotions, but its value shall be converted to **`short int`** or **`unsigned short int`** before printing); or that a following `n` conversion specifier applies to a pointer to a **`short int`** argument.

`l` (ell) Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`long int`** or **`unsigned long int`** argument; that a following `n` conversion specifier applies to a pointer to a **`long int`** argument; that a following `c` conversion specifier applies to a **`wint_t`** argument; that a following `s` conversion specifier applies to a pointer to a **`wchar_t`** argument; or has no effect on a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier.

[322)] The results of all floating conversions of a negative zero, and of negative values that round to zero, include a minus sign.

© ISO 2024 — All rights reserved

`ll` (ell-ell) Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`long long int`** or **`unsigned long long int`** argument; or that a following `n` conversion specifier applies to a pointer to a **`long long int`** argument.

`j` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to an **`intmax_t`** or **`uintmax_t`** argument; or that a following `n` conversion specifier applies to a pointer to an **`intmax_t`** argument.

`z` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`size_t`** or the corresponding signed integer type argument; or that a following `n` conversion specifier applies to a pointer to a signed integer type corresponding to **`size_t`** argument.

`t` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`ptrdiff_t`** or the corresponding unsigned integer type argument; or that a following `n` conversion specifier applies to a pointer to a **`ptrdiff_t`** argument.

`w`*N* Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to an integer argument with a specific width where *N* is a positive decimal integer with no leading zeros (the argument will have been promoted according to the integer promotions, but its value shall be converted to the unpromoted type); or that a following `n` conversion specifier applies to a pointer to an integer type argument with a width of *N* bits. All minimum-width integer types (7.22.1.2) and exact-width integer types (7.22.1.1) defined in the header `<stdint.h>` shall be supported. Other supported values of *N* are implementation-defined.

`wf`*N* Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a fastest minimum-width integer argument with a specific width where *N* is a positive decimal integer with no leading zeros (the argument will have been promoted according to the integer promotions, but its value shall be converted to the unpromoted type); or that a following `n` conversion specifier applies to a pointer to a fastest minimum-width integer type argument with a width of *N* bits. All fastest minimum-width integer types (7.22.1.3) defined in the header `<stdint.h>` shall be supported. Other supported values of *N* are implementation-defined.

`L` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to a **`long double`** argument.

`H` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to a **`_Decimal32`** argument.

`D` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to a **`_Decimal64`** argument.

`DD` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to a **`_Decimal128`** argument.

If a length modifier appears with any conversion specifier other than as specified previously, the behavior is undefined.

8 The conversion specifiers and their meanings are:

`d,i` The **`int`** argument is converted to signed decimal in the style *[-]dddd*. The precision specifies the minimum number of digits to appear; if the value being converted can be represented in fewer digits, it is expanded with leading zeros. The default precision is 1. The result of converting a zero value with a precision of zero is no characters.

`b,B,o,u,x,X` The **`unsigned int`** argument is converted to unsigned binary (`b` or `B`), unsigned octal (`o`), unsigned decimal (`u`), or unsigned hexadecimal notation (`x` or `X`) in the style *dddd*; the letters `abcdef` are used for `x` conversion and the letters `ABCDEF` for `X` conversion. The precision specifies the minimum number of digits to appear; if the value being converted can be represented in fewer digits, it is expanded with leading zeros. The default precision

© ISO 2024 — All rights reserved

is 1. The result of converting a zero value with a precision of zero is no characters. The specifier `B` is optional and provides the same functionality as `b`, except for the **#** flag as previously specified. The **PRIB** macros from `<inttypes.h>` shall only be defined if the implementation follows the specification as given here.

`f,F` A **double** argument representing a floating-point number is converted to decimal notation in the style *[-]ddd.ddd*, where the number of digits after the decimal-point character is equal to the precision specification. If the precision is missing, it is taken as 6; if the precision is zero and the **#** flag is not specified, no decimal-point character appears. If a decimal-point character appears, at least one digit appears before it. The value is rounded to the appropriate number of digits.

A **double** argument representing an infinity is converted in one of the styles *[-]*`inf` or *[-]*`infinity` — which style is implementation-defined. A **double** argument representing a NaN is converted in one of the styles *[-]*`nan` or *[-]*`nan(`*n-char-sequence*`)` — which style, and the meaning of any *n-char-sequence*, is implementation-defined. The `F` conversion specifier produces `INF`, `INFINITY`, or `NAN` instead of `inf`, `infinity`, or `nan`, respectively.[323)]

`e,E` A **double** argument representing a floating-point number is converted in the style *[-]d.ddd*`e`±*dd*, where there is one digit (which is nonzero if the argument is nonzero) before the decimal-point character and the number of digits after it is equal to the precision; if the precision is missing, it is taken as 6; if the precision is zero and the **#** flag is not specified, no decimal-point character appears. The value is rounded to the appropriate number of digits. The `E` conversion specifier produces a number with `E` instead of `e` introducing the exponent. The exponent always contains at least two digits, and only as many more digits as necessary to represent the exponent. If the value is zero, the exponent is zero.

A **double** argument representing an infinity or NaN is converted in the style of an `f` or `F` conversion specifier.

`g,G` A **double** argument representing a floating-point number is converted in style `f` or `e` (or in style `F` or `E` in the case of a `G` conversion specifier), depending on the value converted and the precision. Let P equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style `E` would have an exponent of X:

if $P > X \geq -4$, the conversion is with style `f` (or `F`) and precision $P - (X + 1)$.

otherwise, the conversion is with style `e` (or `E`) and precision $P - 1$.

Finally, unless the **#** flag is used, any trailing zeros are removed from the fractional portion of the result and the decimal-point character is removed if there is no fractional portion remaining.

A **double** argument representing an infinity or NaN is converted in the style of an `f` or `F` conversion specifier.

`a,A` A **double** argument representing a floating-point number is converted in the style *[-]*`0x`*h.hhhh*`p`±*d*, where there is one hexadecimal digit (which is nonzero if the argument is a normalized floating-point number and is otherwise unspecified) before the decimal-point character [324)] and the number of hexadecimal digits after it is equal to the precision; if the

[323)] When applied to infinite and NaN values, the `-`, `+`, and *space* flag characters have their usual meaning; the **#** and `0` flag characters have no effect.

[324)] Binary implementations can choose the hexadecimal digit to the left of the decimal-point character so that subsequent digits align to nibble (4-bit) boundaries. This implementation choice affects numerical values printed with a precision P that is insufficient to represent all values exactly. Implementations with different conventions about the most significant hexadecimal digit will round at different places, affecting the numerical value of the hexadecimal result. For example, possible printed output for the code

```
#include <stdio.h>
/* ... */
double x = 123.0;
printf("%.1a", x);
```

include "`0x1.fp+6` " and "`0xf.6p+3` " whose numerical values are 124 and 123, respectively. Portable code seeking identical numerical results on different platforms should avoid precisions P that require rounding.

 © ISO 2024 — All rights reserved

precision is missing and **FLT_RADIX** is a power of 2, then the precision is sufficient for an exact representation of the value; if the precision is missing and **FLT_RADIX** is not a power of 2, then the precision is sufficient to distinguish[325] values of type **double**, except that trailing zeros may be omitted; if the precision is zero and the **#** flag is not specified, no decimal-point character appears. The letters `abcdef` are used for `a` conversion and the letters `ABCDEF` for `A` conversion. The `A` conversion specifier produces a number with `X` and `P` instead of `x` and `p`. The exponent always contains at least one digit, and only as many more digits as necessary to represent the decimal exponent of 2. If the value is zero, the exponent is zero.

A **double** argument representing an infinity or NaN is converted in the style of an `f` or `F` conversion specifier.

If an `H`, `D`, or `DD` modifier is present and the precision is missing, then for a decimal floating type argument represented by a triple of integers (s, c, q), where n is the number of significant digits in the coefficient c,

— if $-(n + 5) \leq q \leq 0$, use style `f` (or style `F` in the case of an `A` conversion specifier) with formatting precision equal to $-q$,

— otherwise, use style `e` (or style `E` in the case of an `A` conversion specifier) with formatting precision equal to $n - 1$, with the exceptions that if $c = 0$ then the digit-sequence in the exponent-part shall have the value q (rather than 0), and that the exponent is always expressed with the minimum number of digits required to represent its value (the exponent never contains a leading zero).

If the precision P is present (in the conversion specification) and is zero or at least as large as the precision p (5.2.5.3.3) of the decimal floating type, the conversion is as if the precision were missing. If the precision P is present (and nonzero) and less than the precision p of the decimal floating type, the conversion first obtains an intermediate result as follows, where n is the number of significant digits in the coefficient:

— If $n \leq P$, set the intermediate result to the input.

— If $n > P$, round the input value, according to the current rounding direction for decimal floating-point operations, to P decimal digits, with unbounded exponent range, representing the result with a P-digit integer coefficient when in the form (s, c, q).

Convert the intermediate result in the manner described previously for the case where the precision is missing.

`c` If no `l` length modifier is present, the **int** argument is converted to an **unsigned char**, and the resulting character is written.

If an `l` length modifier is present, the **wint_t** argument is converted as if by a call to the **wcrtomb** function with a pointer to storage of at least **MB_CUR_MAX** bytes, the **wint_t** argument converted to **wchar_t**, and an initial shift state.

`s` If no `l` length modifier is present, the argument shall be a pointer to storage of character type.[326] Characters from the storage are written up to (but not including) the terminating null character. If the precision is specified, no more than that many bytes are written. If the precision is not specified or is greater than the size of the storage, the storage shall contain a null character.

If an `l` length modifier is present, the argument shall be a pointer to storage of **wchar_t** type. Wide characters from the storage are converted to multibyte characters (each as if by a call to the **wcrtomb** function, with the conversion state described by an **mbstate_t** object initialized to zero before the first wide character is converted) up to and including

[325] The formatting precision P is sufficient to distinguish values of the source type if $16^P > b^p$ where b (not a power of 2) and p are the base and precision of the source type (5.2.5.3.3). A smaller P potentially suffices depending on the implementation's scheme for determining the digit to the left of the decimal-point character.

[326] No special provisions are made for multibyte characters.

© ISO 2024 — All rights reserved

a terminating null wide character. The resulting multibyte characters are written up to (but not including) the terminating null character (byte). If no precision is specified, the storage shall contain a null wide character. If a precision is specified, no more than that many bytes are written (including shift sequences, if any), and the storage shall contain a null wide character if, to equal the multibyte character sequence length given by the precision, the function would need to access a wide character one past the end of the array. In no case is a partial multibyte character written.[327)]

`p` The argument shall be a pointer to **`void`** or a pointer to a character type. The value of the pointer is converted to a sequence of printing characters, in an implementation-defined manner.

`n` The argument shall be a pointer to signed integer whose type is specified by the length modifier, if any, for the conversion specification, or shall be **`int`** if no length modifier is specified for the conversion specification. The number of characters written to the output stream so far by this call to **`fprintf`** is stored into the integer object pointed to by the argument. No argument is converted, but one is consumed. If the conversion specification includes any flags, a field width, or a precision, the behavior is undefined.

`%` A `%` character is written. No argument is converted. The complete conversion specification shall be `%%`.

9 If a conversion specification is invalid, the behavior is undefined.[328)] **`fprintf`** shall behave as if it uses **`va_arg`** with a type argument naming the type resulting from applying the default argument promotions to the type corresponding to the conversion specification and then converting the result of the **`va_arg`** expansion to the type corresponding to the conversion specification.[329)]

10 In no case does a nonexistent or small field width cause truncation of a field; if the result of a conversion is wider than the field width, the field is expanded to contain the conversion result.

11 For `a` and `A` conversions, if **`FLT_RADIX`** is a power of 2, the value is correctly rounded to a hexadecimal floating number with the given precision.

Recommended practice

12 For `a` and `A` conversions, if **`FLT_RADIX`** is not a power of 2 and the result is not exactly representable in the given precision, the result should be one of the two adjacent numbers in hexadecimal floating style with the given precision, with the extra stipulation that the error should have a correct sign for the current rounding direction.

13 For `e`, `E`, `f`, `F`, `g`, and `G` conversions, if the number of significant decimal digits is at most the maximum value M of the T**`_DECIMAL_DIG`** macros (defined in `<float.h>`), then the result should be correctly rounded.[330)] If the number of significant decimal digits is more than M but the source value is exactly representable with M digits, then the result should be an exact representation with trailing zeros. Otherwise, the source value is bounded by two adjacent decimal strings $L < U$, both having M significant digits; the value of the resultant decimal string D should satisfy $L \leq D \leq U$, with the extra stipulation that the error should have a correct sign for the current rounding direction.

14 The uppercase `B` format specifier is made optional by the previous description, because it used to be available for extensions in previous versions of this document. Implementations that did not use an uppercase `B` as their own extension before are encouraged to implement it as previously described.

Returns

15 The **`fprintf`** function returns the number of characters transmitted, or a negative value if an output or encoding error occurred or if the implementation does not support a specified width length modifier.

[327)] Redundant shift sequences can result if multibyte characters have a state-dependent encoding.
[328)] See "future library directions" (7.33.15).
[329)] The behavior is undefined when the types differ as specified for **`va_arg`** 7.16.1.1.
[330)] For binary-to-decimal conversion, the result format's values are the numbers representable with the given format specifier. The number of significant digits is determined by the format specifier, and in the case of fixed-point conversion by the source value as well.

 © ISO 2024 — All rights reserved

Environmental limits

16 The number of characters that can be produced by any single conversion shall be at least 4095.

17 EXAMPLE 1 To print a date and time in the form “Sunday, July 3, 10:02” followed by π to five decimal places:

```
#include <math.h>
#include <stdio.h>
/* ... */
char *weekday, *month;    // pointers to strings
int day, hour, min;
fprintf(stdout, "%s, %s %d, %.2d:%.2d\n",
      weekday, month, day, hour, min);
fprintf(stdout, "pi = %.5f\n", 4 * atan(1.0));
```

18 EXAMPLE 2 In this example, multibyte characters do not have a state-dependent encoding, and the members of the extended character set that consist of more than one byte each consist of exactly two bytes, the first of which is denoted here by a □ and the second by an uppercase letter.

19 Given the following wide string with length seven,

```
static wchar_t wstr[] = L"□X□Yabc□Z□W";
```

the seven calls

```
fprintf(stdout, "|1234567890123|\n");
fprintf(stdout, "|%13ls|\n", wstr);
fprintf(stdout, "|%-13.9ls|\n", wstr);
fprintf(stdout, "|%13.10ls|\n", wstr);
fprintf(stdout, "|%13.11ls|\n", wstr);
fprintf(stdout, "|%13.15ls|\n", &wstr[2]);
fprintf(stdout, "|%13lc|\n", (wint_t) wstr[5]);
```

will print the following seven lines:

```
|1234567890123|
|  □X□Yabc□Z□W|
|□X□Yabc□Z    |
|    □X□Yabc□Z|
|  □X□Yabc□Z□W|
|      abc□Z□W|
|           □Z|
```

20 EXAMPLE 3 Following are representations of **_Decimal64** arguments as triples (s, c, q) and the corresponding character sequences **fprintf** produces with "%Da":

© ISO 2024 — All rights reserved

$(+1, 123, 0)$	`123`
$(-1, 123, 0)$	`-123`
$(+1, 123, -2)$	`1.23`
$(+1, 123, 1)$	`1.23e+3`
$(-1, 123, 1)$	`-1.23e+3`
$(+1, 123, -8)$	`0.00000123`
$(+1, 123, -9)$	`1.23e-7`
$(+1, 120, -8)$	`0.00000120`
$(+1, 120, -9)$	`1.20e-7`
$(+1, 1234567890123456, 0)$	`1234567890123456`
$(+1, 1234567890123456, 1)$	`1.234567890123456e+16`
$(+1, 1234567890123456, -1)$	`123456789012345.6`
$(+1, 1234567890123456, -21)$	`0.000001234567890123456`
$(+1, 1234567890123456, -22)$	`1.234567890123456e-7`
$(+1, 0, 0)$	`0`
$(-1, 0, 0)$	`-0`
$(+1, 0, -6)$	`0.000000`
$(+1, 0, -7)$	`0e-7`
$(+1, 0, 2)$	`0e+2`
$(+1, 5, -6)$	`0.000005`
$(+1, 50, -7)$	`0.0000050`
$(+1, 5, -7)$	`5e-7`

To illustrate the effects of a precision specification, the sequence:

```
      _Decimal32 x = 6543.00DF;        // (+1, 654300, -2)
      fprintf(stdout, "%Ha\n", x);
      fprintf(stdout, "%.6Ha\n", x);
      fprintf(stdout, "%.5Ha\n", x);
      fprintf(stdout, "%.4Ha\n", x);
      fprintf(stdout, "%.3Ha\n", x);
      fprintf(stdout, "%.2Ha\n", x);
      fprintf(stdout, "%.1Ha\n", x);
      fprintf(stdout, "%.0Ha\n", x);
```

assuming default rounding, results in:

```
6543.00
6543.00
6543.0
6543
6.54e+3
6.5e+3
7e+3
6543.00
```

To illustrate the effects of the exponent range, the sequence:

```
      _Decimal32 x = 9543210e87DF;    // (+1, 9543210, 87)
      _Decimal32 y = 9500000e90DF;    // (+1, 9500000, 90)
      fprintf(stdout, "%.6Ha\n", x);
      fprintf(stdout, "%.5Ha\n", x);
      fprintf(stdout, "%.4Ha\n", x);
      fprintf(stdout, "%.3Ha\n", x);
      fprintf(stdout, "%.2Ha\n", x);
      fprintf(stdout, "%.1Ha\n", x);
      fprintf(stdout, "%.1Ha\n", y);
```

assuming default rounding, results in:

© ISO 2024 — All rights reserved

```
9.54321e+93
9.5432e+93
9.543e+93
9.54e+93
9.5e+93
1e+94
1e+97
```

To further illustrate the effects of the exponent range, the sequence:

```
_Decimal32 x = 9512345e90DF;     // (+1, 9512345, 90)
_Decimal32 y = 9512345e86DF;     // (+1, 9512345, 86)
fprintf(stdout, "%.3Ha\n", x);
fprintf(stdout, "%.2Ha\n", x);
fprintf(stdout, "%.1Ha\n", x);
fprintf(stdout, "%.2Ha\n", y);
```

assuming default rounding, results in:

```
9.51e+96
9.5e+96
1e+97
9.5e+92
```

Forward references: conversion state (7.31.6), the `wcrtomb` function (7.31.6.3.3).

7.23.6.2 The `fscanf` function

Synopsis

1

```
#include <stdio.h>
int fscanf(FILE * restrict stream, const char * restrict format, ...);
```

Description

2 The `fscanf` function reads input from the stream pointed to by `stream`, under control of the string pointed to by `format` that specifies the admissible input sequences and how they are to be converted for assignment, using subsequent arguments as pointers to the objects to receive the converted input. If there are insufficient arguments for the format, the behavior is undefined. If the format is exhausted while arguments remain, the excess arguments are evaluated (as always) but are otherwise ignored.

3 The format shall be a multibyte character sequence, beginning and ending in its initial shift state. The format is composed of zero or more directives: one or more white-space characters, an ordinary multibyte character (neither `%` nor a white-space character), or a conversion specification. Each conversion specification is introduced by the character `%`. After the `%`, the following appear in sequence:

— An optional assignment-suppressing character `*`.

— An optional decimal integer greater than zero that specifies the maximum field width (in characters).

— An optional *length modifier* that specifies the size of the receiving object.

— A *conversion specifier* character that specifies the type of conversion to be applied.

4 The `fscanf` function executes each directive of the format in turn. When all directives have been executed, or if a directive fails (as detailed later in this subclause), the function returns. Failures are described as input failures (due to the occurrence of an encoding error or the unavailability of input characters), or matching failures (due to inappropriate input).

5 A directive composed of white-space character(s) is executed by reading input up to the first non-white-space character (which remains unread), or until no more characters can be read. The directive never fails.

© ISO 2024 — All rights reserved

6 A directive that is an ordinary multibyte character is executed by reading the next characters of the stream. If any of those characters differ from the ones composing the directive, the directive fails and the differing and subsequent characters remain unread. Similarly, if end-of-file, an encoding error, or a read error prevents a character from being read, the directive fails.

7 A directive that is a conversion specification defines a set of matching input sequences, as described further in this subclause for each specifier. A conversion specification is executed in the following steps:

8 Input white-space characters are skipped, unless the specification includes a `[`, `c`, or `n` specifier.[331)]

9 An input item is read from the stream, unless the specification includes an `n` specifier. An input item is defined as the longest sequence of input characters which does not exceed any specified field width and which is, or is a prefix of, a matching input sequence.[332)] The first character, if any, after the input item remains unread. If the length of the input item is zero, the execution of the directive fails; this condition is a matching failure unless end-of-file, an encoding error, or a read error prevented input from the stream, in which case it is an input failure.

10 Except in the case of a `%` specifier, the input item (or, in the case of a `%n` directive, the count of input characters) is converted to a type appropriate to the conversion specifier. If the input item is not a matching sequence, the execution of the directive fails: this condition is a matching failure. Unless assignment suppression was indicated by a `*`, the result of the conversion is placed in the object pointed to by the first argument following the `format` argument that has not already received a conversion result. If this object does not have an appropriate type, or if the result of the conversion cannot be represented in the object, the behavior is undefined.

11 The length modifiers and their meanings are:

`hh` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`signed char`** or **`unsigned char`**.

`h` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`short int`** or **`unsigned short int`**.

`l` (ell) Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`long int`** or **`unsigned long int`**; that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`double`**; or that a following `c`, `s`, or `[` conversion specifier applies to an argument with type pointer to **`wchar_t`**.

`ll` (ell-ell) Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`long long int`** or **`unsigned long long int`**.

`j` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`intmax_t`** or **`uintmax_t`**.

`z` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`size_t`** or the corresponding signed integer type.

`t` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`ptrdiff_t`** or the corresponding unsigned integer type.

`w`*N* Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument which is a pointer to an integer with a specific width where *N* is a positive decimal integer with no leading zeros. All minimum-width integer types (7.22.1.2) and exact-width integer types (7.22.1.1) defined in the header `<stdint.h>` shall be supported. Other supported values of *N* are implementation-defined.

[331)] These white-space characters are not counted against a specified field width.

[332)] **`fscanf`** pushes back at most one input character onto the input stream. Therefore, some sequences that are acceptable to **`strtod`**, **`strtol`**, etc., are unacceptable to **`fscanf`**.

 © ISO 2024 — All rights reserved

`wf`*N* Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument which is a pointer to a fastest minimum-width integer with a specific width where *N* is a positive decimal integer with no leading zeros. All fastest minimum-width integer types (7.22.1.3) defined in the header `<stdint.h>` shall be supported. Other supported values of *N* are implementation-defined.

`L` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`long double`**.

`H` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`_Decimal32`**.

`D` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`_Decimal64`**.

`DD` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`_Decimal128`**.

If a length modifier appears with any conversion specifier other than as specified previously, the behavior is undefined.

12 In the following, the type of the corresponding argument for a conversion specifier shall be a pointer to a type determined by the length modifiers, if any, or specified by the conversion specifier. The conversion specifiers and their meanings are:

`d` Matches an optionally signed decimal integer, whose format is the same as expected for the subject sequence of the **`strtol`** function with the value 10 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`int`**.

`b` Matches an optionally signed binary integer, whose format is the same as expected for the subject sequence of the **`strtoul`** function with the value 2 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`unsigned int`**.

`i` Matches an optionally signed integer, whose format is the same as expected for the subject sequence of the **`strtol`** function with the value 0 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`int`**.

`o` Matches an optionally signed octal integer, whose format is the same as expected for the subject sequence of the **`strtoul`** function with the value 8 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`unsigned int`**.

`u` Matches an optionally signed decimal integer, whose format is the same as expected for the subject sequence of the **`strtoul`** function with the value 10 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`unsigned int`**.

`x` Matches an optionally signed hexadecimal integer, whose format is the same as expected for the subject sequence of the **`strtoul`** function with the value 16 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`unsigned int`**.

`a,e,f,g` Matches an optionally signed floating-point number, infinity, or NaN, whose format is the same as expected for the subject sequence of the **`strtod`** function. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`float`**.

`c` Matches a sequence of characters of exactly the number specified by the field width (1 if no field width is present in the directive).

© ISO 2024 — All rights reserved

333)

If no `l` length modifier is present, the corresponding argument shall be a pointer to **`char`**, **`signed char`**, **`unsigned char`**, or **`void`** that points to storage large enough to accept the sequence. No null character is added.

If an `l` length modifier is present, the input shall be a sequence of multibyte characters that begins in the initial shift state. Each multibyte character in the sequence is converted to a wide character as if by a call to the **`mbrtowc`** function, with the conversion state described by an **`mbstate_t`** object initialized to zero before the first multibyte character is converted. The corresponding argument shall be a pointer to storage of **`wchar_t`** large enough to accept the resulting sequence of wide characters. No null wide character is added.

`s` Matches a sequence of non-white-space characters.[333)]

If no `l` length modifier is present, the corresponding argument shall be a pointer to **`char`**, **`signed char`**, **`unsigned char`**, or **`void`** that points to storage large enough to accept the sequence and a terminating null character, which will be added automatically.

If an `l` length modifier is present, the input shall be a sequence of multibyte characters that begins in the initial shift state. Each multibyte character is converted to a wide character as if by a call to the **`mbrtowc`** function, with the conversion state described by an **`mbstate_t`** object initialized to zero before the first multibyte character is converted. The corresponding argument shall be a pointer to storage of **`wchar_t`** large enough to accept the sequence and the terminating null wide character, which will be added automatically.

`[` Matches a nonempty sequence of characters from a set of expected characters (the *scanset*).[333)]

If no `l` length modifier is present, the corresponding argument shall be a pointer to **`char`**, **`signed char`**, **`unsigned char`**, or **`void`** that points to storage large enough to accept the sequence and a terminating null character, which will be added automatically.

If an `l` length modifier is present, the input shall be a sequence of multibyte characters that begins in the initial shift state. Each multibyte character is converted to a wide character as if by a call to the **`mbrtowc`** function, with the conversion state described by an **`mbstate_t`** object initialized to zero before the first multibyte character is converted. The corresponding argument shall be a pointer that points to storage of **`wchar_t`** large enough to accept the sequence and the terminating null wide character, which will be added automatically.

The conversion specifier includes all subsequent characters in the `format` string, up to and including the matching right bracket (`]`). The characters between the brackets (the *scanlist*) compose the scanset, unless the character after the left bracket is a circumflex (`^`), in which case the scanset contains all characters that do not appear in the scanlist between the circumflex and the right bracket. If the conversion specifier begins with `[]` or `[^]`, the right bracket character is in the scanlist and the next following right bracket character is the matching right bracket that ends the specification; otherwise the first following right bracket character is the one that ends the specification. If a `-` character is in the scanlist and is not the first, nor the second where the first character is a `^`, nor the last character, the behavior is implementation-defined.

`p` Matches an implementation-defined set of sequences, which should be the same as the set of sequences that may be produced by the `%p` conversion of the **`fprintf`** function. The corresponding argument shall be a pointer to a pointer of **`void`**. The input item is converted to a pointer value in an implementation-defined manner. If the input item is a value converted earlier during the same program execution, the pointer that results shall compare equal to that value; otherwise the behavior of the `%p` conversion is undefined.

[333)] No special provisions are made for multibyte characters in the matching rules used by the `c`, `s`, and `[` conversion specifiers — the extent of the input field is determined on a byte-by-byte basis. The resulting field is nevertheless a sequence of multibyte characters that begins in the initial shift state.

© ISO 2024 — All rights reserved

`n` No input is consumed. The corresponding argument shall be a pointer of a signed integer type. The number of characters read from the input stream so far by this call to the **`fscanf`** function is stored into the integer object pointed to by the argument. Execution of a `%n` directive does not increment the assignment count returned at the completion of execution of the **`fscanf`** function. No argument is converted, but one is consumed. If the conversion specification includes an assignment-suppressing character or a field width, the behavior is undefined.

`%` Matches a single `%` character; no conversion or assignment occurs. The complete conversion specification shall be `%%`.

13 If a conversion specification is invalid, the behavior is undefined.[334)]

14 The conversion specifiers `A`, `E`, `F`, `G`, and `X` are also valid and behave the same as, respectively, `a`, `e`, `f`, `g`, and `x`.

15 Trailing white-space characters (including new-line characters) are left unread unless matched by a directive. The success of literal matches and suppressed assignments is not directly determinable other than via the `%n` directive.

Returns

16 The **`fscanf`** function returns the value of the macro **`EOF`** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure or if the implementation does not support a specific width length modifier.

17 EXAMPLE 1 The call:

```
#include <stdio.h>
/* ... */
int n, i; float x; char name[50];
n = fscanf(stdin, "%d%f%s", &i, &x, name);
```

with the input line:

```
25 54.32E-1 thompson
```

will assign to `n` the value 3, to `i` the value 25, to `x` the value 5.432, and to `name` the sequence `thompson\0`.

18 EXAMPLE 2 The call:

```
#include <stdio.h>
/* ... */
int i; float x; char name[50];
fscanf(stdin, "%2d%f%*d %[0123456789]", &i, &x, name);
```

with input:

```
56789 0123 56a72
```

will assign to `i` the value 56 and to `x` the value 789.0, will skip `0123`, and will assign to `name` the sequence `56\0`. The next character read from the input stream will be `a`.

19 EXAMPLE 3 To accept repeatedly from **`stdin`** a quantity, a unit of measure, and an item name:

```
#include <stdio.h>
/* ... */
int count; float quant; char units[21], item[21];
do {
      count = fscanf(stdin, "%f%20s of %20s", &quant, units, item);
```

[334)] See "future library directions" (7.33.15).

© ISO 2024 — All rights reserved

```
            fscanf(stdin,"%*[^\n]");
        } while (!feof(stdin) && !ferror(stdin));
```

20 If the **stdin** stream contains the following lines:

```
2 quarts of oil
-12.8degrees Celsius
lots of luck
10.0LBS     of
dirt
100ergs of energy
```

the execution of the preceding example will be analogous to the following assignments:

```
quant = 2; strcpy(units, "quarts"); strcpy(item, "oil");
count = 3;
quant = -12.8; strcpy(units, "degrees");
count = 2; // "C" fails to match "o"
count = 0; // "l" fails to match "%f"
quant = 10.0; strcpy(units, "LBS"); strcpy(item, "dirt");
count = 3;
count = 0; // "100e" fails to match "%f"
count = EOF;
```

21 EXAMPLE 4 In:

```
#include <stdio.h>
/* ... */
int d1, d2, n1, n2, i;
i = sscanf("123", "%d%n%n%d", &d1, &n1, &n2, &d2);
```

the value 123 is assigned to `d1` and the value 3 to `n1`. Because `%n` can never get an input failure, the value of 3 is also assigned to `n2`. The value of `d2` is not affected. The value 1 is assigned to `i`.

22 EXAMPLE 5 The call:

```
#include <stdio.h>
/* ... */
int n, i;
n = sscanf("foo  %bar  42", "foo%%bar%d", &i);
```

will assign to `n` the value 1 and to `i` the value 42 because input white-space characters are skipped for both the `%` and `d` conversion specifiers.

23 EXAMPLE 6 In these examples, multibyte characters do have a state-dependent encoding, and the members of the extended character set that consist of more than one byte each consist of exactly two bytes, the first of which is denoted here by a □ and the second by an uppercase letter, but are only recognized as such when in the alternate shift state. The shift sequences are denoted by ↑ and ↓, in which the first causes entry into the alternate shift state.

24 After the call:

```
#include <stdio.h>
/* ... */
char str[50];
fscanf(stdin, "a%s", str);
```

with the input line:

```
a↑□X□Y↓ bc
```

`str` will contain ↑□X□Y↓\0 assuming that none of the bytes of the shift sequences (or of the multibyte characters, in the more general case) appears to be a single-byte white-space character.

© ISO 2024 — All rights reserved

25 In contrast, after the call:

```
#include <stdio.h>
#include <stddef.h>
/* ... */
wchar_t wstr[50];
fscanf(stdin, "a%ls", wstr);
```

with the same input line, `wstr` will contain the two wide characters that correspond to □X and □Y and a terminating null wide character.

26 However, the call:

```
#include <stdio.h>
#include <stddef.h>
/* ... */
wchar_t wstr[50];
fscanf(stdin, "a↑□X↓%ls", wstr);
```

with the same input line will return zero due to a matching failure against the ↓ sequence in the format string.

27 Assuming that the first byte of the multibyte character □X is the same as the first byte of the multibyte character □Y, after the call:

```
#include <stdio.h>
#include <stddef.h>
/* ... */
wchar_t wstr[50];
fscanf(stdin, "a↑□Y↓%ls", wstr);
```

with the same input line, zero will again be returned, but **stdin** will be left with a partially consumed multibyte character.

Forward references: the **strtod**, **strtof**, and **strtold** functions (7.24.1.5), the **strtol**, **strtoll**, **strtoul**, and **strtoull** functions (7.24.1.7), conversion state (7.31.6), the **wcrtomb** function (7.31.6.3.3).

7.23.6.3 The printf function

Synopsis

1

```
#include <stdio.h>
int printf(const char * restrict format, ...);
```

Description

2 The **printf** function is equivalent to **fprintf** with the argument **stdout** interposed before the arguments to **printf**.

Returns

3 The **printf** function returns the number of characters transmitted, or a negative value if an output or encoding error occurred.

7.23.6.4 The scanf function

Synopsis

1

```
#include <stdio.h>
int scanf(const char * restrict format, ...);
```

Description

2 The **scanf** function is equivalent to **fscanf** with the argument **stdin** interposed before the arguments to **scanf**.

© ISO 2024 — All rights reserved

Returns

3 The **scanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **scanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.6.5 The snprintf function

Synopsis

1
```
#include <stdio.h>
int snprintf(char * restrict s, size_t n, const char * restrict format, ...);
```

Description

2 The **snprintf** function is equivalent to **fprintf**, except that the output is written into an array (specified by argument s) rather than to a stream. If n is zero, nothing is written, and s may be a null pointer. Otherwise, output characters beyond the n-1st are discarded rather than being written to the array, and a null character is written at the end of the characters actually written into the array. If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **snprintf** function returns the number of characters that would have been written had n been sufficiently large, not counting the terminating null character, or a negative value if an encoding error occurred. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than n.

7.23.6.6 The sprintf function

Synopsis

1
```
#include <stdio.h>
int sprintf(char * restrict s, const char * restrict format, ...);
```

Description

2 The **sprintf** function is equivalent to **fprintf**, except that the output is written into an array (specified by the argument s) rather than to a stream. A null character is written at the end of the characters written; it is not counted as part of the returned value. If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **sprintf** function returns the number of characters written in the array, not counting the terminating null character, or a negative value if an encoding error occurred.

7.23.6.7 The sscanf function

Synopsis

1
```
#include <stdio.h>
int sscanf(const char * restrict s, const char * restrict format, ...);
```

Description

2 The **sscanf** function is equivalent to **fscanf**, except that input is obtained from a string (specified by the argument s) rather than from a stream. Reaching the end of the string is equivalent to encountering end-of-file for the **fscanf** function. If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **sscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **sscanf** function returns the number of input

 © ISO 2024 — All rights reserved

items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.6.8 The `vfprintf` function

Synopsis

1

```
#include <stdarg.h>
#include <stdio.h>
int vfprintf(FILE * restrict stream, const char * restrict format, va_list arg);
```

Description

2 The **vfprintf** function is equivalent to **fprintf**, with the variable argument list replaced by `arg`, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** invocations). The **vfprintf** function does not invoke the **va_end** macro.[335)]

Returns

3 The **vfprintf** function returns the number of characters transmitted, or a negative value if an output or encoding error occurred.

4 EXAMPLE The following shows the use of the **vfprintf** function in a general error-reporting routine.

```
#include <stdarg.h>
#include <stdio.h>

void error(char *function_name, char *format, ...)
{
      va_list args;

      va_start(args, format);
      // print out name of function causing error
      fprintf(stderr, "ERROR in %s: ", function_name);
      // print out remainder of message
      vfprintf(stderr, format, args);
      va_end(args);
}
```

7.23.6.9 The `vfscanf` function

Synopsis

1

```
#include <stdarg.h>
#include <stdio.h>
int vfscanf(FILE * restrict stream, const char * restrict format, va_list arg);
```

Description

2 The **vfscanf** function is equivalent to **fscanf**, with the variable argument list replaced by `arg`, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vfscanf** function does not invoke the **va_end** macro.[335)]

Returns

3 The **vfscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vfscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.6.10 The `vprintf` function

[335)] As the functions **vfprintf**, **vfscanf**, **vprintf**, **vscanf**, **vsnprintf**, **vsprintf**, and **vsscanf** invoke the **va_arg** macro, `arg` after the return has an indeterminate representation.

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdarg.h>
#include <stdio.h>
int vprintf(const char * restrict format, va_list arg);
```

Description

2 The **vprintf** function is equivalent to **printf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vprintf** function does not invoke the **va_end** macro.[335)]

Returns

3 The **vprintf** function returns the number of characters transmitted, or a negative value if an output or encoding error occurred.

7.23.6.11 The vscanf function

Synopsis

1
```
#include <stdarg.h>
#include <stdio.h>
int vscanf(const char * restrict format, va_list arg);
```

Description

2 The **vscanf** function is equivalent to **scanf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vscanf** function does not invoke the **va_end** macro.[335)]

Returns

3 The **vscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.6.12 The vsnprintf function

Synopsis

1
```
#include <stdarg.h>
#include <stdio.h>
int vsnprintf(char * restrict s, size_t n, const char * restrict format, va_list
    arg);
```

Description

2 The **vsnprintf** function is equivalent to **snprintf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vsnprintf** function does not invoke the **va_end** macro.[335)] If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **vsnprintf** function returns the number of characters that would have been written had n been sufficiently large, not counting the terminating null character, or a negative value if an encoding error occurred. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than n.

7.23.6.13 The vsprintf function

Synopsis

1
```
#include <stdarg.h>
#include <stdio.h>
```

 © ISO 2024 — All rights reserved

```
int vsprintf(char * restrict s, const char * restrict format, va_list arg);
```

Description

2 The **vsprintf** function is equivalent to **sprintf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vsprintf** function does not invoke the **va_end** macro.[335] If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **vsprintf** function returns the number of characters written in the array, not counting the terminating null character, or a negative value if an encoding error occurred.

7.23.6.14 The vsscanf function

Synopsis

1
```
#include <stdarg.h>
#include <stdio.h>
int vsscanf(const char * restrict s, const char * restrict format, va_list arg);
```

Description

2 The **vsscanf** function is equivalent to **sscanf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vsscanf** function does not invoke the **va_end** macro.[335]

Returns

3 The **vsscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vsscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.23.7 Character input/output functions

7.23.7.1 The fgetc function

Synopsis

1
```
#include <stdio.h>
int fgetc(FILE *stream);
```

Description

2 If the end-of-file indicator for the input stream pointed to by stream is not set and a next character is present, the **fgetc** function obtains that character as an **unsigned char** converted to an **int** and advances the associated file position indicator for the stream (if defined).

Returns

3 If the end-of-file indicator for the stream is set, or if the stream is at end-of-file, the end-of-file indicator for the stream is set and the **fgetc** function returns **EOF**. Otherwise, the **fgetc** function returns the next character from the input stream pointed to by stream. If a read error occurs, the error indicator for the stream is set and the **fgetc** function returns **EOF**.[336]

7.23.7.2 The fgets function

Synopsis

1
```
#include <stdio.h>
char *fgets(char * restrict s, int n, FILE * restrict stream);
```

[336] An end-of-file and a read error can be distinguished by use of the **feof** and **ferror** functions.

© ISO 2024 — All rights reserved

Description

2 The **fgets** function reads at most one less than the number of characters specified by `n` from the stream pointed to by `stream` into the array pointed to by `s`. No additional characters are read after a new-line character (which is retained) or after end-of-file. A null character is written immediately after the last character read into the array. If `n` is negative or zero, the behavior is undefined.

Returns

3 The **fgets** function returns `s` if successful. If end-of-file is encountered and no characters have been read into the array, the contents of the array remain unchanged and a null pointer is returned. If a read error occurs during the operation, the members of the array have unspecified values and a null pointer is returned.

7.23.7.3 The fputc function

Synopsis

1
```
#include <stdio.h>
int fputc(int c, FILE *stream);
```

Description

2 The **fputc** function writes the character specified by `c` (converted to an **unsigned char**) to the output stream pointed to by `stream`, at the position indicated by the associated file position indicator for the stream (if defined), and advances the indicator appropriately. If the file cannot support positioning requests, or if the stream was opened with append mode, the character is appended to the output stream.

Returns

3 The **fputc** function returns the character written. If a write error occurs, the error indicator for the stream is set and **fputc** returns **EOF**.

7.23.7.4 The fputs function

Synopsis

1
```
#include <stdio.h>
int fputs(const char * restrict s, FILE * restrict stream);
```

Description

2 The **fputs** function writes the string pointed to by `s` to the stream pointed to by `stream`. The terminating null character is not written.

Returns

3 The **fputs** function returns **EOF** if a write error occurs; otherwise it returns a nonnegative value.

7.23.7.5 The getc function

Synopsis

1
```
#include <stdio.h>
int getc(FILE *stream);
```

Description

2 The **getc** function is equivalent to **fgetc**, except that if it is implemented as a macro, it may evaluate `stream` more than once, so the argument should never be an expression with side effects.

Returns

3 The **getc** function returns the next character from the input stream pointed to by `stream`. If the stream is at end-of-file, the end-of-file indicator for the stream is set and **getc** returns **EOF**. If a read error occurs, the error indicator for the stream is set and **getc** returns **EOF**.

 © ISO 2024 — All rights reserved

7.23.7.6 The `getchar` function

Synopsis

1
```
#include <stdio.h>
int getchar(void);
```

Description

2 The **`getchar`** function is equivalent to **`getc`** with the argument **`stdin`**.

Returns

3 The **`getchar`** function returns the next character from the input stream pointed to by **`stdin`**. If the stream is at end-of-file, the end-of-file indicator for the stream is set and **`getchar`** returns **`EOF`**. If a read error occurs, the error indicator for the stream is set and **`getchar`** returns **`EOF`**.

7.23.7.7 The `putc` function

Synopsis

1
```
#include <stdio.h>
int putc(int c, FILE *stream);
```

Description

2 The **`putc`** function is equivalent to **`fputc`**, except that if it is implemented as a macro, it may evaluate `stream` more than once, so that argument should never be an expression with side effects.

Returns

3 The **`putc`** function returns the character written. If a write error occurs, the error indicator for the stream is set and **`putc`** returns **`EOF`**.

7.23.7.8 The `putchar` function

Synopsis

1
```
#include <stdio.h>
int putchar(int c);
```

Description

2 The **`putchar`** function is equivalent to **`putc`** with the second argument **`stdout`**.

Returns

3 The **`putchar`** function returns the character written. If a write error occurs, the error indicator for the stream is set and **`putchar`** returns **`EOF`**.

7.23.7.9 The `puts` function

Synopsis

1
```
#include <stdio.h>
int puts(const char *s);
```

Description

2 The **`puts`** function writes the string pointed to by `s` to the stream pointed to by **`stdout`**, and appends a new-line character to the output. The terminating null character is not written.

Returns

3 The **`puts`** function returns **`EOF`** if a write error occurs; otherwise it returns a nonnegative value.

7.23.7.10 The `ungetc` function

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdio.h>
int ungetc(int c, FILE *stream);
```

Description

2 The **ungetc** function pushes the character specified by `c` (converted to an **unsigned char**) back onto the input stream pointed to by `stream`. Pushed-back characters will be returned by subsequent reads on that stream in the reverse order of their pushing. A successful intervening call (with the stream pointed to by `stream`) to a file positioning function (**fseek**, **fsetpos**, or **rewind**) discards any pushed-back characters for the stream. The external storage corresponding to the stream is unchanged.

3 One character of pushback is guaranteed. If the **ungetc** function is called too many times on the same stream without an intervening read or file positioning operation on that stream, the operation may fail.

4 If the value of `c` equals that of the macro **EOF**, the operation fails and the input stream is unchanged.

5 A successful call to the **ungetc** function clears the end-of-file indicator for the stream. The value of the file position indicator for the stream after reading or discarding all pushed-back characters shall be the same as it was before the characters were pushed back.[337] For a text stream, the value of its file position indicator after a successful call to the **ungetc** function is unspecified until all pushed-back characters are read or discarded. For a binary stream, its file position indicator is decremented by each successful call to the **ungetc** function; if its value was zero before a call, it has an indeterminate representation after the call.[338]

Returns

6 The **ungetc** function returns the character pushed back after conversion, or **EOF** if the operation fails.

Forward references: file positioning functions (7.23.9).

7.23.8 Direct input/output functions

7.23.8.1 The fread function

Synopsis

1
```
#include <stdio.h>
size_t fread(void * restrict ptr, size_t size, size_t nmemb,
      FILE * restrict stream);
```

Description

2 The **fread** function reads, into the array pointed to by `ptr`, up to `nmemb` elements whose size is specified by `size`, from the stream pointed to by `stream`. For each object, `size` calls are made to the **fgetc** function and the results stored, in the order read, in an array of **unsigned char** exactly overlaying the object. The file position indicator for the stream (if defined) is advanced by the number of characters successfully read. If an error occurs, the resulting representation of the file position indicator for the stream is indeterminate. If a partial element is read, its representation is indeterminate.

Returns

3 The **fread** function returns the number of elements successfully read, which may be less than `nmemb` if a read error or end-of-file is encountered. If `size` or `nmemb` is zero, **fread** returns zero and the contents of the array and the state of the stream remain unchanged.

7.23.8.2 The fwrite function

[337] Note that a file positioning function could further modify the file position indicator after discarding any pushed-back characters.
[338] See "future library directions" (7.33.15).

 © ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdio.h>
size_t fwrite(const void * restrict ptr, size_t size, size_t nmemb,
      FILE * restrict stream);
```

Description

2 The **fwrite** function writes, from the array pointed to by ptr, up to nmemb elements whose size is specified by size, to the stream pointed to by stream. For each object, size calls are made to the **fputc** function, taking the values (in order) from an array of **unsigned char** exactly overlaying the object. The file position indicator for the stream (if defined) is advanced by the number of characters successfully written. If an error occurs, the resulting representation of the file position indicator for the stream is indeterminate.

Returns

3 The **fwrite** function returns the number of elements successfully written, which will be less than nmemb only if a write error is encountered. If size or nmemb is zero, **fwrite** returns zero and the state of the stream remains unchanged.

7.23.9 File positioning functions

7.23.9.1 The fgetpos function

Synopsis

1
```
#include <stdio.h>
int fgetpos(FILE * restrict stream, fpos_t * restrict pos);
```

Description

2 The **fgetpos** function stores the current values of the parse state (if any) and file position indicator for the stream pointed to by stream in the object pointed to by pos. The values stored contain unspecified information usable by the **fsetpos** function for repositioning the stream to its position at the time of the call to the **fgetpos** function.

Returns

3 If successful, the **fgetpos** function returns zero; on failure, the **fgetpos** function returns nonzero and stores an implementation-defined positive value in **errno**.

Forward references: the **fsetpos** function (7.23.9.3).

7.23.9.2 The fseek function

Synopsis

1
```
#include <stdio.h>
int fseek(FILE *stream, long int offset, int whence);
```

Description

2 The **fseek** function sets the file position indicator for the stream pointed to by stream. If a read or write error occurs, the error indicator for the stream is set and **fseek** fails.

3 For a binary stream, the new position, measured in characters from the beginning of the file, is obtained by adding offset to the position specified by whence. The specified position is the beginning of the file if whence is **SEEK_SET**, the current value of the file position indicator if **SEEK_CUR**, or end-of-file if **SEEK_END**. A binary stream may or may not meaningfully support **fseek** calls with a whence value of **SEEK_END**.

4 For a text stream, either offset shall be zero, or offset shall be a value returned by an earlier successful call to the **ftell** function on a stream associated with the same file and whence shall be **SEEK_SET**.

5 After determining the new position, a successful call to the **fseek** function undoes any effects of the

© ISO 2024 — All rights reserved

ungetc function on the stream, clears the end-of-file indicator for the stream, and then establishes the new position. After a successful **fseek** call, the next operation on an update stream may be either input or output.

Returns

6 The **fseek** function returns nonzero only for a request that cannot be satisfied.

Forward references: the **ftell** function (7.23.9.4).

7.23.9.3 The fsetpos function

Synopsis

1
```
#include <stdio.h>
int fsetpos(FILE *stream, const fpos_t *pos);
```

Description

2 The **fsetpos** function sets the **mbstate_t** object (if any) and file position indicator for the stream pointed to by `stream` according to the value of the object pointed to by `pos`, which shall be a value obtained from an earlier successful call to the **fgetpos** function on a stream associated with the same file. If a read or write error occurs, the error indicator for the stream is set and **fsetpos** fails.

3 A successful call to the **fsetpos** function undoes any effects of the **ungetc** function on the stream, clears the end-of-file indicator for the stream, and then establishes the new parse state and position. After a successful **fsetpos** call, the next operation on an update stream may be either input or output.

Returns

4 If successful, the **fsetpos** function returns zero; on failure, the **fsetpos** function returns nonzero and stores an implementation-defined positive value in **errno**.

7.23.9.4 The ftell function

Synopsis

1
```
#include <stdio.h>
long int ftell(FILE *stream);
```

Description

2 The **ftell** function obtains the current value of the file position indicator for the stream pointed to by `stream`. For a binary stream, the value is the number of characters from the beginning of the file. For a text stream, its file position indicator contains unspecified information, usable by the **fseek** function for returning the file position indicator for the stream to its position at the time of the **ftell** call; the difference between two such return values is not necessarily a meaningful measure of the number of characters written or read.

Returns

3 If successful, the **ftell** function returns the current value of the file position indicator for the stream. On failure, the **ftell** function returns `-1L` and stores an implementation-defined positive value in **errno**.

7.23.9.5 The rewind function

Synopsis

1
```
#include <stdio.h>
void rewind(FILE *stream);
```

Description

2 The **rewind** function sets the file position indicator for the stream pointed to by `stream` to the beginning of the file. It is equivalent to

 © ISO 2024 — All rights reserved

```
    (void)fseek(stream, 0L, SEEK_SET)
```

except that the error indicator for the stream is also cleared.

Returns

3 The **rewind** function returns no value.

7.23.10 Error-handling functions

7.23.10.1 The clearerr function

Synopsis

1
```
    #include <stdio.h>
    void clearerr(FILE *stream);
```

Description

2 The **clearerr** function clears the end-of-file and error indicators for the stream pointed to by stream.

Returns

3 The **clearerr** function returns no value.

7.23.10.2 The feof function

Synopsis

1
```
    #include <stdio.h>
    int feof(FILE *stream);
```

Description

2 The **feof** function tests the end-of-file indicator for the stream pointed to by stream.

Returns

3 The **feof** function returns nonzero if and only if the end-of-file indicator is set for stream.

© ISO 2024 — All rights reserved

7.23.10.3 The `ferror` function

Synopsis

1
```
#include <stdio.h>
int ferror(FILE *stream);
```

Description

2 The **`ferror`** function tests the error indicator for the stream pointed to by `stream`.

Returns

3 The **`ferror`** function returns nonzero if and only if the error indicator is set for `stream`.

7.23.10.4 The `perror` function

Synopsis

1
```
#include <stdio.h>
void perror(const char *s);
```

Description

2 The **`perror`** function maps the error number in the integer expression **`errno`** to an error message. It writes a sequence of characters to the standard error stream thus: first (if `s` is not a null pointer and the character pointed to by `s` is not the null character), the string pointed to by `s` followed by a colon (`:`) and a space; then an appropriate error message string followed by a new-line character. The contents of the error message strings are the same as those returned by the **`strerror`** function with argument **`errno`**.

Returns

3 The **`perror`** function returns no value.

Forward references: the **`strerror`** function (7.26.6.3).

© ISO 2024 — All rights reserved

7.24 General utilities <stdlib.h>

1 The header <stdlib.h> declares several types and functions of general utility, and defines several macros.[339)]

2 The feature test macro __STDC_VERSION_STDLIB_H__ expands to the token 202311L.

3 The types declared are **size_t** and **wchar_t** (both described in 7.21), **once_flag** (described in 7.28),

```
div_t
```

which is a structure type that is the type of the value returned by the **div** function,

```
ldiv_t
```

which is a structure type that is the type of the value returned by the **ldiv** function, and

```
lldiv_t
```

which is a structure type that is the type of the value returned by the **lldiv** function.

4 The macros defined are **NULL** (described in 7.21); **ONCE_FLAG_INIT** (described in 7.28);

```
EXIT_FAILURE
```

and

```
EXIT_SUCCESS
```

which expand to integer constant expressions that can be used as the argument to the **exit** function to return unsuccessful or successful termination status, respectively, to the host environment;

```
RAND_MAX
```

which expands to an integer constant expression that is the maximum value returned by the **rand** function; and

```
MB_CUR_MAX
```

which expands to a positive integer expression with type **size_t** that is the maximum number of bytes in a multibyte character for the extended character set specified by the current locale (category **LC_CTYPE**), which is never greater than **MB_LEN_MAX**.

5 The function

```
#include <stdlib.h>
void call_once(once_flag *flag, void (*func)(void));
```

is described in 7.28.2.

7.24.1 Numeric conversion functions

1 The functions **atof**, **atoi**, **atol**, and **atoll** are not required to affect the value of the integer expression **errno** on an error. If the value of the result cannot be represented, the behavior is undefined.

7.24.1.1 The atof function

Synopsis

1
```
#include <stdlib.h>
double atof(const char *nptr);
```

[339)] See "future library directions" (7.33.16).

© ISO 2024 — All rights reserved

Description

2 The **atof** function converts the initial portion of the string pointed to by nptr to **double** representation. Except for the behavior on error, it is equivalent to

```
strtod(nptr, nullptr)
```

Returns

3 The **atof** function returns the converted value.

Forward references: the **strtod**, **strtof**, and **strtold** functions (7.24.1.5).

7.24.1.2 The atoi, atol, and atoll functions

Synopsis

1

```
#include <stdlib.h>
int atoi(const char *nptr);
long int atol(const char *nptr);
long long int atoll(const char *nptr);
```

Description

2 The **atoi**, **atol**, and **atoll** functions convert the initial portion of the string pointed to by nptr to **int**, **long int**, and **long long int** representation, respectively. Except for the behavior on error, they are equivalent to

```
atoi:  (int)strtol(nptr, nullptr, 10)
atol:  strtol(nptr, nullptr, 10)
atoll: strtoll(nptr, nullptr, 10)
```

Returns

3 The **atoi**, **atol**, and **atoll** functions return the converted value.

Forward references: the **strtol**, **strtoll**, **strtoul**, and **strtoull** functions (7.24.1.7).

7.24.1.3 The strfromd, strfromf, and strfroml functions

Synopsis

1

```
#include <stdlib.h>
int strfromd(char * restrict s, size_t n, const char * restrict format,
    double fp);
int strfromf(char * restrict s, size_t n, const char * restrict format,
    float fp);
int strfroml(char * restrict s, size_t n, const char * restrict format,
    long double fp);
```

Description

2 The **strfromd**, **strfromf**, and **strfroml** functions are equivalent to **snprintf**(s, n, format, fp) (7.23.6.5), except that the default argument promotions are not applied and the format string shall only contain the character %, an optional precision that does not contain an asterisk *, and one of the conversion specifiers a, A, e, E, f, F, g, or G, which applies to the type (**double**, **float**, or **long double**) indicated by the function suffix (rather than by a length modifier). Use of these functions with any other format string results in undefined behavior.

Returns

3 The **strfromd**, **strfromf**, and **strfroml** functions return the number of characters that would have been written had n been sufficiently large, not counting the terminating null character. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than n.

 © ISO 2024 — All rights reserved

7.24.1.4 The `strfromd`*N* functions

Synopsis

1
```
#include <stdlib.h>
#ifdef __STDC_IEC_60559_DFP__
int strfromd32(char * restrict s, size_t n, const char * restrict format,
    _Decimal32 fp);
int strfromd64(char * restrict s, size_t n, const char * restrict format,
    _Decimal64 fp);
int strfromd128(char * restrict s, size_t n, const char * restrict format,
    _Decimal128 fp);
#endif
```

Description

2 The `strfromd`*N* functions are equivalent to `snprintf(s, n, format, fp)` (7.23.6.5), except the format string contains only the character `%`, an optional precision that does not contain an asterisk `*`, and one of the conversion specifiers `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G`, which applies to the type (**`_Decimal32`**, **`_Decimal64`**, or **`_Decimal128`**) indicated by the function suffix (rather than by a length modifier). Use of these functions with any other format string results in undefined behavior.

Returns

3 The `strfromd`*N* functions return the number of characters that would have been written had `n` been sufficiently large, not counting the terminating null character. Thus, the null-terminated output has been completely written if and only if the returned value is both nonnegative and less than `n`.

7.24.1.5 The `strtod`, `strtof`, and `strtold` functions

Synopsis

1
```
#include <stdlib.h>
double strtod(const char * restrict nptr, char ** restrict endptr);
float strtof(const char * restrict nptr, char ** restrict endptr);
long double strtold(const char * restrict nptr, char ** restrict endptr);
```

Description

2 The **`strtod`**, **`strtof`**, and **`strtold`** functions convert the initial portion of the string pointed to by `nptr` to **`double`**, **`float`**, and **`long double`** representation, respectively. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space characters, a subject sequence resembling a floating constant or representing an infinity or NaN; and a final string of one or more unrecognized characters, including the terminating null character of the input string. Then, they attempt to convert the subject sequence to a floating-point number, and return the result.

3 The expected form of the subject sequence is an optional plus or minus sign, then one of the following:

— a nonempty sequence of decimal digits optionally containing a decimal-point character, then an optional exponent part as defined in 6.4.4.3, excluding any digit separators (6.4.4.2);

— a `0x` or `0X`, then a nonempty sequence of hexadecimal digits optionally containing a decimal-point character, then an optional binary exponent part as defined in 6.4.4.3, excluding any digit separators;

— `INF` or `INFINITY`, ignoring case

— `NAN` or `NAN(`*n-char-sequence*$_{opt}$`)`, ignoring case in the `NAN` part, where:

n-char-sequence:
digit
nondigit
n-char-sequence digit

© ISO 2024 — All rights reserved

n-char-sequence nondigit

The subject sequence is defined as the longest initial subsequence of the input string, starting with the first non-white-space character, that is of the expected form. The subject sequence contains no characters if the input string is not of the expected form.

4 If the subject sequence has the expected form for a floating-point number, the sequence of characters starting with the first digit or the decimal-point character (whichever occurs first) is interpreted as a floating constant according to the rules of 6.4.4.3, except that the decimal-point character is used in place of a period, and that if neither an exponent part nor a decimal-point character appears in a decimal floating-point number, or if a binary exponent part does not appear in a hexadecimal floating-point number, an exponent part of the appropriate type with value zero is assumed to follow the last digit in the string. If the subject sequence begins with a minus sign, the sequence is interpreted as arithmetically negated.[340)]

5 A character sequence `INF` or `INFINITY` is interpreted as an infinity, if representable in the return type, else like a floating constant that is too large for the range of the return type. A character sequence `NAN` or `NAN(`*n-char-sequence*$_{opt}$`)` is interpreted as a quiet NaN, if supported in the return type, else like a subject sequence part that does not have the expected form; the meaning of the n-char sequence is implementation-defined.[341)] A pointer to the final string is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

6 If the subject sequence has the hexadecimal form and **`FLT_RADIX`** is a power of 2, the value resulting from the conversion is correctly rounded.

7 In other than the `"C"` locale, additional locale-specific subject sequence forms may be accepted.

8 If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of `nptr` is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

Recommended practice

9 If the subject sequence has the hexadecimal form, **`FLT_RADIX`** is not a power of 2, and the result is not exactly representable, the result should be one of the two numbers in the appropriate internal format that are adjacent to the hexadecimal floating source value, with the extra stipulation that the error should have a correct sign for the current rounding direction.

10 If the subject sequence has the decimal form and at most M significant digits, where M is the maximum value of the *T*`_DECIMAL_DIG` macros (defined in `<float.h>`), the result should be correctly rounded. If the subject sequence D has the decimal form and more than M significant digits, consider the two bounding, adjacent decimal strings L and U, both having M significant digits, such that the values of L, D, and U satisfy $L \leq D \leq U$. The result should be one of the (equal or adjacent) values that would be obtained by correctly rounding L and U according to the current rounding direction, with the extra stipulation that the error with respect to D should have a correct sign for the current rounding direction.[342)]

Returns

11 The functions return the converted value, if any. If no conversion could be performed, positive or unsigned zero is returned.

12 If the correct value overflows and default rounding is in effect (7.12.1), plus or minus **`HUGE_VAL`**, **`HUGE_VALF`**, or **`HUGE_VALL`** is returned (according to the return type and sign of the value); if the integer expression **`math_errhandling`** & **`MATH_ERRNO`** is nonzero, the integer expression **`errno`** acquires the value of **`ERANGE`**; if the integer expression **`math_errhandling`** & **`MATH_ERREXCEPT`** is nonzero, the "overflow" floating-point exception is raised.

[340)] It is unspecified whether a minus-signed sequence is converted to a negative number directly or by negating the value resulting from converting the corresponding unsigned sequence (see F.5); the two methods could yield different results if rounding is toward positive or negative infinity. In either case, the functions honor the sign of zero if floating-point arithmetic supports signed zeros.

[341)] An implementation can use the n-char sequence to determine extra information to be represented in the NaN's significand.

[342)] M is sufficiently large that L and U will usually correctly round to the same internal floating value, but if not will correctly round to adjacent values.

 © ISO 2024 — All rights reserved

13 If the result underflows (7.12.1), the functions return a value whose magnitude is no greater than the smallest normalized positive number in the return type; if the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero, whether **errno** acquires the value **ERANGE** is implementation-defined; if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero, whether the "underflow" floating-point exception is raised is implementation-defined.

7.24.1.6 The **strtod***N* functions

Synopsis

1
```
#include <stdlib.h>
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 strtod32(const char * restrict nptr, char ** restrict endptr);
_Decimal64 strtod64(const char * restrict nptr, char ** restrict endptr);
_Decimal128 strtod128(const char * restrict nptr, char ** restrict endptr);
#endif
```

Description

2 The **strtod***N* functions convert the initial portion of the string pointed to by `nptr` to decimal floating type representation. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space characters; a subject sequence resembling a floating constant or representing an infinity or NaN; and a final string of one or more unrecognized characters, including the terminating null character of the input string. Then, they attempt to convert the subject sequence to a floating-point number, and return the result.

3 The expected form of the subject sequence is an optional plus or minus sign, then one of the following:

— a nonempty sequence of decimal digits optionally containing a decimal-point character, then an optional exponent part as defined in 6.4.4.3, excluding any digit separators (6.4.4.2)

— a `0x` or `0X`, then a nonempty sequence of hexadecimal digits optionally containing a decimal-point character, then an optional binary exponent part as defined in 6.4.4.3, excluding any digit separators (6.4.4.2)

— `INF` or `INFINITY`, ignoring case

— `NAN` or `NAN(`*d-char-sequence*$_{opt}$`)`, ignoring case in the `NAN` part, where:

d-char-sequence:
digit
nondigit
d-char-sequence digit
d-char-sequence nondigit

The subject sequence is defined as the longest initial subsequence of the input string, starting with the first non-white-space character, that is of the expected form. The subject sequence contains no characters if the input string is not of the expected form.

4 If the subject sequence has the expected form for a floating-point number, the sequence of characters starting with the first digit or the decimal-point character (whichever occurs first) is interpreted as a floating constant according to the rules of 6.4.4.3, except that the decimal-point character is used in place of a period, and that if neither an exponent part nor a decimal-point character appears in a decimal floating-point number, or if a binary exponent part does not appear in a hexadecimal floating-point number, an exponent part of the appropriate type with value zero is assumed to follow the last digit in the string. If the subject sequence begins with a minus sign, the sequence is interpreted as arithmetically negated before rounding and the sign s is set to -1, else s is set to 1.

© ISO 2024 — All rights reserved

5 If the subject sequence has the expected form for a decimal floating-point number, the value resulting from the conversion is correctly rounded and the coefficient c and the quantum exponent q are determined by the rules in 6.4.4.3 for a decimal floating constant of decimal type.

6 If the subject sequence has the expected form for a hexadecimal floating-point number, the value resulting from the conversion is correctly rounded provided the subject sequence has at most M significant hexadecimal digits, where $M \geq \lceil (P-1)/4 \rceil + 1$ is implementation-defined, and P is the maximum precision of the supported radix-2 floating types and binary non-arithmetic interchange formats.[343)] If all subject sequences of hexadecimal form are correctly rounded, M may be regarded as infinite. If the subject sequence has more than M significant hexadecimal digits, the implementation may first round to M significant hexadecimal digits according to the applicable decimal rounding direction mode, signaling exceptions as though converting from a wider format, then correctly round the result of the shortened hexadecimal input to the result type. The preferred quantum exponent for the result is 0 if the hexadecimal number is exactly represented in the decimal type; the preferred quantum exponent for the result is the least possible if the hexadecimal number is not exactly represented in the decimal type.

7 A character sequence `INF` or `INFINITY` is interpreted as an infinity. A character sequence `NAN` or `NAN(`*d-char-sequence*$_{opt}$`)`, is interpreted as a quiet NaN; the meaning of the d-char sequence is implementation-defined.[344)] A pointer to the final string is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

8 In other than the `"C"` locale, additional locale-specific subject sequence forms may be accepted.

9 If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of `nptr` is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

Returns

10 The **`strtod`***N* functions return the converted value, if any. If no conversion could be performed, the value of the triple $(+1, 0, 0)$ is returned. If the correct value overflows:

— the value of the macro **`ERANGE`** is stored in **`errno`** if the integer expression **`math_errhandling`** & **`MATH_ERRNO`** is nonzero;

— the "overflow" floating-point exception is raised if the integer expression **`math_errhandling`** & **`MATH_ERREXCEPT`** is nonzero.

If the result underflows (7.12.1), whether **`errno`** acquires the value **`ERANGE`** if the integer expression **`math_errhandling`** & **`MATH_ERRNO`** is nonzero is implementation-defined; if the integer expression **`math_errhandling`** & **`MATH_ERREXCEPT`** is nonzero, whether the "underflow" floating-point exception is raised is implementation-defined.

11 EXAMPLE Following are subject sequences of the decimal form and the resulting triples (s, c, q) produced by **`strtod64`**. Note that for **`_Decimal64`**, the precision (maximum coefficient length) is 16 and the quantum exponent range is $-398 \leq q \leq 369$.

`"0"`	$(+1, 0, 0)$
`"0.00"`	$(+1, 0, -2)$
`"123"`	$(+1, 123, 0)$
`"-123"`	$(-1, 123, 0)$
`"1.23E3"`	$(+1, 123, 1)$
`"1.23E+3"`	$(+1, 123, 1)$
`"12.3E+7"`	$(+1, 123, 6)$
`"12.0"`	$(+1, 120, -1)$
`"12.3"`	$(+1, 123, -1)$
`"0.00123"`	$(+1, 123, -5)$
`"-1.23E-12"`	$(-1, 123, -14)$

[343)] Non-arithmetic interchange formats are an optional feature in Annex H.

[344)] An implementation may use the d-char sequence to determine extra information to be represented in the NaN's significand.

 © ISO 2024 — All rights reserved

`"1234.5E-4"`	$(+1, 12345, -5)$
`"-0"`	$(-1, 0, 0)$
`"-0.00"`	$(-1, 0, -2)$
`"0E+7"`	$(+1, 0, 7)$
`"-0E-7"`	$(-1, 0, -7)$
`"12345678901234567890"`	$(+1, 1234567890123457, 4)$ or $(+1, 1234567890123456, 4)$ depending on rounding mode
`"1234E-400"`	$(+1, 12, -398)$ or $(+1, 13, -398)$ depending on rounding mode
`"1234E-402"`	$(+1, 0, -398)$ or $(+1, 1, -398)$ depending on rounding mode
`"1000."`	$(+1, 1000, 0)$
`".0001"`	$(+1, 1, -4)$
`"1000.e0"`	$(+1, 1000, 0)$
`".0001e0"`	$(+1, 1, -4)$
`"1000.0"`	$(+1, 10000, -1)$
`"0.0001"`	$(+1, 1, -4)$
`"1000.00"`	$(+1, 100000, -2)$
`"00.0001"`	$(+1, 1, -4)$
`"001000."`	$(+1, 1000, 0)$
`"001000.0"`	$(+1, 10000, -1)$
`"001000.00"`	$(+1, 100000, -2)$
`"00.00"`	$(+1, 0, -2)$
`"00."`	$(+1, 0, 0)$
`".00"`	$(+1, 0, -2)$
`"00.00e-5"`	$(+1, 0, -7)$
`"00.e-5"`	$(+1, 0, -5)$
`".00e-5"`	$(+1, 0, -7)$
`"0x1.8p+4"`	$(+1, 24, 0)$
`"infinite"`	infinity, and a pointer to `"inite"` is stored in the object pointed to by `endptr`, provided `endptr` is not a null pointer

7.24.1.7 The `strtol`, `strtoll`, `strtoul`, and `strtoull` functions

Synopsis

1

```
#include <stdlib.h>
long int strtol(const char * restrict nptr, char ** restrict endptr, int base);
long long int strtoll(const char * restrict nptr, char ** restrict endptr,
    int base);
unsigned long int strtoul(const char * restrict nptr, char ** restrict endptr,
    int base);
unsigned long long int strtoull(const char * restrict nptr,
    char ** restrict endptr, int base);
```

Description

2 The **`strtol`**, **`strtoll`**, **`strtoul`**, and **`strtoull`** functions convert the initial portion of the string pointed to by `nptr` to **`long int`**, **`long long int`**, **`unsigned long int`**, and **`unsigned long long int`** representation, respectively. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space characters, a subject sequence resembling an integer represented in some radix determined by the value of `base`, and a final string of one or more unrecognized characters, including the terminating null character of the input string. Then, they attempt to convert the subject sequence to an integer, and return the result.

3 If the value of `base` is zero, the expected form of the subject sequence is that of an integer constant as described in 6.4.4.2, optionally preceded by a plus or minus sign, but not including an integer suffix or any optional digit separators. If the value of `base` is between 2 and 36 (inclusive), the expected form of the subject sequence is a sequence of letters and digits representing an integer with the radix specified by `base`, optionally preceded by a plus or minus sign, but not including an integer suffix or any optional digit separators. The letters from `a` (or `A`) through `z` (or `Z`) are ascribed the values 10 through 35; only letters and digits whose ascribed values are less than that of `base` are permitted. If the value of base is 2, the characters `0b` or `0B` may optionally precede the sequence of letters and

© ISO 2024 — All rights reserved

digits, following the sign if present. If the value of `base` is 16, the characters `0x` or `0X` may optionally precede the sequence of letters and digits, following the sign if present.

4 The subject sequence is defined as the longest initial subsequence of the input string, starting with the first non-white-space character, that is of the expected form. The subject sequence contains no characters if the input string is empty or consists entirely of white-space characters, or if the first non-white-space character is other than a sign or a permissible letter or digit.

5 If the subject sequence has the expected form and the value of `base` is zero, the sequence of characters starting with the first digit is interpreted as an integer constant according to the rules of 6.4.4.2. If the subject sequence has the expected form and the value of `base` is between 2 and 36, it is used as the base for conversion, ascribing to each letter its value as previously given. If the subject sequence begins with a minus sign, the resulting value is the negative of the converted value; for functions whose return type is an unsigned integer type this action is performed in the return type. A pointer to the final string is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

6 In other than the `"C"` locale, additional locale-specific subject sequence forms may be accepted.

7 If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of `nptr` is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

Returns

8 The **`strtol`**, **`strtoll`**, **`strtoul`**, and **`strtoull`** functions return the converted value, if any. If no conversion could be performed, zero is returned. If the correct value is outside the range of representable values, **`LONG_MIN`**, **`LONG_MAX`**, **`LLONG_MIN`**, **`LLONG_MAX`**, **`ULONG_MAX`**, or **`ULLONG_MAX`** is returned (according to the return type and sign of the value, if any), and the value of the macro **`ERANGE`** is stored in **`errno`**.

7.24.2 Pseudo-random sequence generation functions

7.24.2.1 The `rand` function

Synopsis

1
```
#include <stdlib.h>
int rand(void);
```

Description

2 The **`rand`** function computes a sequence of pseudo-random integers in the range 0 to **`RAND_MAX`** inclusive.

3 The **`rand`** function is not required to avoid data races with other calls to pseudo-random sequence generation functions. The implementation shall behave as if no library function calls the **`rand`** function.

4 NOTE There are no guarantees as to the quality of the random sequence produced and some implementations are known to produce sequences with distressingly non-random low-order bits. Applications with particular requirements should use a generator that is known to be sufficient for their needs.

Returns

5 The **`rand`** function returns a pseudo-random integer.

Environmental limits

6 The value of the **`RAND_MAX`** macro shall be at least 32767.

7.24.2.2 The `srand` function

Synopsis

1
```
#include <stdlib.h>
void srand(unsigned int seed);
```

 © ISO 2024 — All rights reserved

Description

2 The **srand** function uses the argument as a seed for a new sequence of pseudo-random numbers to be returned by subsequent calls to **rand**. If **srand** is then called with the same seed value, the sequence of pseudo-random numbers shall be repeated. If **rand** is called before any calls to **srand** have been made, the same sequence shall be generated as when **srand** is first called with a seed value of 1.

3 The **srand** function is not required to avoid data races with other calls to pseudo-random sequence generation functions. The implementation shall behave as if no library function calls the **srand** function.

Returns

4 The **srand** function returns no value.

5 EXAMPLE The following functions define a portable implementation of **rand** and **srand**.

```
static unsigned long int next = 1;

int rand(void)   //  RAND_MAX assumed to be 32767
{
      next = next * 1103515245 + 12345;
      return (unsigned int)(next/65536) % 32768;
}

void srand(unsigned int seed)
{
      next = seed;
}
```

7.24.3 Memory management functions

1 The order and contiguity of storage allocated by successive calls to the **aligned_alloc**, **calloc**, **malloc**, and **realloc** functions is unspecified. The pointer returned if the allocation succeeds is suitably aligned so that it may be assigned to a pointer to any type of object with a fundamental alignment requirement and size less than or equal to the size requested. It may then be used to access such an object or an array of such objects in the space allocated (until the space is explicitly deallocated). The lifetime of an allocated object extends from the allocation until the deallocation. Each such allocation shall yield a pointer to an object disjoint from any other object. The pointer returned points to the start (lowest byte address) of the allocated space. If the space cannot be allocated, a null pointer is returned. If the size of the space requested is zero, the behavior is implementation-defined: either a null pointer is returned to indicate an error, or the behavior is as if the size were some nonzero value, except that the returned pointer shall not be used to access an object.

2 For purposes of determining the existence of a data race, memory allocation functions behave as though they accessed only memory locations accessible through their arguments and not other static duration storage. These functions may, however, visibly modify the storage that they allocate or deallocate. Calls to these functions that allocate or deallocate a particular region of memory shall occur in a single total order, and each such deallocation call shall synchronize with the next allocation (if any) in this order.

7.24.3.1 The **aligned_alloc** function

Synopsis

1

```
#include <stdlib.h>
void *aligned_alloc(size_t alignment, size_t size);
```

© ISO 2024 — All rights reserved

Description

2 The **aligned_alloc** function allocates space for an object whose alignment is specified by alignment,[345] whose size is specified by size, and whose representation is indeterminate. If the value of alignment is not a valid alignment supported by the implementation the function shall fail by returning a null pointer.

Returns

3 The **aligned_alloc** function returns either a null pointer or a pointer to the allocated space.

7.24.3.2 The calloc function

Synopsis

1
```
#include <stdlib.h>
void *calloc(size_t nmemb, size_t size);
```

Description

2 The **calloc** function allocates space for an array of nmemb objects, each of whose size is size. The space is initialized to all bits zero.[346]

Returns

3 The **calloc** function returns either a pointer to the allocated space or a null pointer if the space cannot be allocated or if the product nmemb * size would wraparound **size_t**.

7.24.3.3 The free function

Synopsis

1
```
#include <stdlib.h>
void free(void *ptr);
```

Description

2 The **free** function causes the space pointed to by ptr to be deallocated, that is, made available for further allocation. If ptr is a null pointer, no action occurs. Otherwise, if the argument does not match a pointer earlier returned by a memory management function, or if the space has been deallocated by a call to **free** or **realloc**, the behavior is undefined.

Returns

3 The **free** function returns no value.

7.24.3.4 The free_sized function

Synopsis

1
```
#include <stdlib.h>
void free_sized(void *ptr, size_t size);
```

Description

2 If ptr is a null pointer or the result obtained from a call to **malloc**, **realloc**, or **calloc**, where size size is equal to the requested allocation size, this function is equivalent to **free**(ptr). Otherwise, the behavior is undefined.

3 NOTE 1 A conforming implementation may ignore size and call **free**.

4 NOTE 2 The result of an **aligned_alloc** call may not be passed to **free_sized**.

Recommended practice

5 Implementations may provide extensions to query the usable size of an allocation, or to determine the usable size of the allocation that would result if a request for some other size were to succeed.

[345] The alignment requirements from 7.24.3 also apply even if the requested alignment is less strict.
[346] Note that this need not be the same as the representation of floating-point zero or a null pointer constant.

 © ISO 2024 — All rights reserved

Such implementations should allow passing the resulting usable size as the `size` parameter, and provide functionality equivalent to **`free`** in such cases.

Returns

6 The **`free_sized`** function returns no value.

7.24.3.5 The `free_aligned_sized` function

Synopsis

1
```
#include <stdlib.h>
void free_aligned_sized(void *ptr, size_t alignment, size_t size);
```

Description

2 If `ptr` is a null pointer or the result obtained from a call to **`aligned_alloc`**, where `alignment` is equal to the requested allocation alignment and `size` is equal to the requested allocation size, this function is equivalent to **`free`**`(ptr)`. Otherwise, the behavior is undefined.

3 NOTE 1 A conforming implementation may ignore `alignment` and `size` and call **`free`**.

4 NOTE 2 The result of an **`malloc`**, **`calloc`**, or **`realloc`** call may not be passed to **`free_aligned_sized`**.

Recommended practice

5 Implementations may provide extensions to query the usable size of an allocation, or to determine the usable size of the allocation that would result if a request for some other size were to succeed. Such implementations should allow passing the resulting usable size as the `size` parameter, and provide functionality equivalent to **`free`** in such cases.

Returns

6 The **`free_aligned_sized`** function returns no value.

7.24.3.6 The `malloc` function

Synopsis

1
```
#include <stdlib.h>
void *malloc(size_t size);
```

Description

2 The **`malloc`** function allocates space for an object whose size is specified by `size` and whose representation is indeterminate.

Returns

3 The **`malloc`** function returns either a null pointer or a pointer to the allocated space.

7.24.3.7 The `realloc` function

Synopsis

1
```
#include <stdlib.h>
void *realloc(void *ptr, size_t size);
```

Description

2 The **`realloc`** function deallocates the old object pointed to by `ptr` and returns a pointer to a new object that has the size specified by `size`. The contents of the new object shall be the same as that of the old object prior to deallocation, up to the lesser of the new and old sizes. Any bytes in the new object beyond the size of the old object have unspecified values.

3 If `ptr` is a null pointer, the **`realloc`** function behaves like the **`malloc`** function for the specified size. Otherwise, if `ptr` does not match a pointer earlier returned by a memory management function, or if the space has been deallocated by a call to the **`free`** or **`realloc`** function, or if the `size` is zero, the

© ISO 2024 — All rights reserved

behavior is undefined. If memory for the new object is not allocated, the old object is not deallocated and its value is unchanged.

Returns

4 The **realloc** function returns a pointer to the new object (which may have the same value as a pointer to the old object), or a null pointer if the new object has not been allocated.

7.24.4 Communication with the environment

7.24.4.1 The abort function

Synopsis

1
```
#include <stdlib.h>
[[noreturn]] void abort(void);
```

Description

2 The **abort** function causes abnormal program termination to occur, unless the signal **SIGABRT** is being caught and the signal handler does not return. Whether open streams with unwritten buffered data are flushed, open streams are closed, or temporary files are removed is implementation-defined. An implementation-defined form of the status *unsuccessful termination* is returned to the host environment by means of the function call **raise**(**SIGABRT**).

Returns

3 The **abort** function does not return to its caller.

7.24.4.2 The atexit function

Synopsis

1
```
#include <stdlib.h>
int atexit(void (*func)(void));
```

Description

2 The **atexit** function registers the function pointed to by `func`, to be called without arguments at normal program termination.[347)] It is unspecified whether a call to the **atexit** function that does not happen before the **exit** function is called will succeed.

Environmental limits

3 The implementation shall support the registration of at least 32 functions.

Returns

4 The **atexit** function returns zero if the registration succeeds, nonzero if it fails.

Forward references: the **at_quick_exit** function (7.24.4.3), the **exit** function (7.24.4.4).

7.24.4.3 The at_quick_exit function

Synopsis

1
```
#include <stdlib.h>
int at_quick_exit(void (*func)(void));
```

Description

2 The **at_quick_exit** function registers the function pointed to by `func`, to be called without arguments should **quick_exit** be called.[348)] It is unspecified whether a call to the **at_quick_exit** function that does not happen before the **quick_exit** function is called will succeed.

[347)] The **atexit** function registrations are distinct from the **at_quick_exit** registrations, so applications potentially need to call both registration functions with the same argument.

[348)] The **at_quick_exit** function registrations are distinct from the **atexit** registrations, so applications potentially need to call both registration functions with the same argument.

 © ISO 2024 — All rights reserved

Environmental limits

3 The implementation shall support the registration of at least 32 functions.

Returns

4 The **at_quick_exit** function returns zero if the registration succeeds, nonzero if it fails.

Forward references: the **quick_exit** function (7.24.4.7).

7.24.4.4 The exit function

Synopsis

1
```
#include <stdlib.h>
[[noreturn]] void exit(int status);
```

Description

2 The **exit** function causes normal program termination to occur. No functions registered by the **at_quick_exit** function are called. If a program calls the **exit** function more than once, or calls the **quick_exit** function in addition to the **exit** function, the behavior is undefined.

3 First, all functions registered by the **atexit** function are called, in the reverse order of their registration,[349] except that a function is called after any previously registered functions that had already been called at the time it was registered. If, during the call to any such function, a call to the **longjmp** function is made that would terminate the call to the registered function, the behavior is undefined.

4 Next, all open streams with unwritten buffered data are flushed, all open streams are closed, and all files created by the **tmpfile** function are removed.

5 Finally, control is returned to the host environment. If the value of `status` is zero or **EXIT_SUCCESS**, an implementation-defined form of the status *successful termination* is returned. If the value of `status` is **EXIT_FAILURE**, an implementation-defined form of the status *unsuccessful termination* is returned. Otherwise the status returned is implementation-defined.

Returns

6 The **exit** function cannot return to its caller.

7.24.4.5 The _Exit function

Synopsis

1
```
#include <stdlib.h>
[[noreturn]] void _Exit(int status);
```

Description

2 The **_Exit** function causes normal program termination to occur and control to be returned to the host environment. No functions registered by the **atexit** function, the **at_quick_exit** function, or signal handlers registered by the **signal** function are called. The status returned to the host environment is determined in the same way as for the **exit** function (7.24.4.4). Whether open streams with unwritten buffered data are flushed, open streams are closed, or temporary files are removed is implementation-defined.

Returns

3 The **_Exit** function cannot return to its caller.

7.24.4.6 The getenv function

Synopsis

1
```
#include <stdlib.h>
char *getenv(const char *name);
```

[349] Each function is called as many times as it was registered, and in the correct order with respect to other registered functions.

© ISO 2024 — All rights reserved

Description

2 The **getenv** function searches an *environment list*, provided by the host environment, for a string that matches the string pointed to by `name`. The set of environment names and the method for altering the environment list are implementation-defined. The **getenv** function is not required to avoid data races with other threads of execution that modify the environment list.[350)]

3 The implementation shall behave as if no library function calls the **getenv** function.

Returns

4 The **getenv** function returns a pointer to a string associated with the matched list member. The string pointed to shall not be modified by the program, but may be overwritten by a subsequent call to the **getenv** function. If the specified `name` cannot be found, a null pointer is returned.

7.24.4.7 The quick_exit function

Synopsis

1
```
#include <stdlib.h>
[[noreturn]] void quick_exit(int status);
```

Description

2 The **quick_exit** function causes normal program termination to occur. No functions registered by the **atexit** function or signal handlers registered by the **signal** function are called. If a program calls the **quick_exit** function more than once, or calls the **exit** function in addition to the **quick_exit** function, the behavior is undefined. If a signal is raised while the **quick_exit** function is executing, the behavior is undefined.

3 The **quick_exit** function first calls all functions registered by the **at_quick_exit** function, in the reverse order of their registration,[351)] except that a function is called after any previously registered functions that had already been called at the time it was registered. If, during the call to any such function, a call to the **longjmp** function is made that would terminate the call to the registered function, the behavior is undefined.

4 Then control is returned to the host environment by means of the function call **_Exit**(`status`).

Returns

5 The **quick_exit** function cannot return to its caller.

7.24.4.8 The system function

Synopsis

1
```
#include <stdlib.h>
int system(const char *string);
```

Description

2 If `string` is a null pointer, the **system** function determines whether the host environment has a *command processor*. If `string` is not a null pointer, the **system** function passes the string pointed to by `string` to that command processor to be executed in a manner which the implementation shall document; this can then cause the program calling **system** to behave in a non-conforming manner or to terminate.

Returns

3 If the argument is a null pointer, the **system** function returns nonzero only if a command processor is available. If the argument is not a null pointer, and the **system** function does return, it returns an implementation-defined value.

[350)] Many implementations provide non-standard functions that modify the environment list.

[351)] Each function is called as many times as it was registered, and in the correct order with respect to other registered functions.

 © ISO 2024 — All rights reserved

7.24.5 Searching and sorting utilities

1 These utilities make use of a comparison function to search or sort arrays of unspecified type. Where an argument declared as **size_t** nmemb specifies the length of the array for a function, nmemb can have the value zero on a call to that function; the comparison function is not called, a search finds no matching element, and sorting performs no rearrangement. Pointer arguments on such a call shall still have valid values, as described in 7.1.4.

2 The implementation shall ensure that the second argument of the comparison function (when called from **bsearch**), or both arguments (when called from **qsort**), are pointers to elements of the array.[352)] The first argument when called from **bsearch** shall equal key.

3 The comparison function shall not alter the contents of the array. The implementation may reorder elements of the array between calls to the comparison function, but shall not alter the contents of any individual element.

4 When the same objects (consisting of size bytes, irrespective of their current positions in the array) are passed more than once to the comparison function, the results shall be consistent with one another. That is, for **qsort** they shall define a total ordering on the array, and for **bsearch** the same object shall always compare the same way with the key.

5 A sequence point occurs immediately before and immediately after each call to the comparison function, and also between any call to the comparison function and any movement of the objects passed as arguments to that call.

7.24.5.1 The bsearch generic function

Synopsis

1
```
#include <stdlib.h>
QVoid *bsearch(const void *key, QVoid *base, size_t nmemb, size_t size,
      int (*compar)(const void *, const void *));
```

Description

2 The **bsearch** generic function searches an array of nmemb objects, the initial element of which is pointed to by base, for an element that matches the object pointed to by key. The size of each element of the array is specified by size.

3 The comparison function pointed to by compar is called with two arguments that point to the key object and to an array element, in that order. The function shall return an integer less than, equal to, or greater than zero if the key object is considered, respectively, to be less than, to match, or to be greater than the array element. The array shall consist of: all the elements that compare less than, all the elements that compare equal to, and all the elements that compare greater than the key object, in that order.[353)]

Returns

4 The **bsearch** generic function returns a pointer to a matching element of the array, or a null pointer if no match is found. If two elements compare as equal, which element is matched is unspecified.

5 The **bsearch** function is generic in the qualification of the type pointed to by the argument base. If this argument is a pointer to a **const**-qualified object type, the returned pointer will be a pointer to **const**-qualified **void**. Otherwise, the argument shall be a pointer to an unqualified object type or a null pointer constant,[354)] and the returned pointer will be a pointer to unqualified **void**.

[352)] That is, if the value passed is p, then the following expressions are always nonzero:

```
((char *)p - (char *)base) % size == 0
(char *)p >= (char *)base
(char *)p  <  (char *)base + nmemb * size
```

[353)] In practice, the entire array is sorted according to the comparison function.

[354)] If the argument is a null pointer and the call is executed, the behavior is undefined.

© ISO 2024 — All rights reserved

The external declaration of **bsearch** has the concrete type:

```
void * (const void *, const void *, size_t, size_t,
      int (*) (const void *, const void *))
```

which supports all correct uses. If a macro definition of this generic function is suppressed to access an actual function, the external declaration with this concrete type is visible.[355)]

7.24.5.2 The qsort function

Synopsis

1
```
#include <stdlib.h>
void qsort(void *base, size_t nmemb, size_t size,
      int (*compar)(const void *, const void *));
```

Description

2 The **qsort** function sorts an array of nmemb objects, the initial element of which is pointed to by base. The size of each object is specified by size.

3 The contents of the array are sorted into ascending order according to a comparison function pointed to by compar, which is called with two arguments that point to the objects being compared. The function shall return an integer less than, equal to, or greater than zero if the first argument is considered to be respectively less than, equal to, or greater than the second.

4 If two elements compare as equal, their order in the resulting sorted array is unspecified.

Returns

5 The **qsort** function returns no value.

7.24.6 Integer arithmetic functions

7.24.6.1 The abs, labs, and llabs functions

Synopsis

1
```
#include <stdlib.h>
int abs(int j);
long int labs(long int j);
long long int llabs(long long int j);
```

Description

2 The **abs**, **labs**, and **llabs** functions compute the absolute value of an integer j. If the result cannot be represented, the behavior is undefined.[356)]

Returns

3 The **abs**, **labs**, and **llabs**, functions return the absolute value.

7.24.6.2 The div, ldiv, and lldiv functions

Synopsis

1
```
#include <stdlib.h>
div_t div(int numer, int denom);
ldiv_t ldiv(long int numer, long int denom);
lldiv_t lldiv(long long int numer, long long int denom);
```

Description

2 The **div**, **ldiv**, and **lldiv**, functions compute numer/denom and numer%denom in a single operation.

[355)] This is an obsolescent feature.
[356)] The absolute value of the most negative number is not representable.

 © ISO 2024 — All rights reserved

Returns

3 The **div**, **ldiv**, and **lldiv** functions return a structure of type **div_t**, **ldiv_t**, and **lldiv_t**, respectively, comprising both the quotient and the remainder. The structures shall contain (in either order) the members quot (the quotient) and rem (the remainder), each of which has the same type as the arguments numer and denom. If either part of the result cannot be represented, the behavior is undefined.

7.24.7 Multibyte/wide character conversion functions

1 The behavior of the multibyte character functions is affected by the **LC_CTYPE** category of the current locale. For a state-dependent encoding, each of the **mbtowc** and **wctomb** functions is placed into its initial conversion state prior to the first call to the function and can be returned to that state by a call for which its character pointer argument, s, is a null pointer. Subsequent calls with s as other than a null pointer cause the internal conversion state of the function to be altered as necessary. It is implementation-defined whether internal conversion state has thread storage duration, and whether a newly created thread has the same state as the current thread at the time of creation, or the initial conversion state. A call with s as a null pointer causes these functions to return a nonzero value if encodings have state dependency, and zero otherwise.[357)] It is implementation-defined whether these functions avoid data races with other calls to the same function.

2 Changing the **LC_CTYPE** category causes the internal object describing the conversion state of the **mbtowc** and **wctomb** functions to have an indeterminate representation.

7.24.7.1 The mblen function

Synopsis

1
```
#include <stdlib.h>
int mblen(const char *s, size_t n);
```

Description

2 If s is not a null pointer, the **mblen** function determines the number of bytes contained in the multibyte character pointed to by s. Except that the conversion state of the **mbtowc** function is not affected, it is equivalent to

```
mbtowc((wchar_t *)0, (const char *)0, 0);
mbtowc((wchar_t *)0, s, n);
```

Returns

3 If s is a null pointer, the **mblen** function returns a nonzero or zero value, if multibyte character encodings, respectively, do or do not have state-dependent encodings. If s is not a null pointer, the **mblen** function either returns 0 (if s points to the null character), or returns the number of bytes that are contained in the multibyte character (if the next n or fewer bytes form a valid multibyte character), or returns -1 (if they do not form a valid multibyte character).

Forward references: the **mbtowc** function (7.24.7.2).

7.24.7.2 The mbtowc function

Synopsis

1
```
#include <stdlib.h>
int mbtowc(wchar_t * restrict pwc, const char * restrict s, size_t n);
```

Description

2 If s is not a null pointer, the **mbtowc** function inspects at most n bytes beginning with the byte pointed to by s to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is

[357)] If the locale employs special bytes to change the shift state, these bytes do not produce separate wide character codes, but are grouped with an adjacent multibyte character.

© ISO 2024 — All rights reserved

complete and valid, it determines the value of the corresponding wide character and then, if `pwc` is not a null pointer, stores that value in the object pointed to by `pwc`. If the corresponding wide character is the null wide character, the function is left in the initial conversion state.

3 The implementation shall behave as if no library function calls the **`mbtowc`** function.

Returns

4 If `s` is a null pointer, the **`mbtowc`** function returns a nonzero or zero value, if multibyte character encodings, respectively, do or do not have state-dependent encodings. If `s` is not a null pointer, the **`mbtowc`** function either returns 0 (if `s` points to the null character), or returns the number of bytes that are contained in the converted multibyte character (if the next `n` or fewer bytes form a valid multibyte character), or returns `-1` (if they do not form a valid multibyte character).

5 In no case will the value returned be greater than `n` or the value of the **`MB_CUR_MAX`** macro.

7.24.7.3 The `wctomb` function

Synopsis

1
```
#include <stdlib.h>
int wctomb(char *s, wchar_t wc);
```

Description

2 The **`wctomb`** function determines the number of bytes needed to represent the multibyte character corresponding to the wide character given by `wc` (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by `s` (if `s` is not a null pointer). At most **`MB_CUR_MAX`** characters are stored. If `wc` is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state, and the function is left in the initial conversion state.

3 The implementation shall behave as if no library function calls the **`wctomb`** function.

Returns

4 If `s` is a null pointer, the **`wctomb`** function returns a nonzero or zero value, if multibyte character encodings, respectively, do or do not have state-dependent encodings. If `s` is not a null pointer, the **`wctomb`** function returns `-1` if the value of `wc` does not correspond to a valid multibyte character, or returns the number of bytes that are contained in the multibyte character corresponding to the value of `wc`.

5 In no case will the value returned be greater than the value of the **`MB_CUR_MAX`** macro.

7.24.8 Multibyte/wide string conversion functions

1 The behavior of the multibyte string functions is affected by the **`LC_CTYPE`** category of the current locale.

7.24.8.1 The `mbstowcs` function

Synopsis

1
```
#include <stdlib.h>
size_t mbstowcs(wchar_t * restrict pwcs, const char * restrict s, size_t n);
```

Description

2 The **`mbstowcs`** function converts a sequence of multibyte characters that begins in the initial shift state from the array pointed to by `s` into a sequence of corresponding wide characters and stores not more than `n` wide characters into the array pointed to by `pwcs`. No multibyte characters that follow a null character (which is converted into a null wide character) will be examined or converted. Each multibyte character is converted as if by a call to the **`mbtowc`** function, except that the conversion state of the **`mbtowc`** function is not affected.

3 No more than `n` elements will be modified in the array pointed to by `pwcs`. If copying takes place between objects that overlap, the behavior is undefined.

 © ISO 2024 — All rights reserved

Returns

4 If an invalid multibyte character is encountered, the **mbstowcs** function returns **(size_t)(-1)**. Otherwise, the **mbstowcs** function returns the number of array elements modified, not including a terminating null wide character, if any.[358)]

7.24.8.2 The wcstombs function

Synopsis

1
```
#include <stdlib.h>
size_t wcstombs(char * restrict s, const wchar_t * restrict pwcs, size_t n);
```

Description

2 The **wcstombs** function converts a sequence of wide characters from the array pointed to by pwcs into a sequence of corresponding multibyte characters that begins in the initial shift state, and stores these multibyte characters into the array pointed to by s, stopping if a multibyte character would exceed the limit of n total bytes or if a null character is stored. Each wide character is converted as if by a call to the **wctomb** function, except that the conversion state of the **wctomb** function is not affected.

3 No more than n bytes will be modified in the array pointed to by s. If copying takes place between objects that overlap, the behavior is undefined.

Returns

4 If a wide character is encountered that does not correspond to a valid multibyte character, the **wcstombs** function returns **(size_t)(-1)**. Otherwise, the **wcstombs** function returns the number of bytes modified, not including a terminating null character, if any.[358)]

7.24.9 Alignment of memory

7.24.9.1 The memalignment function

Synopsis

1
```
#include <stdlib.h>
size_t memalignment(const void *p);
```

Description

2 The **memalignment** function accepts a pointer to any object and returns the maximum alignment satisfied by its address value. The alignment may be an extended alignment and may also be beyond the range supported by the implementation for explicit use by **alignas**.[359)] If so, it will satisfy all alignments usable by the implementation. The value returned can be compared to the result of **alignof**, and if it is greater or equal, the alignment requirement for the type operand is satisfied.

Returns

3 The alignment of the pointer p, which is a power of two. If p is a null pointer, an alignment of zero is returned.

4 NOTE An alignment of zero indicates that the tested pointer cannot be used to access an object of any type.

[358)] The array will not be null-terminated if the value returned is n.

[359)] The actual alignment of an object may be stricter than the alignment requested for an object by **alignas** or (implicitly) by an allocation function, but will always satisfy it.

© ISO 2024 — All rights reserved

7.25 `_Noreturn` `<stdnoreturn.h>`

1 The header `<stdnoreturn.h>` defines the macro

```
noreturn
```

which expands to **`_Noreturn`**.

2 The **`noreturn`** macro and the `<stdnoreturn.h>` header are obsolescent features.

© ISO 2024 — All rights reserved

7.26 String handling <string.h>

7.26.1 String function conventions

1 The header <string.h> declares one type, several functions, several type-generic functions, and defines two macros useful for manipulating arrays of character type and other objects treated as arrays of character type.[360] The type is **size_t** and one of the macros is **NULL** (both described in 7.21). Various methods are used for determining the lengths of the arrays, but in all cases a **char** * or **void** * argument points to the initial (lowest addressed) character of the array. If an array is accessed beyond the end of an object, the behavior is undefined.

2 The macro

```
__STDC_VERSION_STRING_H__
```

is an integer constant expression with a value equivalent to 202311L.

3 Where an argument declared as **size_t** n specifies the length of the array for a function, n can have the value zero on a call to that function. Unless explicitly stated otherwise in the description of a particular function in this subclause, pointer arguments on such a call shall still have valid values, as described in 7.1.4. On such a call, a function that locates a character finds no occurrence, a function that compares two character sequences returns zero, and a function that copies characters copies zero characters.

4 For all functions in this subclause, each character shall be interpreted as if it had the type **unsigned char** (and therefore every possible object representation is valid and has a different value).

7.26.2 Copying functions

7.26.2.1 The memcpy function

Synopsis

1
```
#include <string.h>
void *memcpy(void * restrict s1, const void * restrict s2, size_t n);
```

Description

2 The **memcpy** function copies n characters from the object pointed to by s2 into the object pointed to by s1. If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **memcpy** function returns the value of s1.

7.26.2.2 The memccpy function

Synopsis

1
```
#include <string.h>
void *memccpy(void * restrict s1, const void * restrict s2, int c, size_t n);
```

Description

2 The **memccpy** function copies characters from the object pointed to by s2 into the object pointed to by s1, stopping after the first occurrence of character c (converted to an **unsigned char**) is copied, or after n characters are copied, whichever comes first. If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **memccpy** function returns a pointer to the character after the copy of c in s1, or a null pointer if c was not found in the first n characters of s2.

7.26.2.3 The memmove function

[360] See "future library directions" (7.33.17).

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <string.h>
void *memmove(void *s1, const void *s2, size_t n);
```

Description

2 The **memmove** function copies `n` characters from the object pointed to by `s2` into the object pointed to by `s1`. Copying takes place as if the `n` characters from the object pointed to by `s2` are first copied into a temporary array of `n` characters that does not overlap the objects pointed to by `s1` and `s2`, and then the `n` characters from the temporary array are copied into the object pointed to by `s1`.

Returns

3 The **memmove** function returns the value of `s1`.

7.26.2.4 The strcpy function

Synopsis

1
```
#include <string.h>
char *strcpy(char * restrict s1, const char * restrict s2);
```

Description

2 The **strcpy** function copies the string pointed to by `s2` (including the terminating null character) into the array pointed to by `s1`. If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **strcpy** function returns the value of `s1`.

7.26.2.5 The strncpy function

Synopsis

1
```
#include <string.h>
char *strncpy(char * restrict s1, const char * restrict s2, size_t n);
```

Description

2 The **strncpy** function copies not more than `n` characters (characters that follow a null character are not copied) from the array pointed to by `s2` to the array pointed to by `s1`.[361] If copying takes place between objects that overlap, the behavior is undefined.

3 If the array pointed to by `s2` is a string that is shorter than `n` characters, null characters are appended to the copy in the array pointed to by `s1`, until `n` characters in all have been written.

Returns

4 The **strncpy** function returns the value of `s1`.

7.26.2.6 The strdup function

Synopsis

1
```
#include <string.h>
char *strdup(const char *s);
```

Description

2 The **strdup** function creates a copy of the string pointed to by `s` in a space allocated as if by a call to **malloc**.

[361] Thus, if there is no null character in the first `n` characters of the array pointed to by `s2`, the result will not be null-terminated.

© ISO 2024 — All rights reserved

Returns

3 The **strdup** function returns a pointer to the first character of the duplicate string. The returned pointer can be passed to **free**. If no space can be allocated the **strdup** function returns a null pointer.

7.26.2.7 The strndup function

Synopsis

1
```
#include <string.h>
char *strndup(const char *s, size_t n);
```

Description

2 The **strndup** function creates a string initialized with no more than `n` initial characters of the array pointed to by `s` and up to the first null character, whichever comes first, in a space allocated as if by a call to **malloc**. If the array pointed to by `s` does not contain a null within the first `n` characters, a null is appended to the copy of the array.

Returns

3 The **strndup** function returns a pointer to the first character of the created string. The returned pointer can be passed to **free**. If space cannot be allocated the **strndup** function returns a null pointer.

7.26.3 Concatenation functions

7.26.3.1 The strcat function

Synopsis

1
```
#include <string.h>
char *strcat(char * restrict s1, const char * restrict s2);
```

Description

2 The **strcat** function appends a copy of the string pointed to by `s2` (including the terminating null character) to the end of the string pointed to by `s1`. The initial character of `s2` overwrites the null character at the end of `s1`. If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **strcat** function returns the value of `s1`.

7.26.3.2 The strncat function

Synopsis

1
```
#include <string.h>
char *strncat(char * restrict s1, const char * restrict s2, size_t n);
```

Description

2 The **strncat** function appends not more than `n` characters (a null character and characters that follow it are not appended) from the array pointed to by `s2` to the end of the string pointed to by `s1`. The initial character of `s2` overwrites the null character at the end of `s1`. A terminating null character is always appended to the result.[362)] If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **strncat** function returns the value of `s1`.

Forward references: the **strlen** function (7.26.6.4).

[362)] Thus, the maximum number of characters that can end up in the array pointed to by `s1` is **strlen**`(s1)+n+1`.

© ISO 2024 — All rights reserved

7.26.4 Comparison functions

1 The sign of a nonzero value returned by the comparison functions **memcmp**, **strcmp**, and **strncmp** is determined by the sign of the difference between the values of the first pair of characters (both interpreted as **unsigned char**) that differ in the objects being compared.

7.26.4.1 The memcmp function

Synopsis

1
```
#include <string.h>
int memcmp(const void *s1, const void *s2, size_t n);
```

Description

2 The **memcmp** function compares the first `n` characters of the object pointed to by `s1` to the first `n` characters of the object pointed to by `s2`.[363)]

Returns

3 The **memcmp** function returns an integer greater than, equal to, or less than zero, accordingly as the object pointed to by `s1` is greater than, equal to, or less than the object pointed to by `s2`.

7.26.4.2 The strcmp function

Synopsis

1
```
#include <string.h>
int strcmp(const char *s1, const char *s2);
```

Description

2 The **strcmp** function compares the string pointed to by `s1` to the string pointed to by `s2`.

Returns

3 The **strcmp** function returns an integer greater than, equal to, or less than zero, accordingly as the string pointed to by `s1` is greater than, equal to, or less than the string pointed to by `s2`.

7.26.4.3 The strcoll function

Synopsis

1
```
#include <string.h>
int strcoll(const char *s1, const char *s2);
```

Description

2 The **strcoll** function compares the string pointed to by `s1` to the string pointed to by `s2`, both interpreted as appropriate to the **LC_COLLATE** category of the current locale.

Returns

3 The **strcoll** function returns an integer greater than, equal to, or less than zero, accordingly as the string pointed to by `s1` is greater than, equal to, or less than the string pointed to by `s2` when both are interpreted as appropriate to the current locale.

7.26.4.4 The strncmp function

Synopsis

1
```
#include <string.h>
int strncmp(const char *s1, const char *s2, size_t n);
```

[363)] The unused bytes used as padding for purposes of alignment within structure objects take on unspecified values when a value is stored in the object (see 6.2.6.1). Strings shorter than their allocated space and unions can also cause problems in comparison.

© ISO 2024 — All rights reserved

Description

2 The **strncmp** function compares not more than n characters (characters that follow a null character are not compared) from the array pointed to by s1 to the array pointed to by s2.

Returns

3 The **strncmp** function returns an integer greater than, equal to, or less than zero, accordingly as the possibly null-terminated array pointed to by s1 is greater than, equal to, or less than the possibly null-terminated array pointed to by s2.

7.26.4.5 The strxfrm function

Synopsis

1
```
#include <string.h>
size_t strxfrm(char * restrict s1, const char * restrict s2, size_t n);
```

Description

2 The **strxfrm** function transforms the string pointed to by s2 and places the resulting string into the array pointed to by s1. The transformation is such that if the **strcmp** function is applied to two transformed strings, it returns a value greater than, equal to, or less than zero, corresponding to the result of the **strcoll** function applied to the same two original strings. No more than n characters are placed into the resulting array pointed to by s1, including the terminating null character. If n is zero, s1 is permitted to be a null pointer. If copying takes place between objects that overlap, the behavior is undefined.

Returns

3 The **strxfrm** function returns the length of the transformed string (not including the terminating null character). If the value returned is n or more, the members of the array pointed to by s1 have an indeterminate representation.

4 EXAMPLE The value of the following expression is the size of the array needed to hold the transformation of the string pointed to by s.

```
1 + strxfrm(nullptr, s, 0)
```

7.26.5 Search functions

7.26.5.1 Introduction

1 The stateless search functions in this section (**memchr**, **strchr**, **strpbrk**, **strrchr**, **strstr**) are *generic functions*. These functions are generic in the qualification of the array to be searched and will return a result pointer to an element with the same qualification as the passed array. If the array to be searched is **const**-qualified, the result pointer will be to a **const**-qualified element. If the array to be searched is not **const**-qualified,[364] the result pointer will be to an unqualified element.

2 The external declarations of these generic functions have a concrete function type that returns a pointer to an unqualified element (of type **char** when specified as *QChar*, and **void** when specified as *QVoid*), and accepts a pointer to a **const**-qualified array of the same type to search. This signature supports all correct uses. If a macro definition of any of these generic functions is suppressed to access an actual function, the external declaration with the corresponding concrete type is visible.[365]

3 The **volatile** and **restrict** qualifiers are not accepted on the elements of the array to search.

7.26.5.2 The memchr generic function

Synopsis

1
```
#include <string.h>
QVoid *memchr(QVoid *s, int c, size_t n);
```

[364] The null pointer constant is not a pointer to a **const**-qualified type, and therefore the result expression has the type of a pointer to an unqualified element; however, evaluating such a call is undefined.
[365] This is an obsolescent feature.

© ISO 2024 — All rights reserved

Description

2 The **memchr** generic function locates the first occurrence of c (converted to an **unsigned char**) in the initial n characters (each interpreted as **unsigned char**) of the object pointed to by s. The implementation shall behave as if it reads the characters sequentially and stops as soon as a matching character is found.

Returns

3 The **memchr** generic function returns a pointer to the located character, or a null pointer if the character does not occur in the object.

7.26.5.3 The strchr generic function

Synopsis

1
```
#include <string.h>
QChar *strchr(QChar *s, int c);
```

Description

2 The **strchr** generic function locates the first occurrence of c (converted to a **char**) in the string pointed to by s. The terminating null character is considered to be part of the string.

Returns

3 The **strchr** generic function returns a pointer to the located character, or a null pointer if the character does not occur in the string.

7.26.5.4 The strcspn function

Synopsis

1
```
#include <string.h>
size_t strcspn(const char *s1, const char *s2);
```

Description

2 The **strcspn** function computes the length of the maximum initial segment of the string pointed to by s1 which consists entirely of characters *not* from the string pointed to by s2.

Returns

3 The **strcspn** function returns the length of the segment.

7.26.5.5 The strpbrk generic function

Synopsis

1
```
#include <string.h>
QChar *strpbrk(QChar *s1, const char *s2);
```

Description

2 The **strpbrk** generic function locates the first occurrence in the string pointed to by s1 of any character from the string pointed to by s2.

Returns

3 The **strpbrk** generic function returns a pointer to the character, or a null pointer if no character from s2 occurs in s1.

7.26.5.6 The strrchr generic function

Synopsis

1
```
#include <string.h>
QChar *strrchr(QChar *s, int c);
```


© ISO 2024 — All rights reserved

Description

2 The **strrchr** generic function locates the last occurrence of c (converted to a **char**) in the string pointed to by s. The terminating null character is considered to be part of the string.

Returns

3 The **strrchr** generic function returns a pointer to the character, or a null pointer if c does not occur in the string.

7.26.5.7 The strspn function

Synopsis

1
```
#include <string.h>
size_t strspn(const char *s1, const char *s2);
```

Description

2 The **strspn** function computes the length of the maximum initial segment of the string pointed to by s1 which consists entirely of characters from the string pointed to by s2.

Returns

3 The **strspn** function returns the length of the segment.

7.26.5.8 The strstr generic function

Synopsis

1
```
#include <string.h>
QChar *strstr(QChar *s1, const char *s2);
```

Description

2 The **strstr** generic function locates the first occurrence in the string pointed to by s1 of the sequence of characters (excluding the terminating null character) in the string pointed to by s2.

Returns

3 The **strstr** generic function returns a pointer to the located string, or a null pointer if the string is not found. If s2 points to a string with zero length, the function returns s1.

7.26.5.9 The strtok function

Synopsis

1
```
#include <string.h>
char *strtok(char * restrict s1, const char * restrict s2);
```

Description

2 A sequence of calls to the **strtok** function breaks the string pointed to by s1 into a sequence of tokens, each of which is delimited by a character from the string pointed to by s2. The first call in the sequence has a non-null first argument; subsequent calls in the sequence have a null first argument. If any of the subsequent calls in the sequence is made by a different thread than the first, the behavior is undefined. The separator string pointed to by s2 may be different from call to call.

3 The first call in the sequence searches the string pointed to by s1 for the first character that is *not* contained in the current separator string pointed to by s2. If no such character is found, then there are no tokens in the string pointed to by s1 and the **strtok** function returns a null pointer. If such a character is found, it is the start of the first token.

4 The **strtok** function then searches from there for a character that *is* contained in the current separator string. If no such character is found, the current token extends to the end of the string pointed to by s1, and subsequent searches for a token will return a null pointer. If such a character is found, it is overwritten by a null character, which terminates the current token. The **strtok** function saves a pointer to the following character, from which the next search for a token will start.

© ISO 2024 — All rights reserved

5 Each subsequent call, with a null pointer as the value of the first argument, starts searching from the saved pointer and behaves as described previously.

6 The **strtok** function is not required to avoid data races with other calls to the **strtok** function.[366] The implementation shall behave as if no library function calls the **strtok** function.

Returns

7 The **strtok** function returns a pointer to the first character of a token, or a null pointer if there is no token.

8 EXAMPLE

```
#include <string.h>
static char str[] = "?a???b,,,#c";
char *t;

t = strtok(str, "?");      // t points to the token "a"
t = strtok(nullptr, ",");  // t points to the token "??b"
t = strtok(nullptr, "#,"); // t points to the token "c"
t = strtok(nullptr, "?");  // t is a null pointer
```

Forward references: The **strtok_s** function (K.3.7.4.1).

7.26.6 Miscellaneous functions

7.26.6.1 The memset function

Synopsis

1
```
#include <string.h>
void *memset(void *s, int c, size_t n);
```

Description

2 The **memset** function copies the value of c (converted to an **unsigned char**) into each of the first n characters of the object pointed to by s.

Returns

3 The **memset** function returns the value of s.

7.26.6.2 The memset_explicit function

Synopsis

1
```
#include <string.h>
void *memset_explicit(void *s, int c, size_t n);
```

Description

2 The **memset_explicit** function copies the value of c (converted to an **unsigned char**) into each of the first n characters of the object pointed to by s. The purpose of this function is to make sensitive information stored in the object inaccessible.[367]

Returns

3 The **memset_explicit** function returns the value of s.

7.26.6.3 The strerror function

Synopsis

1
```
#include <string.h>
char *strerror(int errnum);
```

[366] The **strtok_s** function can be used instead to avoid data races.

[367] The intention is that the memory store is always performed (i.e. never elided), regardless of optimizations. This is in contrast to calls to the **memset** function (7.26.6.1)

 © ISO 2024 — All rights reserved

Description

2 The **strerror** function maps the number in errnum to a message string. Typically, the values for errnum come from **errno**, but **strerror** shall map any value of type **int** to a message.

3 The **strerror** function is not required to avoid data races with other calls to the **strerror** function.[368] The implementation shall behave as if no library function calls the **strerror** function.

Returns

4 The **strerror** function returns a pointer to the string, the contents of which are locale-specific. The array pointed to shall not be modified by the program. The behavior is undefined if the returned value is used after a subsequent call to the **strerror** function, or after the thread which called the function to obtain the returned value has exited.

Forward references: The **strerror_s** function (K.3.7.5.2).

7.26.6.4 The strlen function

Synopsis

1
```
#include <string.h>
size_t strlen(const char *s);
```

Description

2 The **strlen** function computes the length of the string pointed to by s.

Returns

3 The **strlen** function returns the number of characters that precede the terminating null character.

[368] The **strerror_s** function can be used instead to avoid data races.

© ISO 2024 — All rights reserved

7.27 Type-generic math <tgmath.h>

1 The header <tgmath.h> includes the headers <math.h> and <complex.h> and defines several type-generic macros.

2 The feature test macro **__STDC_VERSION_TGMATH_H__** expands to the token 202311L.

3 This clause specifies a many-to-one correspondence of functions in <math.h> and <complex.h> with *type-generic macros*.[369] Use of a type-generic macro invokes a corresponding function whose type is determined by the types of the arguments for particular parameters called the *generic parameters*.[370]

4 Of the <math.h> and <complex.h> functions without an **f** (**float**) or **l** (**long double**) suffix, several have one or more parameters whose corresponding real type is **double**. For each such function, except the functions that round result to narrower type (7.12.14) (which are covered subsequently in this subclause) and **modf**, there is a corresponding type-generic macro. The parameters whose corresponding real type is **double** in the function synopsis are generic parameters.

5 Some of the <math.h> functions for decimal floating types have no unsuffixed counterpart. Of these functions with a **d64** suffix, some have one or more parameters whose type is **_Decimal64**. For each such function, except **decodedecd64**, **encodedecd64**, **decodebind64**, and **encodebind64**, there is a corresponding type-generic macro. The parameters whose real type is **_Decimal64** in the function synopsis are generic parameters.

6 If arguments for generic parameters of a type-generic macro are such that some argument has a corresponding real type that is of standard floating type and another argument is of decimal floating type, the behavior is undefined.

7 Except for the macros for functions that round result to a narrower type (7.12.14), use of a type-generic macro invokes a function whose generic parameters have the corresponding real type determined by the types of the arguments for the generic parameters as follows:

— Arguments of integer type are regarded as having type **_Decimal64** if any argument has decimal floating type, and as having type **double** otherwise.

— If the function has exactly one generic parameter, the type determined is the corresponding real type of the argument for the generic parameter.

— If the function has exactly two generic parameters, the type determined is the corresponding real type determined by the usual arithmetic conversions (6.3.1.8) applied to the arguments for the generic parameters.

— If the function has more than two generic parameters, the type determined is the corresponding real type determined by repeatedly applying the usual arithmetic conversions, first to the first two arguments for generic parameters, then to that result type and the next argument for a generic parameter, and so forth until the usual arithmetic conversions have been applied to the last argument for a generic parameter.

If neither <math.h> and <complex.h> define a function whose generic parameters have the determined corresponding real type, the behavior is undefined.

8 For each unsuffixed function in <math.h> for which there is a function in <complex.h> with the same name except for a **c** prefix, the corresponding type-generic macro (for both functions) has the same name as the function in <math.h>. The corresponding type-generic macro for **fabs** and **cabs** is **fabs**.

[369] Like other function-like macros in standard libraries, each type-generic macro can be suppressed to make available the corresponding ordinary function.

[370] If the type of the argument is not compatible with the type of the parameter for the selected function, the behavior is undefined.

 © ISO 2024 — All rights reserved

<math.h> function	<complex.h> function	type-generic macro
acos	**cacos**	**acos**
asin	**casin**	**asin**
atan	**catan**	**atan**
acosh	**cacosh**	**acosh**
asinh	**casinh**	**asinh**
atanh	**catanh**	**atanh**
cos	**ccos**	**cos**
sin	**csin**	**sin**
tan	**ctan**	**tan**
cosh	**ccosh**	**cosh**
sinh	**csinh**	**sinh**
tanh	**ctanh**	**tanh**
exp	**cexp**	**exp**
log	**clog**	**log**
pow	**cpow**	**pow**
sqrt	**csqrt**	**sqrt**
fabs	**cabs**	**fabs**

If at least one argument for a generic parameter is complex, then use of the macro invokes a complex function; otherwise, use of the macro invokes a real function.

9 For each unsuffixed function in <math.h> without a **c**-prefixed counterpart in <complex.h> (except functions that round result to narrower type, **modf**, and **canonicalize**), the corresponding type-generic macro has the same name as the function. These type-generic macros are:

acospi	**exp2**	**fmod**	**log2**	**rootn**
asinpi	**expm1**	**frexp**	**logb**	**roundeven**
atan2pi	**fdim**	**fromfpx**	**logp1**	**round**
atan2	**floor**	**fromfp**	**lrint**	**rsqrt**
atanpi	**fmax**	**hypot**	**lround**	**scalbln**
cbrt	**fmaximum**	**ilogb**	**nearbyint**	**scalbn**
ceil	**fmaximum_mag**	**ldexp**	**nextafter**	**sinpi**
compoundn	**fmaximum_num**	**lgamma**	**nextdown**	**tanpi**
copysign	**fmaximum_mag_num**	**llogb**	**nexttoward**	**tgamma**
cospi	**fma**	**llrint**	**nextup**	**trunc**
erfc	**fmin**	**llround**	**pown**	**ufromfpx**
erf	**fminimum**	**log10p1**	**powr**	**ufromfp**
exp10m1	**fminimum_mag**	**log10**	**remainder**	
exp10	**fminimum_num**	**log1p**	**remquo**	
exp2m1	**fminimum_mag_num**	**log2p1**	**rint**	

If all arguments for generic parameters are real, then use of the macro invokes a real function (provided <math.h> defines a function of the determined type); otherwise, use of the macro is undefined.

10 For each unsuffixed function in <complex.h> that is not a **c**-prefixed counterpart to a function in <math.h>, the corresponding type-generic macro has the same name as the function. These type-generic macros are:

carg	**cimag**	**conj**	**cproj**	**creal**

Use of the macro with any argument of standard floating or complex type invokes a complex function. Use of the macro with an argument of decimal floating type is undefined.

11 The functions that round result to a narrower type have type-generic macros whose names are obtained by omitting any suffix from the function names. Thus, the macros with **f** or **d** prefix are:

© ISO 2024 — All rights reserved

fadd	**fsub**	**fmul**	**fdiv**	**ffma**	**fsqrt**
dadd	**dsub**	**dmul**	**ddiv**	**dfma**	**dsqrt**

and the macros with **d32** or **d64** prefix are:

d32add	**d32sub**	**d32mul**	**d32div**	**d32fma**	**d32sqrt**
d64add	**d64sub**	**d64mul**	**d64div**	**d64fma**	**d64sqrt**

All arguments shall be real. If the macro prefix is **f** or **d**, use of an argument of decimal floating type is undefined. If the macro prefix is **d32** or **d64**, use of an argument of standard floating type is undefined. The function invoked is determined as follows:

— If any argument has type **_Decimal128**, or if the macro prefix is **d64**, the function invoked has the name of the macro, with a **d128** suffix.

— Otherwise, if the macro prefix is **d32**, the function invoked has the name of the macro, with a **d64** suffix.

— Otherwise, if any argument has type **long double**, or if the macro prefix is **d**, the function invoked has the name of the macro, with an **l** suffix.

— Otherwise, the function invoked has the name of the macro (with no suffix).

12 For each **d64**-suffixed function in <math.h>, except **decodedecd64**, **encodedecd64**, **decodebind64**, and **encodebind64**, that does not have an unsuffixed counterpart, the corresponding type-generic macro has the name of the function, but without the suffix. These type-generic macros are:

<math.h> function	type-generic macro
quantized*N*	**quantize**
samequantumd*N*	**samequantum**
quantumd*N*	**quantum**
llquantexpd*N*	**llquantexp**

Use of the macro with an argument of standard floating or complex type or with only integer type arguments is undefined.

13 A type-generic macro corresponding to a function indicated in the table in 7.6.2 is affected by constant rounding modes (7.6.4).

14 NOTE The type-generic macro definition in the example in 6.5.2.1 does not conform to this specification. A conforming macro could be implemented as follows:

```
#define cbrt(X)                 \
_Generic((X),                   \
  long double: _Roundwise_cbrtl, \
  default:     _Roundwise_cbrt,  \
  float:       _Roundwise_cbrtf  \
)(X)
```

where _Roundwise_cbrtl, _Roundwise_cbrt, and _Roundwise_cbrtf are pointers to functions that are equivalent to **cbrtl**, **cbrt**, and **cbrtf**, respectively, but that are guaranteed to be affected by constant rounding modes (7.6.2).

© ISO 2024 — All rights reserved

15 EXAMPLE With the declarations

```
#include <tgmath.h>
int n;
float f;
double d;
long double ld;
float complex fc;
double complex dc;
long double complex ldc;
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 d32;
_Decimal64 d64;
_Decimal128 d128;
#endif
```

functions invoked by use of type-generic macros are shown in the following table:

macro use	invocation
`exp(n)`	`exp(n)`, the function
`acosh(f)`	`acoshf(f)`
`sin(d)`	`sin(d)`, the function
`atan(ld)`	`atanl(ld)`
`log(fc)`	`clogf(fc)`
`sqrt(dc)`	`csqrt(dc)`
`pow(ldc, f)`	`cpowl(ldc, f)`
`remainder(n, n)`	`remainder(n, n)`, the function
`nextafter(d, f)`	`nextafter(d, f)`, the function
`nexttoward(f, ld)`	`nexttowardf(f, ld)`
`copysign(n, ld)`	`copysignl(n, ld)`
`ceil(fc)`	undefined
`rint(dc)`	undefined
`fmaximum(ldc, ld)`	undefined
`carg(n)`	`carg(n)`, the function
`cproj(f)`	`cprojf(f)`
`creal(d)`	`creal(d)`, the function
`cimag(ld)`	`cimagl(ld)`
`fabs(fc)`	`cabsf(fc)`
`carg(dc)`	`carg(dc)`, the function
`cproj(ldc)`	`cprojl(ldc)`
`fsub(f, ld)`	`fsubl(f, ld)`
`fdiv(d, n)`	`fdiv(d, n)`, the function
`dfma(f, d, ld)`	`dfmal(f, d, ld)`
`dadd(f, f)`	`daddl(f, f)`
`dsqrt(dc)`	undefined
`exp(d64)`	`expd64(d64)`
`sqrt(d32)`	`sqrtd32(d32)`
`fmaximum(d64, d128)`	`fmaximumd128(d64, d128)`
`pow(d32, n)`	`powd64(d32, n)`
`remainder(d64, d)`	undefined
`creal(d64)`	undefined
`remquo(d32, d32, &n)`	undefined
`llquantexp(d)`	undefined
`quantize(dc)`	undefined
`samequantum(n, n)`	undefined
`d32sub(d32, d128)`	`d32subd128(d32, d128)`
`d32div(d64, n)`	`d32divd64(d64, n)`
`d64fma(d32, d64, d128)`	`d64fmad128(d32, d64, d128)`
`d64add(d32, d32)`	`d64addd128(d32, d32)`
`d64sqrt(d)`	undefined

© ISO 2024 — All rights reserved

`dadd(n, d64)`	undefined

© ISO 2024 — All rights reserved

7.28 Threads <threads.h>

7.28.1 Introduction

1 The header <threads.h> includes the header <time.h>, defines macros, and declares types, enumeration constants, and functions that support multiple threads of execution.[371)]

2 Implementations that define the macro **__STDC_NO_THREADS__** may not provide this header nor support any of its facilities.

3 The macros are

```
ONCE_FLAG_INIT
```

which expands to a value that can be used to initialize an object of type **once_flag**; and

```
TSS_DTOR_ITERATIONS
```

which expands to an integer constant expression representing the maximum number of times that destructors will be called when a thread terminates.

4 The types are

```
cnd_t
```

which is a complete object type that holds an identifier for a condition variable;

```
thrd_t
```

which is a complete object type that holds an identifier for a thread;

```
tss_t
```

which is a complete object type that holds an identifier for a thread-specific storage pointer;

```
mtx_t
```

which is a complete object type that holds an identifier for a mutex;

```
tss_dtor_t
```

which is the function pointer type **void** (*)(**void***), used for a destructor for a thread-specific storage pointer;

```
thrd_start_t
```

which is the function pointer type **int** (*)(**void***) that is passed to **thrd_create** to create a new thread; and

```
once_flag
```

which is a complete object type that holds a flag for use by **call_once**.

5 The enumeration constants are

```
mtx_plain
```

which is passed to **mtx_init** to create a mutex object that does not support timeout;

```
mtx_recursive
```

[371)] See "future library directions" (7.33.19).

© ISO 2024 — All rights reserved

which is passed to **`mtx_init`** to create a mutex object that supports recursive locking;

```
mtx_timed
```

which is passed to **`mtx_init`** to create a mutex object that supports timeout;

```
thrd_timedout
```

which is returned by a timed wait function to indicate that the time specified in the call was reached without acquiring the requested resource;

```
thrd_success
```

which is returned by a function to indicate that the requested operation succeeded;

```
thrd_busy
```

which is returned by a function to indicate that the requested operation failed because a resource requested by a test and return function is already in use;

```
thrd_error
```

which is returned by a function to indicate that the requested operation failed; and

```
thrd_nomem
```

which is returned by a function to indicate that the requested operation failed because it was unable to allocate memory.

Forward references: date and time (7.29).

7.28.2 Initialization functions

7.28.2.1 The `call_once` function

Synopsis

1
```
#include <threads.h>
void call_once(once_flag *flag, void (*func)(void));
```

Description

2 The **`call_once`** function uses the **`once_flag`** pointed to by `flag` to ensure that `func` is called exactly once, the first time the **`call_once`** function is called with that value of `flag`. Completion of an effective call to the **`call_once`** function synchronizes with all subsequent calls to the **`call_once`** function with the same value of `flag`.

Returns

3 The **`call_once`** function returns no value.

7.28.3 Condition variable functions

7.28.3.1 The `cnd_broadcast` function

Synopsis

1
```
#include <threads.h>
int cnd_broadcast(cnd_t *cond);
```


© ISO 2024 — All rights reserved

Description

2 The **cnd_broadcast** function unblocks all the threads that are blocked on the condition variable pointed to by cond at the time of the call. If no threads are blocked on the condition variable pointed to by cond at the time of the call, the function does nothing.

Returns

3 The **cnd_broadcast** function returns **thrd_success** on success, or **thrd_error** if the request could not be honored.

7.28.3.2 The cnd_destroy function

Synopsis

1
```
#include <threads.h>
void cnd_destroy(cnd_t *cond);
```

Description

2 The **cnd_destroy** function releases all resources used by the condition variable pointed to by cond. The **cnd_destroy** function requires that no threads be blocked waiting for the condition variable pointed to by cond.

Returns

3 The **cnd_destroy** function returns no value.

7.28.3.3 The cnd_init function

Synopsis

1
```
#include <threads.h>
int cnd_init(cnd_t *cond);
```

Description

2 The **cnd_init** function creates a condition variable. If it succeeds it sets the object pointed to by cond to a value that uniquely identifies the newly created condition variable. A thread that calls **cnd_wait** on a newly created condition variable will block.

Returns

3 The **cnd_init** function returns **thrd_success** on success, or **thrd_nomem** if no memory could be allocated for the newly created condition, or **thrd_error** if the request could not be honored.

7.28.3.4 The cnd_signal function

Synopsis

1
```
#include <threads.h>
int cnd_signal(cnd_t *cond);
```

Description

2 The **cnd_signal** function unblocks one of the threads that are blocked on the condition variable pointed to by cond at the time of the call. If no threads are blocked on the condition variable at the time of the call, the function does nothing and returns success.

Returns

3 The **cnd_signal** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

7.28.3.5 The cnd_timedwait function

Synopsis

1
```
#include <threads.h>
int cnd_timedwait(cnd_t * restrict cond, mtx_t * restrict mtx,
      const struct timespec * restrict ts);
```

© ISO 2024 — All rights reserved

Description

2 The **cnd_timedwait** function atomically unlocks the mutex pointed to by mtx and blocks until the condition variable pointed to by cond is signaled by a call to **cnd_signal** or to **cnd_broadcast**, or until after the **TIME_UTC**-based calendar time pointed to by ts, or until it is unblocked due to an unspecified reason. When the calling thread becomes unblocked it locks the object pointed to by mtx before it returns. The **cnd_timedwait** function requires that the mutex pointed to by mtx be locked by the calling thread.

Returns

3 The **cnd_timedwait** function returns **thrd_success** upon success, or **thrd_timedout** if the time specified in the call was reached without acquiring the requested resource, or **thrd_error** if the request could not be honored.

7.28.3.6 The cnd_wait function

Synopsis

1
```
#include <threads.h>
int cnd_wait(cnd_t *cond, mtx_t *mtx);
```

Description

2 The **cnd_wait** function atomically unlocks the mutex pointed to by mtx and blocks until the condition variable pointed to by cond is signaled by a call to **cnd_signal** or to **cnd_broadcast**, or until it is unblocked due to an unspecified reason. When the calling thread becomes unblocked it locks the mutex pointed to by mtx before it returns. The **cnd_wait** function requires that the mutex pointed to by mtx be locked by the calling thread.

Returns

3 The **cnd_wait** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

7.28.4 Mutex functions

1 For purposes of determining the existence of a data race, lock and unlock operations behave as atomic operations. All lock and unlock operations on a particular mutex occur in some particular total order.

2 NOTE This total order can be viewed as the modification order of the mutex.

7.28.4.1 The mtx_destroy function

Synopsis

1
```
#include <threads.h>
void mtx_destroy(mtx_t *mtx);
```

Description

2 The **mtx_destroy** function releases any resources used by the mutex pointed to by mtx. No threads can be blocked waiting for the mutex pointed to by mtx.

Returns

3 The **mtx_destroy** function returns no value.

7.28.4.2 The mtx_init function

Synopsis

1
```
#include <threads.h>
int mtx_init(mtx_t *mtx, int type);
```

 © ISO 2024 — All rights reserved

Description

2 The **mtx_init** function creates a mutex object with properties indicated by type, which shall have one of these values:

mtx_plain for a simple non-recursive mutex,

mtx_timed for a non-recursive mutex that supports timeout,

mtx_plain | **mtx_recursive** for a simple recursive mutex, or

mtx_timed | **mtx_recursive** for a recursive mutex that supports timeout.

3 If the **mtx_init** function succeeds, it sets the mutex pointed to by mtx to a value that uniquely identifies the newly created mutex.

Returns

4 The **mtx_init** function returns **thrd_success** on success, or **thrd_error** if the request could not be honored.

7.28.4.3 The mtx_lock function

Synopsis

1
```
#include <threads.h>
int mtx_lock(mtx_t *mtx);
```

Description

2 The **mtx_lock** function blocks until it locks the mutex pointed to by mtx. If the mutex is non-recursive, it shall not be locked by the calling thread. Prior calls to **mtx_unlock** on the same mutex synchronize with this operation.

Returns

3 The **mtx_lock** function returns **thrd_success** on success, or **thrd_error** if the request could not be honored.

7.28.4.4 The mtx_timedlock function

Synopsis

1
```
#include <threads.h>
int mtx_timedlock(mtx_t * restrict mtx, const struct timespec * restrict ts);
```

Description

2 The **mtx_timedlock** function endeavors to block until it locks the mutex pointed to by mtx or until after the **TIME_UTC**-based calendar time pointed to by ts. The specified mutex shall support timeout. If the operation succeeds, prior calls to **mtx_unlock** on the same mutex synchronize with this operation.

Returns

3 The **mtx_timedlock** function returns **thrd_success** on success, or **thrd_timedout** if the time specified was reached without acquiring the requested resource, or **thrd_error** if the request could not be honored.

7.28.4.5 The mtx_trylock function

Synopsis

1
```
#include <threads.h>
int mtx_trylock(mtx_t *mtx);
```

© ISO 2024 — All rights reserved

Description

2 The **mtx_trylock** function endeavors to lock the mutex pointed to by mtx. If the mutex is already locked, the function returns without blocking. If the operation succeeds, prior calls to **mtx_unlock** on the same mutex synchronize with this operation.

Returns

3 The **mtx_trylock** function returns **thrd_success** on success, or **thrd_busy** if the resource requested is already in use, or **thrd_error** if the request could not be honored. **mtx_trylock** may spuriously fail to lock an unused resource, in which case it returns **thrd_busy**.

7.28.4.6 The mtx_unlock function

Synopsis

1
```
#include <threads.h>
int mtx_unlock(mtx_t *mtx);
```

Description

2 The **mtx_unlock** function unlocks the mutex pointed to by mtx. The mutex pointed to by mtx shall be locked by the calling thread.

Returns

3 The **mtx_unlock** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

7.28.5 Thread functions

7.28.5.1 The thrd_create function

Synopsis

1
```
#include <threads.h>
int thrd_create(thrd_t *thr, thrd_start_t func, void *arg);
```

Description

2 The **thrd_create** function creates a new thread executing func(arg). If the **thrd_create** function succeeds, it sets the object pointed to by thr to the identifier of the newly created thread. (A thread's identifier may be reused for a different thread once the original thread has exited and either been detached or joined to another thread.) The completion of the **thrd_create** function synchronizes with the beginning of the execution of the new thread.

3 Returning from func has the same behavior as invoking **thrd_exit** with the value returned from func.

Returns

4 The **thrd_create** function returns **thrd_success** on success, or **thrd_nomem** if no memory could be allocated for the thread requested, or **thrd_error** if the request could not be honored.

7.28.5.2 The thrd_current function

Synopsis

1
```
#include <threads.h>
thrd_t thrd_current(void);
```

Description

2 The **thrd_current** function identifies the thread that called it.

Returns

3 The **thrd_current** function returns the identifier of the thread that called it.

© ISO 2024 — All rights reserved

7.28.5.3 The `thrd_detach` function

Synopsis

1
```
#include <threads.h>
int thrd_detach(thrd_t thr);
```

Description

2 The **`thrd_detach`** function tells the operating system to dispose of any resources allocated to the thread identified by `thr` when that thread terminates. The thread identified by `thr` shall not have been previously detached or joined with another thread.

Returns

3 The **`thrd_detach`** function returns **`thrd_success`** on success or **`thrd_error`** if the request could not be honored.

7.28.5.4 The `thrd_equal` function

Synopsis

1
```
#include <threads.h>
int thrd_equal(thrd_t thr0, thrd_t thr1);
```

Description

2 The **`thrd_equal`** function will determine whether the thread identified by `thr0` refers to the thread identified by `thr1`.

Returns

3 The **`thrd_equal`** function returns zero if the thread `thr0` and the thread `thr1` refer to different threads. Otherwise the **`thrd_equal`** function returns a nonzero value.

7.28.5.5 The `thrd_exit` function

Synopsis

1
```
#include <threads.h>
[[noreturn]] void thrd_exit(int res);
```

Description

2 For every thread-specific storage key which was created with a non-null destructor and for which the value is non-null, **`thrd_exit`** sets the value associated with the key to a null pointer value and then invokes the destructor with its previous value. The order in which destructors are invoked is unspecified.

3 If after this process there remain keys with both non-null destructors and values, the implementation repeats this process up to **`TSS_DTOR_ITERATIONS`** times.

4 Following this, the **`thrd_exit`** function terminates execution of the calling thread and sets its result code to `res`.

5 The program terminates normally after the last thread has been terminated. The behavior is as if the program called the **`exit`** function with the status **`EXIT_SUCCESS`** at thread termination time.

Returns

6 The **`thrd_exit`** function returns no value.

7.28.5.6 The `thrd_join` function

Synopsis

1
```
#include <threads.h>
int thrd_join(thrd_t thr, int *res);
```

© ISO 2024 — All rights reserved

Description

2 The **thrd_join** function joins the thread identified by thr with the current thread by blocking until the other thread has terminated. If the parameter res is not a null pointer, it stores the thread's result code in the integer pointed to by res. The termination of the other thread synchronizes with the completion of the **thrd_join** function. The thread identified by thr shall not have been previously detached or joined with another thread.

Returns

3 The **thrd_join** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

7.28.5.7 The thrd_sleep function

Synopsis

1
```
#include <threads.h>
int thrd_sleep(const struct timespec *duration, struct timespec *remaining);
```

Description

2 The **thrd_sleep** function suspends execution of the calling thread until either the interval specified by duration has elapsed or a signal which is not being ignored is received. If interrupted by a signal and the remaining argument is not null, the amount of time remaining (the requested interval minus the time actually slept) is stored in the interval it points to. The duration and remaining arguments may point to the same object.

3 The suspension time may be longer than requested because the interval is rounded up to an integer multiple of the sleep resolution or because of the scheduling of other activity by the system. But, except for the case of being interrupted by a signal, the suspension time will not be less than that specified, as measured by the system clock **TIME_UTC**.

Returns

4 The **thrd_sleep** function returns zero if the requested time has elapsed, -1 if it has been interrupted by a signal, or a negative value (which may also be -1) if it fails.

7.28.5.8 The thrd_yield function

Synopsis

1
```
#include <threads.h>
void thrd_yield(void);
```

Description

2 The **thrd_yield** function endeavors to permit other threads to run, even if the current thread would ordinarily continue to run.

Returns

3 The **thrd_yield** function returns no value.

7.28.6 Thread-specific storage functions

7.28.6.1 The tss_create function

Synopsis

1
```
#include <threads.h>
int tss_create(tss_t *key, tss_dtor_t dtor);
```

Description

2 The **tss_create** function creates a thread-specific storage pointer with destructor dtor, which may be null.

 © ISO 2024 — All rights reserved

3 A null pointer value is associated with the newly created key in all existing threads. Upon subsequent thread creation, the value associated with all keys is initialized to a null pointer value in the new thread.

4 Destructors associated with thread-specific storage are not invoked at program termination.

5 The **tss_create** function shall not be called from within a destructor.

Returns

6 If the **tss_create** function is successful, it sets the thread-specific storage pointed to by key to a value that uniquely identifies the newly created pointer and returns **thrd_success**; otherwise, **thrd_error** is returned and the thread-specific storage pointed to by key is set to an indeterminate representation.

7.28.6.2 The tss_delete function

Synopsis

1
```
#include <threads.h>
void tss_delete(tss_t key);
```

Description

2 The **tss_delete** function releases any resources used by the thread-specific storage identified by key. The **tss_delete** function shall only be called with a value for key that was returned by a call to **tss_create** before the thread commenced executing destructors.

3 If **tss_delete** is called while another thread is executing destructors, whether this will affect the number of invocations of the destructor associated with key on that thread is unspecified.

4 Calling **tss_delete** will not result in the invocation of any destructors.

Returns

5 The **tss_delete** function returns no value.

7.28.6.3 The tss_get function

Synopsis

1
```
#include <threads.h>
void *tss_get(tss_t key);
```

Description

2 The **tss_get** function returns the value for the current thread held in the thread-specific storage identified by key. The **tss_get** function shall only be called with a value for key that was returned by a call to **tss_create** before the thread commenced executing destructors.

Returns

3 The **tss_get** function returns the value for the current thread if successful, or zero if unsuccessful.

7.28.6.4 The tss_set function

Synopsis

1
```
#include <threads.h>
int tss_set(tss_t key, void *val);
```

Description

2 The **tss_set** function sets the value for the current thread held in the thread-specific storage identified by key to val. The **tss_set** function shall only be called with a value for key that was returned by a call to **tss_create** before the thread commenced executing destructors.

3 This action will not invoke the destructor associated with the key on the value being replaced.

© ISO 2024 — All rights reserved

Returns

4 The **tss_set** function returns **thrd_success** on success or **thrd_error** if the request could not be honored.

© ISO 2024 — All rights reserved

7.29 Date and time <time.h>

7.29.1 Components of time

1 The header <time.h> defines several macros, and declares types and functions for manipulating time. Many functions deal with a *calendar time* that represents the current date (according to the Gregorian calendar) and time. Some functions deal with *local time*, which is the calendar time expressed for some specific time zone, and with *Daylight Saving Time*, which is a temporary change in the algorithm for determining local time. The local time zone and Daylight Saving Time are implementation-defined.

2 The feature test macro **__STDC_VERSION_TIME_H__** expands to the token 202311L. The other macros defined are **NULL** (described in 7.21);

```
CLOCKS_PER_SEC
```

which expands to an expression with type **clock_t** (described later in this subclause) that is the number per second of the value returned by the **clock** function;

```
TIME_UTC
TIME_MONOTONIC
```

which expand to integer constants greater than 0 designating the calendar time and monotonic time bases, respectively. Additional time base macro definitions, beginning with **TIME_** and an uppercase letter, may also be specified by the implementation;[372] and,

```
TIME_ACTIVE
TIME_THREAD_ACTIVE
```

which, if defined, expand to integer values, designating overall execution and thread-specific active processing time bases, respectively.

3 The definition of macros for time bases other than **TIME_UTC** are optional. If defined, the corresponding time bases are supported by **timespec_get** and **timespec_getres**, and their values are positive. If defined, the value of the optional macro **TIME_ACTIVE** shall be different from the constants **TIME_UTC** and **TIME_MONOTONIC** and shall not change during the same program invocation. The optional macro **TIME_THREAD_ACTIVE** shall not be defined if the implementation does not support threads; its value shall be different from **TIME_UTC**, **TIME_MONOTONIC**, and **TIME_ACTIVE**, it shall be the same for all expansions of the macro for the same thread, and the value provided for one thread shall not be used by a different thread as the base argument of **timespec_get** or **timespec_getres**.

4 The types declared are **size_t** (described in 7.21);

```
clock_t
```

and

```
time_t
```

which are real types capable of representing times;

```
struct timespec
```

which holds an interval specified in seconds and nanoseconds (which may represent a calendar time based on a particular epoch); and

```
struct tm
```

[372] See future library directions (7.33). Implementations can define additional time bases, but are only required to support a real time clock based on UTC.

© ISO 2024 — All rights reserved

which holds the components of a calendar time, called the *broken-down time*.

5 The range and precision of times representable in **clock_t** and **time_t** are implementation-defined. The **timespec** structure shall contain at least the following members, in any order. The semantics of the members and their normal ranges are expressed in the comments.[373)]

```
time_t          tv_sec;  // whole seconds -- ≥ 0
/* see the following */ tv_nsec; // nanoseconds -- [0, 999999999]
```

The **tv_nsec** member shall be an implementation-defined signed integer type capable of representing the range [0, 999999999].

6 The **tm** structure shall contain at least the following members, in any order.[374)] The semantics of the members and their normal ranges are expressed in the comments.

```
int tm_sec;   // seconds after the minute -- [0, 60]
int tm_min;   // minutes after the hour -- [0, 59]
int tm_hour;  // hours since midnight -- [0, 23]
int tm_mday;  // day of the month -- [1, 31]
int tm_mon;   // months since January -- [0, 11]
int tm_year;  // years since 1900
int tm_wday;  // days since Sunday -- [0, 6]
int tm_yday;  // days since January 1 -- [0, 365]
int tm_isdst; // Daylight Saving Time flag
```

The value of **tm_isdst** is positive if Daylight Saving Time is in effect, zero if Daylight Saving Time is not in effect, and negative if the information is not available.

7.29.2 Time manipulation functions

7.29.2.1 The clock function

Synopsis

1
```
#include <time.h>
clock_t clock(void);
```

Description

2 The **clock** function determines the processor time used.

Returns

3 The **clock** function returns the implementation's best approximation of the active processing time associated with the program execution since the beginning of an implementation-defined era related only to the program invocation. To determine the time in seconds, the value returned by the **clock** function should be divided by the value of the macro **CLOCKS_PER_SEC**. If the processor time used is not available, the function returns the value `(clock_t)(-1)`. If the value cannot be represented, the function returns an unspecified value.[375)]

7.29.2.2 The difftime function

Synopsis

1
```
#include <time.h>
double difftime(time_t time1, time_t time0);
```

Description

2 The **difftime** function computes the difference between two calendar times: `time1 - time0`.

[373)] The **tv_sec** member is a linear count of seconds and potentially does not have the normal semantics of a **time_t**.
[374)] The range [0, 60] for **tm_sec** allows for a positive leap second.
[375)] This could be due to overflow of the **clock_t** type.

© ISO 2024 — All rights reserved

Returns

3 The **difftime** function returns the difference expressed in seconds as a **double**.

7.29.2.3 The mktime function

Synopsis

1
```
#include <time.h>
time_t mktime(struct tm *timeptr);
```

Description

2 The **mktime** function converts the broken-down time, expressed as local time, in the structure pointed to by `timeptr` into a calendar time value with the same encoding as that of the values returned by the **time** function. The original values of the **tm_wday** and **tm_yday** components of the structure are ignored, and the original values of the other components are not restricted to the ranges indicated previously. If the local time to be used for the conversion is one that includes Daylight Saving Time adjustments, a positive or zero value for **tm_isdst** causes the **mktime** function to perform the conversion as if Daylight Saving Time, respectively, is or is not in effect for the specified time. A negative value causes it to attempt to determine whether Daylight Saving Time is in effect for the specified time; if it determines that Daylight Saving Time is in effect it produces the same result as an equivalent call with a positive **tm_isdst** value, otherwise it produces the same result as an equivalent call with a **tm_isdst** value of zero.[376)] On successful completion, the components of the structure are set to the same values that would be returned by a call to the **localtime** function with the calculated calendar time as its argument.

Returns

3 The **mktime** function returns the specified calendar time encoded as a value of type **time_t**. If the calendar time cannot be represented in the **time_t** encoding used for the return value or the value to be returned in the **tm_year** component of the structure pointed to by `timeptr` cannot be represented as an **int**, the function returns the value `(time_t)(-1)` and does not change the value of the **tm_wday** component of the structure.

4 EXAMPLE What day of the week is July 4, 2001?

```
#include <stdio.h>
#include <time.h>
static const char *const wday[] = {
      "Sunday", "Monday", "Tuesday", "Wednesday",
      "Thursday", "Friday", "Saturday", "-unknown-"
};
struct tm time_str;
/* ... */

time_str.tm_year  = 2001 - 1900;
time_str.tm_mon   = 7 - 1;
time_str.tm_mday  = 4;
time_str.tm_hour  = 0;
time_str.tm_min   = 0;
time_str.tm_sec   = 1;
time_str.tm_isdst = -1;
time_str.tm_wday  = 7;
mktime(&time_str);
printf("%s\n", wday[time_str.tm_wday]);
```

7.29.2.4 The timegm function

Synopsis

1 [376)] If the broken-down time specifies a time that is either skipped over or repeated when a transition to or from Daylight Saving Time occurs, it is unspecified whether the **mktime** function produces the same result as an equivalent call with a positive **tm_isdst** value or as an equivalent call with a **tm_isdst** value of zero.

© ISO 2024 — All rights reserved

```
#include <time.h>
time_t timegm(struct tm *timeptr);
```

Description

2 The **timegm** function converts the broken-down time, expressed as UTC time, in the structure pointed to by `timeptr` into a calendar time value with the same encoding as that of the values returned by the **time** function. The original values of the **tm_wday** and **tm_yday** components of the structure are ignored, and the original values of the other components are not restricted to the ranges indicated previously. On successful completion, the values of the **tm_wday** and **tm_yday** components of the structure are set appropriately, and the other components are set to represent the specified calendar time, but with their values forced to the ranges indicated previously; the final value of **tm_mday** is not set until **tm_mon** and **tm_year** are determined.

Returns

3 The **timegm** function returns the specified calendar time encoded as a value of type **time_t**. If the calendar time cannot be represented in the **time_t** encoding used for the return value or the value to be returned in the **tm_year** component of the structure pointed to by `timeptr` cannot be represented as an **int**, the function returns the value `(time_t)(-1)` and does not change the value of the **tm_wday** component of the structure.

7.29.2.5 The time function

Synopsis

1
```
#include <time.h>
time_t time(time_t *timer);
```

Description

2 The **time** function determines the current calendar time. The encoding of the value is unspecified.

Returns

3 The **time** function returns the implementation's best approximation to the current calendar time. The value `(time_t)(-1)` is returned if the calendar time is not available. If `timer` is not a null pointer, the return value is also assigned to the object it points to.

7.29.2.6 The timespec_get function

Synopsis

1
```
#include <time.h>
int timespec_get(struct timespec *ts, int base);
```

Description

2 The **timespec_get** function sets the interval pointed to by `ts` to hold the current calendar time based on the specified time base.

3 If `base` is **TIME_UTC**, the **tv_sec** member is set to the number of seconds since an implementation-defined *epoch*, truncated to a whole value and the **tv_nsec** member is set to the integral number of nanoseconds, rounded to the resolution of the system clock.[377] The optional time base **TIME_MONOTONIC** is the same, but the reference point is an implementation-defined time point; different program invocations may or may not refer to the same reference points.[378] For the same program invocation, the results of two calls to **timespec_get** with **TIME_MONOTONIC** such that the first happens before the second shall not be decreasing. It is implementation-defined if **TIME_MONOTONIC** accounts for time during which the execution environment is suspended.[379] For the optional time

[377] Although a **struct timespec** object describes times with nanosecond resolution, the available resolution is system dependent and could even be greater than 1 second.

[378] Commonly, this reference point is the boot time of the execution environment or the start of the execution.

[379] The execution environment may, for example, not be able to track physical time that elapsed during suspension in a low power consumption mode.

© ISO 2024 — All rights reserved

bases **TIME_ACTIVE** and **TIME_THREAD_ACTIVE** the result is similar, but the call measures the amount of active processing time associated with the whole program invocation or with the calling thread, respectively.

Returns

4 If the **timespec_get** function is successful it returns the nonzero value base; otherwise, it returns zero.

Recommended practice

5 It is recommended practice that timing results of calls to **timespec_get** with **TIME_ACTIVE**, if defined, and of calls to **clock** are as close to each other as their types, value ranges, and resolutions (obtained with **timespec_getres** and **CLOCKS_PER_SEC**, respectively) allow. Because of its wider value range and improved indications on error, **timespec_get** with time base **TIME_ACTIVE** should be used instead of **clock** by new code whenever possible.

7.29.2.7 The timespec_getres function

Synopsis

1
```
#include <time.h>
int timespec_getres(struct timespec *ts, int base);
```

Description

2 If ts is non-null and base is supported by the **timespec_get** function, the **timespec_getres** function returns the resolution of the time provided by the **timespec_get** function for base in the **timespec** structure pointed to by ts. For each supported base, multiple calls to the **timespec_getres** function during the same program execution shall have identical results.

Returns

3 If the value base is supported by the **timespec_get** function, the **timespec_getres** function returns the nonzero value base; otherwise, it returns zero.

7.29.3 Time conversion functions

1 Functions with a **_r** suffix place the result of the conversion into the buffer referred by buf and return that pointer. These functions and the function **strftime** shall not be subject to data races, unless the time or calendar state is changed in a multi-thread execution.[380)]

2 Functions **gmtime** and **localtime** are the same as their counterparts suffixed with **_r**. In place of the parameter buf, they use a pointer to one or two broken-down time structures. Similarly, an array of **char** is commonly used by **asctime** and **ctime**. Execution of any of the functions that return a pointer to one of these objects may overwrite the information returned from any previous call to one of these functions that uses the same object. These functions are not reentrant and are not required to avoid data races with each other. Accessing the returned pointer after the thread that called the function that returned it has exited results in undefined behavior. The implementation shall behave as if no other library functions call these functions.

7.29.3.1 The asctime function

Synopsis

1
```
#include <time.h>
[[deprecated]] char *asctime(const struct tm *timeptr);
```

Description

2 This function is obsolescent and should be avoided in new code.

[380)] This does not mean that these functions may not read global state that describes the time and calendar settings of the execution, such as the **LC_TIME** locale or the implementation-defined specification of the local time zone. Only the setting of that state by **setlocale** or by means of implementation-defined functions may constitute races.

© ISO 2024 — All rights reserved

3 The **asctime** function converts the broken-down time in the structure pointed to by timeptr into a string in the form

```
      Sun Sep 16 01:03:52 1973\n\0
```

using the equivalent of the following algorithm.

```
[[deprecated]] char *asctime(const struct tm *timeptr)
{
      static const char wday_name[7][3] = {
            "Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"
      };
      static const char mon_name[12][3] = {
            "Jan", "Feb", "Mar", "Apr", "May", "Jun",
            "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"
      };
      static char result[26];

      snprintf(result, 26, "%.3s %.3s%3d %.2d:%.2d:%.2d %d\n",
            wday_name[timeptr->tm_wday],
            mon_name[timeptr->tm_mon],
            timeptr->tm_mday, timeptr->tm_hour,
            timeptr->tm_min, timeptr->tm_sec,
            1900 + timeptr->tm_year);
      return result;
}
```

4 If any of the members of the broken-down time contain values that are outside their normal ranges,[381] the behavior of the **asctime** function is undefined. Likewise, if the calculated year exceeds four digits or is less than the year 1000, the behavior is undefined.

Returns

5 The **asctime** function returns a pointer to the string.

7.29.3.2 The ctime function

Synopsis

1

```
      #include <time.h>
      [[deprecated]] char *ctime(const time_t *timer);
```

Description

2 This function is obsolescent and should be avoided in new code.

3 The **ctime** function converts the calendar time pointed to by timer to local time in the form of a string. They are equivalent to:

```
      asctime(localtime(timer))
```

Returns

4 The **ctime** function returns the pointer returned by the **asctime** function with that broken-down time as argument.

Forward references: the **localtime** functions (7.29.3.4).

7.29.3.3 The gmtime functions

Synopsis

1

```
      #include <time.h>
```

[381] See 7.29.1.

© ISO 2024 — All rights reserved

```
struct tm *gmtime(const time_t *timer);
struct tm *gmtime_r(const time_t *timer, struct tm *buf);
```

Description

2 The **gmtime** functions convert the calendar time pointed to by timer into a broken-down time, expressed as UTC.

Returns

3 The **gmtime** functions return a pointer to the broken-down time, or a null pointer if the specified time cannot be converted to UTC.

7.29.3.4 The localtime functions

Synopsis

1
```
#include <time.h>
struct tm *localtime(const time_t *timer);
struct tm *localtime_r(const time_t *timer, struct tm *buf);
```

Description

2 The **localtime** functions convert the calendar time pointed to by timer into a broken-down time, expressed as local time.

Returns

3 The **localtime** functions return a pointer to the broken-down time, or a null pointer if the specified time cannot be converted to local time.

7.29.3.5 The strftime function

Synopsis

1
```
#include <time.h>
size_t strftime(char * restrict s, size_t maxsize, const char * restrict format,
      const struct tm * restrict timeptr);
```

Description

2 The **strftime** function places characters into the array pointed to by s as controlled by the string pointed to by format. The format shall be a multibyte character sequence, beginning and ending in its initial shift state. The format string consists of zero or more conversion specifiers and ordinary multibyte characters. A conversion specifier consists of a % character, possibly followed by an E or O modifier character (described later), followed by a character that determines the behavior of the conversion specifier. All ordinary multibyte characters (including the terminating null character) are copied unchanged into the array. If copying takes place between objects that overlap, the behavior is undefined. No more than maxsize characters are placed into the array.

3 Each conversion specifier shall be replaced by appropriate characters as described in the following list. The appropriate characters shall be determined using the **LC_TIME** category of the current locale and by the values of zero or more members of the broken-down time structure pointed to by timeptr, as specified in brackets in the description. If any of the specified values is outside the normal range, the characters stored are unspecified.

- %a is replaced by the locale's abbreviated weekday name. [**tm_wday**]
- %A is replaced by the locale's full weekday name. [**tm_wday**]
- %b is replaced by the locale's abbreviated month name. [**tm_mon**]
- %B is replaced by the locale's full month name. [**tm_mon**]
- %c is replaced by the locale's appropriate date and time representation. [all specified in 7.29.1]
- %C is replaced by the year divided by 100 and truncated to an integer, as a decimal number (00–99). [**tm_year**]

© ISO 2024 — All rights reserved

`%d` is replaced by the day of the month as a decimal number (`01`–`31`). [**`tm_mday`**]

`%D` is equivalent to "`%m/%d/%y`". [**`tm_mon`**, **`tm_mday`**, **`tm_year`**]

`%e` is replaced by the day of the month as a decimal number (`1`–`31`); a single digit is preceded by a space. [**`tm_mday`**]

`%F` is equivalent to `"%Y-%m-%d"` (the ISO 8601 date format, when the year is between 1000 and 9999 inclusive). [**`tm_year`**, **`tm_mon`**, **`tm_mday`**]

`%g` is replaced by the last 2 digits of the week-based year (see further in this subclause) as a decimal number (`00`–`99`). [**`tm_year`**, **`tm_wday`**, **`tm_yday`**]

`%G` is replaced by the week-based year (see further in this subclause) as a decimal number (e.g. 1997). [**`tm_year`**, **`tm_wday`**, **`tm_yday`**]

`%h` is equivalent to "`%b`". [**`tm_mon`**]

`%H` is replaced by the hour (24-hour clock) as a decimal number (`00`–`23`). [**`tm_hour`**]

`%I` is replaced by the hour (12-hour clock) as a decimal number (`01`–`12`). [**`tm_hour`**]

`%j` is replaced by the day of the year as a decimal number (`001`–`366`). [**`tm_yday`**]

`%m` is replaced by the month as a decimal number (`01`–`12`). [**`tm_mon`**]

`%M` is replaced by the minute as a decimal number (`00`–`59`). [**`tm_min`**]

`%n` is replaced by a new-line character.

`%p` is replaced by the locale's equivalent of the AM/PM designations associated with a 12-hour clock. [**`tm_hour`**]

`%r` is replaced by the locale's 12-hour clock time. [**`tm_hour`**, **`tm_min`**, **`tm_sec`**]

`%R` is equivalent to "`%H:%M`". [**`tm_hour`**, **`tm_min`**]

`%S` is replaced by the second as a decimal number (`00`–`60`). [**`tm_sec`**]

`%t` is replaced by a horizontal-tab character.

`%T` is equivalent to "`%H:%M:%S`" (the ISO 8601 time format). [**`tm_hour`**, **`tm_min`**, **`tm_sec`**]

`%u` is replaced by the ISO 8601 weekday as a decimal number (`1`–`7`), where Monday is 1. [**`tm_wday`**]

`%U` is replaced by the week number of the year (the first Sunday as the first day of week 1) as a decimal number (`00`–`53`). [**`tm_year`**, **`tm_wday`**, **`tm_yday`**]

`%V` is replaced by the ISO 8601 week number (see further in this subclause) as a decimal number (`01`–`53`). [**`tm_year`**, **`tm_wday`**, **`tm_yday`**]

`%w` is replaced by the weekday as a decimal number (`0`–`6`), where Sunday is 0. [**`tm_wday`**]

`%W` is replaced by the week number of the year (the first Monday as the first day of week 1) as a decimal number (`00`–`53`). [**`tm_year`**, **`tm_wday`**, **`tm_yday`**]

`%x` is replaced by the locale's appropriate date representation. [all specified in 7.29.1]

`%X` is replaced by the locale's appropriate time representation. [all specified in 7.29.1]

`%y` is replaced by the last 2 digits of the year as a decimal number (`00`–`99`). [**`tm_year`**]

`%Y` is replaced by the year as a decimal number (e.g. `1997`). [**`tm_year`**]

`%z` is replaced by the offset from UTC in the ISO 8601 format "`-0430`" (meaning 4 hours 30 minutes behind UTC, west of Greenwich), or by no characters if no time zone is determinable. [**`tm_isdst`**]

`%Z` is replaced by the locale's time zone name or abbreviation, or by no characters if no time zone is determinable. [**`tm_isdst`**]

`%%` is replaced by `%`.

4 Some conversion specifiers can be modified by the inclusion of an `E` or `O` modifier character to indicate an alternative format or specification. If the alternative format or specification does not exist for the current locale, the modifier is ignored.

© ISO 2024 — All rights reserved

`%Ec` is replaced by the locale's alternative date and time representation.

`%EC` is replaced by the name of the base year (period) in the locale's alternative representation.

`%Ex` is replaced by the locale's alternative date representation.

`%EX` is replaced by the locale's alternative time representation.

`%Ey` is replaced by the offset from `%EC` (year only) in the locale's alternative representation.

`%EY` is replaced by the locale's full alternative year representation.

`%Ob` is replaced by the locale's abbreviated alternative month name.

`%OB` is replaced by the locale's alternative appropriate full month name.

`%Od` is replaced by the day of the month, using the locale's alternative numeric symbols (filled as needed with leading zeros, or with leading spaces if there is no alternative symbol for zero).

`%Oe` is replaced by the day of the month, using the locale's alternative numeric symbols (filled as needed with leading spaces).

`%OH` is replaced by the hour (24-hour clock), using the locale's alternative numeric symbols.

`%OI` is replaced by the hour (12-hour clock), using the locale's alternative numeric symbols.

`%Om` is replaced by the month, using the locale's alternative numeric symbols.

`%OM` is replaced by the minutes, using the locale's alternative numeric symbols.

`%OS` is replaced by the seconds, using the locale's alternative numeric symbols.

`%Ou` is replaced by the ISO 8601 weekday as a number in the locale's alternative representation, where Monday is 1.

`%OU` is replaced by the week number, using the locale's alternative numeric symbols.

`%OV` is replaced by the ISO 8601 week number, using the locale's alternative numeric symbols.

`%Ow` is replaced by the weekday as a number, using the locale's alternative numeric symbols.

`%OW` is replaced by the week number of the year, using the locale's alternative numeric symbols.

`%Oy` is replaced by the last 2 digits of the year, using the locale's alternative numeric symbols.

5 `%g`, `%G`, and `%V` give values according to the ISO 8601 week-based year. In this system, weeks begin on a Monday and week 1 of the year is the week that includes January 4th, which is also the week that includes the first Thursday of the year, and is also the first week that contains at least four days in the year. If the first Monday of January is the 2nd, 3rd, or 4th, the preceding days are part of the last week of the preceding year; thus, for Saturday 2nd January 1999, `%G` is replaced by `1998` and `%V` is replaced by `53`. If December 29th, 30th, or 31st is a Monday, it and any following days are part of week 1 of the following year. Thus, for Tuesday 30th December 1997, `%G` is replaced by `1998` and `%V` is replaced by `01`.

6 If a conversion specifier is not one of the ones previously specified, the behavior is undefined.

7 In the `"C"` locale, the `E` and `O` modifiers are ignored and the replacement strings for the following specifiers are:

`%a` the first three characters of `%A`.

`%A` one of "`Sunday`", "`Monday`", ..., "`Saturday`".

`%b` the first three characters of `%B`.

`%B` one of "`January`", "`February`", ..., "`December`".

`%c` equivalent to "`%a %b %e %T %Y`".

`%p` one of "`AM`" or "`PM`".

`%r` equivalent to "`%I:%M:%S %p`".

`%x` equivalent to "`%m/%d/%y`".

`%X` equivalent to `%T`.

`%Z` implementation-defined.

© ISO 2024 — All rights reserved

Returns

8 If the total number of resulting characters including the terminating null character is not more than `maxsize`, the **`strftime`** function returns the number of characters placed into the array pointed to by `s` not including the terminating null character. Otherwise, zero is returned and the members of the array have an indeterminate representation.

© ISO 2024 — All rights reserved

7.30 Unicode utilities <uchar.h>

1 The header <uchar.h> declares one macro, a few types, and several functions for manipulating Unicode characters.

2 The macro

```
__STDC_VERSION_UCHAR_H__
```

is an integer constant expression with a value equivalent to 202311L.

3 The types declared are **mbstate_t** (described in 7.31.1) and **size_t** (described in 7.21);

```
char8_t
```

which is an unsigned integer type used for 8-bit characters and is the same type as **unsigned char**;

```
char16_t
```

which is an unsigned integer type used for 16-bit characters and is the same type as **uint_least16_t** (described in 7.22.1.2); and

```
char32_t
```

which is an unsigned integer type used for 32-bit characters and is the same type as **uint_least32_t** (also described in 7.22.1.2).

7.30.1 Restartable multibyte/wide character conversion functions

1 These functions have a parameter, ps, of type pointer to **mbstate_t** that points to an object that can completely describe the current conversion state of the associated multibyte character sequence, which the functions alter as necessary. If ps is a null pointer, each function uses its own internal **mbstate_t** object instead, which is initialized prior to the first call to the function to the initial conversion state; the functions are not required to avoid data races with other calls to the same function in this case. It is implementation-defined whether the internal **mbstate_t** object has thread storage duration; if it has thread storage duration, it is initialized to the initial conversion state prior to the first call to the function on the new thread. The implementation behaves as if no library function calls these functions with a null pointer for ps.

2 When used in the functions in this subclause, the encoding of **char8_t**, **char16_t**, and **char32_t** objects, and sequences of such objects, is UTF-8, UTF-16, and UTF-32, respectively. Similarly, the encoding of **char** and **wchar_t**, and sequences of such objects, is the execution and wide execution encodings (6.2.9), respectively.

7.30.1.1 The mbrtoc8 function

Synopsis

1
```
#include <uchar.h>
size_t mbrtoc8(char8_t * restrict pc8, const char * restrict s, size_t n,
      mbstate_t * restrict ps);
```

Description

2 If s is a null pointer, the **mbrtoc8** function is equivalent to the call:

```
mbrtoc8(nullptr, "", 1, ps)
```

In this case, the values of the parameters pc8 and n are ignored.

3 If s is not a null pointer, the **mbrtoc8** function function inspects at most n bytes beginning with the byte pointed to by s to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character

© ISO 2024 — All rights reserved

is complete and valid, it determines the values of the corresponding characters and then, if `pc8` is not a null pointer, stores the value of the first (or only) such character in the object pointed to by `pc8`. Subsequent calls will store successive characters without consuming any additional input until all the characters have been stored. If the corresponding character is the null character, the resulting state described is the initial conversion state.

Returns

4 The **`mbrtoc8`** function returns the first of the following that applies (given the current conversion state):

`0` if the next `n` or fewer bytes complete the multibyte character that corresponds to the null character (which is the value stored).

between `1` *and* `n` *inclusive* if the next `n` or fewer bytes complete a valid multibyte character (which is the value stored); the value returned is the number of bytes that complete the multibyte character.

`(size_t)(-3)` if the next character resulting from a previous call has been stored (no bytes from the input have been consumed by this call).

`(size_t)(-2)` if the next `n` bytes contribute to an incomplete (but potentially valid) multibyte character, and all `n` bytes have been processed (no value is stored).[382)]

`(size_t)(-1)` if an encoding error occurs, in which case the next `n` or fewer bytes do not contribute to a complete and valid multibyte character (no value is stored); the value of the macro **`EILSEQ`** is stored in **`errno`**, and the conversion state is unspecified.

7.30.1.2 The `c8rtomb` function

Synopsis

1
```
#include <uchar.h>
size_t c8rtomb(char * restrict s, char8_t c8, mbstate_t * restrict ps);
```

Description

2 If `s` is a null pointer, the **`c8rtomb`** function is equivalent to the call

```
c8rtomb(buf, u8'\0', ps)
```

where `buf` is an internal buffer.

3 If `s` is not a null pointer, the **`c8rtomb`** function determines the number of bytes needed to represent the multibyte character that corresponds to the character given or completed by `c8` (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by `s`, or stores nothing if `c8` does not represent a complete character. At most **`MB_CUR_MAX`** bytes are stored. If `c8` is a null character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

Returns

4 The **`c8rtomb`** function returns the number of bytes stored in the array object (including any shift sequences). When `c8` is not a valid character, an encoding error occurs: the function stores the value of the macro **`EILSEQ`** in **`errno`** and returns `(size_t)(-1)`; the conversion state is unspecified.

7.30.1.3 The `mbrtoc16` function

[382)] When `n` has at least the value of the **`MB_CUR_MAX`** macro, this case can only occur if `s` points at a sequence of redundant shift sequences (for implementations with state-dependent encodings).

 © ISO 2024 — All rights reserved

Synopsis

1
```
#include <uchar.h>
size_t mbrtoc16(char16_t * restrict pc16, const char * restrict s, size_t n,
      mbstate_t * restrict ps);
```

Description

2 If `s` is a null pointer, the **`mbrtoc16`** function is equivalent to the call:

```
mbrtoc16(nullptr, "", 1, ps)
```

In this case, the values of the parameters `pc16` and `n` are ignored.

3 If `s` is not a null pointer, the **`mbrtoc16`** function inspects at most `n` bytes beginning with the byte pointed to by `s` to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is complete and valid, it determines the values of the corresponding wide characters and then, if `pc16` is not a null pointer, stores the value of the first (or only) such character in the object pointed to by `pc16`. Subsequent calls will store successive wide characters without consuming any additional input until all the characters have been stored. If the corresponding wide character is the null wide character, the resulting state described is the initial conversion state.

Returns

4 The **`mbrtoc16`** function returns the first of the following that applies (given the current conversion state):

`0` if the next `n` or fewer bytes complete the multibyte character that corresponds to the null wide character (which is the value stored).

between `1` *and* `n` *inclusive* if the next `n` or fewer bytes complete a valid multibyte character (which is the value stored); the value returned is the number of bytes that complete the multibyte character.

`(size_t)(-3)` if the next character resulting from a previous call has been stored (no bytes from the input have been consumed by this call).

`(size_t)(-2)` if the next `n` bytes contribute to an incomplete (but potentially valid) multibyte character, and all `n` bytes have been processed (no value is stored).[383)]

`(size_t)(-1)` if an encoding error occurs, in which case the next `n` or fewer bytes do not contribute to a complete and valid multibyte character (no value is stored); the value of the macro **`EILSEQ`** is stored in **`errno`**, and the conversion state is unspecified.

7.30.1.4 The `c16rtomb` function

Synopsis

1
```
#include <uchar.h>
size_t c16rtomb(char * restrict s, char16_t c16, mbstate_t * restrict ps);
```

Description

2 If `s` is a null pointer, the **`c16rtomb`** function is equivalent to the call

```
c16rtomb(buf, u'\0', ps)
```

where `buf` is an internal buffer.

[383)] When `n` has at least the value of the **`MB_CUR_MAX`** macro, this case can only occur if `s` points at a sequence of redundant shift sequences (for implementations with state-dependent encodings).

© ISO 2024 — All rights reserved

3 If `s` is not a null pointer, the **`c16rtomb`** function determines the number of bytes needed to represent the multibyte character that corresponds to the wide character given or completed by `c16` (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by `s`, or stores nothing if `c16` does not represent a complete character. At most **`MB_CUR_MAX`** bytes are stored. If `c16` is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

Returns

4 The **`c16rtomb`** function returns the number of bytes stored in the array object (including any shift sequences). When `c16` is not a valid wide character, an encoding error occurs: the function stores the value of the macro **`EILSEQ`** in **`errno`** and returns `(size_t)(-1)`; the conversion state is unspecified.

7.30.1.5 The `mbrtoc32` function

Synopsis

1
```
#include <uchar.h>
size_t mbrtoc32(char32_t * restrict pc32, const char * restrict s, size_t n,
      mbstate_t * restrict ps);
```

Description

2 If `s` is a null pointer, the **`mbrtoc32`** function is equivalent to the call:

```
mbrtoc32(nullptr, "", 1, ps)
```

In this case, the values of the parameters `pc32` and `n` are ignored.

3 If `s` is not a null pointer, the **`mbrtoc32`** function inspects at most `n` bytes beginning with the byte pointed to by `s` to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is complete and valid, it determines the values of the corresponding wide characters and then, if `pc32` is not a null pointer, stores the value of the first (or only) such character in the object pointed to by `pc32`. Subsequent calls will store successive wide characters without consuming any additional input until all the characters have been stored. If the corresponding wide character is the null wide character, the resulting state described is the initial conversion state.

Returns

4 The **`mbrtoc32`** function returns the first of the following that applies (given the current conversion state):

`0` if the next `n` or fewer bytes complete the multibyte character that corresponds to the null wide character (which is the value stored).

between `1` *and* `n` *inclusive* if the next `n` or fewer bytes complete a valid multibyte character (which is the value stored); the value returned is the number of bytes that complete the multibyte character.

`(size_t)(-3)` if the next character resulting from a previous call has been stored (no bytes from the input have been consumed by this call).

`(size_t)(-2)` if the next `n` bytes contribute to an incomplete (but potentially valid) multibyte character, and all `n` bytes have been processed (no value is stored).[384)]

`(size_t)(-1)` if an encoding error occurs, in which case the next `n` or fewer bytes do not contribute to a complete and valid multibyte character (no value is stored); the value of the macro **`EILSEQ`** is stored in **`errno`**, and the conversion state is unspecified.

[384)] When `n` has at least the value of the **`MB_CUR_MAX`** macro, this case can only occur if `s` points at a sequence of redundant shift sequences (for implementations with state-dependent encodings).

 © ISO 2024 — All rights reserved

7.30.1.6 The `c32rtomb` function

Synopsis

1
```
#include <uchar.h>
size_t c32rtomb(char * restrict s, char32_t c32, mbstate_t * restrict ps);
```

Description

2 If `s` is a null pointer, the **`c32rtomb`** function is equivalent to the call

```
c32rtomb(buf, U'\0', ps)
```

where `buf` is an internal buffer.

3 If `s` is not a null pointer, the **`c32rtomb`** function determines the number of bytes needed to represent the multibyte character that corresponds to the wide character given by `c32` (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by `s`. At most **`MB_CUR_MAX`** bytes are stored. If `c32` is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

Returns

4 The **`c32rtomb`** function returns the number of bytes stored in the array object (including any shift sequences). When `c32` is not a valid wide character, an encoding error occurs: the function stores the value of the macro **`EILSEQ`** in **`errno`** and returns `(size_t)(-1)`;the conversion state is unspecified.

© ISO 2024 — All rights reserved

7.31 Extended multibyte and wide character utilities <wchar.h>

7.31.1 Introduction

1 The header <wchar.h> defines five macros, and declares four data types, one tag, and many functions.[385)]

2 The macro

```
__STDC_VERSION_WCHAR_H__
```

is an integer constant expression with a value equivalent to 202311L.

3 The types declared are **wchar_t** and **size_t** (both described in 7.21);

```
mbstate_t
```

which is a complete object type other than an array type that can hold the conversion state information necessary to convert between sequences of multibyte characters and wide characters;

```
wint_t
```

which is an integer type unchanged by default argument promotions that can hold any value corresponding to members of the extended character set, as well as at least one value that does not correspond to any member of the extended character set (see subsequent **WEOF** description);[386)] and

```
struct tm
```

which is declared as an incomplete structure type (the contents are described in 7.29.1).

4 The macros defined are **NULL** (described in 7.21); **WCHAR_MIN**, **WCHAR_MAX**, and **WCHAR_WIDTH** (described in 7.22); and

```
WEOF
```

which expands to a constant expression of type **wint_t** whose value does not correspond to any member of the extended character set.[387)] It is accepted (and returned) by several functions in this subclause to indicate *end-of-file*, that is, no more input from a stream. It is also used as a wide character value that does not correspond to any member of the extended character set.

5 The functions declared are grouped as follows:

— Functions that perform input and output of wide characters, or multibyte characters, or both;

— Functions that provide wide string numeric conversion;

— Functions that perform general wide string manipulation;

— Functions for wide string date and time conversion; and

— Functions that provide extended capabilities for conversion between multibyte and wide character sequences.

6 Arguments to the functions in this subclause may point to arrays containing **wchar_t** values that do not correspond to members of the extended character set. Such values shall be processed according to the specified semantics, except that it is unspecified whether an encoding error occurs if such a value appears in the format string for a function in 7.31.2 or 7.31.5 and the specified semantics do not require that value to be processed by **wcrtomb**.

7 Unless explicitly stated otherwise, if the execution of a function described in this subclause causes copying to take place between objects that overlap, the behavior is undefined.

[385)] See "future library directions" (7.33.20).
[386)] **wchar_t** and **wint_t** can be the same integer type.
[387)] The value of the macro **WEOF** can differ from that of **EOF** and need not be negative.

 © ISO 2024 — All rights reserved

7.31.2 Formatted wide character input/output functions

1 The formatted wide character input/output functions shall behave as if there is a sequence point after the actions associated with each specifier.[388)]

7.31.2.1 The `fwprintf` function

Synopsis

1
```
#include <stdio.h>
#include <wchar.h>
int fwprintf(FILE * restrict stream, const wchar_t * restrict format, ...);
```

Description

2 The **`fwprintf`** function writes output to the stream pointed to by `stream`, under control of the wide string pointed to by `format` that specifies how subsequent arguments are converted for output. If there are insufficient arguments for the format, the behavior is undefined. If the format is exhausted while arguments remain, the excess arguments are evaluated (as always) but are otherwise ignored. The **`fwprintf`** function returns when the end of the format string is encountered.

3 The format is composed of zero or more directives: ordinary wide characters (not `%`), which are copied unchanged to the output stream; and conversion specifications, each of which results in fetching zero or more subsequent arguments, converting them, if applicable, according to the corresponding conversion specifier, and then writing the result to the output stream.

4 Each conversion specification is introduced by the wide character `%`. After the `%`, the following appear in sequence:

— Zero or more *flags* (in any order) that modify the meaning of the conversion specification.

— An optional minimum *field width*. If the converted value has fewer wide characters than the field width, it is padded with spaces (by default) on the left (or right, if the left adjustment flag, described later, has been given) to the field width. The field width takes the form of an asterisk `*` (described later) or a nonnegative decimal integer.[389)]

— An optional *precision* that gives the minimum number of digits to appear for the `b`, `B`, `d`, `i`, `o`, `u`, `x`, and `X` conversions, the number of digits to appear after the decimal-point wide character for `a`, `A`, `e`, `E`, `f`, and `F` conversions, the maximum number of significant digits for the `g` and `G` conversions, or the maximum number of wide characters to be written for `s` conversions. The precision takes the form of a period (`.`) followed either by an asterisk `*` (described later) or by an optional nonnegative decimal integer; if only the period is specified, the precision is taken as zero. If a precision appears with any other conversion specifier, the behavior is undefined.

— An optional *length modifier* that specifies the size of the argument.

— A *conversion specifier* wide character that specifies the type of conversion to be applied.

5 As noted previously, a field width, or precision, or both, may be indicated by an asterisk. In this case, an **`int`** argument supplies the field width or precision. The arguments specifying field width, or precision, or both, shall appear (in that order) before the argument (if any) to be converted. A negative field width argument is taken as a `-` flag followed by a positive field width. A negative precision argument is taken as if the precision were omitted.

6 The flag wide characters and their meanings are:

`-` The result of the conversion is left-justified within the field. (It is right-justified if this flag is not specified.)

`+` The result of a signed conversion always begins with a plus or minus sign. (It begins with a sign only when a value with a negative sign is converted if this flag is not specified.)[390)]

[388)] The **`fwprintf`** functions perform writes to memory for the `%n` specifier.
[389)] Note that `0` is taken as a flag, not as the beginning of a field width.
[390)] The results of all floating conversions of a negative zero, and of negative values that round to zero, include a minus sign.

© ISO 2024 — All rights reserved

space If the first wide character of a signed conversion is not a sign, or if a signed conversion results in no wide characters, a space is prefixed to the result. If the *space* and `+` flags both appear, the *space* flag is ignored.

`#` The result is converted to an "alternative form". For `o` conversion, it increases the precision, if and only if necessary, to force the first digit of the result to be a zero (if the value and precision are both 0, a single 0 is printed). For `b` conversion, a nonzero result has `0b` prefixed to it. For the optional `B` conversion as described later in this subclause, a nonzero result has `0B` prefixed to it. For `x` (or `X`) conversion, a nonzero result has `0x` (or `0X`) prefixed to it. For `a`, `A`, `e`, `E`, `f`, `F`, `g`, and `G` conversions, the result of converting a floating-point number always contains a decimal-point wide character, even if no digits follow it. (Normally, a decimal-point wide character appears in the result of these conversions only if a digit follows it.) For `g` and `G` conversions, trailing zeros are *not* removed from the result. For other conversions, the behavior is undefined.

`0` For `b`, `B`, `d`, `i`, `o`, `u`, `x`, `X`, `a`, `A`, `e`, `E`, `f`, `F`, `g`, and `G` conversions, leading zeros (following any indication of sign or base) are used to pad to the field width rather than performing space padding, except when converting an infinity or NaN. If the `0` and `-` flags both appear, the `0` flag is ignored. For `b`, `B`, `d`, `i`, `o`, `u`, `x`, and `X` conversions, if a precision is specified, the `0` flag is ignored. For other conversions, the behavior is undefined.

7 The length modifiers and their meanings are:

`hh` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`signed char`** or **`unsigned char`** argument (the argument will have been promoted according to the integer promotions, but its value shall be converted to **`signed char`** or **`unsigned char`** before printing); or that a following `n` conversion specifier applies to a pointer to a **`signed char`** argument.

`h` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`short int`** or **`unsigned short int`** argument (the argument will have been promoted according to the integer promotions, but its value shall be converted to **`short int`** or **`unsigned short int`** before printing); or that a following `n` conversion specifier applies to a pointer to a **`short int`** argument.

`l` (ell) Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`long int`** or **`unsigned long int`** argument; that a following `n` conversion specifier applies to a pointer to a **`long int`** argument; that a following `c` conversion specifier applies to a **`wint_t`** argument; that a following `s` conversion specifier applies to a pointer to a **`wchar_t`** argument; or has no effect on a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier.

`ll` (ell-ell) Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`long long int`** or **`unsigned long long int`** argument; or that a following `n` conversion specifier applies to a pointer to a **`long long int`** argument.

`j` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to an **`intmax_t`** or **`uintmax_t`** argument; or that a following `n` conversion specifier applies to a pointer to an **`intmax_t`** argument.

`z` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`size_t`** or the corresponding signed integer type argument; or that a following `n` conversion specifier applies to a pointer to a signed integer type corresponding to **`size_t`** argument.

`t` Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a **`ptrdiff_t`** or the corresponding unsigned integer type argument; or that a following `n` conversion specifier applies to a pointer to a **`ptrdiff_t`** argument.

© ISO 2024 — All rights reserved

`w`*N* Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to an integer argument with a specific width where *N* is a positive decimal integer with no leading zeros (the argument will have been promoted according to the integer promotions, but its value shall be converted to the unpromoted type); or that a following `n` conversion specifier applies to a pointer to an integer type argument with a width of *N* bits. All minimum-width integer types (7.22.1.2) and exact-width integer types (7.22.1.1) defined in the header `<stdint.h>` shall be supported. Other supported values of *N* are implementation-defined.

`wf`*N* Specifies that a following `b`, `B`, `d`, `i`, `o`, `u`, `x`, or `X` conversion specifier applies to a fastest minimum-width integer argument with a specific width where *N* is a positive decimal integer with no leading zeros (the argument will have been promoted according to the integer promotions, but its value shall be converted to the unpromoted type); or that a following `n` conversion specifier applies to a pointer to a fastest minimum-width integer type argument with a width of *N* bits. All fastest minimum-width integer types (7.22.1.3) defined in the header `<stdint.h>` shall be supported. Other supported values of *N* are implementation-defined.

`L` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to a **`long double`** argument.

`H` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to a **`_Decimal32`** argument.

`D` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to a **`_Decimal64`** argument.

`DD` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to a **`_Decimal128`** argument.

If a length modifier appears with any conversion specifier other than as specified previously, the behavior is undefined.

8 The conversion specifiers and their meanings are:

`d,i` The **`int`** argument is converted to signed decimal in the style *[-]dddd*. The precision specifies the minimum number of digits to appear; if the value being converted can be represented in fewer digits, it is expanded with leading zeros. The default precision is 1. The result of converting a zero value with a precision of zero is no wide characters.

`b,B,o,u,x,X` The **`unsigned int`** argument is converted to unsigned binary (`b` or `B`), unsigned octal (`o`), unsigned decimal (`u`), or unsigned hexadecimal notation (`x` or `X`) in the style *dddd*; the letters `abcdef` are used for `x` conversion and the letters `ABCDEF` for `X` conversion. The precision specifies the minimum number of digits to appear; if the value being converted can be represented in fewer digits, it is expanded with leading zeros. The default precision is 1. The result of converting a zero value with a precision of zero is no wide characters. The specifier `B` is optional and provides the same functionality as `b`, except for the **`#`** flag as previously specified. The **`PRIB`** macros from `<inttypes.h>` shall only be defined if the implementation follows the specification as given here.

`f,F` A **`double`** argument representing a floating-point number is converted to decimal notation in the style *[-]ddd.ddd*, where the number of digits after the decimal-point wide character is equal to the precision specification. If the precision is missing, it is taken as 6; if the precision is zero and the **`#`** flag is not specified, no decimal-point wide character appears. If a decimal-point wide character appears, at least one digit appears before it. The value is rounded to the appropriate number of digits.

A **`double`** argument representing an infinity is converted in one of the styles *[-]*`inf` or *[-]*`infinity` — which style is implementation-defined. A **`double`** argument representing a NaN is converted in one of the styles *[-]*`nan` or *[-]*`nan(`*n-wchar-sequence*`)` — which style, and

© ISO 2024 — All rights reserved

the meaning of any *n-wchar-sequence*, is implementation-defined. The `F` conversion specifier produces `INF`, `INFINITY`, or `NAN` instead of `inf`, `infinity`, or `nan`, respectively.[391)]

`e,E` A **`double`** argument representing a floating-point number is converted in the style *[-]d.dde±dd*, where there is one digit (which is nonzero if the argument is nonzero) before the decimal-point wide character and the number of digits after it is equal to the precision; if the precision is missing, it is taken as 6; if the precision is zero and the **`#`** flag is not specified, no decimal-point wide character appears. The value is rounded to the appropriate number of digits. The `E` conversion specifier produces a number with `E` instead of `e` introducing the exponent. The exponent always contains at least two digits, and only as many more digits as necessary to represent the exponent. If the value is zero, the exponent is zero.

A **`double`** argument representing an infinity or NaN is converted in the style of an `f` or `F` conversion specifier.

`g,G` A **`double`** argument representing a floating-point number is converted in style `f` or `e` (or in style `F` or `E` in the case of a `G` conversion specifier), depending on the value converted and the precision. Let P equal the precision if nonzero, 6 if the precision is omitted, or 1 if the precision is zero. Then, if a conversion with style `E` would have an exponent of X:

if $P > X \geq -4$, the conversion is with style `f` (or `F`) and precision $P - (X + 1)$.

otherwise, the conversion is with style `e` (or `E`) and precision $P - 1$.

Finally, unless the **`#`** flag is used, any trailing zeros are removed from the fractional portion of the result and the decimal-point wide character is removed if there is no fractional portion remaining.

A **`double`** argument representing an infinity or NaN is converted in the style of an `f` or `F` conversion specifier.

`a,A` A **`double`** argument representing a floating-point number is converted in the style *[-]*`0x`*h.hhhh*`p`*±d*, where there is one hexadecimal digit (which is nonzero if the argument is a normalized floating-point number and is otherwise unspecified) before the decimal-point wide character [392)] and the number of hexadecimal digits after it is equal to the precision; if the precision is missing and **`FLT_RADIX`** is a power of 2, then the precision is sufficient for an exact representation of the value; if the precision is missing and **`FLT_RADIX`** is not a power of 2, then the precision is sufficient to distinguish[393)] values of type **`double`**, except that trailing zeros may be omitted; if the precision is zero and the **`#`** flag is not specified, no decimal-point wide character appears. The letters `abcdef` are used for `a` conversion and the letters `ABCDEF` for `A` conversion. The `A` conversion specifier produces a number with `X` and `P` instead of `x` and `p`. The exponent always contains at least one digit, and only as many more digits as necessary to represent the decimal exponent of 2. If the value is zero, the exponent is zero.

[391)] When applied to infinite and NaN values, the `-`, `+`, and *space* flag wide characters have their usual meaning; the **`#`** and `0` flag wide characters have no effect.

[392)] Binary implementations can choose the hexadecimal digit to the left of the decimal-point wide character so that subsequent digits align to nibble (4-bit) boundaries. This implementation choice affects numerical values printed with a precision P that is insufficient to represent all values exactly. Implementations with different conventions about the most significant hexadecimal digit will round at different places, affecting the numerical value of the hexadecimal result. For example, possible printed output for the code

```
#include <stdio.h>
/* ... */
double x = 123.0;
printf("%.1a", x);
```

include "`0x1.fp+6` " and "`0xf.6p+3` " whose numerical values are 124 and 123, respectively. Portable code seeking identical numerical results on different platforms should avoid precisions P that require rounding.

[393)] The formatting precision P is sufficient to distinguish values of the source type if $16^P > b^p$ where b (not a power of 2) and p are the base and precision of the source type (5.2.5.3.3). A smaller P potentially suffices depending on the implementation's scheme for determining the digit to the left of the decimal-point wide character.

© ISO 2024 — All rights reserved

A **double** argument representing an infinity or NaN is converted in the style of an f or F conversion specifier.

If an H, D, or DD modifier is present and the precision is missing, then for a decimal floating type argument represented by a triple of integers (s, c, q), where n is the number of significant digits in the coefficient c,

— if $-(n + 5) \leq q \leq 0$, use style f (or style F in the case of an A conversion specifier) with formatting precision equal to $-q$,

— otherwise, use style e (or style E in the case of an A conversion specifier) with formatting precision equal to $n - 1$, with the exceptions that if $c = 0$ then the digit-sequence in the exponent-part shall have the value q (rather than 0), and that the exponent is always expressed with the minimum number of digits required to represent its value (the exponent never contains a leading zero).

If the precision P is present (in the conversion specification) and is zero or at least as large as the precision p (5.2.5.3.3) of the decimal floating type, the conversion is as if the precision were missing. If the precision P is present (and nonzero) and less than the precision p of the decimal floating type, the conversion first obtains an intermediate result as follows, where n is the number of significant digits in the coefficient:

— If $n \leq P$, set the intermediate result to the input.

— If $n > P$, round the input value, according to the current rounding direction for decimal floating-point operations, to P decimal digits, with unbounded exponent range, representing the result with a P-digit integer coefficient when in the form (s, c, q).

Convert the intermediate result in the manner described previously for the case where the precision is missing.

c — If no l length modifier is present, the **int** argument is converted to a wide character as if by calling **btowc** and the resulting wide character is written.

If an l length modifier is present, the **wint_t** argument is converted to **wchar_t** and written.

s — If no l length modifier is present, the argument shall be a pointer to storage of character type containing a multibyte character sequence beginning in the initial shift state. Characters from the storage are converted as if by repeated calls to the **mbrtowc** function, with the conversion state described by an **mbstate_t** object initialized to zero before the first multibyte character is converted, and written up to (but not including) the terminating null wide character. If the precision is specified, no more than that many wide characters are written. If the precision is not specified or is greater than the size of the converted storage, the converted storage shall contain a null wide character.

If an l length modifier is present, the argument shall be a pointer to storage of **wchar_t** type. Wide characters from the storage are written up to (but not including) a terminating null wide character. If the precision is specified, no more than that many wide characters are written. If the precision is not specified or is greater than the size of the array, the storage shall contain a null wide character.

p — The argument shall be a pointer to **void** or a pointer to a character type. The value of the pointer is converted to a sequence of printing wide characters, in an implementation-defined manner.

n — The argument shall be a pointer to signed integer whose type is specified by the length modifier, if any, for the conversion specification, or shall be **int** if no length modifier is specified for the conversion specification. The number of wide characters written to the output stream so far by this call to **fwprintf** is stored into the integer object pointed to by the argument. No argument is converted, but one is consumed. If the conversion specification includes any flags, a field width, or a precision, the behavior is undefined.

© ISO 2024 — All rights reserved

`%` A `%` wide character is written. No argument is converted. The complete conversion specification shall be `%%`.

9 If a conversion specification is invalid, the behavior is undefined.[394] **`fwprintf`** shall behave as if it uses **`va_arg`** with a type argument naming the type resulting from applying the default argument promotions to the type corresponding to the conversion specification and then converting the result of the **`va_arg`** expansion to the type corresponding to the conversion specification.[395]

10 In no case does a nonexistent or small field width cause truncation of a field; if the result of a conversion is wider than the field width, the field is expanded to contain the conversion result.

11 For `a` and `A` conversions, if **`FLT_RADIX`** is a power of 2, the value is correctly rounded to a hexadecimal floating number with the given precision.

Recommended practice

12 For `a` and `A` conversions, if **`FLT_RADIX`** is not a power of 2 and the result is not exactly representable in the given precision, the result should be one of the two adjacent numbers in hexadecimal floating style with the given precision, with the extra stipulation that the error should have a correct sign for the current rounding direction.

13 For `e`, `E`, `f`, `F`, `g`, and `G` conversions, if the number of significant decimal digits is at most the maximum value M of the T**`_DECIMAL_DIG`** macros (defined in `<float.h>`), then the result should be correctly rounded.[396] If the number of significant decimal digits is more than M but the source value is exactly representable with M digits, then the result should be an exact representation with trailing zeros. Otherwise, the source value is bounded by two adjacent decimal strings $L < U$, both having M significant digits; the value of the resultant decimal string D should satisfy $L \leq D \leq U$, with the extra stipulation that the error should have a correct sign for the current rounding direction.

14 The uppercase `B` format specifier is made optional by the previous description, because it used to be available for extensions in previous versions of this document. Implementations that did not use an uppercase `B` as their own extension before are encouraged to implement it as previously described.

Returns

15 The **`fwprintf`** function returns the number of wide characters transmitted, or a negative value if an output or encoding error occurred or if the implementation does not support a specified width length modifier.

Environmental limits

16 The number of wide characters that can be produced by any single conversion shall be at least 4095.

17 EXAMPLE To print a date and time in the form "Sunday, July 3, 10:02" followed by π to five decimal places:

```
#include <math.h>
#include <stdio.h>
#include <wchar.h>
/* ... */
wchar_t *weekday, *month;  // pointers to wide strings
int day, hour, min;
fwprintf(stdout, L"%ls, %ls %d, %.2d:%.2d\n",
      weekday, month, day, hour, min);
fwprintf(stdout, L"pi = %.5f\n", 4 * atan(1.0));
```

Forward references: the **`btowc`** function (7.31.6.1.1), the **`mbrtowc`** function (7.31.6.3.2).

7.31.2.2 The `fwscanf` function

[394] See "future library directions" (7.33.20).

[395] The behavior is undefined when the types differ as specified for **`va_arg`** 7.16.1.1.

[396] For binary-to-decimal conversion, the result format's values are the numbers representable with the given format specifier. The number of significant digits is determined by the format specifier, and in the case of fixed-point conversion by the source value as well.

 © ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdio.h>
#include <wchar.h>
int fwscanf(FILE * restrict stream, const wchar_t * restrict format, ...);
```

Description

2 The **fwscanf** function reads input from the stream pointed to by `stream`, under control of the wide string pointed to by `format` that specifies the admissible input sequences and how they are to be converted for assignment, using subsequent arguments as pointers to the objects to receive the converted input. If there are insufficient arguments for the format, the behavior is undefined. If the format is exhausted while arguments remain, the excess arguments are evaluated (as always) but are otherwise ignored.

3 The format is composed of zero or more directives: one or more white-space wide characters, an ordinary wide character (neither `%` nor a white-space wide character), or a conversion specification. Each conversion specification is introduced by the wide character `%`. After the `%`, the following appear in sequence:

— An optional assignment-suppressing wide character `*`.

— An optional decimal integer greater than zero that specifies the maximum field width (in wide characters).

— An optional *length modifier* that specifies the size of the receiving object.

— A *conversion specifier* wide character that specifies the type of conversion to be applied.

4 The **fwscanf** function executes each directive of the format in turn. When all directives have been executed, or if a directive fails (as detailed later in this subclause), the function returns. Failures are described as input failures (due to the occurrence of an encoding error or the unavailability of input characters), or matching failures (due to inappropriate input).

5 A directive composed of white-space wide character(s) is executed by reading input up to the first non-white-space wide character (which remains unread), or until no more wide characters can be read. The directive never fails.

6 A directive that is an ordinary wide character is executed by reading the next wide character of the stream. If that wide character differs from the directive, the directive fails and the differing and subsequent wide characters remain unread. Similarly, if end-of-file, an encoding error, or a read error prevents a wide character from being read, the directive fails.

7 A directive that is a conversion specification defines a set of matching input sequences, as described further in this subclause for each specifier. A conversion specification is executed in the following steps:

8 Input white-space wide characters are skipped, unless the specification includes a `[`, `c`, or `n` specifier.[397)]

9 An input item is read from the stream, unless the specification includes an `n` specifier. An input item is defined as the longest sequence of input wide characters which does not exceed any specified field width and which is, or is a prefix of, a matching input sequence.[398)] The first wide character, if any, after the input item remains unread. If the length of the input item is zero, the execution of the directive fails; this condition is a matching failure unless end-of-file, an encoding error, or a read error prevented input from the stream, in which case it is an input failure.

10 Except in the case of a `%` specifier, the input item (or, in the case of a `%n` directive, the count of input wide characters) is converted to a type appropriate to the conversion specifier. If the input item is not a matching sequence, the execution of the directive fails: this condition is a matching failure.

[397)] These white-space wide characters are not counted against a specified field width.

[398)] **fwscanf** pushes back at most one input wide character onto the input stream. Therefore, some sequences that are acceptable to **wcstod**, **wcstol**, etc., are unacceptable to **fwscanf**.

© ISO 2024 — All rights reserved

Unless assignment suppression was indicated by a `*`, the result of the conversion is placed in the object pointed to by the first argument following the `format` argument that has not already received a conversion result. If this object does not have an appropriate type, or if the result of the conversion cannot be represented in the object, the behavior is undefined.

11 The length modifiers and their meanings are:

`hh` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`signed char`** or **`unsigned char`**.

`h` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`short int`** or **`unsigned short int`**.

`l` (ell) Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`long int`** or **`unsigned long int`**; that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`double`**; or that a following `c`, `s`, or `[` conversion specifier applies to an argument with type pointer to **`wchar_t`**.

`ll` (ell-ell) Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`long long int`** or **`unsigned long long int`**.

`j` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`intmax_t`** or **`uintmax_t`**.

`z` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`size_t`** or the corresponding signed integer type.

`t` Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument with type pointer to **`ptrdiff_t`** or the corresponding unsigned integer type.

`w`*N* Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument which is a pointer to an integer with a specific width where *N* is a positive decimal integer with no leading zeros. All minimum-width integer types (7.22.1.2) and exact-width integer types (7.22.1.1) defined in the header `<stdint.h>` shall be supported. Other supported values of *N* are implementation-defined.

`wf`*N* Specifies that a following `b`, `d`, `i`, `o`, `u`, `x`, `X`, or `n` conversion specifier applies to an argument which is a pointer to a fastest minimum-width integer with a specific width where *N* is a positive decimal integer with no leading zeros. All fastest minimum-width integer types (7.22.1.3) defined in the header `<stdint.h>` shall be supported. Other supported values of *N* are implementation-defined.

`L` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`long double`**.

`H` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`_Decimal32`**.

`D` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`_Decimal64`**.

`DD` Specifies that a following `a`, `A`, `e`, `E`, `f`, `F`, `g`, or `G` conversion specifier applies to an argument with type pointer to **`_Decimal128`**.

If a length modifier appears with any conversion specifier other than as specified previously, the behavior is undefined.

12 In the following, the type of the corresponding argument for a conversion specifier shall be a pointer to a type determined by the length modifiers, if any, or specified by the conversion specifier. The conversion specifiers and their meanings are:

 © ISO 2024 — All rights reserved

`d` Matches an optionally signed decimal integer, whose format is the same as expected for the subject sequence of the **`wcstol`** function with the value 10 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`int`**.

`b` Matches an optionally signed binary integer, whose format is the same as expected for the subject sequence of the **`wcstoul`** function with the value 2 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`unsigned int`**.

`i` Matches an optionally signed integer, whose format is the same as expected for the subject sequence of the **`wcstol`** function with the value 0 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`int`**.

`o` Matches an optionally signed octal integer, whose format is the same as expected for the subject sequence of the **`wcstoul`** function with the value 8 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`unsigned int`**.

`u` Matches an optionally signed decimal integer, whose format is the same as expected for the subject sequence of the **`wcstoul`** function with the value 10 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`unsigned int`**.

`x` Matches an optionally signed hexadecimal integer, whose format is the same as expected for the subject sequence of the **`wcstoul`** function with the value 16 for the `base` argument. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`unsigned int`**.

`a,e,f,g` Matches an optionally signed floating-point number, infinity, or NaN, whose format is the same as expected for the subject sequence of the **`wcstod`** function. Unless a length modifier is specified, the corresponding argument shall be a pointer to **`float`**.

`c` Matches a sequence of wide characters of exactly the number specified by the field width (1 if no field width is present in the directive).

If no `l` length modifier is present, characters from the input field are converted as if by repeated calls to the **`wcrtomb`** function, with the conversion state described by an **`mbstate_t`** object initialized to zero before the first wide character is converted. The corresponding argument shall be a pointer to **`char`**, **`signed char`**, **`unsigned char`**, or **`void`** that points to storage large enough to accept the sequence. No null character is added.

If an `l` length modifier is present, the corresponding argument shall be a pointer to storage of **`wchar_t`** large enough to accept the sequence. No null wide character is added.

`s` Matches a sequence of non-white-space wide characters.

If no `l` length modifier is present, characters from the input field are converted as if by repeated calls to the **`wcrtomb`** function, with the conversion state described by an **`mbstate_t`** object initialized to zero before the first wide character is converted. The corresponding argument shall be a pointer to **`char`**, **`signed char`**, **`unsigned char`**, or **`void`** that points to storage large enough to accept the sequence and a terminating null character, which will be added automatically.

If an `l` length modifier is present, the corresponding argument shall be a pointer to storage of **`wchar_t`** large enough to accept the sequence and the terminating null wide character, which will be added automatically.

`[` Matches a nonempty sequence of wide characters from a set of expected characters (the *scanset*).

© ISO 2024 — All rights reserved

If no `l` length modifier is present, characters from the input field are converted as if by repeated calls to the **`wcrtomb`** function, with the conversion state described by an **`mbstate_t`** object initialized to zero before the first wide character is converted. The corresponding argument shall be a pointer to **`char`**, **`signed char`**, **`unsigned char`**, or **`void`** that points to storage large enough to accept the sequence and a terminating null character, which will be added automatically.

If an `l` length modifier is present, the corresponding argument shall be a pointer that points to storage of **`wchar_t`** large enough to accept the sequence and the terminating null wide character, which will be added automatically.

The conversion specifier includes all subsequent wide characters in the `format` string, up to and including the matching right bracket (`]`). The wide characters between the brackets (the *scanlist*) compose the scanset, unless the wide character after the left bracket is a circumflex (`^`), in which case the scanset contains all wide characters that do not appear in the scanlist between the circumflex and the right bracket. If the conversion specifier begins with `[]` or `[^]`, the right bracket wide character is in the scanlist and the next following right bracket wide character is the matching right bracket that ends the specification; otherwise the first following right bracket wide character is the one that ends the specification. If a `-` wide character is in the scanlist and is not the first, nor the second where the first wide character is a `^`, nor the last character, the behavior is implementation-defined.

`p` Matches an implementation-defined set of sequences, which should be the same as the set of sequences that may be produced by the `%p` conversion of the **`fwprintf`** function. The corresponding argument shall be a pointer to a pointer of **`void`**. The input item is converted to a pointer value in an implementation-defined manner. If the input item is a value converted earlier during the same program execution, the pointer that results shall compare equal to that value; otherwise the behavior of the `%p` conversion is undefined.

`n` No input is consumed. The corresponding argument shall be a pointer of a signed integer type. The number of wide characters read from the input stream so far by this call to the **`fwscanf`** function is stored into the integer object pointed to by the argument. Execution of a `%n` directive does not increment the assignment count returned at the completion of execution of the **`fwscanf`** function. No argument is converted, but one is consumed. If the conversion specification includes an assignment-suppressing wide character or a field width, the behavior is undefined.

`%` Matches a single `%` wide character; no conversion or assignment occurs. The complete conversion specification shall be `%%`.

13 If a conversion specification is invalid, the behavior is undefined.[399)]

14 The conversion specifiers `A`, `E`, `F`, `G`, and `X` are also valid and behave the same as, respectively, `a`, `e`, `f`, `g`, and `x`.

15 Trailing white-space wide characters (including new-line wide characters) are left unread unless matched by a directive. The success of literal matches and suppressed assignments is not directly determinable other than via the `%n` directive.

Returns

16 The **`fwscanf`** function returns the value of the macro **`EOF`** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure or if the implementation does not support a specific width length modifier.

17 EXAMPLE 1 The call:

```
#include <stdio.h>
```

[399)] See “future library directions” (7.33.20).

© ISO 2024 — All rights reserved

```
#include <wchar.h>
/* ... */
int n, i; float x; wchar_t name[50];
n = fwscanf(stdin, L"%d%f%ls", &i, &x, name);
```

with the input line:

```
25 54.32E-1 thompson
```

will assign to `n` the value 3, to `i` the value 25, to `x` the value 5.432, and to `name` the sequence `thompson\0`.

18 EXAMPLE 2 The call:

```
#include <stdio.h>
#include <wchar.h>
/* ... */
int i; float x; double y;
fwscanf(stdin, L"%2d%f%*d %lf", &i, &x, &y);
```

with input:

```
56789 0123 56a72
```

will assign to `i` the value 56 and to `x` the value 789.0, will skip past `0123`, and will assign to `y` the value 56.0. The next wide character read from the input stream will be `a`.

Forward references: the **`wcstod`**, **`wcstof`**, and **`wcstold`** functions (7.31.4.1.2), the **`wcstol`**, **`wcstoll`**, **`wcstoul`**, and **`wcstoull`** functions (7.31.4.1.4), the **`wcrtomb`** function (7.31.6.3.3).

7.31.2.3 The `swprintf` function

Synopsis

1

```
#include <wchar.h>
int swprintf(wchar_t * restrict s, size_t n, const wchar_t * restrict format,
      ...);
```

Description

2 The **`swprintf`** function is equivalent to **`fwprintf`**, except that the argument `s` specifies an array of wide characters into which the generated output is to be written, rather than written to a stream. No more than `n` wide characters are written, including a terminating null wide character, which is always added (unless `n` is zero).

Returns

3 The **`swprintf`** function returns the number of wide characters written in the array, not counting the terminating null wide character, or a negative value if an encoding error occurred or if `n` or more wide characters were requested to be written.

7.31.2.4 The `swscanf` function

Synopsis

1

```
#include <wchar.h>
int swscanf(const wchar_t * restrict s, const wchar_t * restrict format, ...);
```

Description

2 The **`swscanf`** function is equivalent to **`fwscanf`**, except that the argument `s` specifies a wide string from which the input is to be obtained, rather than from a stream. Reaching the end of the wide string is equivalent to encountering end-of-file for the **`fwscanf`** function.

© ISO 2024 — All rights reserved

Returns

3 The **swscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **swscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.2.5 The vfwprintf function

Synopsis

1
```
#include <stdarg.h>
#include <stdio.h>
#include <wchar.h>
int vfwprintf(FILE * restrict stream, const wchar_t * restrict format,
      va_list arg);
```

Description

2 The **vfwprintf** function is equivalent to **fwprintf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** invocations). The **vfwprintf** function does not invoke the **va_end** macro.[400)]

Returns

3 The **vfwprintf** function returns the number of wide characters transmitted, or a negative value if an output or encoding error occurred.

4 EXAMPLE The following shows the use of the **vfwprintf** function in a general error-reporting routine.

```
#include <stdarg.h>
#include <stdio.h>
#include <wchar.h>

void error(char *function_name, wchar_t *format, ...)
{
      va_list args;

      va_start(args, format);
      // print out name of function causing error
      fwprintf(stderr, L"ERROR in %s: ", function_name);
      // print out remainder of message
      vfwprintf(stderr, format, args);
      va_end(args);
}
```

7.31.2.6 The vfwscanf function

Synopsis

1
```
#include <stdarg.h>
#include <stdio.h>
#include <wchar.h>
int vfwscanf(FILE * restrict stream, const wchar_t * restrict format,
      va_list arg);
```

Description

2 The **vfwscanf** function is equivalent to **fwscanf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vfwscanf** function does not invoke the **va_end** macro.[400)]

[400)] As the functions **vfwprintf**, **vswprintf**, **vfwscanf**, **vwprintf**, **vwscanf**, and **vswscanf** invoke the **va_arg** macro, the representation of arg after the return is indeterminate.

 © ISO 2024 — All rights reserved

Returns

3 The **vfwscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vfwscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.2.7 The vswprintf function

Synopsis

1
```
#include <stdarg.h>
#include <wchar.h>
int vswprintf(wchar_t * restrict s, size_t n, const wchar_t * restrict format,
      va_list arg);
```

Description

2 The **vswprintf** function is equivalent to **swprintf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vswprintf** function does not invoke the **va_end** macro.[400)]

Returns

3 The **vswprintf** function returns the number of wide characters written in the array, not counting the terminating null wide character, or a negative value if an encoding error occurred or if n or more wide characters were requested to be generated.

7.31.2.8 The vswscanf function

Synopsis

1
```
#include <stdarg.h>
#include <wchar.h>
int vswscanf(const wchar_t * restrict s, const wchar_t * restrict format,
      va_list arg);
```

Description

2 The **vswscanf** function is equivalent to **swscanf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vswscanf** function does not invoke the **va_end** macro.[400)]

Returns

3 The **vswscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vswscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.2.9 The vwprintf function

Synopsis

1
```
#include <stdarg.h>
#include <wchar.h>
int vwprintf(const wchar_t * restrict format, va_list arg);
```

Description

2 The **vwprintf** function is equivalent to **wprintf**, with the variable argument list replaced by arg, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vwprintf** function does not invoke the **va_end** macro.[400)]

© ISO 2024 — All rights reserved

Returns

3 The **vwprintf** function returns the number of wide characters transmitted, or a negative value if an output or encoding error occurred.

7.31.2.10 The vwscanf function

Synopsis

1
```
#include <stdarg.h>
#include <wchar.h>
int vwscanf(const wchar_t * restrict format, va_list arg);
```

Description

2 The **vwscanf** function is equivalent to **wscanf**, with the variable argument list replaced by **arg**, which shall have been initialized by the **va_start** macro (and possibly subsequent **va_arg** calls). The **vwscanf** function does not invoke the **va_end** macro.[400)]

Returns

3 The **vwscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **vwscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.2.11 The wprintf function

Synopsis

1
```
#include <wchar.h>
int wprintf(const wchar_t * restrict format, ...);
```

Description

2 The **wprintf** function is equivalent to **fwprintf** with the argument **stdout** interposed before the arguments to **wprintf**.

Returns

3 The **wprintf** function returns the number of wide characters transmitted, or a negative value if an output or encoding error occurred.

7.31.2.12 The wscanf function

Synopsis

1
```
#include <wchar.h>
int wscanf(const wchar_t * restrict format, ...);
```

Description

2 The **wscanf** function is equivalent to **fwscanf** with the argument **stdin** interposed before the arguments to **wscanf**.

Returns

3 The **wscanf** function returns the value of the macro **EOF** if an input failure occurs before the first conversion (if any) has completed. Otherwise, the **wscanf** function returns the number of input items assigned, which can be fewer than provided for, or even zero, in the event of an early matching failure.

7.31.3 Wide character input/output functions

7.31.3.1 The fgetwc function

Synopsis

1
```
#include <stdio.h>
```

 © ISO 2024 — All rights reserved

```
#include <wchar.h>
wint_t fgetwc(FILE *stream);
```

Description

2 If the end-of-file indicator for the input stream pointed to by `stream` is not set and a next wide character is present, the **`fgetwc`** function obtains that wide character as a **`wchar_t`** converted to a **`wint_t`** and advances the associated file position indicator for the stream (if defined).

Returns

3 If the end-of-file indicator for the stream is set, or if the stream is at end-of-file, the end-of-file indicator for the stream is set and the **`fgetwc`** function returns **`WEOF`**. Otherwise, the **`fgetwc`** function returns the next wide character from the input stream pointed to by `stream`. If a read error occurs, the error indicator for the stream is set and the **`fgetwc`** function returns **`WEOF`**. If an encoding error occurs (including too few bytes), the error indicator for the stream is set and the value of the macro **`EILSEQ`** is stored in **`errno`** and the **`fgetwc`** function returns **`WEOF`**.[401)]

7.31.3.2 The `fgetws` function

Synopsis

1
```
#include <stdio.h>
#include <wchar.h>
wchar_t *fgetws(wchar_t * restrict s, int n, FILE * restrict stream);
```

Description

2 The **`fgetws`** function reads at most one less than the number of wide characters specified by `n` from the stream pointed to by `stream` into the array pointed to by `s`. No additional wide characters are read after a new-line wide character (which is retained) or after end-of-file. A null wide character is written immediately after the last wide character read into the array. If `n` is negative or zero, the behavior is undefined.

Returns

3 The **`fgetws`** function returns `s` if successful. If end-of-file is encountered and no characters have been read into the array, the contents of the array remain unchanged and a null pointer is returned. If a read or encoding error occurs during the operation, the array members have an indeterminate representation and a null pointer is returned.

7.31.3.3 The `fputwc` function

Synopsis

1
```
#include <stdio.h>
#include <wchar.h>
wint_t fputwc(wchar_t c, FILE *stream);
```

Description

2 The **`fputwc`** function writes the wide character specified by `c` to the output stream pointed to by `stream`, at the position indicated by the associated file position indicator for the stream (if defined), and advances the indicator appropriately. If the file cannot support positioning requests, or if the stream was opened with append mode, the character is appended to the output stream.

Returns

3 The **`fputwc`** function returns the wide character written. If a write error occurs, the error indicator for the stream is set and **`fputwc`** returns **`WEOF`**. If an encoding error occurs, the error indicator for the stream is set and the value of the macro **`EILSEQ`** is stored in **`errno`** and **`fputwc`** returns **`WEOF`**.

7.31.3.4 The `fputws` function

[401)] An end-of-file and a read error can be distinguished by use of the **`feof`** and **`ferror`** functions. Also, **`errno`** will be set to **`EILSEQ`** by input/output functions only if an encoding error occurs.

© ISO 2024 — All rights reserved

Synopsis

1
```
#include <stdio.h>
#include <wchar.h>
int fputws(const wchar_t * restrict s, FILE * restrict stream);
```

Description

2 The **fputws** function writes the wide string pointed to by `s` to the stream pointed to by `stream`. The terminating null wide character is not written.

Returns

3 The **fputws** function returns **EOF** if a write or encoding error occurs; otherwise, it returns a nonnegative value.

7.31.3.5 The fwide function

Synopsis

1
```
#include <stdio.h>
#include <wchar.h>
int fwide(FILE *stream, int mode);
```

Description

2 The **fwide** function determines the orientation of the stream pointed to by `stream`. If `mode` is greater than zero, the function first attempts to make the stream wide oriented. If `mode` is less than zero, the function first attempts to make the stream byte oriented.[402)] Otherwise, `mode` is zero and the function does not alter the orientation of the stream.

Returns

3 The **fwide** function returns a value greater than zero if, after the call, the stream has wide orientation, a value less than zero if the stream has byte orientation, or zero if the stream has no orientation.

7.31.3.6 The getwc function

Synopsis

1
```
#include <stdio.h>
#include <wchar.h>
wint_t getwc(FILE *stream);
```

Description

2 The **getwc** function is equivalent to **fgetwc**, except that if it is implemented as a macro, it may evaluate `stream` more than once, so the argument should never be an expression with side effects.

Returns

3 The **getwc** function returns the next wide character from the input stream pointed to by `stream`, or **WEOF**.

7.31.3.7 The getwchar function

Synopsis

1
```
#include <wchar.h>
wint_t getwchar(void);
```

Description

2 The **getwchar** function is equivalent to **getwc** with the argument **stdin**.

[402)] If the orientation of the stream has already been determined, **fwide** does not change it.

© ISO 2024 — All rights reserved

Returns

3 The **getwchar** function returns the next wide character from the input stream pointed to by **stdin**, or **WEOF**.

7.31.3.8 The putwc function

Synopsis

1
```
#include <stdio.h>
#include <wchar.h>
wint_t putwc(wchar_t c, FILE *stream);
```

Description

2 The **putwc** function is equivalent to **fputwc**, except that if it is implemented as a macro, it may evaluate stream more than once, so that argument should never be an expression with side effects.

Returns

3 The **putwc** function returns the wide character written, or **WEOF**.

7.31.3.9 The putwchar function

Synopsis

1
```
#include <wchar.h>
wint_t putwchar(wchar_t c);
```

Description

2 The **putwchar** function is equivalent to **putwc** with the second argument **stdout**.

Returns

3 The **putwchar** function returns the character written, or **WEOF**.

7.31.3.10 The ungetwc function

Synopsis

1
```
#include <stdio.h>
#include <wchar.h>
wint_t ungetwc(wint_t c, FILE *stream);
```

Description

2 The **ungetwc** function pushes the wide character specified by c back onto the input stream pointed to by stream. Pushed-back wide characters will be returned by subsequent reads on that stream in the reverse order of their pushing. A successful intervening call (with the stream pointed to by stream) to a file positioning function (**fseek**, **fsetpos**, or **rewind**) discards any pushed-back wide characters for the stream. The external storage corresponding to the stream is unchanged.

3 One wide character of pushback is guaranteed, even if the call to the **ungetwc** function follows just after a call to a formatted wide character input function **fwscanf**, **vfwscanf**, **vwscanf**, or **wscanf**. If the **ungetwc** function is called too many times on the same stream without an intervening read or file positioning operation on that stream, the operation may fail.

4 If the value of c equals that of the macro **WEOF**, the operation fails and the input stream is unchanged.

5 A successful call to the **ungetwc** function clears the end-of-file indicator for the stream. The value of the file position indicator for the stream after reading or discarding all pushed-back wide characters is the same as it was before the wide characters were pushed back.[403)] For a text or binary stream, the value of its file position indicator after a successful call to the **ungetwc** function is unspecified until all pushed-back wide characters are read or discarded.

[403)] Note that a file positioning function could further modify the file position indicator after discarding any pushed-back wide characters.

© ISO 2024 — All rights reserved

Returns

6 The **ungetwc** function returns the wide character pushed back, or **WEOF** if the operation fails.

7.31.4 General wide string utilities

1 The header <wchar.h> declares functions for wide string manipulation. Various methods are used for determining the lengths of the arrays, but in all cases a **wchar_t*** argument points to the initial (lowest addressed) element of the array. If an array is accessed beyond the end of an object, the behavior is undefined.

2 Where an argument declared as **size_t** n determines the length of the array for a function, n can have the value zero on a call to that function. Unless explicitly stated otherwise in the description of a particular function in this subclause, pointer arguments on such a call shall still have valid values, as described in 7.1.4. On such a call, a function that locates a wide character finds no occurrence, a function that compares two wide character sequences returns zero, and a function that copies wide characters copies zero wide characters.

7.31.4.1 Wide string numeric conversion functions

7.31.4.1.1 General

1 This subclause describes wide string analogs of the **strtod** family of functions (7.24.1.5, 7.24.1.6).[404)]

7.31.4.1.2 The wcstod, wcstof, and wcstold functions

Synopsis

1
```
#include <wchar.h>
double wcstod(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
float wcstof(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
long double wcstold(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
```

Description

2 The **wcstod**, **wcstof**, and **wcstold** functions convert the initial portion of the wide string pointed to by nptr to **double**, **float**, and **long double** representation, respectively. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space wide characters, a subject sequence resembling a floating constant or representing an infinity or NaN; and a final wide string of one or more unrecognized wide characters, including the terminating null wide character of the input wide string. Then, they attempt to convert the subject sequence to a floating-point number, and return the result.

3 The expected form of the subject sequence is an optional plus or minus sign, then one of the following:

— a nonempty sequence of decimal digits optionally containing a decimal-point wide character,

[404)] Wide string analogs of the **strfromd** family of functions (7.24.1.5, 7.24.1.6) are not provided because those conversions can be done by using **mbstowcs** (7.24.8.1) to convert the result of **strfromd**, **strfromf**, and similar to wide string. For example, the following converts **double** d to wide string ws with at most n-1 non-null wide characters, using style g formatting, and computes the number nc of wide characters that would have been written had n been sufficiently large, not counting the terminating null wide character.

```
#include <stdlib.h>
const size_t n = 20;
double d;
//...
// convert d to single-byte character string s
char s[n];
int nc = strfromd(s, n, "%g", d);
// convert s (regarded as a multi-byte character
// string) to wide string ws
wchar_t ws[n];
(void)mbstowcs(ws, s, n);
```

 © ISO 2024 — All rights reserved

then an optional exponent part as defined for the corresponding single-byte characters in 6.4.4.3, excluding any digit separators (6.4.4.2);

— a `0x` or `0X`, then a nonempty sequence of hexadecimal digits optionally containing a decimal-point wide character, then an optional binary exponent part as defined in 6.4.4.3, excluding any digit separators (6.4.4.2);

— `INF` or `INFINITY`, or any other wide string equivalent except for case

— `NAN` or `NAN(`*n-wchar-sequence*$_{opt}$`)`, or any other wide string equivalent except for case in the `NAN` part, where:
n-wchar-sequence:
digit
nondigit
n-wchar-sequence digit
n-wchar-sequence nondigit

The subject sequence is defined as the longest initial subsequence of the input wide string, starting with the first non-white-space wide character, that is of the expected form. The subject sequence contains no wide characters if the input wide string is not of the expected form.

4 If the subject sequence has the expected form for a floating-point number, the sequence of wide characters starting with the first digit or the decimal-point wide character (whichever occurs first) is interpreted as a floating constant according to the rules of 6.4.4.3, except that the decimal-point wide character is used in place of a period, and that if neither an exponent part nor a decimal-point wide character appears in a decimal floating-point number, or if a binary exponent part does not appear in a hexadecimal floating-point number, an exponent part of the appropriate type with value zero is assumed to follow the last digit in the string.

5 If the subject sequence begins with a minus sign, the sequence is interpreted as arithmetically negated.[405)]

6 A wide character sequence `INF` or `INFINITY` is interpreted as an infinity, if representable in the return type, else like a floating constant that is too large for the range of the return type. A wide character sequence `NAN` or `NAN(`*n-wchar-sequence*$_{opt}$`)` is interpreted as a quiet NaN, if supported in the return type, else like a subject sequence part that does not have the expected form; the meaning of the n-wchar sequence is implementation-defined.[406)]

7 A pointer to the final wide string is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

8 If the subject sequence has the hexadecimal form and **`FLT_RADIX`** is a power of 2, the value resulting from the conversion is correctly rounded.

9 In other than the `"C"` locale, additional locale-specific subject sequence forms may be accepted.

10 If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of `nptr` is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

Recommended practice

11 If the subject sequence has the hexadecimal form, **`FLT_RADIX`** is not a power of 2, and the result is not exactly representable, the result should be one of the two numbers in the appropriate internal format that are adjacent to the hexadecimal floating source value, with the extra stipulation that the error should have a correct sign for the current rounding direction.

[405)] It is unspecified whether a minus-signed sequence is converted to a negative number directly or by negating the value resulting from converting the corresponding unsigned sequence (see F.5); the two methods could yield different results if rounding is toward positive or negative infinity. In either case, the functions honor the sign of zero if floating-point arithmetic supports signed zeros.

[406)] An implementation can use the n-wchar sequence to determine extra information to be represented in the NaN's significand.

© ISO 2024 — All rights reserved

12 If the subject sequence has the decimal form and at most M significant digits, where M is the maximum value of the T`_DECIMAL_DIG` macros (defined in `<float.h>`), the result should be correctly rounded. If the subject sequence D has the decimal form and more than M significant digits, consider the two bounding, adjacent decimal strings L and U, both having M significant digits, such that the values of L, D, and U satisfy $L \leq D \leq U$. The result should be one of the (equal or adjacent) values that would be obtained by correctly rounding L and U according to the current rounding direction, with the extra stipulation that the error with respect to D should have a correct sign for the current rounding direction.[407)]

Returns

13 The functions return the converted value, if any. If no conversion could be performed, positive or unsigned zero is returned.

14 If the correct value overflows and default rounding is in effect (7.12.1), plus or minus **HUGE_VAL**, **HUGE_VALF**, or **HUGE_VALL** is returned (according to the return type and sign of the value); if the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero, the integer expression **errno** acquires the value of **ERANGE**; if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero, the "overflow" floating-point exception is raised.

15 If the result underflows (7.12.1), the functions return a value whose magnitude is no greater than the smallest normalized positive number in the return type; if the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero, whether **errno** acquires the value **ERANGE** is implementation-defined; if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero, whether the "underflow" floating-point exception is raised is implementation-defined.

7.31.4.1.3 The wcstod*N* functions

Synopsis

1
```
#include <wchar.h>
#ifdef __STDC_IEC_60559_DFP__
_Decimal32 wcstod32(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
_Decimal64 wcstod64(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
_Decimal128 wcstod128(const wchar_t * restrict nptr, wchar_t ** restrict endptr);
#endif
```

Description

2 The **wcstod***N* functions convert the initial portion of the wide string pointed to by `nptr` to decimal floating type representation. First, they decompose the input wide string into three parts: an initial, possibly empty, sequence of white-space wide characters; a subject sequence resembling a floating constant or representing an infinity or NaN; and a final wide string of one or more unrecognized wide characters, including the terminating null wide character of the input wide string. Then, they attempt to convert the subject sequence to a floating-point number, and return the result.

3 The expected form of the subject sequence is an optional plus or minus sign, then one of the following:

— a nonempty sequence of decimal digits optionally containing a decimal-point wide character, then an optional exponent part as defined in 6.4.4.3, excluding any digit separators (6.4.4.2)

— a `0x` or `0X`, then a nonempty sequence of hexadecimal digits optionally containing a decimal-point wide character, then an optional binary exponent part as defined in 6.4.4.3, excluding any digit separators (6.4.4.2)

— `INF` or `INFINITY`, ignoring case

— `NAN` or `NAN(`*d-wchar-sequence*$_{opt}$`)`, ignoring case in the `NAN` part, where:

[407)] M is sufficiently large that L and `U` will usually correctly round to the same internal floating value, but if not will correctly round to adjacent values.

 © ISO 2024 — All rights reserved

d-wchar-sequence:
digit
nondigit
d-wchar-sequence digit
d-wchar-sequence nondigit

The subject sequence is defined as the longest initial subsequence of the input wide string, starting with the first non-white-space wide character, that is of the expected form. The subject sequence contains no wide characters if the input wide string is not of the expected form.

4 If the subject sequence has the expected form for a floating-point number, the sequence of wide characters starting with the first digit or the decimal-point wide character (whichever occurs first) is interpreted as a floating constant according to the rules of 6.4.4.3, except that the decimal-point wide character is used in place of a period, and that if neither an exponent part nor a decimal-point wide character appears in a decimal floating-point number, or if a binary exponent part does not appear in a hexadecimal floating-point number, an exponent part of the appropriate type with value zero is assumed to follow the last digit in the wide string. If the subject sequence begins with a minus sign, the sequence is interpreted as arithmetically negated before rounding and the sign s is set to -1, else s is set to 1.

5 If the subject sequence has the expected form for a decimal floating-point number, the value resulting from the conversion is correctly rounded and the coefficient c and the quantum exponent q are determined by the rules in 6.4.4.3 for a decimal floating constant of decimal type.

6 If the subject sequence has the expected form for a hexadecimal floating-point number, the value resulting from the conversion is correctly rounded provided the subject sequence has at most M significant hexadecimal digits, where $M \geq \lceil (P-1)/4 \rceil + 1$ is implementation-defined, and P is the maximum precision of the supported radix-2 floating types and binary non-arithmetic interchange formats.[408] If all subject sequences of hexadecimal form are correctly rounded, M may be regarded as infinite. If the subject sequence has more than M significant hexadecimal digits, the implementation may first round to M significant hexadecimal digits according to the applicable decimal rounding direction mode, signaling exceptions as though converting from a wider format, then correctly round the result of the shortened hexadecimal input to the result type. The preferred quantum exponent for the result is 0 if the hexadecimal number is exactly represented in the decimal type; the preferred quantum exponent for the result is the least possible if the hexadecimal number is not exactly represented in the decimal type.

7 A wide character sequence `INF` or `INFINITY` is interpreted as an infinity. A wide character sequence `NAN` or `NAN(`*d-wchar-sequence*$_{opt}$`)`, is interpreted as a quiet NaN; the meaning of the d-wchar sequence is implementation-defined.[409] A pointer to the final wide string is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

8 In other than the `"C"` locale, additional locale-specific subject sequence forms may be accepted.

9 If the subject sequence is empty or does not have the expected form, no conversion is performed; the value of `nptr` is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

Returns

10 The **`wcstod`***N* functions return the converted value, if any. If no conversion could be performed, the value of the triple $(+1, 0, 0)$ is returned. If the correct value overflows:

— the value of the macro **`ERANGE`** is stored in **`errno`** if the integer expression **`math_errhandling`** & **`MATH_ERRNO`** is nonzero;

[408] Non-arithmetic interchange formats are an optional feature in Annex H.
[409] An implementation may use the d-wchar sequence to determine extra information to be represented in the NaN's significand.

© ISO 2024 — All rights reserved

— the "overflow" floating-point exception is raised if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero.

If the result underflows (7.12.1), whether **errno** acquires the value **ERANGE** if the integer expression **math_errhandling** & **MATH_ERRNO** is nonzero is implementation-defined; if the integer expression **math_errhandling** & **MATH_ERREXCEPT** is nonzero, whether the "underflow" floating-point exception is raised is implementation-defined.

7.31.4.1.4 The wcstol, wcstoll, wcstoul, and wcstoull functions

Synopsis

1

```
#include <wchar.h>
long int wcstol(const wchar_t * restrict nptr, wchar_t ** restrict endptr,
      int base);
long long int wcstoll(const wchar_t * restrict nptr, wchar_t ** restrict endptr,
      int base);
unsigned long int wcstoul(const wchar_t * restrict nptr,
      wchar_t ** restrict endptr, int base);
unsigned long long int wcstoull(const wchar_t * restrict nptr,
      wchar_t ** restrict endptr, int base);
```

Description

2 The **wcstol**, **wcstoll**, **wcstoul**, and **wcstoull** functions convert the initial portion of the wide string pointed to by nptr to **long int**, **long long int**, **unsigned long int**, and **unsigned long long int** representation, respectively. First, they decompose the input string into three parts: an initial, possibly empty, sequence of white-space wide characters, a subject sequence resembling an integer represented in some radix determined by the value of base, and a final wide string of one or more unrecognized wide characters, including the terminating null wide character of the input wide string. Then, they attempt to convert the subject sequence to an integer, and return the result.

3 If the value of base is zero, the expected form of the subject sequence is that of an integer constant as described for the corresponding single-byte characters in 6.4.4.2, optionally preceded by a plus or minus sign, but not including an integer suffix or any optional digit separators (6.4.4.2). If the value of base is between 2 and 36 (inclusive), the expected form of the subject sequence is a sequence of letters and digits representing an integer with the radix specified by base, optionally preceded by a plus or minus sign, but not including an integer suffix or any optional digit separators. The letters from a (or A) through z (or Z) are ascribed the values 10 through 35; only letters and digits whose ascribed values are less than that of base are permitted. If the value of base is 2, the characters 0b or 0B may optionally precede the sequence of letters and digits, following the sign if present. If the value of base is 16, the wide characters 0x or 0X may optionally precede the sequence of letters and digits, following the sign if present.

4 The subject sequence is defined as the longest initial subsequence of the input wide string, starting with the first non-white-space wide character, that is of the expected form. The subject sequence contains no wide characters if the input wide string is empty or consists entirely of white-space wide characters, or if the first non-white-space wide character is other than a sign or a permissible letter or digit.

5 If the subject sequence has the expected form and the value of base is zero, the sequence of wide characters starting with the first digit is interpreted as an integer constant according to the rules of 6.4.4.2. If the subject sequence has the expected form and the value of base is between 2 and 36, it is used as the base for conversion, ascribing to each letter its value as previously given. If the subject sequence begins with a minus sign, the resulting value is the negative of the converted value; for functions whose return type is an unsigned integer type this action is performed in the return type. A pointer to the final wide string is stored in the object pointed to by endptr, provided that endptr is not a null pointer.

6 In other than the "C" locale, additional locale-specific subject sequence forms may be accepted.

7 If the subject sequence is empty or does not have the expected form, no conversion is performed; the

 © ISO 2024 — All rights reserved

value of `nptr` is stored in the object pointed to by `endptr`, provided that `endptr` is not a null pointer.

Returns

8 The **`wcstol`**, **`wcstoll`**, **`wcstoul`**, and **`wcstoull`** functions return the converted value, if any. If no conversion could be performed, zero is returned. If the correct value is outside the range of representable values, **`LONG_MIN`**, **`LONG_MAX`**, **`LLONG_MIN`**, **`LLONG_MAX`**, **`ULONG_MAX`**, or **`ULLONG_MAX`** is returned (according to the return type sign of the value, if any), and the value of the macro **`ERANGE`** is stored in **`errno`**.

7.31.4.2 Wide string copying functions

7.31.4.2.1 The `wcscpy` function

Synopsis

1
```
#include <wchar.h>
wchar_t *wcscpy(wchar_t * restrict s1, const wchar_t * restrict s2);
```

Description

2 The **`wcscpy`** function copies the wide string pointed to by `s2` (including the terminating null wide character) into the array pointed to by `s1`.

Returns

3 The **`wcscpy`** function returns the value of `s1`.

7.31.4.2.2 The `wcsncpy` function

Synopsis

1
```
#include <wchar.h>
wchar_t *wcsncpy(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
```

Description

2 The **`wcsncpy`** function copies not more than `n` wide characters (those that follow a null wide character are not copied) from the array pointed to by `s2` to the array pointed to by `s1`.[410)]

3 If the array pointed to by `s2` is a wide string that is shorter than `n` wide characters, null wide characters are appended to the copy in the array pointed to by `s1`, until `n` wide characters in all have been written.

Returns

4 The **`wcsncpy`** function returns the value of `s1`.

7.31.4.2.3 The `wmemcpy` function

Synopsis

1
```
#include <wchar.h>
wchar_t *wmemcpy(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
```

Description

2 The **`wmemcpy`** function copies `n` wide characters from the object pointed to by `s2` to the object pointed to by `s1`.

Returns

3 The **`wmemcpy`** function returns the value of `s1`.

7.31.4.2.4 The `wmemmove` function

[410)] Thus, if there is no null wide character in the first `n` wide characters of the array pointed to by `s2`, the result will not be null-terminated.

© ISO 2024 — All rights reserved

Synopsis

```
#include <wchar.h>
wchar_t *wmemmove(wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

2 The **wmemmove** function copies n wide characters from the object pointed to by s2 to the object pointed to by s1. Copying takes place as if the n wide characters from the object pointed to by s2 are first copied into a temporary array of n wide characters that does not overlap the objects pointed to by s1 or s2, and then the n wide characters from the temporary array are copied into the object pointed to by s1.

Returns

3 The **wmemmove** function returns the value of s1.

7.31.4.3 Wide string concatenation functions

7.31.4.3.1 The wcscat function

Synopsis

```
#include <wchar.h>
wchar_t *wcscat(wchar_t * restrict s1, const wchar_t * restrict s2);
```

Description

2 The **wcscat** function appends a copy of the wide string pointed to by s2 (including the terminating null wide character) to the end of the wide string pointed to by s1. The initial wide character of s2 overwrites the null wide character at the end of s1.

Returns

3 The **wcscat** function returns the value of s1.

7.31.4.3.2 The wcsncat function

Synopsis

```
#include <wchar.h>
wchar_t *wcsncat(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
```

Description

2 The **wcsncat** function appends not more than n wide characters (a null wide character and those that follow it are not appended) from the array pointed to by s2 to the end of the wide string pointed to by s1. The initial wide character of s2 overwrites the null wide character at the end of s1. A terminating null wide character is always appended to the result.[411)]

Returns

3 The **wcsncat** function returns the value of s1.

7.31.4.4 Wide string comparison functions

1 Unless explicitly stated otherwise, the functions described in this subclause order two wide characters the same way as two integers of the underlying integer type designated by **wchar_t**.

7.31.4.4.1 The wcscmp function

Synopsis

```
#include <wchar.h>
int wcscmp(const wchar_t *s1, const wchar_t *s2);
```

[411)] Thus, the maximum number of wide characters that can end up in the array pointed to by s1 is **wcslen**(s1)+n+1.

 © ISO 2024 — All rights reserved

Description

2 The **wcscmp** function compares the wide string pointed to by s1 to the wide string pointed to by s2.

Returns

3 The **wcscmp** function returns an integer greater than, equal to, or less than zero, accordingly as the wide string pointed to by s1 is greater than, equal to, or less than the wide string pointed to by s2.

7.31.4.4.2 The wcscoll function

Synopsis

1
```
#include <wchar.h>
int wcscoll(const wchar_t *s1, const wchar_t *s2);
```

Description

2 The **wcscoll** function compares the wide string pointed to by s1 to the wide string pointed to by s2, both interpreted as appropriate to the **LC_COLLATE** category of the current locale.

Returns

3 The **wcscoll** function returns an integer greater than, equal to, or less than zero, accordingly as the wide string pointed to by s1 is greater than, equal to, or less than the wide string pointed to by s2 when both are interpreted as appropriate to the current locale.

7.31.4.4.3 The wcsncmp function

Synopsis

1
```
#include <wchar.h>
int wcsncmp(const wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

2 The **wcsncmp** function compares not more than n wide characters (those that follow a null wide character are not compared) from the array pointed to by s1 to the array pointed to by s2.

Returns

3 The **wcsncmp** function returns an integer greater than, equal to, or less than zero, accordingly as the possibly null-terminated array pointed to by s1 is greater than, equal to, or less than the possibly null-terminated array pointed to by s2.

7.31.4.4.4 The wcsxfrm function

Synopsis

1
```
#include <wchar.h>
size_t wcsxfrm(wchar_t * restrict s1, const wchar_t * restrict s2, size_t n);
```

Description

2 The **wcsxfrm** function transforms the wide string pointed to by s2 and places the resulting wide string into the array pointed to by s1. The transformation is such that if the **wcscmp** function is applied to two transformed wide strings, it returns a value greater than, equal to, or less than zero, corresponding to the result of the **wcscoll** function applied to the same two original wide strings. No more than n wide characters are placed into the resulting array pointed to by s1, including the terminating null wide character. If n is zero, s1 is permitted to be a null pointer.

Returns

3 The **wcsxfrm** function returns the length of the transformed wide string (not including the terminating null wide character). If the value returned is n or greater, the members of the array pointed to by s1 have an indeterminate representation.

4 EXAMPLE The value of the following expression is the length of the array needed to hold the transformation of the wide string pointed to by s:

© ISO 2024 — All rights reserved

```
    1 + wcsxfrm(nullptr, s, 0)
```

7.31.4.4.5 The wmemcmp function

Synopsis

1
```
#include <wchar.h>
int wmemcmp(const wchar_t *s1, const wchar_t *s2, size_t n);
```

Description

2 The **wmemcmp** function compares the first n wide characters of the object pointed to by s1 to the first n wide characters of the object pointed to by s2.

Returns

3 The **wmemcmp** function returns an integer greater than, equal to, or less than zero, accordingly as the object pointed to by s1 is greater than, equal to, or less than the object pointed to by s2.

7.31.4.5 Wide string search functions

7.31.4.5.1 Introduction

1 The stateless search functions in this section (**wcschr**, **wcspbrk**, **wcsrchr**, **wmemchr**, **wcsstr**) are *generic functions*. These functions are generic in the qualification of the array to be searched and will return a result pointer to an element with the same qualification as the passed array. If the array to be searched is **const**-qualified, the result pointer will be to a **const**-qualified element. If the array to be searched is not **const**-qualified,[412)] the result pointer will be to an unqualified element.

2 The external declarations of these generic functions have a concrete function type that returns a pointer to an unqualified element of type **wchar_t** (indicated by *QWchar_t*), and accepts a pointer to a **const**-qualified array of the same type to search. This signature supports all correct uses. If a macro definition of any of these generic functions is suppressed to access an actual function, the external declaration with this concrete type is visible.[413)]

3 The **volatile** and **restrict** qualifiers are not accepted on the elements of the array to search.

7.31.4.5.2 The wcschr generic function

Synopsis

1
```
#include <wchar.h>
QWchar_t *wcschr(QWchar_t *s, wchar_t c);
```

Description

2 The **wcschr** generic function locates the first occurrence of c in the wide string pointed to by s. The terminating null wide character is considered to be part of the wide string.

Returns

3 The **wcschr** generic function returns a pointer to the located wide character, or a null pointer if the wide character does not occur in the wide string.

7.31.4.5.3 The wcscspn function

Synopsis

1
```
#include <wchar.h>
size_t wcscspn(const wchar_t *s1, const wchar_t *s2);
```

[412)] The null pointer constant is not a pointer to a **const**-qualified type, and therefore the result expression has the type of a pointer to an unqualified element; however, evaluating such a call is undefined.
[413)] This is an obsolescent feature.

 © ISO 2024 — All rights reserved

Description

2 The **wcscspn** function computes the length of the maximum initial segment of the wide string pointed to by `s1` which consists entirely of wide characters *not* from the wide string pointed to by `s2`.

Returns

3 The **wcscspn** function returns the length of the segment.

7.31.4.5.4 The wcspbrk generic function

Synopsis

1
```
#include <wchar.h>
QWchar_t *wcspbrk(QWchar_t *s1, const wchar_t *s2);
```

Description

2 The **wcspbrk** generic function locates the first occurrence in the wide string pointed to by `s1` of any wide character from the wide string pointed to by `s2`.

Returns

3 The **wcspbrk** generic function returns a pointer to the wide character in `s1`, or a null pointer if no wide character from `s2` occurs in `s1`.

7.31.4.5.5 The wcsrchr generic function

Synopsis

1
```
#include <wchar.h>
QWchar_t *wcsrchr(QWchar_t *s, wchar_t c);
```

Description

2 The **wcsrchr** generic function locates the last occurrence of `c` in the wide string pointed to by `s`. The terminating null wide character is considered to be part of the wide string.

Returns

3 The **wcsrchr** generic function returns a pointer to the wide character, or a null pointer if `c` does not occur in the wide string.

7.31.4.5.6 The wcsspn function

Synopsis

1
```
#include <wchar.h>
size_t wcsspn(const wchar_t *s1, const wchar_t *s2);
```

Description

2 The **wcsspn** function computes the length of the maximum initial segment of the wide string pointed to by `s1` which consists entirely of wide characters from the wide string pointed to by `s2`.

Returns

3 The **wcsspn** function returns the length of the segment.

7.31.4.5.7 The wcsstr generic function

Synopsis

1
```
#include <wchar.h>
QWchar_t *wcsstr(QWchar_t *s1, const wchar_t *s2);
```

© ISO 2024 — All rights reserved

Description

2 The **wcsstr** generic function locates the first occurrence in the wide string pointed to by s1 of the sequence of wide characters (excluding the terminating null wide character) in the wide string pointed to by s2.

Returns

3 The **wcsstr** generic function returns a pointer to the located wide string, or a null pointer if the wide string is not found. If s2 points to a wide string with zero length, the function returns s1.

© ISO 2024 — All rights reserved

7.31.4.5.8 The `wcstok` function

Synopsis

1
```
#include <wchar.h>
wchar_t *wcstok(wchar_t * restrict s1, const wchar_t * restrict s2,
      wchar_t ** restrict ptr);
```

Description

2 A sequence of calls to the **wcstok** function breaks the wide string pointed to by `s1` into a sequence of tokens, each of which is delimited by a wide character from the wide string pointed to by `s2`. The third argument points to a caller-provided **wchar_t** pointer into which the **wcstok** function stores information necessary for it to continue scanning the same wide string.

3 The first call in a sequence has a non-null first argument and stores an initial value in the object pointed to by `ptr`. Subsequent calls in the sequence have a null first argument and the object pointed to by `ptr` is required to have the value stored by the previous call in the sequence, which is then updated. The separator wide string pointed to by `s2` may be different from call to call.

4 The first call in the sequence searches the wide string pointed to by `s1` for the first wide character that is *not* contained in the current separator wide string pointed to by `s2`. If no such wide character is found, then there are no tokens in the wide string pointed to by `s1` and the **wcstok** function returns a null pointer. If such a wide character is found, it is the start of the first token.

5 The **wcstok** function then searches from there for a wide character that *is* contained in the current separator wide string. If no such wide character is found, the current token extends to the end of the wide string pointed to by `s1`, and subsequent searches in the same wide string for a token return a null pointer. If such a wide character is found, it is overwritten by a null wide character, which terminates the current token.

6 In all cases, the **wcstok** function stores sufficient information in the pointer pointed to by `ptr` so that subsequent calls, with a null pointer for `s1` and the unmodified pointer value for `ptr`, shall start searching just past the element overwritten by a null wide character (if any).

Returns

7 The **wcstok** function returns a pointer to the first wide character of a token, or a null pointer if there is no token.

8 EXAMPLE

```
#include <wchar.h>
static wchar_t str1[] = L"?a???b,,,#c";
static wchar_t str2[] = L"\t \t";
wchar_t *t, *ptr1, *ptr2;

t = wcstok(str1, L"?", &ptr1);     // t points to the token L"a"
t = wcstok(nullptr, L",", &ptr1);  // t points to the token L"??b"
t = wcstok(str2, L" \t", &ptr2);   // t is a null pointer
t = wcstok(nullptr, L"#,", &ptr1); // t points to the token L"c"
t = wcstok(nullptr, L"?", &ptr1);  // t is a null pointer
```

7.31.4.5.9 The `wmemchr` generic function

Synopsis

1
```
#include <wchar.h>
QWchar_t *wmemchr(QWchar_t *s, wchar_t c, size_t n);
```

Description

2 The **wmemchr** generic function locates the first occurrence of `c` in the initial `n` wide characters of the object pointed to by `s`.

© ISO 2024 — All rights reserved

Returns

3 The **wmemchr** generic function returns a pointer to the located wide character, or a null pointer if the wide character does not occur in the object.

7.31.4.6 Miscellaneous functions

7.31.4.6.1 The wcslen function

Synopsis

1
```
#include <wchar.h>
size_t wcslen(const wchar_t *s);
```

Description

2 The **wcslen** function computes the length of the wide string pointed to by s.

Returns

3 The **wcslen** function returns the number of wide characters that precede the terminating null wide character.

7.31.4.6.2 The wmemset function

Synopsis

1
```
#include <wchar.h>
wchar_t *wmemset(wchar_t *s, wchar_t c, size_t n);
```

Description

2 The **wmemset** function copies the value of c into each of the first n wide characters of the object pointed to by s.

Returns

3 The **wmemset** function returns the value of s.

7.31.5 Wide character time conversion functions

7.31.5.1 The wcsftime function

Synopsis

1
```
#include <time.h>
#include <wchar.h>
size_t wcsftime(wchar_t * restrict s, size_t maxsize,
      const wchar_t * restrict format, const struct tm * restrict timeptr);
```

Description

2 The **wcsftime** function is equivalent to the **strftime** function, except that:

— The argument s points to the initial element of an array of wide characters into which the generated output is to be placed.

— The argument maxsize indicates the limiting number of wide characters.

— The argument format is a wide string and the conversion specifiers are replaced by corresponding sequences of wide characters.

— The return value indicates the number of wide characters.

Returns

3 If the total number of resulting wide characters including the terminating null wide character is not more than maxsize, the **wcsftime** function returns the number of wide characters placed into the array pointed to by s not including the terminating null wide character. Otherwise, zero is returned and the members of the array have an indeterminate representation.

© ISO 2024 — All rights reserved

7.31.6 Extended multibyte/wide character conversion utilities

1 The header <wchar.h> declares an extended set of functions useful for conversion between multibyte characters and wide characters.

2 Most of the following functions — those that are listed as "restartable", 7.31.6.3 and 7.31.6.4 — take as a last argument a pointer to an object of type **mbstate_t** that is used to describe the current *conversion state* from a particular multibyte character sequence to a wide character sequence (or the reverse) under the rules of a particular setting for the **LC_CTYPE** category of the current locale.

3 The initial conversion state corresponds, for a conversion in either direction, to the beginning of a new multibyte character in the initial shift state. A zero-valued **mbstate_t** object is (at least) one way to describe an initial conversion state. A zero-valued **mbstate_t** object can be used to initiate conversion involving any multibyte character sequence, in any **LC_CTYPE** category setting. If an **mbstate_t** object has been altered by any of the functions described in this subclause, and is then used with a different multibyte character sequence, or in the other conversion direction, or with a different **LC_CTYPE** category setting than on earlier function calls, the behavior is undefined.[414)]

4 On entry, each function takes the described conversion state (either internal or pointed to by an argument) as current. The conversion state described by the referenced object is altered as needed to track the shift state, and the position within a multibyte character, for the associated multibyte character sequence.

7.31.6.1 Single-byte/wide character conversion functions

7.31.6.1.1 The btowc function

Synopsis

1
```
#include <wchar.h>
wint_t btowc(int c);
```

Description

2 The **btowc** function determines whether c constitutes a valid single-byte character in the initial shift state.

Returns

3 The **btowc** function returns **WEOF** if c has the value **EOF** or if (**unsigned char**)c does not constitute a valid single-byte character in the initial shift state. Otherwise, it returns the wide character representation of that character.

7.31.6.1.2 The wctob function

Synopsis

1
```
#include <wchar.h>
int wctob(wint_t c);
```

Description

2 The **wctob** function determines whether c corresponds to a member of the extended character set whose multibyte character representation is a single byte when in the initial shift state.

Returns

3 The **wctob** function returns **EOF** if c does not correspond to a multibyte character with length one in the initial shift state. Otherwise, it returns the single-byte representation of that character as an **unsigned char** converted to an **int**.

7.31.6.2 Conversion state functions

7.31.6.2.1 The mbsinit function

Synopsis

1
```
#include <wchar.h>
```

[414)] Thus, a particular **mbstate_t** object can be used, for example, with both the **mbrtowc** and **mbsrtowcs** functions as long as they are used to step sequentially through the same multibyte character string.

© ISO 2024 — All rights reserved

```
int mbsinit(const mbstate_t *ps);
```

Description

2 If `ps` is not a null pointer, the **`mbsinit`** function determines whether the referenced **`mbstate_t`** object describes an initial conversion state.

Returns

3 The **`mbsinit`** function returns nonzero if `ps` is a null pointer or if the referenced object describes an initial conversion state; otherwise, it returns zero.

7.31.6.3 Restartable multibyte/wide character conversion functions

1 These functions differ from the corresponding multibyte character functions of 7.24.7 (**`mblen`**, **`mbtowc`**, and **`wctomb`**) in that they have an extra parameter, `ps`, of type pointer to **`mbstate_t`** that points to an object that can completely describe the current conversion state of the associated multibyte character sequence. If `ps` is a null pointer, each function uses its own internal **`mbstate_t`** object instead, which is initialized prior to the first call to the function to the initial conversion state; the functions are not required to avoid data races with other calls to the same function in this case. It is implementation-defined whether the internal **`mbstate_t`** object has thread storage duration; if it has thread storage duration, it is initialized to the initial conversion state prior to the first call to the function on the new thread. The implementation behaves as if no library function calls these functions with a null pointer for `ps`.

2 Also unlike their corresponding functions, the return value does not represent whether the encoding is state-dependent.

7.31.6.3.1 The `mbrlen` function

Synopsis

1
```
#include <wchar.h>
size_t mbrlen(const char * restrict s, size_t n, mbstate_t * restrict ps);
```

Description

2 The **`mbrlen`** function is equivalent to the call:

```
mbrtowc(nullptr, s, n, ps != nullptr ? ps: &internal)
```

where `internal` is the **`mbstate_t`** object for the **`mbrlen`** function, except that the expression designated by `ps` is evaluated only once.

Returns

3 The **`mbrlen`** function returns a value between zero and `n`, inclusive, `(size_t)(-2)`, or `(size_t)(-1)`.

Forward references: the **`mbrtowc`** function (7.31.6.3.2).

7.31.6.3.2 The `mbrtowc` function

Synopsis

1
```
#include <wchar.h>
size_t mbrtowc(wchar_t * restrict pwc, const char * restrict s, size_t n,
      mbstate_t * restrict ps);
```

Description

2 If `s` is a null pointer, the **`mbrtowc`** function is equivalent to the call:

```
mbrtowc(nullptr, "", 1, ps)
```

In this case, the values of the parameters `pwc` and `n` are ignored.

 © ISO 2024 — All rights reserved

3 If `s` is not a null pointer, the **`mbrtowc`** function inspects at most `n` bytes beginning with the byte pointed to by `s` to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is complete and valid, it determines the value of the corresponding wide character and then, if `pwc` is not a null pointer, stores that value in the object pointed to by `pwc`. If the corresponding wide character is the null wide character, the resulting state described is the initial conversion state.

Returns

4 The **`mbrtowc`** function returns the first of the following that applies (given the current conversion state):

`0` if the next `n` or fewer bytes complete the multibyte character that corresponds to the null wide character (which is the value stored).

between `1` *and* `n` *inclusive* if the next `n` or fewer bytes complete a valid multibyte character (which is the value stored); the value returned is the number of bytes that complete the multibyte character.

`(`**`size_t`**`)(-2)` if the next `n` bytes contribute to an incomplete (but potentially valid) multibyte character, and all `n` bytes have been processed (no value is stored).[415)]

`(`**`size_t`**`)(-1)` if an encoding error occurs, in which case the next `n` or fewer bytes do not contribute to a complete and valid multibyte character (no value is stored); the value of the macro **`EILSEQ`** is stored in **`errno`**, and the conversion state is unspecified.

7.31.6.3.3 The `wcrtomb` function

Synopsis

1
```
#include <wchar.h>
size_t wcrtomb(char * restrict s, wchar_t wc, mbstate_t * restrict ps);
```

Description

2 If `s` is a null pointer, the **`wcrtomb`** function is equivalent to the call

```
wcrtomb(buf, L'\0', ps)
```

where `buf` is an internal buffer.

3 If `s` is not a null pointer, the **`wcrtomb`** function determines the number of bytes needed to represent the multibyte character that corresponds to the wide character given by `wc` (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by `s`. At most **`MB_CUR_MAX`** bytes are stored. If `wc` is a null wide character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

Returns

4 The **`wcrtomb`** function returns the number of bytes stored in the array object (including any shift sequences). When `wc` is not a valid wide character, an encoding error occurs: the function stores the value of the macro **`EILSEQ`** in **`errno`** and returns `(`**`size_t`**`)(-1)`; the conversion state is unspecified.

7.31.6.4 Restartable multibyte/wide string conversion functions

1 These functions differ from the corresponding multibyte string functions of 7.24.8 (**`mbstowcs`** and **`wcstombs`**) in that they have an extra parameter, `ps`, of type pointer to **`mbstate_t`** that points to an object that can completely describe the current conversion state of the associated multibyte character sequence. If `ps` is a null pointer, each function uses its own internal **`mbstate_t`** object

[415)] When `n` has at least the value of the **`MB_CUR_MAX`** macro, this case can only occur if `s` points at a sequence of redundant shift sequences (for implementations with state-dependent encodings).

© ISO 2024 — All rights reserved

instead, which is initialized prior to the first call to the function to the initial conversion state; the functions are not required to avoid data races with other calls to the same function in this case. It is implementation-defined whether the internal **mbstate_t** object has thread storage duration; if it has thread storage duration, it is initialized to the initial conversion state prior to the first call to the function on the new thread. The implementation behaves as if no library function calls these functions with a null pointer for ps.

2 Also unlike their corresponding functions, the conversion source parameter, src, has a pointer-to-pointer type. When the function is storing the results of conversions (that is, when dst is not a null pointer), the pointer object pointed to by this parameter is updated to reflect the amount of the source processed by that invocation.

7.31.6.4.1 The mbsrtowcs function

Synopsis

1
```
#include <wchar.h>
size_t mbsrtowcs(wchar_t * restrict dst, const char ** restrict src, size_t len,
      mbstate_t * restrict ps);
```

Description

2 The **mbsrtowcs** function converts a sequence of multibyte characters that begins in the conversion state described by the object pointed to by ps, from the array indirectly pointed to by src into a sequence of corresponding wide characters. If dst is not a null pointer, the converted characters are stored into the array pointed to by dst. Conversion continues up to and including a terminating null character, which is also stored. Conversion stops earlier in two cases: when a sequence of bytes is encountered that does not form a valid multibyte character, or (if dst is not a null pointer) when len wide characters have been stored into the array pointed to by dst.[416] Each conversion takes place as if by a call to the **mbrtowc** function.

3 If dst is not a null pointer, the pointer object pointed to by src is assigned either a null pointer (if conversion stopped due to reaching a terminating null character) or the address just past the last multibyte character converted (if any). If conversion stopped due to reaching a terminating null character and if dst is not a null pointer, the resulting state described is the initial conversion state.

Returns

4 If the input conversion encounters a sequence of bytes that do not form a valid multibyte character, an encoding error occurs: the **mbsrtowcs** function stores the value of the macro **EILSEQ** in **errno** and returns `(size_t)(-1)`; the conversion state is unspecified. Otherwise, it returns the number of multibyte characters successfully converted, not including the terminating null character (if any).

7.31.6.4.2 The wcsrtombs function

Synopsis

1
```
#include <wchar.h>
size_t wcsrtombs(char * restrict dst, const wchar_t ** restrict src, size_t len,
      mbstate_t * restrict ps);
```

Description

2 The **wcsrtombs** function converts a sequence of wide characters from the array indirectly pointed to by src into a sequence of corresponding multibyte characters that begins in the conversion state described by the object pointed to by ps. If dst is not a null pointer, the converted characters are then stored into the array pointed to by dst. Conversion continues up to and including a terminating null wide character, which is also stored. Conversion stops earlier in two cases: when a wide character is reached that does not correspond to a valid multibyte character, or (if dst is not a null pointer) when the next multibyte character would exceed the limit of len total bytes to be stored into the

[416] Thus, the value of len is ignored if dst is a null pointer.

 © ISO 2024 — All rights reserved

array pointed to by `dst`. Each conversion takes place as if by a call to the **`wcrtomb`** function.[417)]

3 If `dst` is not a null pointer, the pointer object pointed to by `src` is assigned either a null pointer (if conversion stopped due to reaching a terminating null wide character) or the address just past the last wide character converted (if any). If conversion stopped due to reaching a terminating null wide character, the resulting state described is the initial conversion state.

Returns

4 If conversion stops because a wide character is reached that does not correspond to a valid multibyte character, an encoding error occurs: the **`wcsrtombs`** function stores the value of the macro **`EILSEQ`** in **`errno`** and returns `(size_t)(-1)`; the conversion state is unspecified. Otherwise, it returns the number of bytes in the resulting multibyte character sequence, not including the terminating null character (if any).

[417)] If conversion stops because a terminating null wide character has been reached, the bytes stored include those necessary to reach the initial shift state immediately before the null byte.

© ISO 2024 — All rights reserved

7.32 Wide character classification and mapping utilities <wctype.h>

7.32.1 Introduction

1 The header <wctype.h> defines one macro, and declares three data types and many functions.[418)]

2 The types declared are **wint_t** described in 7.31.1;

```
wctrans_t
```

which is a scalar type that can hold values which represent locale-specific character mappings; and

```
wctype_t
```

which is a scalar type that can hold values which represent locale-specific character classifications.

3 The macro defined is **WEOF** (described in 7.31.1).

4 The functions declared are grouped as follows:

— Functions that provide wide character classification;

— Extensible functions that provide wide character classification;

— Functions that provide wide character case mapping;

— Extensible functions that provide wide character mapping.

5 For all functions described in this subclause that accept an argument of type **wint_t**, the value shall be representable as a **wchar_t** or shall equal the value of the macro **WEOF**. If this argument has any other value, the behavior is undefined.

6 The behavior of these functions is affected by the **LC_CTYPE** category of the current locale.

7.32.2 Wide character classification utilities

1 The header <wctype.h> declares several functions useful for classifying wide characters.

2 The term *printing wide character* refers to a member of a locale-specific set of wide characters, each of which occupies at least one printing position on a display device. The term *control wide character* refers to a member of a locale-specific set of wide characters that are not printing wide characters.

7.32.2.1 Wide character classification functions

1 The functions in this subclause return nonzero (true) if and only if the value of the argument `wc` conforms to that in the description of the function.

2 Each of the following functions returns true for each wide character that corresponds (as if by a call to the **wctob** function) to a single-byte character for which the corresponding character classification function from 7.4.2 returns true, except that the **iswgraph** and **iswpunct** functions may differ with respect to wide characters other than `L' '` that are both printing and white-space wide characters.[419)]

Forward references: the **wctob** function (7.31.6.1.2).

[418)] See "future library directions" (7.33.21).

[419)] For example, if the expression **isalpha**(**wctob**(`wc`)) evaluates to true, then the call **iswalpha**(`wc`) also returns true. But, if the expression **isgraph**(**wctob**(`wc`)) evaluates to true (which cannot occur for `wc == L' '` of course), then either **iswgraph**(`wc`) or **iswprint**(`wc`)&& **iswspace**(`wc`) is true, but not both.

 © ISO 2024 — All rights reserved

7.32.2.1.1 The `iswalnum` function

Synopsis

1
```
#include <wctype.h>
int iswalnum(wint_t wc);
```

Description

2 The **`iswalnum`** function tests for any wide character for which **`iswalpha`** or **`iswdigit`** is true.

7.32.2.1.2 The `iswalpha` function

Synopsis

1
```
#include <wctype.h>
int iswalpha(wint_t wc);
```

Description

2 The **`iswalpha`** function tests for any wide character for which **`iswupper`** or **`iswlower`** is true, or any wide character that is one of a locale-specific set of alphabetic wide characters for which none of **`iswcntrl`**, **`iswdigit`**, **`iswpunct`**, or **`iswspace`** is true.[420)]

7.32.2.1.3 The `iswblank` function

Synopsis

1
```
#include <wctype.h>
int iswblank(wint_t wc);
```

Description

2 The **`iswblank`** function tests for any wide character that is a standard blank wide character or is one of a locale-specific set of wide characters for which **`iswspace`** is true and that is used to separate words within a line of text. The standard blank wide characters are the following: space (`L' '`), and horizontal tab (`L'\t'`). In the `"C"` locale, **`iswblank`** returns true only for the standard blank characters.

7.32.2.1.4 The `iswcntrl` function

Synopsis

1
```
#include <wctype.h>
int iswcntrl(wint_t wc);
```

Description

2 The **`iswcntrl`** function tests for any control wide character.

7.32.2.1.5 The `iswdigit` function

Synopsis

1
```
#include <wctype.h>
int iswdigit(wint_t wc);
```

Description

2 The **`iswdigit`** function tests for any wide character that corresponds to a decimal-digit character (as defined in 5.2.1).

[420)] The functions **`iswlower`** and **`iswupper`** test true or false separately for each of these additional wide characters; all four combinations are possible.

© ISO 2024 — All rights reserved

7.32.2.1.6 The `iswgraph` function

Synopsis

1
```
#include <wctype.h>
int iswgraph(wint_t wc);
```

Description

2 The **iswgraph** function tests for any wide character for which **iswprint** is true and **iswspace** is false.[421)]

7.32.2.1.7 The `iswlower` function

Synopsis

1
```
#include <wctype.h>
int iswlower(wint_t wc);
```

Description

2 The **iswlower** function tests for any wide character that corresponds to a lowercase letter or is one of a locale-specific set of wide characters for which none of **iswcntrl**, **iswdigit**, **iswpunct**, or **iswspace** is true.

7.32.2.1.8 The `iswprint` function

Synopsis

1
```
#include <wctype.h>
int iswprint(wint_t wc);
```

Description

2 The **iswprint** function tests for any printing wide character.

7.32.2.1.9 The `iswpunct` function

Synopsis

1
```
#include <wctype.h>
int iswpunct(wint_t wc);
```

Description

2 The **iswpunct** function tests for any printing wide character that is one of a locale-specific set of punctuation wide characters for which neither **iswspace** nor **iswalnum** is true.[421)]

7.32.2.1.10 The `iswspace` function

Synopsis

1
```
#include <wctype.h>
int iswspace(wint_t wc);
```

Description

2 The **iswspace** function tests for any wide character that corresponds to a locale-specific set of white-space wide characters for which none of **iswalnum**, **iswgraph**, or **iswpunct** is true.

7.32.2.1.11 The `iswupper` function

Synopsis

1
```
#include <wctype.h>
int iswupper(wint_t wc);
```

[421)] Note that the behavior of the **iswgraph** and **iswpunct** functions can differ from their corresponding functions in 7.4.2 with respect to printing, white-space, single-byte execution characters other than `' '`.

 © ISO 2024 — All rights reserved

Description

2 The **iswupper** function tests for any wide character that corresponds to an uppercase letter or is one of a locale-specific set of wide characters for which none of **iswcntrl**, **iswdigit**, **iswpunct**, or **iswspace** is true.

7.32.2.1.12 The iswxdigit function

Synopsis

1
```
#include <wctype.h>
int iswxdigit(wint_t wc);
```

Description

2 The **iswxdigit** function tests for any wide character that corresponds to a hexadecimal-digit character (as defined in 6.4.4.2).

7.32.2.2 Extensible wide character classification functions

1 The functions **wctype** and **iswctype** provide extensible wide character classification as well as testing equivalent to that performed by the functions described in the previous subclause (7.32.2.1).

7.32.2.2.1 The iswctype function

Synopsis

1
```
#include <wctype.h>
int iswctype(wint_t wc, wctype_t desc);
```

Description

2 The **iswctype** function determines whether the wide character `wc` has the property described by `desc`. The current setting of the **LC_CTYPE** category shall be the same as during the call to **wctype** that returned the value `desc`.

3 Each of the following expressions has a truth-value equivalent to the call to the wide character classification function (7.32.2.1) in the comment that follows the expression:

```
iswctype(wc, wctype("alnum"))    // iswalnum(wc)
iswctype(wc, wctype("alpha"))    // iswalpha(wc)
iswctype(wc, wctype("blank"))    // iswblank(wc)
iswctype(wc, wctype("cntrl"))    // iswcntrl(wc)
iswctype(wc, wctype("digit"))    // iswdigit(wc)
iswctype(wc, wctype("graph"))    // iswgraph(wc)
iswctype(wc, wctype("lower"))    // iswlower(wc)
iswctype(wc, wctype("print"))    // iswprint(wc)
iswctype(wc, wctype("punct"))    // iswpunct(wc)
iswctype(wc, wctype("space"))    // iswspace(wc)
iswctype(wc, wctype("upper"))    // iswupper(wc)
iswctype(wc, wctype("xdigit"))   // iswxdigit(wc)
```

Returns

4 The **iswctype** function returns nonzero (true) if and only if the value of the wide character `wc` has the property described by `desc`. If `desc` is zero, the **iswctype** function returns zero (false).

Forward references: the **wctype** function (7.32.2.2.2).

7.32.2.2.2 The wctype function

Synopsis

1
```
#include <wctype.h>
wctype_t wctype(const char *property);
```

© ISO 2024 — All rights reserved

Description

2 The **wctype** function constructs a value with type **wctype_t** that describes a class of wide characters identified by the string argument property.

3 The strings listed in the description of the **iswctype** function shall be valid in all locales as property arguments to the **wctype** function.

Returns

4 If property identifies a valid class of wide characters according to the **LC_CTYPE** category of the current locale, the **wctype** function returns a nonzero value that is valid as the second argument to the **iswctype** function; otherwise, it returns zero.

7.32.3 Wide character case mapping utilities

1 The header <wctype.h> declares several functions useful for mapping wide characters.

7.32.3.1 Wide character case mapping functions

7.32.3.1.1 The towlower function

Synopsis

1
```
#include <wctype.h>
wint_t towlower(wint_t wc);
```

Description

2 The **towlower** function converts an uppercase letter to a corresponding lowercase letter.

Returns

3 If the argument is a wide character for which **iswupper** is true and there are one or more corresponding wide characters, as specified by the current locale, for which **iswlower** is true, the **towlower** function returns one of the corresponding wide characters (always the same one for any given locale); otherwise, the argument is returned unchanged.

7.32.3.1.2 The towupper function

Synopsis

1
```
#include <wctype.h>
wint_t towupper(wint_t wc);
```

Description

2 The **towupper** function converts a lowercase letter to a corresponding uppercase letter.

Returns

3 If the argument is a wide character for which **iswlower** is true and there are one or more corresponding wide characters, as specified by the current locale, for which **iswupper** is true, the **towupper** function returns one of the corresponding wide characters (always the same one for any given locale); otherwise, the argument is returned unchanged.

7.32.3.2 Extensible wide character case mapping functions

1 The functions **wctrans** and **towctrans** provide extensible wide character mapping as well as case mapping equivalent to that performed by the functions described in the previous subclause (7.32.3.1).

7.32.3.2.1 The towctrans function

Synopsis

1
```
#include <wctype.h>
wint_t towctrans(wint_t wc, wctrans_t desc);
```

 © ISO 2024 — All rights reserved

Description

2 The **towctrans** function maps the wide character wc using the mapping described by desc. The current setting of the **LC_CTYPE** category shall be the same as during the call to **wctrans** that returned the value desc.

3 Each of the following expressions behaves the same as the call to the wide character case mapping function (7.32.3.1) in the comment that follows the expression:

```
towctrans(wc, wctrans("tolower"))    // towlower(wc)
towctrans(wc, wctrans("toupper"))    // towupper(wc)
```

Returns

4 The **towctrans** function returns the mapped value of wc using the mapping described by desc. If desc is zero, the **towctrans** function returns the value of wc.

7.32.3.2.2 The wctrans function

Synopsis

1
```
#include <wctype.h>
wctrans_t wctrans(const char *property);
```

Description

2 The **wctrans** function constructs a value with type **wctrans_t** that describes a mapping between wide characters identified by the string argument property.

3 The strings listed in the description of the **towctrans** function shall be valid in all locales as property arguments to the **wctrans** function.

Returns

4 If property identifies a valid mapping of wide characters according to the **LC_CTYPE** category of the current locale, the **wctrans** function returns a nonzero value that is valid as the second argument to the **towctrans** function; otherwise, it returns zero.

© ISO 2024 — All rights reserved

7.33 Future library directions

1 Although grouped under individual headers, all the external names identified as reserved identifiers or potentially reserved identifiers in this subclause remain so regardless of which headers are included in the program.

7.33.1 Complex arithmetic <complex.h>

1 The function names

cacospi	**cexp10m1**	**clog10**	**crootn**
casinpi	**cexp10**	**clog1p**	**crsqrt**
catanpi	**cexp2m1**	**clog2p1**	**csinpi**
ccompoundn	**cexp2**	**clog2**	**ctanpi**
ccospi	**cexpm1**	**clogp1**	**ctgamma**
cerfc	**clgamma**	**cpown**	
cerf	**clog10p1**	**cpowr**	

and the same names suffixed with **f** or **l** are potentially reserved identifiers and may be added to the declarations in the <complex.h> header.

7.33.2 Character handling <ctype.h>

1 Function names that begin with either **is** or **to**, and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <ctype.h> header.

7.33.3 Errors <errno.h>

1 Macros that begin with **E** and a digit or **E** and an uppercase letter may be added to the macros defined in the <errno.h> header by a future revision of this document or by an implementation.

7.33.4 Floating-point environment <fenv.h>

1 Macros that begin with **FE_** and an uppercase letter may be added to the macros defined in the <fenv.h> header by a future revision of this document or by an implementation.

7.33.5 Characteristics of floating types <float.h>

1 Macros that begin with **DBL_**, **DEC32_**, **DEC64_**, **DEC128_**, **DEC_**, **FLT_**, or **LDBL_** and an uppercase letter are potentially reserved identifiers and may be added to the macros defined in the <float.h> header.

2 Use of the **DECIMAL_DIG** macro is an obsolescent feature. A similar type-specific macro, such as **LDBL_DECIMAL_DIG**, can be used instead.

3 The use of **FLT_HAS_SUBNORM**, **DBL_HAS_SUBNORM**, and **LDBL_HAS_SUBNORM** macros is an obsolescent feature.

7.33.6 Format conversion of integer types <inttypes.h>

1 Macros that begin with either **PRI** or **SCN**, and either a lowercase letter, B, or X are potentially reserved identifiers and may be added to the macros defined in the <inttypes.h> header.

2 Function names that begin with **str**, or **wcs** and a lowercase letter are potentially reserved identifiers may be added to the declarations in the <inttypes.h> header.

7.33.7 Localization <locale.h>

1 Macros that begin with **LC_** and an uppercase letter may be added to the macros defined in the <locale.h> header by a future revision of this document or by an implementation.

7.33.8 Mathematics <math.h>

1 Macros that begin with **FP_** and an uppercase letter may be added to the macros defined in the <math.h> header by a future revision of this document or by an implementation.

© ISO 2024 — All rights reserved

2 Macros that begin with **MATH_** and an uppercase letter are potentially reserved identifiers and may be added to the macros in the <math.h> header.

3 Function names that begin with **is** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <math.h> header.

4 Function names that begin with **cr_** are potentially reserved identifiers and may be added to the <math.h> header. The **cr_** prefix is intended to indicate a correctly rounded version of the function.

5 Use of the macros **INFINITY**, **DEC_INFINITY**, **NAN**, and **DEC_NAN** in <math.h> is an obsolescent feature. Instead, use the same macros in <float.h>.

7.33.9 Signal handling <signal.h>

1 Macros that begin with either **SIG** and an uppercase letter or **SIG_** and an uppercase letter may be added to the macros defined in the <signal.h> header by a future revision of this document or by an implementation.

7.33.10 Atomics <stdatomic.h>

1 Macros that begin with **ATOMIC_** and an uppercase letter are potentially reserved identifiers and may be added to the macros defined in the <stdatomic.h> header. Typedef names that begin with either **atomic_** or **memory_**, and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <stdatomic.h> header. Enumeration constants that begin with **memory_order_** and a lowercase letter are potentially reserved identifiers and may be added to the definition of the **memory_order** type in the <stdatomic.h> header. Function names that begin with **atomic_** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <stdatomic.h> header.

7.33.11 Boolean type and values <stdbool.h>

1 The macro **__bool_true_false_are_defined** is an obsolescent feature.

7.33.12 Bit and byte utilities <stdbit.h>

1 Type and function names that begin with **stdc_** are potentially reserved identifiers and may be added to the declarations in the <stdbit.h> header.

7.33.13 Checked Arithmetic Functions <stdckdint.h>

1 Type and function names that begin with **ckd_** are potentially reserved identifiers and may be added to the declarations in the <stdckdint.h> header.

7.33.14 Integer types <stdint.h>

1 Typedef names beginning with **int** or **uint** and ending with **_t** are potentially reserved identifiers and may be added to the types defined in the <stdint.h> header. Macro names beginning with **INT** or **UINT** and ending with **_MAX**, **_MIN**, **_WIDTH**, or **_C** are potentially reserved identifiers and may be added to the macros defined in the <stdint.h> header.

7.33.15 Input/output <stdio.h>

1 Lowercase letters may be added to the conversion specifiers and length modifiers in **fprintf** and **fscanf**. Other characters may be used in extensions. The specifier B for **printf** may become mandatory in future versions of this document.

2 The use of **ungetc** on a binary stream where the file position indicator is zero prior to the call is an obsolescent feature.

7.33.16 General utilities <stdlib.h>

1 Function names that begin with **str** or **wcs** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <stdlib.h> header.

2 Suppressing the macro definition of **bsearch** to access the actual function is an obsolescent feature.

© ISO 2024 — All rights reserved

7.33.17 String handling <string.h>

1 Function names that begin with **str**, **mem**, or **wcs** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <string.h> header.

2 Suppressing the macro definitions of **memchr**, **strchr**, **strpbrk**, **strrchr**, or **strstr** to access the corresponding actual function is an obsolescent feature.

7.33.18 Date and time <time.h>

1 Macros beginning with **TIME_** and an uppercase letter may be added to the macros in the <time.h> header by a future revision of this document or by an implementation.

2 The time bases **TIME_MONOTONIC**, **TIME_ACTIVE** and **TIME_THREAD_ACTIVE** may become mandatory in future versions of this standard.

7.33.19 Threads <threads.h>

1 Function names, type names, and enumeration constants that begin with either **cnd_**, **mtx_**, **thrd_**, or **tss_**, and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <threads.h> header.

7.33.20 Extended multibyte and wide character utilities <wchar.h>

1 Function names that begin with **wcs** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <wchar.h> header.

2 Lowercase letters may be added to the conversion specifiers and length modifiers in **fwprintf** and **fwscanf**. Other characters may be used in extensions.

3 Suppressing the macro definitions of **wcschr**, **wcspbrk**, **wcsrchr**, **wmemchr**, or **wcsstr** to access the corresponding actual function is an obsolescent feature.

7.33.21 Wide character classification and mapping utilities <wctype.h>

1 Function names that begin with **is** or **to** and a lowercase letter are potentially reserved identifiers and may be added to the declarations in the <wctype.h> header.

© ISO 2024 — All rights reserved